Faculty Evaluation System

Admin Module — Source Code

20 files

Table of Contents

1. admin/admin-styles.css
2. admin/dashboard.html
3. admin/js/core/admin-core.js
4. admin/js/core/admin-nav.js
5. admin/js/core/admin-router.js
6. admin/js/core/admin-scoring.js
7. admin/js/modules/adminEval.js
8. admin/js/modules/adminQuestions.js
9. admin/js/modules/admindept.js
10. admin/js/modules/adminfeatures.js
11. admin/js/modules/adminrate.js
12. admin/js/records/admin-bulk.js
13. admin/js/records/admin-courses.js
14. admin/js/records/admin-students.js
15. admin/js/records/admin-subjects.js
16. admin/js/records/admin-teachers.js
17. admin/js/reports/admin-annex.js
18. admin/js/reports/admin-fer.js
19. admin/js/reports/admin-reports.js
20. admin/js/reports/admin-sef.js

admin/admin-styles.css

2159 lines

```css
/* Graduate School and Custom Department Styles */
.dept-GS { background: linear-gradient(135deg, #4c1d95, #8b5cf6); }
.dept-custom { background: linear-gradient(135deg, #0f766e, #14b8a6); }

/* ============================================================
   TEACHER EXPANDABLE ROW STYLES
   ============================================================ */
.teacher-row {
    cursor: pointer;
    transition: background 0.15s;
}

.teacher-row:hover {
    background: var(--primary-light);
}

.teacher-details-row td {
    padding: 0 !important;
}

.teacher-details-cell {
    padding: 0 !important;
}

.teacher-details-container {
    background: #f8fafc;
    border-top: 1px solid var(--border);
    border-bottom: 1px solid var(--border);
    animation: fadeIn 0.2s ease;
}

/* Report teacher row hover (Annex D click) */
.report-teacher-row {
    cursor: pointer;
    transition: background 0.15s;
}

.report-teacher-row:hover {
    background: var(--primary-light);
}

/* ============================================================
   ANNEX D PRINT STYLES
   ============================================================ */
@media print {
    /* Printed forms: inputs and date pickers print as plain text on a ruled line. */
    [data-print-field] {
        border-bottom: 1px solid #333 !important;
    }
    input[type="date"] {
        border: none !important;
        border-bottom: 1px solid #333 !important;
        -webkit-appearance: none;
        appearance: none;
        color: #000 !important;
        background: transparent !important;
    }
    input[type="date"]::-webkit-calendar-picker-indicator,
    input[type="date"]::-webkit-inner-spin-button,
    input[type="date"]::-webkit-clear-button {
        display: none !important;
    }

    .sidebar,
    .page-header,
    .btn,
    .modal-close,
    .sidebar-footer,
    .mobile-menu-btn {
        display: none !important;
    }
    .main-content {
        margin-left: 0 !important;
        padding: 0 !important;
    }
    .modal-overlay {
        position: relative !important;
        display: block !important;
        background: white !important;
    }
    .modal {
        position: relative !important;
        transform: none !important;
        margin: 0 !important;
        width: 100% !important;
        max-width: 100% !important;
        box-shadow: none !important;
    }
}

/* Department tag inline */
.dept-tag-inline {
    display: inline-block;
    padding: 2px 8px;
    background: var(--primary-light);
    color: var(--primary);
    border-radius: 12px;
    font-size: 0.7rem;
    font-weight: 500;
}
```

```css

/* Improved table responsiveness */
.table-wrap {
    overflow-x: auto;
    -webkit-overflow-scrolling: touch;
    border-radius: var(--radius);
}

.data-table {
    width: 100%;
    border-collapse: collapse;
    font-size: 0.8rem;
}

.data-table th,
.data-table td {
    padding: 10px 12px;
    text-align: left;
    border-bottom: 1px solid var(--border);
    vertical-align: middle;
}

.data-table th {
    background: var(--bg);
    font-weight: 700;
    color: var(--muted-foreground);
    white-space: normal;
    word-break: break-word;
    vertical-align: bottom;
}

/* Class details expandable row */
.class-details-row {
    background: #f8fafc;
}

.class-details-row td {
    padding: 0;
}

.class-details-container {
    padding: 16px;
    border-top: 1px solid var(--border);
}

/* Progress bar improvements */
.progress-bar {
    background: #e2e8f0;
    border-radius: 9999px;
    height: 8px;
    overflow: hidden;
}

.progress-fill {
    background: var(--primary);
    height: 100%;
    border-radius: 9999px;
    transition: width 0.3s ease;
}

/* Score display improvements */
.score-chip {
    display: inline-block;
    padding: 2px 8px;
    border-radius: 20px;
    font-size: 0.7rem;
    font-weight: 600;
}

.score-chip.high {
    background: #dcfce7;
    color: #166534;
}

.score-chip.medium {
    background: #fef3c7;
    color: #92400e;
}

.score-chip.low {
    background: #fee2e2;
    color: #991b1b;
}

:root {
  --sidebar-w: 240px;
}

#app {
  display: flex;
  min-height: 100vh;
}

body {
  margin: 0;
  padding: 0;
  min-height: 100vh;
  background-color: #f8fafc;
  position: relative;
}

body::before {
  content: "";
  position: fixed;
  top: 0;
```

```
  left: 0;
  width: 100%;
  height: 100%;
  background-image: url('../nwssu_logo.jpg');
  background-position: center;
  background-repeat: no-repeat;
  background-size: 60%;
  background-color: #e2e8f0;
  background-blend-mode: darken;
  opacity: 0.4;
  z-index: -1;
  pointer-events: none;
}

/* Sidebar Styles */
.sidebar {
  width: var(--sidebar-w);
  background: #064e3b;
  position: fixed;
  top: 0;
  left: 0;
  bottom: 0;
  display: flex;
  flex-direction: column;
  z-index: 100;
  transition: transform 0.3s ease;
}

.sidebar-brand {
  padding: 16px 14px 14px;
  border-bottom: 1px solid rgba(255,255,255,0.08);
  display: flex;
  align-items: center;
  gap: 10px;
}

.sidebar-brand-text h2 {
  font-size: 0.82rem;
  font-weight: 700;
  color: #fff;
  margin: 0;
}

.sidebar-brand-text p {
  font-size: 0.65rem;
  color: #86efac;
  margin-top: 2px;
}

.sidebar-nav {
  flex: 1;
  padding: 12px 10px;
  overflow-y: auto;
  scrollbar-width: none;
  -ms-overflow-style: none;
}

.nav-section-label {
  font-size: 0.62rem;
  font-weight: 700;
  color: #86efac;
  letter-spacing: 0.1em;
  text-transform: uppercase;
  padding: 8px 10px 4px;
}

.nav-item {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 9px 10px;
  border-radius: 7px;
  cursor: pointer;
  color: #94a3b8;
  font-size: 0.82rem;
  font-weight: 500;
  transition: all 0.15s;
  margin-bottom: 2px;
}

.nav-item:hover {
  background: rgba(255,255,255,0.06);
  color: #fff;
}

.nav-item.active {
  background: #059669;
  color: #fff;
}

.sidebar-footer {
  padding: 12px 10px;
  border-top: 1px solid rgba(255,255,255,0.08);
}

.logout-btn {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 9px 10px;
  border-radius: 7px;
  cursor: pointer;
  color: #f87171;
  font-size: 0.82rem;
  font-weight: 600;
```

```
  transition: all 0.15s;
  width: 100%;
  background: none;
  border: none;
  font-family: 'Plus Jakarta Sans', sans-serif;
}

.logout-btn:hover {
  background: rgba(248,113,113,0.1);
}

/* Main Content */
.main-content {
  margin-left: var(--sidebar-w);
  flex: 1;
  padding: 28px 32px;
  min-height: 100vh;
}

.page {
  display: none;
}

.page.active {
  display: block;
  animation: fadeIn 0.2s ease;
}

.page-header {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  margin-bottom: 24px;
  flex-wrap: wrap;
  gap: 12px;
}

.page-title h1 {
  font-size: 1.5rem;
  font-weight: 800;
  color: var(--ink);
  letter-spacing: -0.02em;
}

.page-title p {
  color: var(--muted);
  font-size: 0.82rem;
  margin-top: 3px;
}

/* Stats Cards */
.stats-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(160px, 1fr));
  gap: 16px;
  margin-bottom: 24px;
}

.stat-card {
  background: var(--white);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  padding: 18px 20px;
  box-shadow: var(--shadow);
}

/* ==== TERM (HISTORY) SELECTOR ==== */
.term-selector {
  position: relative;
  display: inline-flex;
  align-items: center;
  gap: 8px;
  background: #fff;
  border: 1px solid var(--border);
  border-radius: 10px;
  padding: 0 10px 0 12px;
  box-shadow: 0 1px 2px rgba(0,0,0,0.05);
  transition: border-color 0.15s, box-shadow 0.15s;
}
.term-selector:hover { border-color: var(--primary); box-shadow: 0 2px 10px rgba(0,0,0,0.08); }
.term-selector .term-cal { color: var(--primary); flex-shrink: 0; }
.term-selector select {
  appearance: none;
  -webkit-appearance: none;
  border: none;
  background: transparent;
  font-family: inherit;
  font-size: 0.82rem;
  font-weight: 600;
  color: var(--ink);
  padding: 9px 26px 9px 0;
  cursor: pointer;
  outline: none;
}
.term-selector .term-chevron {
  position: absolute;
  right: 9px;
  color: var(--muted);
  pointer-events: none;
}

/* ==== REPORTS LOADING STATE ==== */
.reports-loading {
  display: flex;
  flex-direction: column;
```

```
  align-items: center;
  justify-content: center;
  gap: 14px;
  padding: 64px 20px;
  animation: rptfade 0.2s ease;
}
@keyframes rptfade { from { opacity: 0; } to { opacity: 1; } }
.reports-spinner {
  width: 40px;
  height: 40px;
  border: 3px solid var(--primary-light);
  border-top-color: var(--primary);
  border-radius: 50%;
  animation: rptspin 0.7s linear infinite;
}
@keyframes rptspin { to { transform: rotate(360deg); } }
.reports-loading-text { font-size: 0.9rem; font-weight: 700; color: var(--ink); }
.reports-loading-sub { font-size: 0.75rem; color: var(--muted); }

.stat-card-clickable {
  cursor: pointer;
  transition: transform 0.15s, box-shadow 0.15s;
}

.stat-card-clickable:hover {
  transform: translateY(-2px);
  box-shadow: 0 6px 18px rgba(0,0,0,0.10);
}

.stat-card-clickable:focus,
.stat-card-clickable:active {
  outline: none;
}

.stat-icon {
  width: 40px;
  height: 40px;
  border-radius: 10px;
  display: flex;
  align-items: center;
  justify-content: center;
  margin-bottom: 12px;
}

.stat-card .value {
  font-size: 1.8rem;
  font-weight: 800;
  letter-spacing: -0.03em;
  line-height: 1;
}

.stat-card .label {
  font-size: 0.75rem;
  color: var(--muted);
  font-weight: 500;
  margin-top: 4px;
}

.period-card {
  background: var(--white);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  padding: 20px;
  margin-bottom: 24px;
  box-shadow: var(--shadow);
}

.sem-card {
  border: 2px solid var(--border);
  border-radius: var(--radius);
  padding: 16px 18px;
  margin-bottom: 10px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  transition: all 0.15s;
}

.sem-card.active-sem {
  border-color: var(--success);
  background: var(--success-light);
}

/* Badges */
.eval-deadline-banner {
  background: var(--warning-light);
  border: 1px solid #fcd34d;
  border-radius: 8px;
  padding: 10px 14px;
  font-size: 0.78rem;
  color: var(--warning);
  font-weight: 600;
  margin-bottom: 16px;
  display: flex;
  align-items: center;
  gap: 8px;
}

.closed-banner {
  background: var(--danger-light);
  border: 1px solid #fca5a5;
  border-radius: 8px;
  padding: 10px 14px;
  font-size: 0.78rem;
  color: var(--danger);
```

```
  font-weight: 600;
  margin-bottom: 16px;
  display: flex;
  align-items: center;
  gap: 8px;
}

/* Rank Badges */
.rank-badge {
  width: 32px;
  height: 32px;
  border-radius: 50%;
  background: var(--primary-light);
  color: var(--primary);
  display: inline-flex;
  align-items: center;
  justify-content: center;
  font-size: 0.8rem;
  font-weight: 800;
}

.rank-badge.rank-1 {
  background: #fef3c7;
  color: #d97706;
}

.rank-badge.rank-2 {
  background: #f1f5f9;
  color: #475569;
}

.rank-badge.rank-3 {
  background: #fef2f2;
  color: #dc2626;
}

/* Audit Log */
.audit-row {
  display: flex;
  gap: 12px;
  padding: 12px 0;
  border-bottom: 1px solid var(--border);
  align-items: flex-start;
}

.audit-row:last-child {
  border-bottom: none;
}

.audit-dot {
  width: 8px;
  height: 8px;
  border-radius: 50%;
  background: var(--primary);
  margin-top: 5px;
  flex-shrink: 0;
}

.audit-time {
  font-size: 0.7rem;
  color: var(--muted);
  font-family: 'JetBrains Mono', monospace;
}

/* Score Display */
.score-breakdown {
  background: var(--bg);
  border-radius: 8px;
  padding: 14px;
  margin-top: 12px;
}

.score-row {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 6px 0;
  font-size: 0.8rem;
  border-bottom: 1px solid var(--border);
}

.score-row:last-child {
  border-bottom: none;
}

.total-score-display {
  text-align: center;
  padding: 20px;
  background: var(--primary-light);
  border-radius: var(--radius);
  margin-top: 16px;
}

.total-score-display .big-score {
  font-size: 3rem;
  font-weight: 800;
  color: var(--primary);
  letter-spacing: -0.04em;
  line-height: 1;
}

.total-score-display .out-of {
  font-size: 0.85rem;
  color: var(--muted);
  margin-top: 4px;
```

```css
}

.total-score-display .remarks-badge {
  display: inline-block;
  margin-top: 10px;
  padding: 5px 16px;
  border-radius: 20px;
  font-size: 0.8rem;
  font-weight: 700;
  background: var(--primary);
  color: #fff;
}

/* Modal */
.confirm-modal .confirm-icon {
  width: 52px;
  height: 52px;
  border-radius: 50%;
  display: flex;
  align-items: center;
  justify-content: center;
  margin: 0 auto 14px;
}

.checklist {
  list-style: none;
}

.checklist li {
  display: flex;
  align-items: center;
  gap: 8px;
  padding: 5px 0;
  font-size: 0.8rem;
}

.overlay-bg {
  position: fixed;
  inset: 0;
  background: rgba(0,0,0,0.5);
  z-index: 99;
  display: none;
}

.overlay-bg.open {
  display: block;
}

.mobile-menu-btn {
  display: none;
  align-items: center;
  justify-content: center;
  background: none;
  border: none;
  cursor: pointer;
  color: var(--ink);
  padding: 6px;
  border-radius: 6px;
}

/* Department Styles */
.dept-nav-item {
  justify-content: space-between;
  padding: 7px 10px;
}

.dept-nav-item > span:first-of-type {
  font-weight: 700;
  font-size: 0.8rem;
  letter-spacing: 0.03em;
}

.dept-tag {
  font-size: 0.62rem;
  color: #86efac;
  font-weight: 500;
  opacity: 0.75;
  margin-left: auto;
  white-space: nowrap;
}

.dept-nav-item.active .dept-tag {
  opacity: 1;
  color: #bbf7d0;
}

.dept-banner {
  border-radius: 12px;
  padding: 24px 28px;
  margin-bottom: 24px;
  display: flex;
  align-items: center;
  gap: 20px;
  color: #fff;
  position: relative;
  overflow: hidden;
}

.dept-banner::before {
  content: '';
  position: absolute;
  right: -20px;
  top: -20px;
  width: 140px;
  height: 140px;
```

```
  border-radius: 50%;
  background: rgba(255,255,255,0.07);
}

.dept-banner::after {
  content: '';
  position: absolute;
  right: 40px;
  bottom: -30px;
  width: 90px;
  height: 90px;
  border-radius: 50%;
  background: rgba(255,255,255,0.05);
}

.dept-banner-icon {
  width: 56px;
  height: 56px;
  border-radius: 14px;
  background: rgba(255,255,255,0.15);
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 1.6rem;
  flex-shrink: 0;
  z-index: 1;
}

.dept-banner-info {
  z-index: 1;
}

.dept-banner-info h2 {
  font-size: 1.3rem;
  font-weight: 800;
  margin: 0 0 3px;
  letter-spacing: -0.01em;
}

.dept-banner-info p {
  font-size: 0.78rem;
  opacity: 1;
  margin: 0;
}

.dept-stats-row {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(140px, 1fr));
  gap: 14px;
  margin-bottom: 24px;
}

.dept-stat {
  background: var(--white);
  border: 1px solid var(--border);
  border-radius: 10px;
  padding: 16px 18px;
  box-shadow: var(--shadow);
  display: flex;
  align-items: center;
  gap: 12px;
}

.dept-stat-icon {
  width: 38px;
  height: 38px;
  border-radius: 9px;
  display: flex;
  align-items: center;
  justify-content: center;
  flex-shrink: 0;
  font-size: 1.1rem;
}

.dept-stat-val {
  font-size: 1.5rem;
  font-weight: 800;
  line-height: 1;
  letter-spacing: -0.02em;
}

.dept-stat-label {
  font-size: 0.7rem;
  color: var(--muted);
  font-weight: 500;
  margin-top: 2px;
}

.dept-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 20px;
}

.dept-table-section {
  background: var(--white);
  border: 1px solid var(--border);
  border-radius: 12px;
  overflow: hidden;
  box-shadow: var(--shadow);
}

.dept-table-header {
  padding: 14px 18px;
  border-bottom: 1px solid var(--border);
```

```
  display: flex;
  align-items: center;
  justify-content: space-between;
}

.dept-table-header h4 {
  font-size: 0.88rem;
  font-weight: 700;
  margin: 0;
}

.dept-table-header .count-badge {
  background: var(--primary-light);
  color: var(--primary);
  font-size: 0.72rem;
  font-weight: 700;
  padding: 2px 10px;
  border-radius: 20px;
}

/* Department Colors */
.dept-COED { background: linear-gradient(135deg, #1d4ed8, #3b82f6); }
.dept-CCJS { background: linear-gradient(135deg, #4a4a4a, #9ca3af); }
.dept-CCIS { background: linear-gradient(135deg, #854d0e, #eab308); }
.dept-CON  { background: linear-gradient(135deg, #064e3b, #059669); }
.dept-CEA  { background: linear-gradient(135deg, #9a3412, #ea580c); }
.dept-COM  { background: linear-gradient(135deg, #0c4a6e, #0ea5e9); }
.dept-CAT  { background: linear-gradient(135deg, #4c0519, #be123c); }

/* Responsive */
@media (max-width: 768px) {
  .sidebar { transform: translateX(-100%); }
  .sidebar.open { transform: translateX(0); }
  .main-content { margin-left: 0; padding: 16px; }
  .stats-grid { grid-template-columns: repeat(2, 1fr); }
  .mobile-menu-btn { display: flex !important; }
  .dept-grid { grid-template-columns: 1fr; }
}

@keyframes fadeIn {
  from { opacity: 0; transform: translateY(10px); }
  to { opacity: 1; transform: translateY(0); }
}

@keyframes slideIn {
  from { transform: translateX(100%); opacity: 0; }
  to { transform: translateX(0); opacity: 1; }
}


/** for department logo*/
/* Department banner icon - supports both text and images */
.dept-banner-icon {
    width: 56px;
    height: 56px;
    border-radius: 14px;
    background: rgba(255,255,255,0.15);
    display: flex;
    align-items: center;
    justify-content: center;
    flex-shrink: 0;
    z-index: 1;
}

.dept-banner-icon img {
    width: 40px;
    height: 40px;
    border-radius: 8px;
    object-fit: cover;
}


/* ============================================================
   STUDENT DEPT GROUP HEADERS
   ============================================================ */
.dept-group-header-row td {
    padding: 0 !important;
    border-bottom: none !important;
}

.dept-group-header {
    display: flex;
    align-items: center;
    gap: 8px;
    padding: 10px 14px;
    background: linear-gradient(90deg, var(--primary-light) 0%, transparent 100%);
    color: var(--primary);
    font-size: 0.78rem;
    font-weight: 700;
    border-bottom: 1px solid var(--border);
    margin-top: 4px;
}

.dept-group-count {
    margin-left: auto;
    font-size: 0.68rem;
    font-weight: 600;
    color: var(--muted);
    background: var(--bg);
    padding: 2px 8px;
    border-radius: 10px;
    border: 1px solid var(--border);
}

.dept-group-student-row td {
```

```
    padding-left: 18px !important;
}

/* ============================================================
   DEPT FILTER PILLS (students + feedback pages)
   ============================================================ */
.student-dept-filter-btn {
    display: inline-flex;
    align-items: center;
    gap: 5px;
    padding: 5px 12px;
    border: 1px solid var(--border);
    border-radius: 20px;
    background: var(--white);
    color: var(--muted-foreground);
    font-size: 0.75rem;
    font-weight: 600;
    cursor: pointer;
    transition: all 0.15s;
    font-family: 'Plus Jakarta Sans', sans-serif;
}

.student-dept-filter-btn:hover {
    border-color: var(--primary);
    color: var(--primary);
    background: var(--primary-light);
}

.student-dept-filter-btn.active {
    background: var(--primary);
    color: #fff;
    border-color: var(--primary);
}

.dept-filter-count {
    background: rgba(255,255,255,0.25);
    border-radius: 10px;
    padding: 1px 6px;
    font-size: 0.68rem;
}

.student-dept-filter-btn.active .dept-filter-count {
    background: rgba(255,255,255,0.3);
}

/* ============================================================
   DEPT MANAGEMENT PAGE CARDS
   ============================================================ */
.dept-manage-grid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(300px, 1fr));
    gap: 16px;
}

.dept-manage-card {
    border-radius: 14px;
    color: #fff;
    overflow: hidden;
    position: relative;
    box-shadow: 0 4px 16px rgba(0,0,0,0.12);
    display: flex;
    flex-direction: column;
    height: 100%;
}

.dept-manage-card-header {
    display: flex;
    align-items: center;
    gap: 12px;
    padding: 18px 18px 10px;
}

.dept-manage-icon {
    width: 48px;
    height: 48px;
    border-radius: 12px;
    background: rgba(255,255,255,0.15);
    display: flex;
    align-items: center;
    justify-content: center;
    flex-shrink: 0;
}

.dept-manage-labels {
    flex: 1;
}

.dept-manage-code {
    font-size: 1.1rem;
    font-weight: 800;
    letter-spacing: 0.02em;
    line-height: 1;
}

.dept-manage-name {
    font-size: 0.88rem;
    font-weight: 700;
    line-height: 1.3;
    opacity: 1;
    margin-top: 0;
}

.dept-manage-remove-btn {
    background: rgba(255,255,255,0.15);
    border: none;
```

```
    color: #fff;
    width: 26px;
    height: 26px;
    border-radius: 50%;
    display: flex;
    align-items: center;
    justify-content: center;
    cursor: pointer;
    transition: background 0.15s;
    flex-shrink: 0;
}

.dept-manage-remove-btn:hover {
    background: rgba(255,0,0,0.3);
}

.dept-manage-default-badge {
    font-size: 0.62rem;
    font-weight: 700;
    background: rgba(255,255,255,0.2);
    padding: 2px 8px;
    border-radius: 10px;
    flex-shrink: 0;
}

.dept-manage-desc {
    font-size: 0.72rem;
    opacity: 0.8;
    padding: 0 18px 12px;
    line-height: 1.5;
}

.dept-manage-stats {
    display: flex;
    background: rgba(0,0,0,0.15);
    padding: 10px 0;
    margin-top: auto; /* pushes stats + action bar to the bottom of the card */
}

.dept-manage-stat {
    flex: 1;
    text-align: center;
    display: flex;
    flex-direction: column;
    gap: 2px;
    border-right: 1px solid rgba(255,255,255,0.1);
}

.dept-manage-stat:last-child {
    border-right: none;
}

.dept-ms-val {
    font-size: 1.2rem;
    font-weight: 800;
    line-height: 1;
}

.dept-ms-label {
    font-size: 0.62rem;
    opacity: 0.75;
    font-weight: 500;
}

.dept-manage-view-btn {
    display: none; /* replaced by dmc-action-bar */
}

/* ── Card action bar (View / Edit / Delete) — dark overlay, equal-width buttons. */
.dmc-action-bar {
    display: flex;
    border-top: 1px solid rgba(0,0,0,0.14);
}

.dmc-action-btn {
    flex: 1 1 0;
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 6px;
    padding: 11px 6px;
    background: rgba(0,0,0,0.20);
    border: none;
    border-right: 1px solid rgba(255,255,255,0.12);
    color: #fff;
    font-size: 0.72rem;
    font-weight: 600;
    line-height: 1;
    cursor: pointer;
    transition: background 0.15s;
    font-family: 'Plus Jakarta Sans', sans-serif;
    white-space: nowrap;
}

.dmc-action-btn svg {
    flex-shrink: 0;
    display: block;
}

.dmc-action-btn:last-child {
    border-right: none;
}

.dmc-view-btn:hover  { background: rgba(0,0,0,0.34); }
.dmc-edit-btn2:hover { background: rgba(0,0,0,0.34); }
```

```
.dmc-del-btn:hover   { background: rgba(220,38,38,0.60); }

/* ============================================================
   DEPT MANAGEMENT — LIST LAYOUT (mirrors the grid cards)
   ============================================================ */
.dept-manage-list {
    display: flex;
    flex-direction: column;
    gap: 10px;
}

.dml-row {
    display: flex;
    align-items: center;
    gap: 14px;
    padding: 12px 16px;
    background: #fff;
    border: 1px solid var(--border);
    border-radius: 12px;
    box-shadow: 0 2px 8px rgba(0,0,0,0.05);
    transition: box-shadow 0.15s;
}

.dml-row:hover {
    box-shadow: 0 4px 14px rgba(0,0,0,0.09);
}

.dml-icon {
    width: 44px;
    height: 44px;
    border-radius: 10px;
    flex-shrink: 0;
    background: var(--primary-light);
    display: flex;
    align-items: center;
    justify-content: center;
    overflow: hidden;
}

.dml-icon img {
    width: 100%;
    height: 100%;
    object-fit: cover;
    border-radius: 9px;
}

.dml-info {
    flex: 1;
    min-width: 0;
}

.dml-name {
    font-size: 0.86rem;
    font-weight: 700;
    color: var(--ink);
    display: flex;
    align-items: center;
    flex-wrap: wrap;
}

.dml-badge {
    font-size: 0.62rem;
    font-weight: 700;
    background: #e5e7eb;
    color: #6b7280;
    padding: 2px 8px;
    border-radius: 10px;
}

.dml-badge.custom {
    background: var(--primary-light);
    color: var(--primary);
}

.dml-meta {
    font-size: 0.72rem;
    color: var(--muted-foreground);
    display: flex;
    gap: 8px;
    align-items: center;
    margin-top: 2px;
    white-space: nowrap;
    overflow: hidden;
    text-overflow: ellipsis;
}

.dml-code {
    font-family: 'JetBrains Mono', monospace;
    font-weight: 700;
    color: var(--primary);
    flex-shrink: 0;
}

.dml-stats {
    display: flex;
    gap: 18px;
    padding: 0 14px;
    flex-shrink: 0;
}

.dml-stat {
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: 1px;
```

```
}

.dml-stat-val {
    font-size: 1rem;
    font-weight: 800;
    color: var(--ink);
    line-height: 1;
}

.dml-stat-label {
    font-size: 0.6rem;
    color: var(--muted-foreground);
    font-weight: 600;
}

/* Same as the grid's dmc-action-btn, themed for a white row. */
.dml-actions {
    display: flex;
    gap: 6px;
    flex-shrink: 0;
    border-left: 1px solid var(--border);
    padding-left: 14px;
}

.dml-action-btn {
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 5px;
    padding: 7px 12px;
    border-radius: 8px;
    border: 1px solid var(--border);
    background: #fff;
    font-size: 0.72rem;
    font-weight: 600;
    cursor: pointer;
    transition: background 0.15s, border-color 0.15s, color 0.15s;
    font-family: 'Plus Jakarta Sans', sans-serif;
    color: var(--muted-foreground);
}

.dml-action-btn.dml-view:hover {
    background: var(--primary-light);
    border-color: var(--primary);
    color: var(--primary);
}

.dml-action-btn.dml-edit {
    color: var(--primary);
}

.dml-action-btn.dml-edit:hover {
    background: var(--primary-light);
    border-color: var(--primary);
}

.dml-action-btn.dml-del {
    color: #dc2626;
}

.dml-action-btn.dml-del:hover {
    background: #fef2f2;
    border-color: #dc2626;
}

/* ============================================================
   FEEDBACK PAGE
   ============================================================ */
.feedback-teacher-card {
    background: var(--white);
    border: 1px solid var(--border);
    border-radius: 12px;
    margin-bottom: 16px;
    overflow: hidden;
    box-shadow: var(--shadow);
}

.feedback-teacher-header {
    display: flex;
    align-items: center;
    gap: 14px;
    padding: 16px 20px;
    border-bottom: 1px solid var(--border);
    background: var(--bg);
}

.feedback-teacher-avatar {
    width: 44px;
    height: 44px;
    border-radius: 12px;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 1.2rem;
    font-weight: 800;
    color: #fff;
    flex-shrink: 0;
}

.feedback-teacher-info {
    flex: 1;
}

.feedback-print-btn {
    display: inline-flex;
```

```
    align-items: center;
    gap: 6px;
    flex-shrink: 0;
    padding: 7px 12px;
    font-size: 0.74rem;
    font-weight: 600;
    color: var(--primary);
    background: var(--primary-light);
    border: 1px solid transparent;
    border-radius: 8px;
    cursor: pointer;
    transition: background 0.15s, border-color 0.15s;
    font-family: 'Plus Jakarta Sans', sans-serif;
}

.feedback-print-btn:hover {
    background: #fff;
    border-color: var(--primary);
}

/* Print-scope chooser overlay (all students vs one specific student) */
.fb-print-overlay {
    position: fixed;
    inset: 0;
    background: rgba(15,23,42,0.45);
    display: flex;
    align-items: center;
    justify-content: center;
    z-index: 9999;
}

.fb-print-dialog {
    background: #fff;
    border-radius: 12px;
    padding: 22px 24px;
    width: min(440px, 92vw);
    box-shadow: 0 20px 50px rgba(0,0,0,0.25);
    font-family: 'Plus Jakarta Sans', sans-serif;
}

.fb-print-dialog h3 {
    margin: 0 0 4px;
    font-size: 1.05rem;
    color: var(--ink);
}

.fb-print-dialog p {
    margin: 0 0 12px;
    font-size: 0.82rem;
    color: var(--muted);
}

.fb-print-select {
    width: 100%;
    padding: 9px 12px;
    border: 1px solid var(--border);
    border-radius: 8px;
    font-size: 0.85rem;
    background: #fff;
    color: var(--ink);
    margin-bottom: 16px;
}

.fb-print-actions {
    display: flex;
    justify-content: flex-end;
    gap: 8px;
}

.fb-print-actions button {
    padding: 8px 18px;
    border-radius: 8px;
    font-size: 0.82rem;
    font-weight: 600;
    cursor: pointer;
    font-family: 'Plus Jakarta Sans', sans-serif;
}

.fb-print-cancel {
    background: #fff;
    border: 1px solid var(--border);
    color: var(--muted);
}

.fb-print-go {
    background: var(--primary);
    border: 1px solid var(--primary);
    color: #fff;
}

.feedback-teacher-name {
    font-size: 0.95rem;
    font-weight: 700;
    color: var(--ink);
    margin-bottom: 4px;
}

.feedback-teacher-meta {
    display: flex;
    align-items: center;
    flex-wrap: wrap;
    gap: 6px;
}

.feedback-comments-list {
```

```
    padding: 4px 0;
}

/* Per-subject grouping inside a teacher card */
.feedback-subject-group {
    border-top: 1px solid var(--border);
}

.feedback-subject-group:first-child {
    border-top: none;
}

.feedback-subject-header {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 10px;
    flex-wrap: wrap;
    padding: 10px 20px;
    background: #f8fafc;
}

.feedback-subject-stats {
    font-size: 0.72rem;
    color: var(--muted);
    font-weight: 600;
}

.feedback-header-actions {
    display: flex;
    align-items: center;
    gap: 8px;
    flex-shrink: 0;
    flex-wrap: wrap;
    justify-content: flex-end;
}

.feedback-viewall-btn {
    font-size: 0.74rem;
    font-weight: 600;
    color: var(--primary);
    background: transparent;
    border: 1px solid var(--border);
    border-radius: 8px;
    padding: 7px 14px;
    cursor: pointer;
    transition: background 0.15s, border-color 0.15s;
    font-family: 'Plus Jakarta Sans', sans-serif;
    white-space: nowrap;
}

.feedback-viewall-btn:hover {
    background: var(--primary-light);
    border-color: var(--primary);
}

.feedback-item-actions {
    display: flex;
    align-items: center;
    justify-content: flex-end;
    gap: 8px;
    margin-top: 8px;
}

.feedback-printone-btn {
    display: inline-flex;
    align-items: center;
    gap: 5px;
    font-size: 0.7rem;
    font-weight: 600;
    color: var(--muted);
    background: transparent;
    border: 1px solid var(--border);
    border-radius: 6px;
    padding: 5px 10px;
    cursor: pointer;
    transition: background 0.15s, border-color 0.15s, color 0.15s;
    font-family: 'Plus Jakarta Sans', sans-serif;
}

.feedback-printone-btn:hover {
    background: var(--primary-light);
    border-color: var(--primary);
    color: var(--primary);
}

.feedback-comment-item {
    padding: 14px 20px;
    border-bottom: 1px solid var(--border);
}

.feedback-comment-item:last-child {
    border-bottom: none;
}

.feedback-comment-meta {
    display: flex;
    align-items: center;
    justify-content: space-between;
    margin-bottom: 8px;
    flex-wrap: wrap;
    gap: 6px;
}

.feedback-subject-badge {
```

```
    font-size: 0.7rem;
    font-weight: 700;
    background: var(--primary-light);
    color: var(--primary);
    padding: 2px 10px;
    border-radius: 10px;
}

.feedback-timestamp {
    font-size: 0.68rem;
    color: var(--muted);
    font-family: 'JetBrains Mono', monospace;
}

.feedback-comment-text {
    font-size: 0.85rem;
    color: var(--ink);
    line-height: 1.6;
    font-style: italic;
    background: #f8fafc;
    border-left: 3px solid var(--primary);
    padding: 8px 12px;
    border-radius: 0 6px 6px 0;
    margin-bottom: 6px;
}

.feedback-comment-text.feedback-no-comment {
    color: var(--muted);
    background: #f9fafb;
    border-left-color: #cbd5e1;
    font-style: italic;
    opacity: 0.85;
}

.feedback-comment-from {
    font-size: 0.75rem;
    color: var(--muted);
    font-weight: 500;
    text-align: right;
}

.feedback-anon-tag {
    display: inline-block;
    font-size: 0.6rem;
    font-weight: 700;
    text-transform: uppercase;
    letter-spacing: 0.03em;
    color: #64748b;
    background: #eef2f5;
    border-radius: 8px;
    padding: 1px 7px;
    margin-left: 6px;
    vertical-align: middle;
}

.feedback-rating-row {
    display: flex;
    align-items: center;
    justify-content: flex-end;
    margin-top: 8px;
}

.feedback-view-rating-btn {
    font-size: 0.7rem;
    font-weight: 600;
    color: var(--primary);
    background: var(--primary-light);
    border: 1px solid transparent;
    border-radius: 6px;
    padding: 4px 10px;
    cursor: pointer;
    transition: background .15s, border-color .15s;
}

.feedback-view-rating-btn:hover {
    background: #fff;
    border-color: var(--primary);
}

.feedback-rating-panel {
    margin-top: 8px;
    padding: 10px 12px;
    background: #f8fafc;
    border: 1px solid var(--border);
    border-radius: 8px;
}

.feedback-rating-items {
    display: flex;
    flex-direction: column;
    gap: 5px;
    margin-bottom: 8px;
}

.feedback-rating-item {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 10px;
    font-size: 0.75rem;
}

.feedback-rating-item-label {
    color: var(--ink);
}
```

```css

.feedback-rating-item-score {
    font-weight: 700;
    color: var(--primary);
    font-family: 'JetBrains Mono', monospace;
    flex-shrink: 0;
}

.feedback-rating-total {
    font-size: 0.78rem;
    color: var(--ink);
    font-weight: 700;
    padding-top: 8px;
    border-top: 1px dashed var(--border);
}

.feedback-stat-chip {
    font-size: 0.78rem;
    color: var(--muted);
    font-weight: 600;
    padding: 4px 12px;
    background: var(--bg);
    border: 1px solid var(--border);
    border-radius: 20px;
}

/* Responsive adjustments */
@media (max-width: 768px) {
    .dept-manage-grid { grid-template-columns: 1fr; }
}
/* ===== DEPT FILTER PILLS (Students / Teachers / Subjects) ===== */
.dept-filter-pill {
    display: inline-flex;
    align-items: center;
    gap: 5px;
    padding: 5px 14px;
    border-radius: 20px;
    border: 1.5px solid var(--border);
    background: #fff;
    font-size: 0.75rem;
    font-weight: 600;
    color: var(--text-secondary);
    cursor: pointer;
    transition: all 0.15s;
    white-space: nowrap;
    font-family: 'Plus Jakarta Sans', sans-serif;
}

.dept-filter-pill:hover {
    border-color: var(--primary);
    color: var(--primary);
    background: var(--primary-light, #eff6ff);
}

.dept-filter-pill.active {
    background: var(--primary);
    border-color: var(--primary);
    color: #fff;
    box-shadow: 0 2px 8px rgba(37,99,235,0.18);
}

.dept-filter-pill .dept-filter-count {
    background: rgba(0,0,0,0.1);
    border-radius: 10px;
    padding: 1px 6px;
    font-size: 0.68rem;
}

.dept-filter-pill.active .dept-filter-count {
    background: rgba(255,255,255,0.3);
}

/* ============================================================
   REPORTS PAGE — PILL DEPT FILTER + SINGLE TABLES
   (pills reuse .student-dept-filter-btn / .dept-filter-count)
   ============================================================ */

/* Pill bar row — kept here since it is unique to Reports */
.rpt-pill-bar {
    display: flex;
    flex-wrap: wrap;
    gap: 6px;
    margin-bottom: 18px;
}

/* View All button row at the bottom of a table card */
.rpt-viewall-btn {
    font-size: 0.8rem;
    color: var(--primary);
    border-color: var(--primary);
}
.rpt-viewall-btn:hover {
    background: var(--primary-light, #eff6ff);
}

/* ============================================================
   TEACHER PAGE — STICKY SECTION TABS
   ============================================================ */
.teacher-tab-bar {
    display: flex;
    gap: 0;
    border-bottom: 2px solid var(--border);
    margin-bottom: 20px;
    position: sticky;
    top: 0;
```

```
    background: #f8fafc;
    z-index: 10;
    padding-top: 4px;
}

.teacher-tab {
    display: inline-flex;
    align-items: center;
    gap: 7px;
    padding: 10px 20px;
    border: none;
    background: none;
    font-size: 0.83rem;
    font-weight: 600;
    color: var(--muted);
    cursor: pointer;
    border-bottom: 3px solid transparent;
    margin-bottom: -2px;
    transition: color 0.15s, border-color 0.15s;
    font-family: 'Plus Jakarta Sans', sans-serif;
}

.teacher-tab:hover {
    color: var(--primary);
}

.teacher-tab-active {
    color: var(--primary) !important;
    border-bottom-color: var(--primary) !important;
}

.teacher-tab-count {
    background: var(--border);
    color: var(--muted);
    border-radius: 10px;
    padding: 1px 7px;
    font-size: 0.68rem;
    font-weight: 700;
    transition: background 0.15s, color 0.15s;
}

.teacher-tab-active .teacher-tab-count {
    background: var(--primary);
    color: #fff;
}

/* ============================================================
   TEACHER PAGE — COLLAPSIBLE CARD HEADER
   ============================================================ */
.teacher-card-header {
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: 14px 20px;
    cursor: pointer;
    border-bottom: 1px solid var(--border);
    user-select: none;
    transition: background 0.15s;
}

.teacher-card-header:hover {
    background: var(--primary-light, #eff6ff);
}

.teacher-collapse-icon {
    transition: transform 0.22s ease;
    color: var(--muted);
    flex-shrink: 0;
}

.teacher-collapse-icon.collapsed {
    transform: rotate(180deg);
}

.teacher-section-body {
    overflow: hidden;
    transition: max-height 0.28s ease, opacity 0.22s ease;
    max-height: 9999px;
    opacity: 1;
}

.teacher-section-body.collapsed {
    max-height: 0;
    opacity: 0;
}
/* Editable fields in printed forms (Annex C / D): dashed on screen, solid in print. */
[data-print-field] {
    /* Solid rule, always drawn, even when empty. */
    border-bottom: 1px solid #333;
    min-height: 1.15em;
    line-height: 1.35;
}
/* Only on hover/focus does it advertise that it is editable. */
[data-print-field]:hover  { background: #f8fafc; }
[data-print-field]:focus  { background: #fff7ed; border-bottom-color: #d97706; }
/* Placeholder text for an empty field, so the line is labelled rather than bare. */
[data-print-field]:empty::before {
    content: attr(data-placeholder);
    color: #94a3b8;
    font-style: italic;
}
@media print {
    [data-print-field]:empty::before { content: ''; }
}
```

```css
/* ===== TEACHER PICKER (Add/Edit Subject modal) ===== */
.tp-wrap { display: flex; flex-direction: column; gap: 8px; }

.tp-selected { min-height: 42px; }
.tp-chosen {
    display: flex; align-items: center; gap: 8px;
    padding: 9px 12px; border-radius: 8px;
    background: var(--primary-light, #eff6ff);
    border: 1px solid var(--primary, #2563eb);
}
.tp-chosen > div { flex: 1; display: flex; align-items: center; gap: 7px; flex-wrap: wrap; font-size: 0.85rem; }
.tp-tid { font-family: 'JetBrains Mono', monospace; font-size: 0.72rem; color: var(--muted); }
.tp-outside { font-size: 0.68rem; color: #b45309; background: #fef3c7; padding: 1px 7px; border-radius: 20px; }
.tp-clear { border: none; background: transparent; cursor: pointer; color: var(--muted); font-size: 0.9rem; padding: 2px 6px; }
.tp-clear:hover { color: var(--danger, #dc2626); }
.tp-empty {
    padding: 9px 12px; border-radius: 8px; font-size: 0.8rem; color: var(--muted);
    background: var(--surface2, #f8fafc); border: 1px dashed var(--border, #e2e8f0);
}

.tp-search { font-size: 0.85rem; }

.tp-pills { display: flex; flex-wrap: wrap; gap: 6px; }
/* Pills reuse the Teachers page filter-bar classes. */

.tp-list {
    max-height: 230px; overflow-y: auto;
    border: 1px solid var(--border, #e2e8f0); border-radius: 8px;
}
.tp-item {
    display: flex; align-items: center; gap: 10px; width: 100%;
    padding: 9px 11px; border: none; background: #fff; cursor: pointer;
    text-align: left; border-bottom: 1px solid var(--border, #e2e8f0);
}
.tp-item:last-child { border-bottom: none; }
.tp-item:hover { background: var(--surface2, #f8fafc); }
.tp-item.selected { background: var(--primary-light, #eff6ff); }
.tp-avatar {
    width: 30px; height: 30px; border-radius: 50%; flex-shrink: 0;
    background: var(--primary, #2563eb); color: #fff;
    display: flex; align-items: center; justify-content: center;
    font-size: 0.78rem; font-weight: 700;
}
.tp-info { display: flex; flex-direction: column; flex: 1; min-width: 0; }
.tp-name { font-size: 0.83rem; font-weight: 600; }
.tp-meta { font-size: 0.71rem; color: var(--muted); }
.tp-check { color: var(--primary, #2563eb); font-weight: 700; }
.tp-none { padding: 22px; text-align: center; font-size: 0.82rem; color: var(--muted); }
.tp-load {
    font-size: 0.66rem; font-weight: 600; white-space: nowrap;
    padding: 2px 8px; border-radius: 20px;
    background: var(--surface2, #f1f5f9); color: var(--muted, #64748b);
}
.tp-load.free { background: #dcfce7; color: #15803d; }
/* Page-toolbar buttons (Reports header): white so they read as buttons. */
.btn-toolbar {
    background: #fff;
    border: 1px solid var(--border, #e2e8f0);
    color: var(--foreground, #0f172a);
    box-shadow: 0 1px 2px rgba(15, 23, 42, 0.05);
}
.btn-toolbar:hover {
    background: var(--primary-light, #eff6ff);
    border-color: var(--primary, #2563eb);
    color: var(--primary, #2563eb);
}


/* ==== REPORTS & ANALYTICS HEADER ==== */
.reports-header {
    align-items: center;
    justify-content: flex-start;
    gap: 18px;
    flex-wrap: wrap;
}
.reports-header .page-title {
    flex: 0 0 auto;
    margin: 0;
}

.reports-toolbar {
    display: flex;
    align-items: center;
    justify-content: flex-start;
    gap: 8px;
    /* Basis 0, not auto, so the toolbar can shrink before wrapping. */
    flex: 1 1 0%;
    min-width: 0;
    flex-wrap: nowrap;
}

/* Term selector and Actions button keep their natural width. */
.reports-toolbar > * { flex: 0 0 auto; }

/* Narrow viewports: toolbar on its own full-width row. */
@media (max-width: 900px) {
    .reports-toolbar { flex: 1 0 100%; flex-wrap: wrap; }
}

/* ==== REPORTS HEADER "ACTIONS" MENU ==== */
.reports-actions { position: relative; }

.reports-actions-btn { display: inline-flex; align-items: center; gap: 6px; }
.reports-actions-btn .ra-chevron {
    color: var(--muted, #64748b);
```

```
    margin-left: 2px;
    transition: transform 0.15s ease;
}
.reports-actions.open .ra-chevron { transform: rotate(180deg); }
.reports-actions.open .reports-actions-btn {
    border-color: var(--primary, #2563eb);
    color: var(--primary, #2563eb);
    background: var(--primary-light, #eff6ff);
}

.reports-actions-menu {
    display: none;
    position: absolute;
    top: calc(100% + 6px);
    left: 0;
    min-width: 300px;
    padding: 6px;
    background: #fff;
    border: 1px solid var(--border, #e2e8f0);
    border-radius: 12px;
    box-shadow: 0 14px 34px rgba(15, 23, 42, 0.14);
    /* Above page content but below the sidebar (z-index 100) and any modal. */
    z-index: 60;
}
.reports-actions.open .reports-actions-menu {
    display: block;
    animation: fadeIn 0.14s ease;
}

.reports-menu-group {
    padding: 8px 10px 4px;
    font-size: 0.66rem;
    font-weight: 800;
    letter-spacing: 0.07em;
    text-transform: uppercase;
    color: var(--muted, #64748b);
}
.reports-menu-sep {
    height: 1px;
    background: var(--border, #e2e8f0);
    margin: 6px 4px;
}

.reports-menu-item {
    display: flex;
    align-items: flex-start;
    gap: 10px;
    width: 100%;
    padding: 9px 10px;
    border: none;
    border-radius: 8px;
    background: none;
    font-family: inherit;
    color: var(--ink, #0f172a);
    text-align: left;
    cursor: pointer;
    transition: background 0.12s, color 0.12s;
}
.reports-menu-item:hover,
.reports-menu-item:focus-visible {
    background: var(--primary-light, #eff6ff);
    color: var(--primary, #2563eb);
    outline: none;
}
.reports-menu-item svg { flex-shrink: 0; margin-top: 1px; }

.reports-menu-text { display: flex; flex-direction: column; gap: 2px; min-width: 0; }
.reports-menu-label { font-size: 0.82rem; font-weight: 600; }
/* Menu item descriptions shown outright. */
.reports-menu-desc {
    font-size: 0.72rem;
    font-weight: 500;
    color: var(--muted, #64748b);
    line-height: 1.35;
}
.reports-menu-item:hover .reports-menu-desc { color: inherit; opacity: 0.85; }

@media print {
    .reports-actions { display: none !important; }
}
/* ============================================================
   SKELETON LOADING (only for genuinely empty containers)
   ============================================================ */
.skel {
  background: linear-gradient(90deg, #eef2f0 25%, #e2e9e5 37%, #eef2f0 63%);
  background-size: 400% 100%;
  animation: skelPulse 1.4s ease infinite;
  border-radius: 4px;
  display: inline-block;
}
@keyframes skelPulse {
  0%   { background-position: 100% 50%; }
  100% { background-position: 0 50%; }
}
.skel-line   { height: 11px; width: 100%; }
.skel-line.sm{ height: 9px; }
.skel-pill   { height: 20px; width: 62px; border-radius: 999px; }
.skel-circle { height: 38px; width: 38px; border-radius: 10px; }

.skel-card {
  background: #fff; border: 1px solid var(--border, #e5e7eb);
  border-radius: 12px; padding: 18px;
  display: flex; flex-direction: column; gap: 12px;
}
```

```
/* Respect reduced-motion: the placeholder holds still. */
@media (prefers-reduced-motion: reduce) {
  .skel { animation: none; }
}

/* ===== Multiple teachers per subject: the chosen list ===== */
.tp-chosen + .tp-chosen { margin-top: 8px; }
.tp-secs { margin-top: 7px; }
.tp-secs-label { display: block; font-size: 0.72rem; color: var(--muted, #64748b);
                 font-weight: 700; margin-bottom: 5px; }
.tp-sec-chips { display: flex; flex-wrap: wrap; gap: 6px; }
/* One chip per section, per teacher; "All students" when none are selected. */
.tp-sec { font-size: 0.74rem; font-weight: 600; cursor: pointer;
          padding: 4px 11px; border-radius: 999px;
          border: 1px solid var(--border, #e2e8f0);
          background: #fff; color: var(--muted, #64748b); transition: all .12s; }
.tp-sec:hover { border-color: var(--primary, #059669); color: var(--primary, #059669); }
.tp-sec.is-on { background: var(--primary, #059669); border-color: var(--primary, #059669);
                color: #fff; }

/* ===== Subjects table: the teacher cell opens Teachers & Sections ===== */
.tcell { cursor: pointer; border-radius: 8px; transition: background .12s; }
.tcell:hover, .tcell:focus-visible { background: #f0fdf4; outline: none; }
.tcell:hover .tcell-names, .tcell:focus-visible .tcell-names { color: var(--primary, #059669); }
.tcell-names { display: block; line-height: 1.45; }
/* First two names listed; the rest are counted. */
.tcell-more { display: inline-block; margin-top: 3px; font-size: 0.7rem; font-weight: 700;
              color: var(--primary, #059669); background: #ecfdf5;
              padding: 1px 7px; border-radius: 999px; }
.tcell-none { color: var(--muted, #64748b); font-style: italic; font-size: 0.78rem; }

/* ===== Subjects table: MULTIPLE tag ===== */
.tcell-multi { display: inline-block; font-size: 0.7rem; font-weight: 800; letter-spacing: 0.06em;
               color: var(--primary, #059669); background: #ecfdf5;
               border: 1px solid #a7f3d0; padding: 2px 9px; border-radius: 999px; }
.tcell-count { display: block; margin-top: 3px; font-size: 0.7rem; color: var(--muted, #64748b); }

/* ===== Who teaches this subject ===== */
.tl-summary { display: flex; gap: 18px; font-size: 0.78rem; color: var(--muted, #64748b);
              padding-bottom: 10px; margin-bottom: 10px; border-bottom: 1px solid var(--border, #e2e8f0); }
.tl-summary strong { color: var(--ink, #0f172a); font-size: 0.95rem; }
/* One button per teacher, not a stack of text. */
.tl-card { display: flex; align-items: center; gap: 12px; width: 100%; text-align: left;
           padding: 12px 14px; margin-bottom: 8px; cursor: pointer;
           background: #fff; border: 1px solid var(--border, #e2e8f0); border-radius: 12px;
           transition: all .12s; }
.tl-card:hover { border-color: var(--primary, #059669); background: #f0fdf4;
                 transform: translateY(-1px); }
.tl-avatar { flex-shrink: 0; width: 38px; height: 38px; border-radius: 50%;
             display: flex; align-items: center; justify-content: center;
             background: linear-gradient(135deg, #064e3b, #059669); color: #fff;
             font-weight: 700; font-size: 0.95rem; }
.tl-main { flex: 1; min-width: 0; }
.tl-name { display: block; font-weight: 700; font-size: 0.88rem; color: var(--ink, #0f172a); }
.tl-meta { display: block; font-size: 0.72rem; color: var(--muted, #64748b); margin-top: 1px; }
.tl-secs { display: flex; flex-wrap: wrap; gap: 5px; margin-top: 6px; }
.tl-sec { font-size: 0.68rem; font-weight: 700; padding: 2px 8px; border-radius: 999px;
          background: #ecfdf5; color: #047857; }
.tl-sec-all { background: #f1f5f9; color: #64748b; }
.tl-right { flex-shrink: 0; text-align: right; }
.tl-roll { display: block; font-size: 0.74rem; font-weight: 700; color: var(--ink, #0f172a); }
.tl-go { display: block; font-size: 0.7rem; color: var(--primary, #059669); margin-top: 3px; }
.tl-empty { padding: 24px; text-align: center; color: var(--muted, #64748b); font-size: 0.84rem; }
/* The teacher opened from the list, so the right card is obvious. */
.tp-chosen.is-focus { box-shadow: 0 0 0 2px var(--primary, #059669); }
```

admin/dashboard.html

1664 lines

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8"/>
<meta name="viewport" content="width=device-width, initial-scale=1.0"/>
<title>Admin Panel — Teacher Evaluation System</title>
<link rel="stylesheet" href="../shared/css/styles.css"/>
<link rel="stylesheet" href="admin-styles.css"/>


<script src="https://www.gstatic.com/firebasejs/10.12.0/firebase-app-compat.js"></script>
<script src="https://www.gstatic.com/firebasejs/10.12.0/firebase-firestore-compat.js"></script>
<script src="https://www.gstatic.com/firebasejs/10.12.0/firebase-auth-compat.js"></script>
<script src="https://www.gstatic.com/firebasejs/10.12.0/firebase-functions-compat.js"></script>

<!-- Shared Firebase config (single source of truth) -->
<script src="../shared/js/firebaseAPI.js"></script>

<script>
function _dashFatal(msg){
  document.documentElement.style.visibility='visible';
  var b=document.createElement('div');
  b.style.cssText='position:fixed;inset:0;z-index:99999;background:#0d1117;color:#e6edf3;display:flex;align-items:center;justify-content:center;paddi
ng:24px;font-family:system-ui,sans-serif;text-align:center;';
  b.innerHTML='<div style="max-width:540px;"><div style="font-size:2rem;margin-bottom:10px;">\u26A0\uFE0F</div><div style="font-size:1.05rem;font-wei
ght:700;margin-bottom:10px;">Dashboard could not start</div><div style="font-size:0.92rem;line-height:1.6;color:#8b949e;">'+msg+'</div><a href="../in
dex.html" style="display:inline-block;margin-top:16px;color:#3fb950;text-decoration:none;">\u2190 Back to login</a></div>';
  document.body.appendChild(b);
}

var db, fbAuth;
// Fallback config so the dashboard works even if shared/js/firebaseAPI.js did not load.
window.firebaseConfig = window.firebaseConfig || {
  apiKey: "AIzaSyCUD1waV1kPYFKj1zRA7ANVjQkhdP7NJic",
  authDomain: "studenteval-937f6.firebaseapp.com",
  projectId: "studenteval-937f6",
  storageBucket: "studenteval-937f6.firebasestorage.app",
  messagingSenderId: "899672493371",
  appId: "1:899672493371:web:600723b0d16bb879067b1e",
  measurementId: "G-D5WD5LB2E5"
};
try {
  firebase.initializeApp(window.firebaseConfig);
  window.db     = firebase.firestore();

  // Fall back to long polling where networks break Firestore's streaming.
  // Must run before any read or write.
  try {
    window.db.settings({ experimentalAutoDetectLongPolling: true, merge: true });
  } catch (e) {
    console.warn('Could not set the Firestore transport option:', e && e.message);
  }
  window.fbAuth = firebase.auth();
  db = window.db; fbAuth = window.fbAuth;
} catch (e) {
  _dashFatal(e.message);
  throw e;
}

// Keep STRICT_ADMIN in sync with index.html. true = only users with an
// /admins/{uid} doc pass (needs deployed firestore.rules).
const STRICT_ADMIN = true;
document.documentElement.style.visibility = 'hidden';

// Safety net: never leave the page silently blank.
var _authWatchdog = setTimeout(function(){
  _dashFatal('No response from Firebase. If your browser address bar starts with <b>file://</b>, the login session does not carry between pages. Run
the site through a local web server (VS Code \u201CLive Server\u201D, or <code>python -m http.server</code>) or deploy to Firebase Hosting, then log
in again.');
}, 8000);

// Persist the session across the login \u2192 dashboard navigation.
try { fbAuth.setPersistence(firebase.auth.Auth.Persistence.LOCAL).catch(function(){}); } catch(e){}

fbAuth.onAuthStateChanged(async (user) => {
  clearTimeout(_authWatchdog);
  if (!user) {
    // No session yet: retry ONCE, since it can lag right after login.
    if (!sessionStorage.getItem('_dashRetried')) {
      sessionStorage.setItem('_dashRetried', '1');
      setTimeout(function(){ window.location.reload(); }, 1500);
      return;
    }
    sessionStorage.removeItem('_dashRetried');
    if (location.protocol === 'file:') {
      _dashFatal('Your sign-in worked, but the browser did not keep the session because this page was opened directly (file://). Run the site through
a local web server (e.g. VS Code \u201CLive Server\u201D) so the address starts with http://, then log in again.');
      return;
    }
    _dashFatal('No active sign-in session was found here (protocol: ' + location.protocol + ').<br><br>'
      + 'Your login worked, but the session did not carry from the login page to this page. Make sure BOTH pages load from the <b>same address</b> \u
2014 e.g. both <code>http://localhost:5500/...</code>. If the login page is http:// but this one opened as file:// (or a different port), that breaks
the session.<br><br>'
      + '<a href="../index.html" style="color:#3fb950;">\u2190 Back to login</a>');
    return;
  }
  sessionStorage.removeItem('_dashRetried');
  // Three outcomes: 'unknown' (Firestore unreachable) must not sign the admin out.
  let verdict = 'unknown';
  try {
```

```
    const ref = db.collection('admins').doc(user.uid);
    const doc = await ref.get();
    if (doc.exists) {
      verdict = 'yes';
    } else {
      try {
        await ref.set({ role: 'admin', email: user.email || '', createdAt: new Date().toISOString() }, { merge: true });
        verdict = 'yes';
      } catch (writeErr) {
        verdict = (writeErr && writeErr.code === 'permission-denied') ? 'no' : 'unknown';
        console.warn('Could not create admins doc:', writeErr && writeErr.message);
      }
    }
  } catch (readErr) {
    verdict = (readErr && readErr.code === 'permission-denied') ? 'no' : 'unknown';
    console.warn('Could not read admins doc:', readErr && readErr.message);
  }

  if (verdict === 'no' && STRICT_ADMIN) {
    await fbAuth.signOut();
    window.location.replace('../index.html');
    return;
  }
  if (verdict === 'unknown') {
    if (!window._adminVerified) {
      _dashFatal('Could not reach Firestore to confirm your administrator account.<br><br>'
        + 'This is usually a network problem rather than a permissions one. '
        + 'Check your connection and reload.<br><br>'
        + '<a href="../index.html" style="color:#3fb950;">&#8592; Back to login</a>');
      return;
    }
    console.warn('Admin check skipped this time (offline); keeping the verified session.');
  } else {
    window._adminVerified = true;
  }
  window._adminUser = { uid: user.uid, email: user.email };
  document.documentElement.style.visibility = 'visible';
});

function setData(key, val) {
  localStorage.setItem(key, JSON.stringify(val));
  syncCollectionToFirestore(key, val);
}

async function syncCollectionToFirestore(key, val) {
  const MAP = {
    students: 'students', teachers: 'teachers', subjects: 'subjects',
    evaluations: 'evaluations', schoolYears: 'schoolYears',
    auditLog: 'auditLog', evalPeriod: 'settings'
  };
  // Discard writes to the legacy 'customDepartments' key (no MAP entry).
  if (key === 'customDepartments') return;

  try {
    if (key === 'departments') {
      // Write every department as its own document in the 'departments' collection
      const batch = db.batch();
      Object.entries(val).forEach(([code, cfg]) => {
        batch.set(db.collection('departments').doc(code), { code, ...cfg });
      });
      await batch.commit();
      return;
    }
    // customCourses: one doc per dept, courses/{DEPT_CODE} → { dept, courses: [...] }
    if (key === 'customCourses') {
      const coursesMap = val || {};
      const entries = Object.entries(coursesMap);
      if (entries.length > 0) {
        const batch = db.batch();
        entries.forEach(([deptCode, list]) => {
          batch.set(db.collection('courses').doc(deptCode), {
            dept: deptCode,
            courses: Array.isArray(list) ? list : []
          });
        });
        await batch.commit();
        console.log('✅ Synced customCourses to Firestore courses collection:', Object.keys(coursesMap));
      }
      return;
    }
    const col = MAP[key];
    if (!col) return;
    if (Array.isArray(val)) {
      let items = val;
      // Audit log is append-only in the rules; send only new entries.
      if (col === 'auditLog') {
        let sent = [];
        try { sent = JSON.parse(localStorage.getItem('_auditSyncedIds') || '[]'); } catch (e) {}
        const sentSet = new Set(sent);
        items = val.filter(it => it && it.id && !sentSet.has(it.id));
        if (!items.length) return;
        localStorage.setItem('_auditSyncedIds',
          JSON.stringify(items.map(it => it.id).concat(sent).slice(0, 600)));
      }
      if (!items.length) return;
      const batch = db.batch();
      items.forEach(item => { if (item.id) batch.set(db.collection(col).doc(item.id), item); });
      await batch.commit();
    } else {
      await db.collection(col).doc(key).set(val);
    }
  } catch(e) { console.warn('Firestore sync error:', e.message); }
}
```

```
// LEGACY CREDENTIAL PURGE: remove the old plaintext 'adminCreds' from localStorage.
(function purgeLegacyCredentials() {
  ['adminCreds', 'adminUser', 'adminPassword', 'currentAdmin'].forEach(function (k) {
    try { if (localStorage.getItem(k) !== null) { localStorage.removeItem(k); console.warn('Removed legacy credential key:', k); } }
    catch (e) {}
  });
})();

let _staleCollections = [];

// Whether the last Firestore pull was COMPLETE; adminrate.js checks it before syncing.
window._firestoreLoadClean = false;

async function loadFromFirebase() {
  updateFirebaseStatus('loading');
  _staleCollections = [];
  window._firestoreLoadClean = false;   // re-earned on every load
  // Placeholders for EMPTY containers while fetching.
  if (typeof showSkeleton === 'function') showSkeleton();
  try {
    const collections = ['students','teachers','subjects','evaluations','schoolYears','auditLog'];
    for (const col of collections) {
      // Per-collection try/catch so one denied read doesn't skip the rest.
      let snap;
      try {
        // Timeout so a blocked read can't hang forever.
        snap = await Promise.race([
          db.collection(col).get(),
          new Promise((_, reject) =>
            setTimeout(() => reject(new Error('timed out after 10s')), 10000))
        ]);
      } catch (e) {
        console.warn('Could not load "' + col + '" from Firestore:', e.code || e.message,
                     '— keeping whatever is already in localStorage.');
        // Remember which collections failed, for the status badge.
        _staleCollections.push(col);
        continue;
      }
      if (!snap.empty) {
        const docs = snap.docs.map(d => {
          const data = {id: d.id, ...d.data()};
          // Ensure teachers always have a facultyType field (old records may lack it)
          if (col === 'teachers' && !data.facultyType) data.facultyType = 'regular';
          return data;
        });
        // Passwords go to the in-memory vault (admin-core.js), not storage.
        if ((col === 'students' || col === 'teachers') && typeof _stripPasswords === 'function') {
          localStorage.setItem(col, JSON.stringify(_stripPasswords(col, docs)));
          if (window._pwVaultReady) window._pwVaultReady[col] = true;
        } else {
          localStorage.setItem(col, JSON.stringify(docs));
        }
      }
    }
    const evalPeriodDoc = await db.collection('settings').doc('evalPeriod').get();
    if (evalPeriodDoc.exists) localStorage.setItem('evalPeriod', JSON.stringify(evalPeriodDoc.data()));
    // Admin auth is Firebase Auth only; 'adminCreds' is never read.
    // Load customCourses from Firestore 'courses' on startup.
    const coursesSnap = await db.collection('courses').get();

    // Drop (and delete) courses of tombstoned departments.
    const deadDepts = new Set();
    try {
      (await db.collection('deletedDepts').get()).forEach(t => deadDepts.add(t.id));
    } catch (e) { console.warn('Could not read the deleted-department list:', e.message); }

    if (!coursesSnap.empty) {
      const customCourses = {};
      coursesSnap.docs.forEach(d => {
        if (deadDepts.has(d.id)) {
          d.ref.delete().catch(() => {});      // a stray copy - clean it up
          return;
        }
        const data = d.data();
        customCourses[d.id] = Array.isArray(data.courses) ? data.courses : [];
      });
      localStorage.setItem('customCourses', JSON.stringify(customCourses));
      if (typeof reloadCoursesByDept === 'function') reloadCoursesByDept();
    } else if (deadDepts.size) {
      // No courses in Firestore: still strip deleted departments locally.
      try {
        const local = JSON.parse(localStorage.getItem('customCourses') || '{}');
        let changed = false;
        deadDepts.forEach(code => { if (local[code]) { delete local[code]; changed = true; } });
        if (changed) localStorage.setItem('customCourses', JSON.stringify(local));
      } catch (e) {}
    }

    // Load all departments from Firestore into localStorage
    const deptSnap = await db.collection('departments').get();
    if (!deptSnap.empty) {
      // Read whatever is already in localStorage (may have offline edits)
      let localDepts = {};
      try { localDepts = JSON.parse(localStorage.getItem('departments') || '{}'); } catch(e) {}

      const deptsFromDB = {};
      deptSnap.docs.forEach(d => {
        const { code, ...cfg } = d.data();
        const localCfg = localDepts[d.id] || {};

        // Keep a local-only custom color and push it to Firestore.
        const localHasCustomColor = localCfg.colorClass === 'dept-custom' && localCfg.color;
        const firestoreHasCustomColor = cfg.colorClass === 'dept-custom' && cfg.color;

        if (localHasCustomColor && !firestoreHasCustomColor) {
```

```
          // Local has a newer color — use it and save it back to Firestore
          cfg.color = localCfg.color;
          cfg.colorClass = 'dept-custom';
          db.collection('departments').doc(d.id).set({ code: d.id, ...cfg })
            .then(() => console.log('✅ Pushed offline color to Firestore:', d.id, cfg.color))
            .catch(e => console.warn('❌ Failed to push offline color:', e));
        } else if (localHasCustomColor && firestoreHasCustomColor) {
          // Both have custom colors — local wins (most recent user action)
          cfg.color = localCfg.color;
          cfg.colorClass = 'dept-custom';
        } else {
          // Firestore is authoritative — normalize colorClass if needed
          if (cfg.color && cfg.colorClass && cfg.colorClass !== 'dept-custom') {
            cfg.colorClass = 'dept-custom';
          }
        }

        deptsFromDB[d.id] = cfg;
      });

      // Local-only departments: re-sync ones not yet confirmed by Firestore,
      // drop tombstoned ones (local and Firestore tombstones).
      let tombstones = [];
      try { tombstones = JSON.parse(localStorage.getItem('deletedDepts') || '[]'); } catch (e) {}
      try {
        const tombSnap = await db.collection('deletedDepts').get();
        tombSnap.forEach(t => { if (tombstones.indexOf(t.id) === -1) tombstones.push(t.id); });
      } catch (e) {
        console.warn('Could not read the deleted-department list:', e.message);
      }

      // Tombstoned departments are dropped even if Firestore still has them.
      tombstones.forEach(code => { delete deptsFromDB[code]; });

      Object.entries(localDepts).forEach(([code, cfg]) => {
        if (!deptsFromDB[code] && cfg.isCustom) {
          if (tombstones.indexOf(code) !== -1) {
            console.log('Skipping deleted department:', code);
            return;                          // stays deleted
          }
          deptsFromDB[code] = cfg;
          // Re-attempt the Firestore write in case it was lost in the race
          db.collection('departments').doc(code).set({ code, ...cfg })
            .then(() => console.log('✅ Re-synced pending local dept to Firestore:', code))
            .catch(e => console.warn('❌ Failed to re-sync pending dept:', code, e));
        }
      });

      // The Firestore tombstone is the lasting record; addDepartment() clears it.

      localStorage.setItem('departments', JSON.stringify(deptsFromDB));
    } else {
      // First ever load — seed Firestore from defaults via getDepartments()
      if (typeof getDepartments === 'function') {
        const seed = getDepartments();
        const batch = db.batch();
        Object.entries(seed).forEach(([code, cfg]) => {
          batch.set(db.collection('departments').doc(code), { code, ...cfg });
        });
        await batch.commit();
      }
    }

    // Only a complete pull makes the local copy safe to push back up.
    if (!_staleCollections.length) window._firestoreLoadClean = true;
    window._firestoreLoadAttempted = true;
    // Stamp the time of a CLEAN pull only, so a partial load never looks fresh.
    if (!_staleCollections.length) {
      try { localStorage.setItem('_lastSync', new Date().toISOString()); } catch (e) {}
    }
    updateFirebaseStatus(_staleCollections.length ? 'stale' : 'connected');
    // Re-run the facultyType migration after Firebase data loads.
    if (typeof migrateSupervisorRecords === 'function') migrateSupervisorRecords();
    // Re-sync sidebar + dept cards after Firestore departments load.
    if (typeof refreshDeptConfig === 'function') refreshDeptConfig();
    if (typeof renderDeptManagePage === 'function') renderDeptManagePage();
    _rerenderCurrentPage();
  } catch(e) {
    window._firestoreLoadAttempted = true;
    console.warn('Firebase load failed, using local data:', e.message);
    updateFirebaseStatus('error');
    _rerenderCurrentPage();
  }
}

// "just now" / "6 minutes ago" / a date - for the badge tooltip.
function _lastSyncText() {
  let t;
  try { t = localStorage.getItem('_lastSync'); } catch (e) {}
  if (!t) return 'in this session';
  const mins = Math.round((Date.now() - new Date(t).getTime()) / 60000);
  if (mins < 1)   return 'just now';
  if (mins === 1) return '1 minute ago';
  if (mins < 60)  return mins + ' minutes ago';
  const hrs = Math.round(mins / 60);
  if (hrs < 24)   return hrs + (hrs === 1 ? ' hour ago' : ' hours ago');
  return 'on ' + new Date(t).toLocaleString();
}

function updateFirebaseStatus(status) {
  const el = document.getElementById('fbStatusBadge');
  if (!el) return;
  const cfg = {
    loading:   { icon: '⏳', text: 'Connecting...',  color: '#f59e0b', tip: 'Loading data from Firestore' },
    connected: { icon: '🔥', text: 'Firebase Live',  color: '#16a34a', tip: 'All data loaded from Firestore ' + _lastSyncText() + '. Click to refres
```

```
h.' },
    stale:     { icon: '⚠', text: 'Data may be old', color: '#d97706',
                 tip: 'Could not load: ' + _staleCollections.join(', ') + '. Showing the copy saved ' + _lastSyncText() + '. Click to retry.' },
    error:     { icon: '⚠', text: 'Offline Mode',   color: '#dc2626', tip: 'No connection. Showing the copy saved ' + _lastSyncText() + '. Click to
retry.' }
  };
  const c = cfg[status] || cfg.error;
  // Clickable badge to reload data.
  el.style.cursor = status === 'loading' ? 'default' : 'pointer';
  el.title = c.tip;
  el.onclick = status === 'loading' ? null : function () { refreshFromCloud(); };
  el.innerHTML = `<span style="color:${c.color};font-size:0.72rem;font-weight:600;">${c.icon} ${c.text}</span>`;
}

// Manual re-pull. Re-renders the page you are actually on afterwards.
window.refreshFromCloud = async function () {
  showToast('Refreshing from Firestore…', 'info');
  // Forced refresh: show placeholders even over real data.
  if (typeof showSkeleton === 'function') showSkeleton(true);
  await loadFromFirebase();
  _liveRetries = 0;
  startRosterListeners();                    // restarts them if they had stopped
  // Put back anything the redraw did not replace (see clearStuckSkeletons).
  if (typeof clearStuckSkeletons === 'function') clearStuckSkeletons();
  showToast(_staleCollections.length
      ? 'Still could not load: ' + _staleCollections.join(', ')
      : 'Data refreshed.', _staleCollections.length ? 'warning' : 'success');
};

// Redraw the open page after a reload.
function _rerenderCurrentPage() {
  // Open page from the DOM: the .active "page-<name>" element.
  const el = document.querySelector('.page.active');
  const page = el ? el.id.replace(/^page-/, '') : 'dashboard';
  const fn = {
    dashboard:  'renderDashboard',  schoolYear: 'renderSchoolYear', students: 'renderStudents',
    teachers:   'renderTeachers',   subjects:   'renderSubjects',   reports:  'renderReports',
    evalControl:'renderEvalControl',auditLog:   'renderAuditLog',   feedback: 'renderFeedbackPage',
    supervisor: 'renderSupervisorList', rptViewAll: 'renderRptViewAll',
    dept: 'renderDeptPage', deptManage: 'renderDeptManagePage'
  }[page];
  // Department pages need window.currentDept to redraw.
  const arg = (page === 'dept') ? window.currentDept : undefined;
  if (page === 'dept' && !arg) { if (typeof renderDashboard === 'function') renderDashboard(); return; }
  if (fn && typeof window[fn] === 'function') { try { window[fn](arg); return; } catch (e) {} }
  if (typeof renderDashboard === 'function') renderDashboard();
}

// LIVE DATA (students, teachers, evaluations): read-only listeners keep this
// browser current. They never call setData().

// Pages redrawn per collection. Reports / "View full list" get a notice
// instead, to keep their filters.
const LIVE_COLLECTIONS = {
  // Students/teachers also redraw evaluation pages (countedEvaluations).
  students:    { redraw: ['students', 'dashboard', 'supervisor', 'dept', 'evalControl', 'feedback'] },
  teachers:    { redraw: ['teachers', 'dashboard', 'supervisor', 'dept', 'evalControl', 'feedback'] },
  evaluations: { redraw: ['dashboard', 'evalControl', 'dept', 'supervisor', 'feedback'],
                 notify: ['reports', 'rptViewAll'] }
};

let _liveUnsubs = null;
let _liveRedrawTimer = null;
let _liveRetries = 0;
let _liveChanged = new Set();        // collections changed since the last redraw
let _liveNewEvals = 0;               // evaluations added since the last redraw
let _liveLastNotice = 0;

function startRosterListeners() {
  if (_liveUnsubs || !db) return;                       // once per page
  // Wait for sign-in before starting listeners.
  if (!fbAuth.currentUser) {
    const off = fbAuth.onAuthStateChanged(u => { if (u) { off(); startRosterListeners(); } });
    return;
  }
  _liveUnsubs = Object.keys(LIVE_COLLECTIONS).map(col => {
    let first = true;
    return db.collection(col).onSnapshot(snap => {
      const docs = snap.docs.map(d => {
        const data = { id: d.id, ...d.data() };
        // Same default loadFromFirebase() applies to old teacher records.
        if (col === 'teachers' && !data.facultyType) data.facultyType = 'regular';
        return data;
      });
      if (col === 'students' || col === 'teachers') {
        // Passwords go to the in-memory vault; only a stripped copy is stored.
        localStorage.setItem(col, JSON.stringify(_stripPasswords(col, docs)));
        if (window._pwVaultReady) window._pwVaultReady[col] = true;
      } else {
        localStorage.setItem(col, JSON.stringify(docs));
      }
      // The first snapshot is simply everything that already exists - not news.
      if (!first && col === 'evaluations') {
        _liveNewEvals += snap.docChanges().filter(c => c.type === 'added').length;
      }
      first = false;
      _liveChanged.add(col);
      _scheduleLiveRedraw();
    }, err => {
      console.warn('Live updates for ' + col + ' stopped:', (err && (err.code || err.message)));
      // Stop all and retry shortly (a few tries); the status badge restarts them.
      if (_liveUnsubs) { _liveUnsubs.forEach(u => { try { u(); } catch (e) {} }); _liveUnsubs = null; }
      if (_liveRetries++ < 3) setTimeout(startRosterListeners, 5000);
    });
  });
```

```
}

// Debounced for bursts of snapshots.
function _scheduleLiveRedraw() {
  clearTimeout(_liveRedrawTimer);
  _liveRedrawTimer = setTimeout(_liveRedraw, 400);
}

function _liveRedraw() {
  const el = document.querySelector('.page.active');
  const page = el ? el.id.replace(/^page-/, '') : '';

  // Don't redraw while a field has focus (except Students/Teachers search).
  const focused = document.activeElement;
  const typingHere = focused && el && el.contains(focused) &&
                     /^(INPUT|TEXTAREA|SELECT)$/.test(focused.tagName) &&
                     !/^(studentSearchInput|teacherSearchInput)$/.test(focused.id);
  // Nor while a pop-up is open, which would close open dropdowns.
  const popupOpen = document.querySelector(
    '.fb-print-overlay, .modal-overlay.open, .modal-overlay.active, .modal.open, #sefAuditOverlay');
  if (typingHere || popupOpen) { _liveRedrawTimer = setTimeout(_liveRedraw, 1500); return; }

  const changed = _liveChanged;
  const newEvals = _liveNewEvals;
  _liveChanged = new Set();
  _liveNewEvals = 0;

  const redraw = [...changed].some(c => (LIVE_COLLECTIONS[c].redraw || []).includes(page));
  const notify = [...changed].some(c => (LIVE_COLLECTIONS[c].notify || []).includes(page));
  const search = id => (document.getElementById(id) || {}).value || '';

  try {
    if (redraw) {
      if (page === 'students' && typeof renderStudents === 'function') renderStudents(search('studentSearchInput'));
      else if (page === 'teachers' && typeof renderTeachers === 'function') renderTeachers(search('teacherSearchInput'));
      else _rerenderCurrentPage();
    } else if (notify && newEvals > 0 && Date.now() - _liveLastNotice > 30000) {
      _liveLastNotice = Date.now();
      showToast(newEvals === 1 ? 'A new evaluation came in. Reopen this report to include it.'
                               : newEvals + ' new evaluations came in. Reopen this report to include them.', 'info');
    }
  } catch (e) { console.warn('Live redraw failed:', e); }
}

window.addEventListener('load', async () => {
  await loadFromFirebase();
  startRosterListeners();
});
</script>

<style>
/* Only critical overrides that can't be in external CSS */
#app { display: flex; min-height: 100vh; }
/* Collapsible dept section */
.dept-section-toggle { transition: opacity 0.15s; }
.dept-section-toggle:hover { opacity: 0.8; }
#deptNavList .nav-item { animation: none; }
/* Annex tab styles */
.annex-tab {
  padding: 10px 18px;
  border: none;
  background: none;
  font-size: 0.82rem;
  font-weight: 600;
  color: var(--muted);
  cursor: pointer;
  border-bottom: 3px solid transparent;
  margin-bottom: -2px;
  transition: color 0.15s, border-color 0.15s;
}
.annex-tab:hover { color: var(--primary); }
.annex-tab-active { color: var(--primary) !important; border-bottom-color: var(--primary) !important; }
/* Clickable ID / Name headers on the Faculty and Supervisors tables. */
.th-sort { cursor: pointer; user-select: none; white-space: nowrap; }
.th-sort:hover { color: var(--primary); }
.th-sort .sort-ind { font-size: 0.7em; margin-left: 2px; opacity: 0.85; }
</style>
</head>
<body>
<div id="app">
  <div class="overlay-bg" id="sidebarOverlay" onclick="closeSidebar()"></div>

  <aside class="sidebar" id="adminSidebar">
    <div class="sidebar-brand">
      <div class="sidebar-brand-text">
        <h2>NwSSU Admin</h2>
        <p>Faculty Evaluation System</p>
      </div>
    </div>
    <nav class="sidebar-nav">
      <div class="nav-section-label">Main</div>
      <div class="nav-item active" onclick="showPage('dashboard')">
        <svg width="16" height="16" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><rect x="3" y="3" width="7" height="7"/><r
ect x="14" y="3" width="7" height="7"/><rect x="3" y="14" width="7" height="7"/><rect x="14" y="14" width="7" height="7"/></svg>Dashboard
      </div>
      <div class="nav-item" onclick="showPage('schoolYear')">
        <svg width="16" height="16" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><rect x="3" y="4" width="18" height="18" r
x="2"/><path d="M16 2v4M8 2v4M3 10h18"/></svg>School Year & Semester
      </div>

      <div class="nav-section-label">Management</div>
      <div class="nav-item" onclick="showPage('students')">
        <svg width="16" height="16" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M17 21v-2a4 4 0 00-4-4H5a4 4 0 00
-4 4v2"/><circle cx="9" cy="7" r="4"/><path d="M23 21v-2a4 4 0 00-3-3.87M16 3.13a4 4 0 010 7.75"/></svg>Students
      </div>
```

```
      <div class="nav-item" onclick="showPage('teachers')">
        <svg width="16" height="16" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M20 21v-2a4 4 0 00-4-4H8a4 4 0 00
-4 4v2"/><circle cx="12" cy="7" r="4"/></svg>Teachers
      </div>
      <div class="nav-item" onclick="showPage('subjects')">
        <svg width="16" height="16" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M4 19.5A2.5 2.5 0 016.5 17H20"/><
path d="M6.5 2H20v20H6.5A2.5 2.5 0 014 19.5v-15A2.5 2.5 0 016.5 2z"/></svg>Subjects
      </div>

      <div class="nav-item" id="nav-deptManage" onclick="showDeptManagePage()">
        <svg width="16" height="16" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><rect x="2" y="7" width="20" height="14" r
x="2"/><path d="M16 7V5a2 2 0 00-2-2h-4a2 2 0 00-2 2v2"/><line x1="12" y1="12" x2="12" y2="16"/><line x1="10" y1="14" x2="14" y2="14"/></svg>Departme
nts
      </div>
      <div class="nav-item" id="nav-feedback" onclick="showFeedbackPage()">
        <svg width="16" height="16" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M21 15a2 2 0 01-2 2H7l-4 4V5a2 2
0 012-2h14a2 2 0 012 2z"/></svg>Feedback
      </div>

      <div class="nav-section-label">Evaluation</div>
      <div class="nav-item" onclick="showPage('evalControl')">
        <svg width="16" height="16" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M9 11l3 3L22 4"/><path d="M21 12v
7a2 2 0 01-2 2H5a2 2 0 01-2-2V5a2 2 0 012-2h11"/></svg>Evaluation Control
      </div>
      <div class="nav-item" onclick="showPage('reports')">
        <svg width="16" height="16" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><line x1="18" y1="20" x2="18" y2="10"/><li
ne x1="12" y1="20" x2="12" y2="4"/><line x1="6" y1="20" x2="6" y2="14"/></svg>Reports & Analytics
      </div>









      <div class="nav-item" onclick="showPage('auditLog')">
        <svg width="16" height="16" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M14 2H6a2 2 0 00-2 2v16a2 2 0 002
2h12a2 2 0 002-2V8z"/><polyline points="14 2 14 8 20 8"/><line x1="16" y1="13" x2="8" y2="13"/><line x1="16" y1="17" x2="8" y2="17"/></svg>Audit Log
      </div>



      <div id="deptNavSection">
        <div class="nav-section-label dept-section-toggle" onclick="toggleDeptNav()" style="cursor:pointer;display:flex;align-items:center;justify-co
ntent:space-between;user-select:none;">
          <span>Departments</span>
          <svg id="deptNavChevron" width="12" height="12" fill="none" stroke="currentColor" stroke-width="2.5" viewBox="0 0 24 24" style="transition:
transform 0.2s;margin-right:4px;"><polyline points="6 9 12 15 18 9"/></svg>
        </div>
        <div id="deptNavList"></div>
      </div>
    </nav>

    <div class="sidebar-footer">
      <div id="fbStatusBadge" style="padding:6px 10px 8px;"><span style="color:#f59e0b;font-size:0.72rem;font-weight:600;">⏳ Initializing...</span>
</div>
      <button class="logout-btn" onclick="doLogout()">
        <svg width="16" height="16" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M9 21H5a2 2 0 01-2-2V5a2 2 0 012-
2h4"/><polyline points="16 17 21 12 16 7"/><line x1="21" y1="12" x2="9" y2="12"/></svg>Logout
      </button>
    </div>
  </aside>

  <main class="main-content">
    <div style="display:flex;align-items:center;gap:12px;margin-bottom:20px;">
      <button class="mobile-menu-btn" onclick="toggleSidebar()">
        <svg width="20" height="20" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><line x1="3" y1="12" x2="21" y2="12"/><lin
e x1="3" y1="6" x2="21" y2="6"/><line x1="3" y1="18" x2="21" y2="18"/></svg>
      </button>
    </div>

    <!-- DASHBOARD -->
    <div class="page active" id="page-dashboard">
      <div class="page-header"><div class="page-title"><h1>Dashboard</h1><p>Overview of the teacher evaluation system</p></div></div>
      <div class="stats-grid" id="dashStats"></div>
      <div id="dashDeptRatings"></div>
      <div class="period-card" id="dashPeriodCard"></div>
    </div>

    <!-- SCHOOL YEAR -->
    <div class="page" id="page-schoolYear">
      <div class="page-header">
        <div class="page-title"><h1>School Year & Semester</h1><p>Manage academic periods</p></div>
        <button class="btn btn-primary" onclick="openModal('addSYModal')">
          <svg width="14" height="14" fill="none" stroke="currentColor" stroke-width="2.5" viewBox="0 0 24 24"><line x1="12" y1="5" x2="12" y2="19"/>
<line x1="5" y1="12" x2="19" y2="12"/></svg>Add School Year
        </button>
      </div>
      <div id="syList"></div>
    </div>

    <!-- STUDENTS - Grouped by Department with dept filter -->
    <div class="page" id="page-students">
      <div class="page-header">
        <div class="page-title"><h1>Student Management</h1><p>Add, edit, and manage student accounts</p></div>
        <div style="display:flex;gap:8px;">
          <button class="btn btn-ghost" onclick="openModal('bulkUploadStudentModal')">
            <svg width="14" height="14" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M21 15v4a2 2 0 01-2 2H5a2 2 0
01-2-2v-4"/><polyline points="17 8 12 3 7 8"/><line x1="12" y1="3" x2="12" y2="15"/></svg>Bulk Upload
          </button>
          <button class="btn btn-primary" onclick="openAddStudentModal()">
            <svg width="14" height="14" fill="none" stroke="currentColor" stroke-width="2.5" viewBox="0 0 24 24"><line x1="12" y1="5" x2="12" y2="1
9"/><line x1="5" y1="12" x2="19" y2="12"/></svg>Add Student
          </button>
        </div>
      </div>
      <div class="card"><div class="card-body">
        <div style="display:flex;gap:10px;flex-wrap:wrap;align-items:center;margin-bottom:12px;">
          <div class="search-box" style="flex:1;min-width:200px;margin-bottom:0;">
```

```
            <svg class="search-icon" width="14" height="14" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><circle cx="11" cy
="11" r="8"/><line x1="21" y1="21" x2="16.65" y2="16.65"/></svg>
            <input type="text" id="studentSearchInput" placeholder="Search by name, ID, course, year, section (e.g. bscs 2nd c)..." oninput="renderSt
udents(this.value)"/>
          </div>
        </div>
        <div id="studentDeptFilterBar" style="display:flex;gap:6px;flex-wrap:wrap;margin-bottom:14px;"></div>
        <div class="table-wrap">
          <table>
            <thead>
              <tr>
                <th>ID Number</th>
                <th>Name</th>
                <th>Course</th>
                <th>Year</th>
                <th>Section</th>
                <th>Department</th>
                <th>Status</th>
                <th>Actions</th>
              </tr>
            </thead>
            <tbody id="studentsTbody"></tbody>
          </table>
        </div>
      </div></div>
    </div>

    <!-- TEACHERS - Split Faculty / Supervisor with dept filter pills -->
    <div class="page" id="page-teachers">
  <div class="page-header">
    <div class="page-title"><h1>Teacher Management</h1><p>Add, edit, and manage teacher records</p></div>
    <div style="display:flex;gap:8px;">
      <button class="btn btn-ghost" onclick="openModal('bulkUploadTeacherModal')">
        <svg width="14" height="14" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M21 15v4a2 2 0 01-2 2H5a2 2 0 01-
2-2v-4"/><polyline points="17 8 12 3 7 8"/><line x1="12" y1="3" x2="12" y2="15"/></svg>Bulk Upload
      </button>
      <button class="btn btn-primary" id="teacherPageAddBtn" onclick="openAddTeacherModal()">
        <svg width="14" height="14" fill="none" stroke="currentColor" stroke-width="2.5" viewBox="0 0 24 24"><line x1="12" y1="5" x2="12" y2="19"/><l
ine x1="5" y1="12" x2="19" y2="12"/></svg>Add Teacher
      </button>
    </div>
  </div>

  <!-- ===== STICKY SECTION TABS ===== -->
  <div class="teacher-tab-bar" id="teacherTabBar">
    <button class="teacher-tab teacher-tab-active" id="teacherTabFaculty" onclick="switchTeacherTab('faculty')">
      <svg width="14" height="14" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M20 21v-2a4 4 0 00-4-4H8a4 4 0 00-4
4v2"/><circle cx="12" cy="7" r="4"/></svg>
      Faculty
      <span class="teacher-tab-count" id="teacherTabFacultyCount">0</span>
    </button>
    <button class="teacher-tab" id="teacherTabSupervisors" onclick="switchTeacherTab('supervisors')">
      <svg width="14" height="14" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M12 22s8-4 8-10V5l-8-3-8 3v7c0 6 8
10 8 10z"/></svg>
      Supervisors
      <span class="teacher-tab-count" id="teacherTabSupervisorsCount">0</span>
    </button>
  </div>

  <!-- ===== FACULTY PANEL ===== -->
  <div id="teacherPanelFaculty">
    <div class="card" style="margin-bottom:20px;">
      <!-- Collapsible header -->
      <div class="teacher-card-header" onclick="toggleTeacherSection('faculty')">
        <div style="display:flex;align-items:center;gap:8px;">
          <svg width="16" height="16" fill="none" stroke="var(--primary)" stroke-width="2" viewBox="0 0 24 24"><path d="M20 21v-2a4 4 0 00-4-4H8a4 4
0 00-4 4v2"/><circle cx="12" cy="7" r="4"/></svg>
          <span style="font-size:0.95rem;font-weight:700;">Faculty</span>
          <span class="dept-group-count" id="facultyHeaderCount" style="margin-left:0;">0</span>
        </div>
        <svg class="teacher-collapse-icon" id="facultyCollapseIcon" width="16" height="16" fill="none" stroke="currentColor" stroke-width="2.5" viewB
ox="0 0 24 24"><polyline points="18 15 12 9 6 15"/></svg>
      </div>
      <div class="card-body" id="facultyCardBody">
        <div class="search-box" style="margin-bottom:12px;">
          <svg class="search-icon" width="14" height="14" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><circle cx="11" cy
="11" r="8"/><line x1="21" y1="21" x2="16.65" y2="16.65"/></svg>
          <input type="text" id="teacherSearchInput" placeholder="Search by name, ID, or department (e.g. CCIS, Nursing)..." oninput="renderTeachers
(this.value)"/>
        </div>
        <div id="teacherDeptFilterBar" style="display:flex;gap:6px;flex-wrap:wrap;margin-bottom:14px;"></div>
        <select id="teacherDeptFilter" style="display:none;" onchange="renderTeachers()"><option value="">All Departments</option></select>
        <select id="teacherTypeFilter" style="display:none;"><option value="regular">Regular Faculty</option></select>
        <div class="table-wrap">
          <table>
            <thead>
              <tr>
                <th class="th-sort" onclick="setTeacherSort('id')" title="Sort by ID">ID <span class="sort-ind" data-sort="id"></span></th><th class
="th-sort" onclick="setTeacherSort('name')" title="Sort by name">Name <span class="sort-ind" data-sort="name"></span></th><th>Department</th><th>Subj
ects</th><th>Status</th><th>Actions</th>
              </tr>
            </thead>
            <tbody id="teachersTbody"></tbody>
          </table>
        </div>
      </div>
    </div>
  </div>

  <!-- ===== SUPERVISORS PANEL ===== -->
  <div id="teacherPanelSupervisors" style="display:none;">
    <div class="card">
      <!-- Collapsible header -->
      <div class="teacher-card-header" onclick="toggleTeacherSection('supervisors')">
        <div style="display:flex;align-items:center;gap:8px;">
```

```
          <svg width="16" height="16" fill="none" stroke="var(--warning)" stroke-width="2" viewBox="0 0 24 24"><path d="M12 22s8-4 8-10V5l-8-3-8 3v7c
0 6 8 10 8 10z"/></svg>
          <span style="font-size:0.95rem;font-weight:700;">Supervisors</span>
          <span class="dept-group-count" id="supervisorHeaderCount" style="margin-left:0;">0</span>
        </div>
        <div style="display:flex;align-items:center;gap:8px;" onclick="event.stopPropagation()">
          <button class="btn btn-primary btn-sm" onclick="openAddSupervisorModal()" style="font-size:0.8rem;padding:6px 14px;">
            <svg width="13" height="13" fill="none" stroke="currentColor" stroke-width="2.5" viewBox="0 0 24 24" style="margin-right:4px;"><line x1
="12" y1="5" x2="12" y2="19"/><line x1="5" y1="12" x2="19" y2="12"/></svg>Add Supervisor
          </button>
          <svg class="teacher-collapse-icon" id="supervisorsCollapseIcon" width="16" height="16" fill="none" stroke="currentColor" stroke-width="2.5"
viewBox="0 0 24 24"><polyline points="18 15 12 9 6 15"/></svg>
        </div>
      </div>
      <div class="card-body" id="supervisorsCardBody">
        <div class="search-box" style="margin-bottom:12px;">
          <svg class="search-icon" width="14" height="14" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><circle cx="11" cy
="11" r="8"/><line x1="21" y1="21" x2="16.65" y2="16.65"/></svg>
          <input type="text" id="supervisorSearchInput" placeholder="Search supervisor by name, ID, or department..." oninput="renderSupervisorTable
(this.value)"/>
        </div>
        <div id="supervisorDeptFilterBar" style="display:flex;gap:6px;flex-wrap:wrap;margin-bottom:14px;"></div>
        <select id="supervisorDeptFilterHidden" style="display:none;"></select>
        <div class="table-wrap">
          <table>
            <thead>
              <tr>
                <th class="th-sort" onclick="setTeacherSort('id')" title="Sort by ID">ID <span class="sort-ind" data-sort="id"></span></th><th class
="th-sort" onclick="setTeacherSort('name')" title="Sort by name">Name <span class="sort-ind" data-sort="name"></span></th><th>Department</th><th>Role
</th><th>Subjects</th><th>Status</th><th>Actions</th>
              </tr>
            </thead>
            <tbody id="supervisorsTbody"></tbody>
          </table>
        </div>
      </div>
    </div>
  </div>

    </div>

    <!-- SUBJECTS - Fixed with correct columns -->
    <div class="page" id="page-subjects">
      <div class="page-header">
        <div class="page-title"><h1>Subject Management</h1><p>Manage subjects and enrollments</p></div>
        <div style="display:flex;gap:8px;">
          <button class="btn btn-ghost" onclick="openModal('bulkUploadSubjectModal')">
            <svg width="14" height="14" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M21 15v4a2 2 0 01-2 2H5a2 2 0
01-2-2v-4"/><polyline points="17 8 12 3 7 8"/><line x1="12" y1="3" x2="12" y2="15"/></svg>Bulk Upload
          </button>
          <button class="btn btn-primary" onclick="openAddSubjectModal()">
            <svg width="14" height="14" fill="none" stroke="currentColor" stroke-width="2.5" viewBox="0 0 24 24"><line x1="12" y1="5" x2="12" y2="1
9"/><line x1="5" y1="12" x2="19" y2="12"/></svg>Add Subject
          </button>
        </div>
      </div>
      <div class="card"><div class="card-body">
        <div style="display:flex;gap:10px;flex-wrap:wrap;align-items:center;margin-bottom:12px;">
          <div class="search-box" style="flex:1;min-width:200px;margin-bottom:0;">
            <svg class="search-icon" width="14" height="14" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><circle cx="11" cy
="11" r="8"/><line x1="21" y1="21" x2="16.65" y2="16.65"/></svg>
            <input type="text" placeholder="Search subjects..." oninput="renderSubjects(this.value)"/>
          </div>
        </div>
        <div id="subjectDeptFilterBar" style="display:flex;gap:6px;flex-wrap:wrap;margin-bottom:14px;"></div>
        <select id="subjectDeptFilter" style="display:none;" onchange="renderSubjects()"><option value="">All Departments</option></select>
        <div class="table-wrap">
          <table>
            <thead>
              <tr>
                <th>Code</th>
                <th>Subject Name</th>
                <th>Department</th>
                <th>Category</th>
                <th>Teacher</th>
                <th>Enrolled</th>
                <th>Actions</th>
              </tr>
            </thead>
            <tbody id="subjectsTbody"></tbody>
          </table>
        </div>
      </div></div>
    </div>

    <!-- EVAL CONTROL -->
    <div class="page" id="page-evalControl">
      <div class="page-header"><div class="page-title"><h1>Evaluation Control</h1><p>Manage periods and track submissions</p></div></div>
      <div id="evalControlContent"></div>
    </div>

    <!-- REPORTS -->
    <div class="page" id="page-reports">
      <!-- Title and toolbar share one row; sizing lives in .reports-header / .reports-toolbar. -->
      <div class="page-header reports-header">
        <div class="page-title"><h1>Reports & Analytics</h1><p>View teacher ratings and evaluation results</p></div>
        <div class="reports-toolbar">
          <div class="term-selector">
            <svg class="term-cal" width="15" height="15" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><rect x="3" y="4" wid
th="18" height="18" rx="2"/><line x1="16" y1="2" x2="16" y2="6"/><line x1="8" y1="2" x2="8" y2="6"/><line x1="3" y1="10" x2="21" y2="10"/></svg>
            <select id="reportTermSelect" onchange="setReportTerm(this.value)" title="View ratings for a school year & semester"></select>
            <svg class="term-chevron" width="14" height="14" fill="none" stroke="currentColor" stroke-width="2.5" viewBox="0 0 24 24"><polyline point
s="6 9 12 15 18 9"/></svg>
          </div>
          <!-- Sliders icon, since the hamburger is already the mobile sidebar toggle. -->
```

```
          <div class="reports-actions" id="reportsActions">
            <button type="button" class="btn btn-toolbar reports-actions-btn" id="reportsActionsBtn"
                    onclick="toggleReportsMenu(event)"
                    aria-haspopup="true" aria-expanded="false" aria-controls="reportsActionsMenu"
                    title="Report actions and setup">
              <svg width="14" height="14" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><line x1="4" y1="21" x2="4" y2="14"/
><line x1="4" y1="10" x2="4" y2="3"/><line x1="12" y1="21" x2="12" y2="12"/><line x1="12" y1="8" x2="12" y2="3"/><line x1="20" y1="21" x2="20" y2="1
6"/><line x1="20" y1="12" x2="20" y2="3"/><line x1="1" y1="14" x2="7" y2="14"/><line x1="9" y1="8" x2="15" y2="8"/><line x1="17" y1="16" x2="23" y2
="16"/></svg>Actions
              <svg class="ra-chevron" width="13" height="13" fill="none" stroke="currentColor" stroke-width="2.5" viewBox="0 0 24 24"><polyline point
s="6 9 12 15 18 9"/></svg>
            </button>
            <div class="reports-actions-menu" id="reportsActionsMenu" role="menu" aria-labelledby="reportsActionsBtn">
              <div class="reports-menu-group">Generate</div>
              <button type="button" class="reports-menu-item" role="menuitem" onclick="closeReportsMenu();generateFERReport()">
                <svg width="15" height="15" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M14 2H6a2 2 0 00-2 2v16a2
2 0 002 2h12a2 2 0 002-2V8z"/><polyline points="14 2 14 8 20 8"/><line x1="16" y1="13" x2="8" y2="13"/><line x1="16" y1="17" x2="8" y2="17"/></svg>
                <span class="reports-menu-text"><span class="reports-menu-label">Generate FER</span><span class="reports-menu-desc">Institutional sum
mary for the President / VPAA (CMO &sect;6.8)</span></span>
              </button>
              <button type="button" class="reports-menu-item" role="menuitem" onclick="closeReportsMenu();exportReport()">
                <svg width="15" height="15" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M21 15v4a2 2 0 01-2 2H5a2
2 0 01-2-2v-4"/><polyline points="7 10 12 15 17 10"/><line x1="12" y1="15" x2="12" y2="3"/></svg>
                <span class="reports-menu-text"><span class="reports-menu-label">Export PDF</span><span class="reports-menu-desc">Print or save the c
urrent term&rsquo;s SET and SEF ratings as a PDF</span></span>
              </button>
              <div class="reports-menu-sep"></div>
              <div class="reports-menu-group">Setup</div>
              <button type="button" class="reports-menu-item" role="menuitem" onclick="closeReportsMenu();openQuestionsModal('SET')">
                <svg width="15" height="15" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M9.09 9a3 3 0 015.83 1c0
2-3 3-3 3"/><line x1="12" y1="17" x2="12.01" y2="17"/><circle cx="12" cy="12" r="10"/></svg>
                <span class="reports-menu-text"><span class="reports-menu-label">Questions</span><span class="reports-menu-desc">View or revise the S
ET and SEF questions</span></span>
              </button>
              <button type="button" class="reports-menu-item" role="menuitem" onclick="closeReportsMenu();openSEFAudit()">
                <svg width="15" height="15" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><circle cx="11" cy="11" r="7"/><li
ne x1="21" y1="21" x2="16.65" y2="16.65"/></svg>
                <span class="reports-menu-text"><span class="reports-menu-label">SEF Audit</span><span class="reports-menu-desc">Audit &amp;amp; clean su
pervisor (SEF) evaluation records</span></span>
              </button>
              <button type="button" class="reports-menu-item" role="menuitem" onclick="closeReportsMenu();openSignatoriesModal()">
                <svg width="15" height="15" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M12 19l7-7 3 3-7 7-3-3z"/
><path d="M18 13l-1.5-7.5L2 2l3.5 14.5L13 18l5-5z"/></svg>
                <span class="reports-menu-text"><span class="reports-menu-label">Signatories</span><span class="reports-menu-desc">Names printed as P
repared by / Reviewed by / Approved by</span></span>
              </button>
            </div>
          </div>
          <!-- Backup and Restore live elsewhere: they are destructive maintenance actions. -->
          <input type="file" id="restoreFileInput" accept="application/json,.json" style="display:none;" onchange="importSystemBackup(this)"/>
        </div>
      </div>
      <div id="reportsContent"></div>
    </div>

    <!-- REPORTS FULL LIST PAGE -->
    <div class="page" id="page-rptViewAll">
      <div id="rptViewAllContent"></div>
    </div>

    <!-- supervisor-->
     <div class="page" id="page-supervisor">
      <div class="page-header">
        <div class="page-title">
          <h1>Supervisor Management</h1>
          <p>Assigned supervisors who conduct SEF ratings — grouped by department</p>
        </div>
      </div>
      <div class="card">
        <div class="card-body">
          <div style="display:flex;gap:10px;flex-wrap:wrap;align-items:center;margin-bottom:12px;">
            <div class="search-box" style="flex:1;min-width:200px;margin-bottom:0;">
              <svg class="search-icon" width="14" height="14" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><circle cx="11"
cy="11" r="8"/><line x1="21" y1="21" x2="16.65" y2="16.65"/></svg>
              <input type="text" placeholder="Search supervisor by name or ID..." oninput="renderSupervisorList(this.value)"/>
            </div>
          </div>
          <div id="supervisorPageDeptFilterBar" style="display:flex;gap:6px;flex-wrap:wrap;margin-bottom:14px;"></div>
          <div class="table-wrap">
            <table>
              <thead>
                <tr>
                  <th>Name</th>
                  <th>Department</th>
                  <th>Role</th>
                  <th>SEF Status</th>
                  <th>Login Info</th>
                  <th>Actions</th>
                </tr>
              </thead>
              <tbody id="supervisorTbody"></tbody>
            </table>
          </div>
        </div>
      </div>
    </div>

    <!-- AUDIT LOG -->
    <div class="page" id="page-auditLog">
      <div class="page-header"><div class="page-title"><h1>Audit Log</h1><p>Track all administrative actions</p></div></div>
      <div class="card"><div class="card-body">
        <div class="search-box">
          <svg class="search-icon" width="14" height="14" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><circle cx="11" cy
="11" r="8"/><line x1="21" y1="21" x2="16.65" y2="16.65"/></svg>
          <input type="text" placeholder="Search audit logs..." oninput="renderAuditLog(this.value)"/>
```

```
        </div>
        <div id="auditLogList"></div>
      </div></div>
    </div>

    <!-- DEPARTMENTS PAGE -->
    <div class="page" id="page-dept">
      <div class="page-header">
        <div class="page-title">
          <h1 id="deptPageTitle">Department</h1>
          <p id="deptPageSubtitle">View teachers, subjects, and evaluations for this department</p>
        </div>
      </div>
      <div class="dept-banner dept-CCIS" id="deptBanner">
        <div class="dept-banner-icon" id="deptBannerIcon">🏫</div>
        <div class="dept-banner-info">
          <h2 id="deptBannerName">Department</h2>
          <p id="deptBannerDesc">Loading...</p>
        </div>
      </div>
      <div class="dept-stats-row" id="deptStatsRow"></div>
      <div class="dept-grid">
        <div class="dept-table-section">
          <div class="dept-table-header">
            <div style="display:flex;align-items:center;gap:8px;">
              <h4>Faculty</h4>
              <span class="count-badge" id="deptTeacherCount">0</span>
            </div>
            <button class="btn btn-ghost btn-sm" onclick="showDeptFullList(window.currentDept, 'teachers')" title="Open the full faculty list">View a
ll</button>
          </div>
          <div class="table-wrap">
            <table>
              <thead>
                <tr><th>ID</th><th>Name</th><th>Status</th><th>Avg Score</th></tr>
              </thead>
              <tbody id="deptTeachersTbody"></tbody>
            </table>
          </div>
        </div>
        <div class="dept-table-section">
          <div class="dept-table-header">
            <div style="display:flex;align-items:center;gap:8px;">
              <h4>Subjects</h4>
              <span class="count-badge" id="deptSubjectCount">0</span>
            </div>
            <button class="btn btn-ghost btn-sm" onclick="showDeptFullList(window.currentDept, 'subjects')" title="Open the full subjects list">View
all</button>
          </div>
          <div class="table-wrap">
            <table>
              <thead>
                <tr><th>Code</th><th>Name</th><th>Type</th><th>Teacher</th><th>Enrolled</th></tr>
              </thead>
              <tbody id="deptSubjectsTbody"></tbody>
            </table>
          </div>
        </div>
      </div>

      <!-- Dept Supervisors Section (shown only when dept has supervisors) -->
      <div id="deptSupervisorsSection" style="display:none;margin-top:20px;">
        <div class="card">
          <div class="card-body">
            <div class="dept-table-header" style="margin-bottom:12px;">
              <h4 style="font-size:0.95rem;font-weight:700;display:flex;align-items:center;gap:8px;">
                <svg width="15" height="15" fill="none" stroke="var(--warning)" stroke-width="2" viewBox="0 0 24 24"><path d="M12 22s8-4 8-10V5l-8-3-
8 3v7c0 6 8 10 8 10z"/></svg>
                Department Supervisors
              </h4>
              <span class="count-badge" id="deptSupervisorCount">0</span>
            </div>
            <p style="font-size:0.75rem;color:var(--muted);margin-bottom:12px;">Supervisors assigned to this department. They log in using their Teac
her ID to conduct SEF ratings.</p>
            <div class="table-wrap">
              <table>
                <thead>
                  <tr><th>ID</th><th>Name</th><th>Role</th><th>Status</th><th>SEF Conducted</th></tr>
                </thead>
                <tbody id="deptSupervisorsTbody"></tbody>
              </table>
            </div>
          </div>
        </div>
      </div>
    </div>

    <!-- DEPARTMENT MANAGEMENT PAGE -->
    <div class="page" id="page-deptManage">
      <div class="page-header">
        <div class="page-title"><h1>Department Management</h1><p>Create and manage college departments</p></div>
        <div style="display:flex;align-items:center;gap:8px;">
          <button class="btn btn-ghost" onclick="openManageDeptsModal()">
            <svg width="14" height="14" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M4 6h16M4 10h16M4 14h10M4 18h
7"/></svg>Manage Courses
          </button>
          <!-- Layout toggle -->
          <button id="dmLayoutGrid" class="btn btn-ghost btn-sm dm-layout-toggle active" title="Grid view" onclick="setDeptManageLayout('grid')">
            <svg width="14" height="14" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><rect x="3" y="3" width="7" height
="7"/><rect x="14" y="3" width="7" height="7"/><rect x="3" y="14" width="7" height="7"/><rect x="14" y="14" width="7" height="7"/></svg>
          </button>
          <button id="dmLayoutList" class="btn btn-ghost btn-sm dm-layout-toggle" title="List view" onclick="setDeptManageLayout('list')">
            <svg width="14" height="14" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><line x1="3" y1="6" x2="21" y2="6"/><l
ine x1="3" y1="12" x2="21" y2="12"/><line x1="3" y1="18" x2="21" y2="18"/></svg>
          </button>
```

```
        </div>
      </div>

      <!-- Add New Department Form -->
      <div class="card" style="margin-bottom:24px;">
        <div class="card-header-bar" style="padding:16px 20px 0; display:flex; align-items:center; justify-content:space-between;">
          <h3 style="font-size:1rem;font-weight:700;margin:0 0 4px;">Add New Department</h3>
        </div>
        <div class="card-body">
          <div class="form-row">
            <div class="form-group">
              <label class="form-label">Full Name <span style="color:var(--danger)">*</span></label>
              <input class="form-control" id="dmName" placeholder="e.g. College of Arts &amp; Sciences"/>
            </div>
            <div class="form-group">
              <label class="form-label">Short Name</label>
              <input class="form-control" id="dmShort" placeholder="e.g. CAS (auto-used as code)"/>
            </div>
          </div>
          <div class="form-row">
            <!-- Icon — image upload only -->
            <div class="form-group" style="flex:0 0 auto; min-width:200px;">
              <label class="form-label">Icon (optional)</label>
              <div style="display:flex;align-items:center;gap:10px;">
                <div id="dmIconPreview" style="width:38px;height:38px;border-radius:8px;background:var(--primary-light);display:flex;align-items:cent
er;justify-content:center;border:1px dashed var(--border);flex-shrink:0;overflow:hidden;">
                  <svg width="18" height="18" fill="none" stroke="var(--muted)" stroke-width="1.5" viewBox="0 0 24 24"><rect x="3" y="3" width="18" h
eight="18" rx="3"/><circle cx="8.5" cy="8.5" r="1.5"/><polyline points="21 15 16 10 5 21"/></svg>
                </div>
                <label style="display:inline-flex;align-items:center;gap:5px;padding:4px 10px;border:1px solid var(--border);border-radius:6px;font-s
ize:0.73rem;cursor:pointer;background:#fff;">
                  <svg width="11" height="11" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M21 15v4a2 2 0 01-2 2H5
a2 2 0 01-2-2v-4"/><polyline points="17 8 12 3 7 8"/><line x1="12" y1="3" x2="12" y2="15"/></svg>
                  Upload Image
                  <input type="file" accept="image/*" style="display:none;" onchange="dmHandleImageUpload(event)">
                </label>
              </div>
              <input type="hidden" id="dmIcon" value=""/>
            </div>
            <!-- Color picker -->
            <div class="form-group">
              <label class="form-label">Department Color</label>
              <div style="display:flex;flex-wrap:wrap;gap:7px;align-items:center;" id="dmColorPicker"><!-- filled by JS --></div>
              <input type="hidden" id="dmColor" value="#3b82f6"/>
            </div>
            <div class="form-group" style="flex:2;">
              <label class="form-label">Description</label>
              <input class="form-control" id="dmDesc" placeholder="Short description of this college/department"/>
            </div>
          </div>
          <button class="btn btn-primary" onclick="saveDeptFromManagePage()">
            <svg width="14" height="14" fill="none" stroke="currentColor" stroke-width="2.5" viewBox="0 0 24 24"><line x1="12" y1="5" x2="12" y2="1
9"/><line x1="5" y1="12" x2="19" y2="12"/></svg>Add Department
          </button>
        </div>
      </div>

      <!-- Department Cards/List -->
      <div id="deptManageGrid" class="dept-manage-grid"></div>
    </div>

    <!-- EDIT DEPT MODAL -->
    <div class="modal-overlay" id="editDeptModal">
      <div class="modal" style="max-width:520px;">
        <div class="modal-header">
          <h3 id="editDeptModalTitle">Edit Department</h3>
          <button class="modal-close" onclick="closeModal('editDeptModal')"><svg width="16" height="16" fill="none" stroke="currentColor" stroke-widt
h="2" viewBox="0 0 24 24"><line x1="18" y1="6" x2="6" y2="18"/><line x1="6" y1="6" x2="18" y2="18"/></svg></button>
        </div>
        <div class="modal-body" id="editDeptModalBody"></div>
      </div>
    </div>

    <!-- FEEDBACK PAGE -->
    <div class="page" id="page-feedback">
      <div class="page-header">
        <div class="page-title"><h1>Student Feedback</h1><p>Student comments submitted during instructor evaluations</p></div>
        <div style="display:flex;gap:10px;align-items:center;">
          <div id="feedbackStats"></div>
          <!-- Only shown when there is something to clean up (renderFeedbackPage). -->
          <button class="btn btn-ghost btn-sm" id="feedbackPurgeBtn" onclick="purgeOrphanFeedback()" style="display:none;color:var(--danger,#dc2626);
border-color:#fecaca;"></button>
        </div>
      </div>

      <!-- Dept filter pills -->
      <div id="feedbackDeptBar" style="display:flex;gap:6px;flex-wrap:wrap;margin-bottom:16px;"></div>

      <!-- Search -->
      <div class="search-box" style="margin-bottom:20px;">
        <svg class="search-icon" width="14" height="14" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><circle cx="11" cy="1
1" r="8"/><line x1="21" y1="21" x2="16.65" y2="16.65"/></svg>
        <input type="text" placeholder="Search by teacher or subject..." oninput="filterFeedback(null, this.value)"/>
      </div>

      <div id="feedbackList"></div>
    </div>

    <!-- MODALS -->
    <div class="modal-overlay" id="addSYModal">
      <div class="modal">
        <div class="modal-header"><h3>Add School Year</h3><button class="modal-close" onclick="closeModal('addSYModal')"><svg width="16" height="16"
fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><line x1="18" y1="6" x2="6" y2="18"/><line x1="6" y1="6" x2="18" y2="18"/></sv
g></button></div>
        <div class="modal-body"><div class="form-group"><label class="form-label">School Year</label><input class="form-control" id="syYearInput" pla
```

```
ceholder="2025-2026"/></div></div>
        <div class="modal-footer"><button class="btn btn-ghost" onclick="closeModal('addSYModal')">Cancel</button><button class="btn btn-primary" onc
lick="addSchoolYear()">Add</button></div>
      </div>
    </div>

    <div class="modal-overlay" id="addStudentModal">
      <div class="modal">
        <div class="modal-header"><h3 id="studentModalTitle">Add Student</h3><button class="modal-close" onclick="closeModal('addStudentModal')"><svg
width="16" height="16" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><line x1="18" y1="6" x2="6" y2="18"/><line x1="6" y1
="6" x2="18" y2="18"/></svg></button></div>
        <div class="modal-body">
          <div class="form-row">
            <div class="form-group"><label class="form-label">ID Number</label><input class="form-control" id="stuId" placeholder="24-00001 or 2025-9
902-1" maxlength="12" autocomplete="off" spellcheck="false"/></div>
            <div class="form-group"><label class="form-label">First Name</label><input class="form-control" id="stuFirst" placeholder="Juan"/></div>
          </div>
          <div class="form-row">
            <div class="form-group"><label class="form-label">Middle Name</label><input class="form-control" id="stuMiddle" placeholder="Miguel"/></d
iv>
            <div class="form-group"><label class="form-label">Last Name</label><input class="form-control" id="stuLast" placeholder="Dela Cruz"/></di
v>
          </div>
          <div class="form-group" style="max-width:180px;">
            <label class="form-label">Suffix</label><input class="form-control" id="stuSuffix" placeholder="Jr. (optional)"/>
          </div>
          <input type="hidden" id="stuName"/>
          <div class="form-group">
            <label class="form-label">Department</label>
            <select class="form-control" id="stuDept" onchange="loadStuCourses(this.value)">
              <option value="">-- Select Department --</option>
            </select>
          </div>
          <div class="form-group">
            <label class="form-label" style="display:flex;align-items:center;gap:8px;">
              Course <span class="dept-tag-inline" id="stuCourseDeptChip" style="display:none;"></span>
            </label>
            <select class="form-control" id="stuCourse">
              <option value="">— Select a department first —</option>
            </select>
          </div>
          <div class="form-row">
            <div class="form-group"><label class="form-label">Year Level</label><select class="form-control" id="stuYear"><option>1st Year</option><o
ption>2nd Year</option><option>3rd Year</option><option>4th Year</option><option>5th Year</option></select></div>
            <div class="form-group"><label class="form-label">Section</label><input class="form-control" id="stuSection" placeholder="A" style="text-
transform:uppercase;" autocapitalize="characters" maxlength="8"/></div>
          </div>
          <div class="form-group"><label class="form-label">Password (default: ID number)</label><input class="form-control" id="stuPass" placeholder
="Leave blank to use ID number"/></div>
        </div>
        <div class="modal-footer"><button class="btn btn-ghost" onclick="closeModal('addStudentModal')">Cancel</button><button class="btn btn-primar
y" id="saveStudentBtn" onclick="saveStudent()">Add Student</button></div>
      </div>
    </div>

    <div class="modal-overlay" id="resetPassModal">
      <div class="modal">
        <div class="modal-header"><h3>Reset Password</h3><button class="modal-close" onclick="closeModal('resetPassModal')"><svg width="16" height="1
6" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><line x1="18" y1="6" x2="6" y2="18"/><line x1="6" y1="6" x2="18" y2="18"/>
</svg></button></div>
        <div class="modal-body">
          <p style="font-size:0.82rem;color:var(--muted);margin-bottom:16px;">Resetting for: <strong id="resetPassName"></strong></p>
          <div class="form-group"><label class="form-label">New Password</label><input class="form-control" id="resetPassInput" placeholder="Leave bl
ank to reset to their Student ID"/></div>
          <p style="font-size:0.76rem;color:var(--muted);line-height:1.5;margin:0;">

          </p>
        </div>
        <div class="modal-footer"><input type="checkbox" id="forceResetCheck" checked style="display:none;"/><button class="btn btn-ghost" onclick="c
loseModal('resetPassModal')">Cancel</button><button class="btn btn-primary" onclick="saveResetPass()">Reset</button></div>
      </div>
    </div>

    <div class="modal-overlay" id="addTeacherModal">
      <div class="modal">
        <div class="modal-header"><h3 id="teacherModalTitle">Add Teacher</h3><button class="modal-close" onclick="closeModal('addTeacherModal')"><svg
width="16" height="16" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><line x1="18" y1="6" x2="6" y2="18"/><line x1="6" y1
="6" x2="18" y2="18"/></svg></button></div>
        <div class="modal-body">
          <div class="form-row">
            <div class="form-group"><label class="form-label">Teacher ID</label><input class="form-control" id="tchId" placeholder="T001"/></div>
            <div class="form-group">
              <label class="form-label">Title</label>
              <select class="form-control" id="tchTitle">
                <option value="">— None —</option>
                <option>Prof.</option><option>Dr.</option><option>Engr.</option>
                <option>Atty.</option><option>Mr.</option><option>Ms.</option><option>Mrs.</option>
              </select>
            </div>
          </div>
          <div class="form-row">
            <div class="form-group"><label class="form-label">First Name</label><input class="form-control" id="tchFirst" placeholder="Juan"/></div>
            <div class="form-group"><label class="form-label">Middle Name</label><input class="form-control" id="tchMiddle" placeholder="Miguel"/></d
iv>
          </div>
          <div class="form-row">
            <div class="form-group"><label class="form-label">Last Name</label><input class="form-control" id="tchLast" placeholder="Santos"/></div>
            <div class="form-group"><label class="form-label">Suffix</label><input class="form-control" id="tchSuffix" placeholder="Jr. (optional)"/>
</div>
          </div>
          <input type="hidden" id="tchName"/>
          <div class="form-row">
            <div class="form-group">
              <label class="form-label">Department</label>
              <select class="form-control" id="tchDept">
```

```
              <option value="">-- Select Department --</option>
            </select>
          </div>
          <div class="form-group">
            <!-- Printed on Annex C section A ("Current Faculty Rank"). -->
            <label class="form-label">Current Faculty Rank</label>
            <select class="form-control" id="tchRank">
              <option value="">-- Select Rank --</option>
              <optgroup label="Instructor">
                <option>Instructor I</option><option>Instructor II</option><option>Instructor III</option>
              </optgroup>
              <optgroup label="Assistant Professor">
                <option>Assistant Professor I</option><option>Assistant Professor II</option>
                <option>Assistant Professor III</option><option>Assistant Professor IV</option>
              </optgroup>
              <optgroup label="Associate Professor">
                <option>Associate Professor I</option><option>Associate Professor II</option>
                <option>Associate Professor III</option><option>Associate Professor IV</option>
                <option>Associate Professor V</option>
              </optgroup>
              <optgroup label="Professor">
                <option>Professor I</option><option>Professor II</option><option>Professor III</option>
                <option>Professor IV</option><option>Professor V</option><option>Professor VI</option>
              </optgroup>
              <option>Part-time Lecturer</option>
            </select>
          </div>
        </div>
        <!-- Hidden field = source of truth. saveTeacher() reads this, NOT the select. -->
        <input type="hidden" id="tchFacultyTypeHidden" value="regular"/>
        <div class="form-group" id="tchFacultyTypeGroup">
          <label class="form-label">Faculty Type</label>
          <select class="form-control" id="tchFacultyType" onchange="onFacultyTypeChange(this.value)">
            <option value="regular">Regular Faculty</option>
            <option value="supervisor">Supervisor (Dean / Chairperson)</option>
          </select>
        </div>
        <div class="form-group" id="tchDeptRoleGroup" style="display:none;">
          <label class="form-label">Supervisory Assignments</label>
          <small style="color:var(--muted);font-size:0.72rem;display:block;margin-bottom:8px;">
            One row per department this person supervises. A Dean of COED who also
            chairs a CCIS program gets two rows, with a different role in each.
          </small>

          <div id="tchSupRows"></div>

          <button type="button" class="btn btn-ghost btn-sm" style="margin-top:6px;" onclick="addSupRow()">
            + Add department
          </button>

          <!-- Legacy single role; mirrors the FIRST row above. -->
          <input type="hidden" id="tchDeptRole"/>

          <small style="color:var(--muted);font-size:0.72rem;margin-top:8px;display:block;">
            Supervisors log in using their Teacher ID and the password below.
          </small>
        </div>
        <!-- Password field — only visible when facultyType is supervisor -->
        <div class="form-group" id="tchPasswordGroup" style="display:none;">
          <label class="form-label">Supervisor Login Password <span style="font-weight:400;color:var(--muted);font-size:0.75rem;">&mdash; </span></
label>
          <div style="display:flex;gap:8px;align-items:center;">
            <input class="form-control" id="tchPassword" type="text" placeholder="Leave blank to keep current password" style="flex:1;"/>
            <button type="button" class="btn btn-ghost btn-sm" style="white-space:nowrap;font-size:0.75rem;" onclick="resetPasswordToId()">🔑 Rese
t to ID</button>
          </div>
          <small style="color:var(--muted);font-size:0.72rem;margin-top:4px;display:block;">
            When adding a new supervisor, leave blank to auto-set password to their Teacher ID.

          </small>
        </div>
      </div>
      <div class="modal-footer"><button class="btn btn-ghost" onclick="closeModal('addTeacherModal')">Cancel</button><button class="btn btn-primar
y" id="saveTeacherBtn" onclick="saveTeacher()">Add Teacher</button></div>
    </div>
  </div>

  <div class="modal-overlay" id="addSubjectModal">
    <div class="modal">
      <div class="modal-header"><h3 id="subjectModalTitle">Add Subject</h3><button class="modal-close" onclick="closeModal('addSubjectModal')"><svg
width="16" height="16" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><line x1="18" y1="6" x2="6" y2="18"/><line x1="6" y1
="6" x2="18" y2="18"/></svg></button></div>
      <div class="modal-body">
        <div class="form-row">
          <div class="form-group"><label class="form-label">Subject Code</label><input class="form-control" id="subCode" placeholder="CS301"/></div
>
          <div class="form-group"><label class="form-label">Subject Name</label><input class="form-control" id="subName" placeholder="Data Structur
es"/></div>
        </div>
        <!-- Category and Year Level share one line. -->
        <div class="form-row">
          <div class="form-group"><label class="form-label">Category</label><select class="form-control" id="subCategory"><option value="">-- Selec
t Category --</option><option value="Major">Major</option><option value="Minor">Minor</option><option value="GE">General Education</option><option va
lue="PE">Physical Education</option><option value="NSTP">NSTP</option><option value="Elective">Elective</option></select></div>
          <div class="form-group">
            <label class="form-label">Year Level</label>
            <select class="form-control" id="subYear">
              <option value="">Any year level</option>
              <option>1st Year</option><option>2nd Year</option><option>3rd Year</option>
              <option>4th Year</option><option>5th Year</option>
            </select>
          </div>
        </div>
        <div class="form-group">
          <label class="form-label">Department</label>
```

```
            <select class="form-control" id="subDept" onchange="populateTeacherSelect(); renderSubCourseChips();">
              <option value="">-- Select Department --</option>
            </select>
          </div>

          <!-- Multi-select: a subject can be shared by several courses. Depends on Department. -->
          <div class="form-group">
            <label class="form-label" style="display:flex;align-items:center;gap:8px;flex-wrap:wrap;">
              Offered to Course(s)
              <span id="subCourseSummary" class="sc-summary"></span>
              <span id="subCourseActions" style="margin-left:auto;display:flex;gap:6px;"></span>
            </label>
            <div id="subCourseChips" class="sc-chips"></div>
            <small style="color:var(--muted);font-size:0.71rem;display:block;margin-top:5px;">
              Tap each course this subject is offered to. Leaving all unselected means
              every course in the department &mdash; use that for GE, PE and NSTP.
            </small>
          </div>

          <div class="form-row">
        <div class="form-group">
          <label class="form-label">Load Type</label>
          <select class="form-control" id="subLoad">
            <option value="Regular">Regular Load</option>
            <option value="Overload">Overload</option>
          </select>
        </div>
        <div class="form-group" style="display:flex; align-items:flex-end; padding-bottom:10px;">
          <label style="display:flex; align-items:center; gap:8px; cursor:pointer; font-size:0.8rem; font-weight:600;">
            <input type="checkbox" id="subIsLab" style="accent-color:var(--primary)"> Laboratory School Class
          </label>
        </div>
      </div>

          <div class="form-group">
            <label class="form-label">Assign Teacher</label>
            <!-- Search-and-filter picker; hidden input keeps id `subTeacher` for saveSubject(). -->
            <input type="hidden" id="subTeacher" value=""/>
            <div id="teacherPicker" class="tp-wrap"></div>
          </div>
        </div>
        <div class="modal-footer"><button class="btn btn-ghost" onclick="closeModal('addSubjectModal')">Cancel</button><button class="btn btn-primar
y" id="saveSubjectBtn" onclick="saveSubject()">Add Subject</button></div>
      </div>
    </div>

    <!-- WHO TEACHES THIS SUBJECT - the list behind the MULTIPLE tag, plus enrolment. -->
    <div class="modal-overlay" id="subjectTeacherListModal">
      <div class="modal modal-lg">
        <div class="modal-header">
          <h3>Teachers of this Subject</h3>
          <button class="modal-close" onclick="closeModal('subjectTeacherListModal')"><svg width="16" height="16" fill="none" stroke="currentColor" s
troke-width="2" viewBox="0 0 24 24"><line x1="18" y1="6" x2="6" y2="18"/><line x1="6" y1="6" x2="18" y2="18"/></svg></button>
        </div>
        <div class="modal-body">
          <p style="font-size:0.82rem;color:var(--muted);margin-bottom:12px;">
            Subject: <strong id="tlSubjectName"></strong>
          </p>
          <div id="teacherListBody"></div>
        </div>
        <div class="modal-footer" style="justify-content:space-between;">
          <div style="display:flex;gap:8px;">
            <button class="btn btn-ghost" onclick="tlEnroll()">Enroll students</button>
            <button class="btn btn-ghost" onclick="tlManageTeachers()">Add or remove teachers</button>
          </div>
          <button class="btn btn-primary" onclick="closeModal('subjectTeacherListModal')">Done</button>
        </div>
      </div>
    </div>

    <!-- Teachers & sections for ONE subject (after students are enrolled). -->
    <div class="modal-overlay" id="subjectTeachersModal">
      <div class="modal modal-lg">
        <div class="modal-header">
          <h3>Teachers &amp; Sections</h3>
          <button class="modal-close" onclick="closeModal('subjectTeachersModal')"><svg width="16" height="16" fill="none" stroke="currentColor" stro
ke-width="2" viewBox="0 0 24 24"><line x1="18" y1="6" x2="6" y2="18"/><line x1="6" y1="6" x2="18" y2="18"/></svg></button>
        </div>
        <div class="modal-body">
          <p style="font-size:0.82rem;color:var(--muted);margin-bottom:6px;">
            Subject: <strong id="stSubjectName"></strong>
          </p>
          <p style="font-size:0.78rem;color:var(--muted);margin-bottom:14px;">
            Tap a teacher to add or remove them. Tick the sections a teacher takes;
            leave all unticked and every student in the subject rates them.
          </p>
          <div id="teacherPicker2" class="tp-wrap"></div>
        </div>
        <div class="modal-footer">
          <button class="btn btn-ghost" onclick="closeModal('subjectTeachersModal')">Cancel</button>
          <button class="btn btn-primary" onclick="saveSubjectTeachers()">Save</button>
        </div>
      </div>
    </div>

    <div class="modal-overlay" id="enrollModal">
      <div class="modal modal-lg">
        <div class="modal-header"><h3>Manage Enrollment</h3><button class="modal-close" onclick="closeModal('enrollModal')"><svg width="16" height="1
6" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><line x1="18" y1="6" x2="6" y2="18"/><line x1="6" y1="6" x2="18" y2="18"/>
</svg></button></div>
        <div class="modal-body">
          <p style="font-size:0.82rem;color:var(--muted);margin-bottom:14px;">Subject: <strong id="enrollSubjectName"></strong></p>
          <div id="enrollDeptFilter"></div>
          <div class="search-box" style="margin-bottom:10px;">
            <svg class="search-icon" width="14" height="14" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><circle cx="11" cy
```

```
="11" r="8"/><line x1="21" y1="21" x2="16.65" y2="16.65"/></svg>
            <input type="text" id="enrollSearchInput" placeholder="Search any student by name or ID (for irregulars)..." oninput="filterEnrollList
()"/>
          </div>
          <div style="max-height:340px;overflow-y:auto;" id="enrollList"></div>
        </div>
        <div class="modal-footer" style="display:flex;align-items:center;justify-content:space-between;gap:12px;">
          <span style="font-size:0.78rem;color:var(--muted);"><strong id="enrollCount">0</strong> student(s) enrolled</span>
          <span style="display:flex;gap:8px;">
            <button class="btn btn-ghost" onclick="closeModal('enrollModal')">Cancel</button>
            <button class="btn btn-primary" onclick="saveEnrollment()">Save Enrollment</button>
          </span>
        </div>
      </div>
    </div>

    <div class="modal-overlay" id="confirmModal">
      <div class="modal confirm-modal">
        <div class="modal-body" style="text-align:center;padding:28px 24px;">
          <div class="confirm-icon" style="background:var(--danger-light);"><svg width="24" height="24" fill="none" stroke="var(--danger)" stroke-wid
th="2" viewBox="0 0 24 24"><polyline points="3 6 5 6 21 6"/><path d="M19 6l-1 14a2 2 0 01-2 2H8a2 2 0 01-2-2L5 6"/></svg></div>
          <h3 style="margin-bottom:8px;" id="confirmTitle">Delete Record</h3>
          <p style="font-size:0.82rem;color:var(--muted);" id="confirmMsg"></p>
        </div>
        <div class="modal-footer"><button class="btn btn-ghost" onclick="closeModal('confirmModal')">Cancel</button><button class="btn btn-danger" id
="confirmActionBtn">Delete</button></div>
      </div>
    </div>

    <div class="modal-overlay" id="viewReportModal">
      <div class="modal modal-lg">
        <div class="modal-header"><h3 id="reportModalTitle">Teacher Report</h3><button class="modal-close" onclick="closeModal('viewReportModal')"><s
vg width="16" height="16" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><line x1="18" y1="6" x2="6" y2="18"/><line x1="6" y1
="6" x2="18" y2="18"/></svg></button></div>
        <div class="modal-body" id="reportModalBody"></div>
      </div>
    </div>

    <!-- DEPT ENROLLED STUDENTS MODAL -->
    <div class="modal-overlay" id="deptEnrolledModal">
      <div class="modal modal-lg">
        <div class="modal-header">
          <div>
            <h3 id="deptEnrolledTitle">Enrolled Students</h3>
            <span id="deptEnrolledCount" style="font-size:0.75rem;color:var(--muted);font-weight:500;"></span>
          </div>
          <button class="modal-close" onclick="closeModal('deptEnrolledModal')"><svg width="16" height="16" fill="none" stroke="currentColor" stroke-
width="2" viewBox="0 0 24 24"><line x1="18" y1="6" x2="6" y2="18"/><line x1="6" y1="6" x2="18" y2="18"/></svg></button>
        </div>
        <div class="modal-body">
          <div class="search-box" style="margin-bottom:12px;">
            <svg class="search-icon" width="14" height="14" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><circle cx="11" cy
="11" r="8"/><line x1="21" y1="21" x2="16.65" y2="16.65"/></svg>
            <input type="text" placeholder="Search by name, ID, year, or section..." oninput="filterDeptEnrolledStudents(this.value)"/>
          </div>
          <div class="table-wrap">
            <table>
              <thead>
                <tr>
                  <th>ID Number</th>
                  <th>Name</th>
                  <th>Year</th>
                  <th>Section</th>
                  <th>Enrolled Subjects</th>
                  <th>Status</th>
                </tr>
              </thead>
              <tbody id="deptEnrolledTbody"></tbody>
            </table>
          </div>
        </div>
        <div class="modal-footer">
          <button class="btn btn-ghost" onclick="closeModal('deptEnrolledModal')">Close</button>
        </div>
      </div>
    </div>

    <!-- ANNEX C & D MODAL -->
    <div class="modal-overlay" id="annexDModal">
      <div class="modal modal-lg">
        <div class="modal-header">
          <div>
            <h3 id="annexDModalTitle">Faculty Evaluation Reports</h3>
            <span style="font-size:0.72rem;color:var(--muted);" id="annexModalSubtitle"></span>
          </div>
          <div style="display:flex;gap:8px;align-items:center;">
            <button class="btn btn-ghost btn-sm" onclick="printActiveAnnex()">🖨️ Print</button>
            <button class="modal-close" onclick="closeModal('annexDModal')"><svg width="16" height="16" fill="none" stroke="currentColor" stroke-widt
h="2" viewBox="0 0 24 24"><line x1="18" y1="6" x2="6" y2="18"/><line x1="6" y1="6" x2="18" y2="18"/></svg></button>
          </div>
        </div>
        <div style="display:flex;gap:0;border-bottom:2px solid var(--border);padding:0 20px;">
          <button id="annexTabC" class="annex-tab annex-tab-active" onclick="switchAnnexTab('C')">📊 Annex C — IFER</button>
          <button id="annexTabD" class="annex-tab" onclick="switchAnnexTab('D')">📋 Annex D — FEDAF</button>
        </div>
        <div class="modal-body" id="annexDModalBody"></div>
      </div>
    </div>

    <!-- BULK UPLOAD STUDENTS MODAL -->
    <div class="modal-overlay" id="bulkUploadStudentModal">
      <div class="modal modal-lg">
        <div class="modal-header">
          <h3>Bulk Upload Students</h3>
          <button class="modal-close" onclick="closeModal('bulkUploadStudentModal')"><svg width="16" height="16" fill="none" stroke="currentColor" st
```

```
roke-width="2" viewBox="0 0 24 24"><line x1="18" y1="6" x2="6" y2="18"/><line x1="6" y1="6" x2="18" y2="18"/></svg></button>
        </div>
        <div class="modal-body">
          <div style="background:#f0fdf4;border:1px solid #bbf7d0;border-radius:8px;padding:14px;margin-bottom:16px;font-size:0.8rem;">
            <strong>CSV Format:</strong> ID Number, Full Name, Year Level, Section, Course, Department, Subject Codes (semicolon-separated)<br>
            <code style="display:block;margin-top:6px;color:#166534;background:#dcfce7;padding:6px 10px;border-radius:4px;">24-00001,Juan dela Cruz,1
st Year,A,Bachelor of Science in Information Technology (BSIT),CCIS,CS301;CS302</code>
            <span style="color:var(--muted);">ID Number must be YY-NNNNN (e.g. 24-00001) or YYYY-N-N (e.g. 2025-10118-1, 2025-9902-1, 2024-652-1 or 2
024-56-1) &mdash; rows in any other format are skipped.  |  Year Level options: 1st Year / 2nd Year / 3rd Year / 4th Year / 5th Year  
|  Course column can be full name or short code</span>
          </div>
          <div style="border:2px dashed var(--border);border-radius:10px;padding:28px;text-align:center;margin-bottom:14px;cursor:pointer;" onclick
="document.getElementById('bulkStudentFile').click()" id="bulkStudentDropZone">
            <svg width="32" height="32" fill="none" stroke="var(--muted)" stroke-width="1.5" viewBox="0 0 24 24" style="margin-bottom:8px;"><path d
="M21 15v4a2 2 0 01-2 2H5a2 2 0 01-2-2v-4"/><polyline points="17 8 12 3 7 8"/><line x1="12" y1="3" x2="12" y2="15"/></svg>
            <p style="color:var(--muted);font-size:0.82rem;margin:0;">Click to select CSV file or drag & drop here</p>
            <input type="file" id="bulkStudentFile" accept=".csv" style="display:none;" onchange="previewBulkStudents(this)"/>
          </div>
          <div id="bulkStudentPreview"></div>
        </div>
        <div class="modal-footer">
          <button class="btn btn-ghost" onclick="closeModal('bulkUploadStudentModal')">Cancel</button>
          <button class="btn btn-primary" id="bulkStudentImportBtn" onclick="importBulkStudents()" style="display:none;">Import Students</button>
        </div>
      </div>
    </div>

    <!-- BULK UPLOAD TEACHERS MODAL -->
    <div class="modal-overlay" id="bulkUploadTeacherModal">
      <div class="modal modal-lg">
        <div class="modal-header">
          <h3>Bulk Upload Teachers</h3>
          <button class="modal-close" onclick="closeModal('bulkUploadTeacherModal')"><svg width="16" height="16" fill="none" stroke="currentColor" st
roke-width="2" viewBox="0 0 24 24"><line x1="18" y1="6" x2="6" y2="18"/><line x1="6" y1="6" x2="18" y2="18"/></svg></button>
        </div>
        <div class="modal-body">
          <div style="background:#eff6ff;border:1px solid #bfdbfe;border-radius:8px;padding:14px;margin-bottom:16px;font-size:0.8rem;">
            <strong>CSV Format:</strong> Teacher ID, Full Name, Department, Faculty Type (regular/supervisor), Subject Codes (semicolon-separated)<br
>
            <code style="display:block;margin-top:6px;color:#1e40af;background:#dbeafe;padding:6px 10px;border-radius:4px;">T001,Prof. Maria Santos,C
CIS,regular,CS301;CS302</code>
          </div>
          <div style="border:2px dashed var(--border);border-radius:10px;padding:28px;text-align:center;margin-bottom:14px;cursor:pointer;" onclick
="document.getElementById('bulkTeacherFile').click()" id="bulkTeacherDropZone">
            <svg width="32" height="32" fill="none" stroke="var(--muted)" stroke-width="1.5" viewBox="0 0 24 24" style="margin-bottom:8px;"><path d
="M21 15v4a2 2 0 01-2 2H5a2 2 0 01-2-2v-4"/><polyline points="17 8 12 3 7 8"/><line x1="12" y1="3" x2="12" y2="15"/></svg>
            <p style="color:var(--muted);font-size:0.82rem;margin:0;">Click to select CSV file or drag & drop here</p>
            <input type="file" id="bulkTeacherFile" accept=".csv" style="display:none;" onchange="previewBulkTeachers(this)"/>
          </div>
          <div id="bulkTeacherPreview"></div>
        </div>
        <div class="modal-footer">
          <button class="btn btn-ghost" onclick="closeModal('bulkUploadTeacherModal')">Cancel</button>
          <button class="btn btn-primary" id="bulkTeacherImportBtn" onclick="importBulkTeachers()" style="display:none;">Import Teachers</button>
        </div>
      </div>
    </div>

    <!-- BULK UPLOAD SUBJECTS MODAL -->
    <div class="modal-overlay" id="bulkUploadSubjectModal">
      <div class="modal modal-lg">
        <div class="modal-header">
          <h3>Bulk Upload Subjects</h3>
          <button class="modal-close" onclick="closeModal('bulkUploadSubjectModal')"><svg width="16" height="16" fill="none" stroke="currentColor" st
roke-width="2" viewBox="0 0 24 24"><line x1="18" y1="6" x2="6" y2="18"/><line x1="6" y1="6" x2="18" y2="18"/></svg></button>
        </div>
        <div class="modal-body">
          <div style="background:#fdf4ff;border:1px solid #e9d5ff;border-radius:8px;padding:14px;margin-bottom:16px;font-size:0.8rem;">
            <strong>CSV Format:</strong> Subject Code, Subject Name, Department, Category, Load Type, Teacher ID (TID), Student IDs (semicolon-separa
ted SIDs)<br>
            <code style="display:block;margin-top:6px;color:#6b21a8;background:#f3e8ff;padding:6px 10px;border-radius:4px;">CS101,Introduction to Com
puting,CCIS,GE,Regular,T001,S001;S002;S003</code>
            <span style="color:var(--muted);">Teacher ID and Student IDs are optional — leave blank if not yet assigned. Duplicate subject codes are
skipped automatically. Matched students are auto-enrolled.</span>
          </div>
          <div style="border:2px dashed var(--border);border-radius:10px;padding:28px;text-align:center;margin-bottom:14px;cursor:pointer;" onclick
="document.getElementById('bulkSubjectFile').click()" id="bulkSubjectDropZone">
            <svg width="32" height="32" fill="none" stroke="var(--muted)" stroke-width="1.5" viewBox="0 0 24 24" style="margin-bottom:8px;"><path d
="M21 15v4a2 2 0 01-2 2H5a2 2 0 01-2-2v-4"/><polyline points="17 8 12 3 7 8"/><line x1="12" y1="3" x2="12" y2="15"/></svg>
            <p style="color:var(--muted);font-size:0.82rem;margin:0;">Click to select CSV file or drag &amp;amp; drop here</p>
            <input type="file" id="bulkSubjectFile" accept=".csv" style="display:none;" onchange="previewBulkSubjects(this)"/>
          </div>
          <div id="bulkSubjectPreview"></div>
        </div>
        <div class="modal-footer">
          <button class="btn btn-ghost" onclick="closeModal('bulkUploadSubjectModal')">Cancel</button>
          <button class="btn btn-primary" id="bulkSubjectImportBtn" onclick="importBulkSubjects()" style="display:none;">Import Subjects</button>
        </div>
      </div>
    </div>

    <div class="toast-container" id="toastContainer"></div>

    <script src="https://cdnjs.cloudflare.com/ajax/libs/xlsx/0.18.5/xlsx.full.min.js"></script>
    <!-- shared/js/data.js removed (unused, held demo passwords). -->
    <!-- LOAD ORDER MATTERS - do not reorder. The LAST definition wins.
         Owners of overridden functions:
           renderTeachers, renderSubjects, renderStudents, saveSubject,
           populateTeacherSelect, syncCollectionToFirestore,
           checkAndAutoFinalize ........................ modules/adminrate.js
           renderDeptPage*, showPage*, pill,
           openEditDeptModal, saveEditDept ........... modules/adminfeatures.js
           exportEnhancedReport ........................ modules/adminEval.js
           mcLoadDept* ................................. modules/admindept.js
         (* wraps the earlier definition - keep it. Earlier render* definitions
```

```
        in js/records/ must also stay: adminrate.js checks they exist.) -->
    <script src="js/core/admin-core.js"></script>
    <script src="js/core/admin-scoring.js"></script>
    <script src="js/core/admin-nav.js"></script>

    <script src="js/records/admin-students.js"></script>
    <script src="js/records/admin-teachers.js"></script>
    <script src="js/records/admin-subjects.js"></script>

    <script src="js/reports/admin-reports.js"></script>
    <script src="js/records/admin-bulk.js"></script>
    <script src="js/reports/admin-annex.js"></script>
    <script src="js/reports/admin-fer.js"></script>
    <script src="js/reports/admin-sef.js"></script>
    <script src="js/records/admin-courses.js"></script>

    <script src="js/modules/admindept.js"></script>
    <script src="js/modules/adminrate.js"></script>
    <script src="js/modules/adminEval.js"></script>
    <script src="js/modules/adminfeatures.js"></script>
    <script src="js/modules/adminQuestions.js"></script>

    <script src="js/core/admin-router.js"></script>



  </main>
</div>
</body>
</html>
```

admin/js/core/admin-core.js

553 lines

```
// admin-core.js - storage, sorting, audit, toast, password reset.
// Loads first; everything else depends on getData/setData.

if (!sessionStorage.getItem('adminLoggedIn')) {
  window.location.href = '../index.html';
}

// Reads localStorage (Firestore loads once at boot).
// Passwords are kept in memory only (_pwVault), never in localStorage;
// getData() re-attaches them so t.password / s.password still work.
const _PW_KEYS = ['students', 'teachers'];
window._pwVault = { students: {}, teachers: {} };
// Set once a collection's passwords have loaded from Firestore this session.
window._pwVaultReady = { students: false, teachers: false };

// Record the passwords from `list` into the vault; return a copy without them.
window._stripPasswords = function (key, list) {
    if (_PW_KEYS.indexOf(key) === -1 || !Array.isArray(list)) return list;
    return list.map(function (rec) {
        if (!rec || typeof rec !== 'object') return rec;
        if (rec.id && typeof rec.password === 'string') window._pwVault[key][rec.id] = rec.password;
        if (!('password' in rec)) return rec;
        const copy = Object.assign({}, rec);
        delete copy.password;
        return copy;
    });
};

// opts.includeHidden: evaluations only. By default, ratings from deleted
// evaluators are skipped; pass true for the full list (backups, writes).
window.getData = function(key, defaultValue = [], opts) {
    const data = localStorage.getItem(key);
    if (!data) return defaultValue;
    let parsed;
    try { parsed = JSON.parse(data); } catch (e) { return defaultValue; }

    if (key === 'evaluations' && Array.isArray(parsed) && !(opts && opts.includeHidden)) {
        return window.countedEvaluations(parsed);
    }

    // Migrate old copies that still hold passwords into the vault, once.
    if (_PW_KEYS.indexOf(key) !== -1 && Array.isArray(parsed) && parsed.some(r => r && 'password' in r)) {
        const clean = window._stripPasswords(key, parsed);
        try { localStorage.setItem(key, JSON.stringify(clean)); } catch (e) {}
    }
    if (_PW_KEYS.indexOf(key) === -1 || !Array.isArray(parsed)) return parsed;

    // Put the in-memory passwords back on what callers get.
    const vault = window._pwVault[key];
    return parsed.map(function (rec) {
        if (rec && rec.id && vault[rec.id] !== undefined && !('password' in rec)) {
            return Object.assign({}, rec, { password: vault[rec.id] });
        }
        return rec;
    });
};

window.setData = function(key, value) {
    // Refuse to save students/teachers before passwords load, or a save
    // could reset a real password to the default.
    if (_PW_KEYS.indexOf(key) !== -1 && !window._pwVaultReady[key]) {
        console.warn('Save of ' + key + ' blocked: passwords not loaded from Firestore yet.');
        // Only warn once a load has actually been tried and failed.
        if (window._firestoreLoadAttempted && typeof showToast === 'function') {
            showToast('Could not load the current ' + key + ' from the database yet. ' +
                      'Click the status badge to refresh, then try again.', 'warning');
        }
        return;
    }
    localStorage.setItem(key, JSON.stringify(window._stripPasswords(key, value)));
    if (typeof syncCollectionToFirestore === 'function') {
        syncCollectionToFirestore(key, value);          // Firestore keeps the full record
    }
};

// PER-RECORD SAVES (students and teachers): write only changed fields
// on the changed record, so stale tabs can't overwrite newer data.

// localStorage only. Same password guard as setData().
window.setLocalData = function(key, value) {
    if (_PW_KEYS.indexOf(key) !== -1 && !window._pwVaultReady[key]) {
        console.warn('Save of ' + key + ' blocked: passwords not loaded from Firestore yet.');
        // Silent during start-up, same as setData().
        if (window._firestoreLoadAttempted && typeof showToast === 'function') {
            showToast('The ' + key + ' list has not finished loading from the database. ' +
                      'Click the status badge to refresh, then try again.', 'warning');
        }
        return false;
    }
    localStorage.setItem(key, JSON.stringify(window._stripPasswords(key, value)));
    return true;
};

// Deep copy taken BEFORE editing, to diff changed fields on save.
window.cloneRecord = function(rec) {
    return rec ? JSON.parse(JSON.stringify(rec)) : null;
};

// Fields that differ between two versions; removed fields become deletes.
function _changedFields(before, after) {
```

```
    const out = {};
    const keys = new Set(Object.keys(before || {}).concat(Object.keys(after || {})));
    keys.forEach(function (k) {
        if (k === 'id') return;
        const a = before ? before[k] : undefined;
        const b = after[k];
        if (JSON.stringify(a) === JSON.stringify(b)) return;
        out[k] = (b === undefined) ? firebase.firestore.FieldValue.delete() : b;
    });
    return out;
}

// Writes one record. `before` null = new record (written whole);
// otherwise only changed fields are sent.
window.writeRecordToCloud = async function(col, before, after) {
    if (!after || !after.id) return false;
    if (typeof firebase === 'undefined' || !firebase.firestore) {
        showToast('Saved on this device only - the database is not reachable.', 'warning');
        return false;
    }
    const ref = firebase.firestore().collection(col).doc(after.id);
    try {
        if (!before) {
            await ref.set(JSON.parse(JSON.stringify(after)));   // drops undefined fields
            return true;
        }
        const changes = _changedFields(before, after);
        if (!Object.keys(changes).length) return true;
        await ref.update(changes);
        return true;
    } catch (e) {
        console.error('Write to ' + col + '/' + after.id + ' failed:', e);
        if (e && e.code === 'not-found') {
            // Deliberately NOT recreated - it was deleted elsewhere.
            showToast((after.name || 'This record') + ' no longer exists in the database ' +
                      '(it was removed elsewhere). Refresh to see the current list.', 'error');
        } else {
            showToast('Saved on this device, but the database write failed: ' + e.message, 'error');
        }
        return false;
    }
};

// Local copy + database. `changes` = [{ before, after }]. Resolves true
// only when every write reached Firestore.
window.commitRecords = async function(key, list, changes) {
    if (!window.setLocalData(key, list)) return false;
    const results = await Promise.all(changes.map(function (c) {
        return window.writeRecordToCloud(key, c.before, c.after);
    }));
    return results.every(Boolean);
};

// Bulk version of commitRecords. Batches of 400; database first, local
// copy only after success.
window.commitRecordsBatch = async function(key, list, changes) {
    if (_PW_KEYS.indexOf(key) !== -1 && !window._pwVaultReady[key]) {
        return window.setLocalData(key, list);          // refuses, with the message
    }
    if (typeof firebase === 'undefined' || !firebase.firestore) {
        showToast('The database is not reachable - nothing was imported.', 'error');
        return false;
    }
    const fdb = firebase.firestore();
    try {
        for (let i = 0; i < changes.length; i += 400) {
            const batch = fdb.batch();
            changes.slice(i, i + 400).forEach(function (c) {
                const ref = fdb.collection(key).doc(c.after.id);
                if (!c.before) {
                    batch.set(ref, JSON.parse(JSON.stringify(c.after)));
                } else {
                    const ch = _changedFields(c.before, c.after);
                    if (Object.keys(ch).length) batch.update(ref, ch);
                }
            });
            await batch.commit();
        }
    } catch (e) {
        console.error('Batch write to ' + key + ' failed:', e);
        showToast('Import stopped: ' + e.message + '. Refresh, then import the same file again - rows that were already saved will be skipped.', 'err
or');
        return false;
    }
    return window.setLocalData(key, list);
};

// All DB records with this ID (deleted included), for duplicate checks.
// Throws when the database is unreachable.
window.findRosterRecords = async function(col, field, value) {
    const snap = await firebase.firestore().collection(col).where(field, '==', value).get();
    return snap.docs.map(function (d) { return Object.assign({}, d.data(), { id: d.id }); });
};

// Same, for many IDs (chunks of 30). Returns { id: [records] }.
window.findRosterRecordsMany = async function(col, field, values) {
    const unique = Array.from(new Set(values.filter(Boolean)));
    const byValue = {};
    for (let i = 0; i < unique.length; i += 30) {
        const chunk = unique.slice(i, i + 30);
        const snap = await firebase.firestore().collection(col).where(field, 'in', chunk).get();
        snap.docs.forEach(function (d) {
            const rec = Object.assign({}, d.data(), { id: d.id });
            (byValue[rec[field]] = byValue[rec[field]] || []).push(rec);
        });
```

```
    }
    return byValue;
};

// RATINGS FROM DELETED PEOPLE: kept in Firestore but not counted.
// Missing evaluator records count as deleted; inactive ones still count.
// student SET -> studentId; supervisor SEF -> supervisorId or studentId.

function _evaluatorOf(e) {
    return e.evaluatorType === 'supervisor'
        ? { col: 'teachers', id: e.supervisorId || e.studentId || '' }
        : { col: 'students', id: e.studentId || '' };
}

// Live student/teacher IDs, cached until either list changes.
let _rosterIdsCache = { s: null, t: null, ids: null };
function _liveEvaluatorIds() {
    const rawS = localStorage.getItem('students') || '[]';
    const rawT = localStorage.getItem('teachers') || '[]';
    if (_rosterIdsCache.ids && _rosterIdsCache.s === rawS && _rosterIdsCache.t === rawT) {
        return _rosterIdsCache.ids;
    }
    const pick = function (raw) {
        let list = [];
        try { list = JSON.parse(raw) || []; } catch (e) {}
        return {
            loaded: list.length > 0,
            live: new Set(list.filter(function (r) { return r && r.deleted !== true; })
                              .map(function (r) { return r.id; }))
        };
    };
    const ids = { students: pick(rawS), teachers: pick(rawT) };
    _rosterIdsCache = { s: rawS, t: rawT, ids: ids };
    return ids;
}

// True when the evaluator was deleted or no longer exists.
// Nothing is hidden until both lists have loaded.
window.isEvaluationHidden = function(e) {
    if (!e) return false;
    const who = _evaluatorOf(e);
    if (!who.id) return false;                  // e.g. admin-entered SEF: no evaluator
    const roster = _liveEvaluatorIds()[who.col];
    return roster.loaded && !roster.live.has(who.id);
};

window.countedEvaluations = function(list) {
    return list.filter(function (e) { return !window.isEvaluationHidden(e); });
};

// Evaluations a person submitted, counted or not - for the delete message.
window.evaluationsSubmittedBy = function(docId, kind) {
    return getData('evaluations', [], { includeHidden: true }).filter(function (e) {
        const who = _evaluatorOf(e);
        return who.id === docId && who.col === (kind === 'teacher' ? 'teachers' : 'students');
    });
};

// Explicit delete. The sync helper has no delete path.
window.deleteDocFromFirestore = async function(collection, docId) {
    try {
        if (typeof firebase === 'undefined' || !firebase.firestore) return false;
        await firebase.firestore().collection(collection).doc(docId).delete();
        console.log('Deleted ' + collection + '/' + docId + ' from Firestore');
        return true;
    } catch (e) {
        console.warn('Firestore delete failed for ' + collection + '/' + docId + ':', e.message);
        if (typeof showToast === 'function') {
            showToast('Removed here, but the database copy could not be deleted. Check your rules.', 'error');
        }
        return false;
    }
};

window.yearRank = function(year) {
    const m = String(year || '').match(/\d+/);
    return m ? parseInt(m[0], 10) : 99;
};

// ALPHABETICAL ORDER: by surname, then first name. Titles and suffixes
// are skipped; particles like "Dela" stay with the surname.
const _SORT_TITLES   = ['dr', 'mr', 'mrs', 'ms', 'miss', 'engr', 'prof', 'atty', 'sir', 'maam'];
const _SORT_SUFFIXES = ['jr', 'sr', 'ii', 'iii', 'iv', 'v'];
const _SORT_PARTICLES = ['de', 'del', 'dela', 'delos', 'de los', 'da', 'di', 'san', 'sta', 'sto', 'van', 'von'];
function _surnameFirst(fullName) {
    const bare = w => w.toLowerCase().replace(/[.,']/g, '');
    let words = String(fullName || '').trim().split(/\s+/).filter(Boolean);
    while (words.length > 1 && _SORT_TITLES.indexOf(bare(words[0])) !== -1) words.shift();
    while (words.length > 1 && _SORT_SUFFIXES.indexOf(bare(words[words.length - 1])) !== -1) words.pop();
    if (words.length < 2) return words.join(' ');
    let start = words.length - 1;                           // the last word is the surname...
    while (start > 1 && _SORT_PARTICLES.indexOf(bare(words[start - 1])) !== -1) start--;  // ...plus "Dela", "De" etc.
    return words.slice(start).concat(words.slice(0, start)).join(' ');
}

window.personSortKey = function(p) {
    if (!p) return '';
    if (p.lastName) {
        return [p.lastName, p.firstName, p.middleName].filter(Boolean).join(' ');
    }
    return _surnameFirst(p.name);
};

// Name for lists: "Lastname, Firstname Middlename" (suffix kept, title dropped).
window.listName = function(p) {
```

```
    if (!p) return '';
    if (p.lastName) {
        const rest = [p.firstName, p.middleName, p.suffix].filter(Boolean).join(' ');
        return rest ? p.lastName + ', ' + rest : p.lastName;
    }
    const bare = w => w.toLowerCase().replace(/[.,']/g, '');
    let words = String(p.name || '').trim().split(/\s+/).filter(Boolean);
    while (words.length > 1 && _SORT_TITLES.indexOf(bare(words[0])) !== -1) words.shift();
    let suffix = '';
    if (words.length > 1 && _SORT_SUFFIXES.indexOf(bare(words[words.length - 1])) !== -1) suffix = words.pop();
    if (words.length < 2) return words.concat(suffix ? [suffix] : []).join(' ');
    let start = words.length - 1;
    while (start > 1 && _SORT_PARTICLES.indexOf(bare(words[start - 1])) !== -1) start--;
    const rest = words.slice(0, start).concat(suffix ? [suffix] : []).join(' ');
    return words.slice(start).join(' ') + ', ' + rest;
};

window.byName = function(a, b) {
    return personSortKey(a).localeCompare(personSortKey(b), undefined, { sensitivity: 'base', numeric: true });
};

// Sort: year level, section, name.
window.byYearThenName = function(a, b) {
    const ya = yearRank(a && a.year), yb = yearRank(b && b.year);
    if (ya !== yb) return ya - yb;
    const sa = String((a && a.section) || ''), sb = String((b && b.section) || '');
    if (sa !== sb) return sa.localeCompare(sb, undefined, { sensitivity: 'base', numeric: true });
    return byName(a, b);
};

// COURSE NAMES: normalizeCourseName (for saving) and courseKey
// (lower-cased, for comparing) give every course one spelling.
window.normalizeCourseName = function(s) {
    return String(s || '')
        .replace(/\s+/g, ' ')
        .replace(/\(\s+/g, '(')
        .replace(/\s+\)/g, ')')
        .trim();
};
window.courseKey = function(s) { return window.normalizeCourseName(s).toLowerCase(); };

// Sort: course first, then year/section/name. No-course records last.
window.byCourseThenYear = function(a, b) {
    const label = function(x) {
        const raw = String((x && x.course) || '');
        return (typeof courseShorthand === 'function') ? courseShorthand(raw) : raw.toUpperCase();
    };
    const ca = label(a), cb = label(b);
    if (!ca !== !cb) return ca ? -1 : 1;
    if (ca !== cb) return ca.localeCompare(cb, undefined, { sensitivity: 'base', numeric: true });
    return byYearThenName(a, b);
};

// Audit trail (CMO 19 5.2).
window.addAudit = function(action, detail) {
    const log = getData('auditLog', []);
    log.unshift({
        id: 'audit' + Date.now(),
        action: action,
        detail: detail,
        timestamp: new Date().toISOString().slice(0, 19).replace('T', ' ')
    });
    while (log.length > 500) log.pop();
    setData('auditLog', log);
};

window.showToast = function(message, type = 'info') {
    let toastContainer = document.getElementById('toastContainer');
    if (!toastContainer) {
        toastContainer = document.createElement('div');
        toastContainer.id = 'toastContainer';
        toastContainer.style.cssText = 'position:fixed;bottom:20px;right:20px;z-index:9999;display:flex;flex-direction:column;gap:10px;';
        document.body.appendChild(toastContainer);
    }

    const toast = document.createElement('div');
    const colors = {
        success: '#16a34a',
        error: '#dc2626',
        info: '#3b82f6',
        warning: '#f59e0b'
    };
    toast.style.cssText = `background:${colors[type] || colors.info};color:#fff;padding:12px 20px;border-radius:8px;font-size:0.875rem;box-shadow:0 4
px 12px rgba(0,0,0,0.15);animation:slideIn 0.3s ease;`;
    toast.textContent = message;
    toastContainer.appendChild(toast);

    setTimeout(() => {
        toast.style.opacity = '0';
        toast.style.transition = 'opacity 0.3s';
        setTimeout(() => toast.remove(), 300);
    }, 3000);
};

if (!document.querySelector('#toastStyles')) {
    const style = document.createElement('style');
    style.id = 'toastStyles';
    style.textContent = `
        @keyframes slideIn {
            from { transform: translateX(100%); opacity: 0; }
            to { transform: translateX(0); opacity: 1; }
        }
    `;
    document.head.appendChild(style);
}
```

```
// PASSWORD RESET. The roster password field is the credential.
window.resetLoginPassword = async function (kind, record, newPass) {
    const loginId = String(kind === 'student' ? record.sid : record.tid || '').trim();
    const wanted  = String(newPass || '').trim() || loginId;

    const col  = kind === 'student' ? 'students' : 'teachers';
    const rows = getData(col, []);
    const idx  = rows.findIndex(r => r.id === record.id);
    if (idx === -1) { showToast('Record not found.', 'error'); return false; }

    const before = window.cloneRecord(rows[idx]);
    rows[idx].password = wanted;
    rows[idx].forceReset = false;
    // Only this record, only these two fields - not the whole list.
    const ok = await window.commitRecords(col, rows, [{ before: before, after: rows[idx] }]);
    if (!ok) return false;

    addAudit('Reset Password', record.name + ' (' + loginId + ')');
    showToast(record.name + ' can now sign in with: ' + wanted, 'success');
    return true;
};
// SKELETON LOADING: shown only where a container is genuinely empty;
// real renders overwrite it.

window.skelLine = function (w) {
  return `<span class="skel skel-line" style="width:${w || '100%'};"></span>`;
};

// Four placeholder stat cards matching the dashboard's real ones.
function _skelStatCards(n) {
  return Array.from({ length: n || 4 }, () => `
    <div class="skel-card">
      <span class="skel skel-circle"></span>
      <span class="skel skel-line" style="width:42%;height:20px;"></span>
      <span class="skel skel-line sm" style="width:64%;"></span>
    </div>`).join('');
}

// Placeholder rows; column count comes from the table header.
function _skelRows(cols, n) {
  const widths = ['70%','55%','45%','60%','40%','50%'];
  return Array.from({ length: n }, () => '<tr>' +
    Array.from({ length: cols }, (_, c) =>
      `<td style="padding:12px 10px;">${skelLine(widths[c % widths.length])}</td>`).join('') +
    '</tr>').join('');
}

// Filter pills (Students, Teachers, Subjects, Departments).
function _skelPills(n) {
  return Array.from({ length: n || 5 }, () =>
    '<span class="skel skel-pill" style="margin-right:6px;"></span>').join('');
}

// List-row placeholders for non-table panels.
function _skelList(n) {
  return Array.from({ length: n || 5 }, () => `
    <div style="display:flex;align-items:center;gap:12px;padding:12px 4px;border-bottom:1px solid var(--border,#eef2f0);">
      <span class="skel skel-circle" style="height:30px;width:30px;"></span>
      <span style="flex:1;display:flex;flex-direction:column;gap:6px;">
        <span class="skel skel-line" style="width:38%;"></span>
        <span class="skel skel-line sm" style="width:22%;"></span>
      </span>
      <span class="skel skel-pill"></span>
    </div>`).join('');
}

// Card panel placeholder with a heading and a table.
function _skelPanel() {
  return `
    <div class="skel-card" style="gap:16px;">
      <span class="skel skel-line" style="width:30%;height:16px;"></span>
      <div>${_skelPills(4)}</div>
      ${_skelList(5)}
    </div>`;
}

// Every container the page scripts fill, keyed by element id.
const SKEL_TARGETS = {
  dashStats:                () => _skelStatCards(4),
  syList:                   () => _skelList(4),
  studentDeptFilterBar:     () => _skelPills(6),
  teacherDeptFilterBar:     () => _skelPills(6),
  supervisorDeptFilterBar:  () => _skelPills(4),
  subjectDeptFilterBar:     () => _skelPills(6),
  supervisorPageDeptFilterBar:() => _skelPills(4),
  evalControlContent:       () => _skelPanel(),
  reportsContent:           () => _skelPanel(),
  rptViewAllContent:        () => _skelPanel(),
  auditLogList:             () => _skelList(6),
  deptManageGrid:           () => _skelStatCards(6),
  deptStatsRow:             () => _skelStatCards(4),
  feedbackStats:            () => _skelStatCards(3),
  feedbackDeptBar:          () => _skelPills(6),
  feedbackList:             () => _skelList(4)
};

// Containers overwritten by a forced skeleton, with their old content.
window._skelRestore = [];

window.showSkeleton = function (force) {
  try {
    // Only the page currently open.
    const page = document.querySelector('.page.active');
    if (!page) return;
    if (force) window._skelRestore = [];
```

```
    const fill = (el, html) => {
      if (force) window._skelRestore.push({ el: el, html: el.innerHTML });
      el.innerHTML = html;
    };

    // Named containers first - these are the panels that are not tables.
    Object.keys(SKEL_TARGETS).forEach(id => {
      const el = document.getElementById(id);
      if (!el || !page.contains(el)) return;
      if (!force && el.children.length) return;   // never cover real content
      fill(el, SKEL_TARGETS[id]());
    });

    page.querySelectorAll('table').forEach(tbl => {
      const body = tbl.querySelector('tbody');
      if (!body) return;
      if (!force && body.children.length) return;   // never cover real rows
      const cols = tbl.querySelectorAll('thead th').length || 4;
      fill(body, _skelRows(cols, 6));
    });
  } catch (e) {
    // Purely cosmetic. A failure here must never stop the data load.
    console.warn('Skeleton render skipped:', e && e.message);
  }
};

// After a forced refresh, restore any container its page didn't redraw.
window.clearStuckSkeletons = function () {
  try {
    (window._skelRestore || []).forEach(r => {
      if (r.el && r.el.querySelector('.skel')) r.el.innerHTML = r.html;
    });
  } catch (e) { /* cosmetic */ }
  window._skelRestore = [];
};
```

admin/js/core/admin-nav.js

204 lines

```
// admin-nav.js - page switching and the dashboard summary cards.

let editStudentId = null, editTeacherId = null, editSubjectId = null;
let _pendingFacultyType = "regular";
let enrollSubjectId = null, resetPassStudentId = null;

function showPage(page) {
  document.querySelectorAll('.page').forEach(p => p.classList.remove('active'));
  document.querySelectorAll('.nav-item').forEach(n => n.classList.remove('active'));

  const pageEl = document.getElementById('page-' + page);
  if (pageEl) pageEl.classList.add('active');

  document.querySelectorAll('.nav-item').forEach(n => {
    if (n.getAttribute('onclick') && n.getAttribute('onclick').includes(`'${page}'`)) n.classList.add('active');
  });

  const renders = {
    dashboard: renderDashboard,
    schoolYear: renderSchoolYear,
    students: renderStudents,
    teachers: renderTeachers,
    subjects: renderSubjects,
    evalControl: renderEvalControl,
    reports: renderReports,
    auditLog: renderAuditLog,
    supervisor: renderSupervisorList,
    rptViewAll: renderRptViewAll
  };

  if (renders[page]) renders[page]();
  closeSidebar();
}

function closeSidebar() {
  document.getElementById('adminSidebar').classList.remove('open');
  document.getElementById('sidebarOverlay').classList.remove('open');
}

function toggleSidebar() {
  document.getElementById('adminSidebar').classList.toggle('open');
  document.getElementById('sidebarOverlay').classList.toggle('open');
}

function doLogout() {
  sessionStorage.removeItem('adminLoggedIn');
  const done = () => window.location.replace('../index.html');
  try {
    if (window.fbAuth) { window.fbAuth.signOut().finally(done); }
    else if (typeof firebase !== 'undefined' && firebase.auth) { firebase.auth().signOut().finally(done); }
    else done();
  } catch (e) { done(); }
}

// Evaluations on the dashboard card: active term only, split into
// SET and SEF (as CMO 19 reports them).
function _countDashEvals() {
  const liveTeachers = new Set(getData('teachers', []).filter(t => !t.deleted).map(t => t.id));
  const inTerm = (typeof evalInActiveTerm === 'function') ? evalInActiveTerm : () => true;

  let set = 0, sef = 0;
  getData('evaluations', []).forEach(e => {
    if (!inTerm(e)) return;
    // A rating for a removed faculty member is history, not current activity.
    if (e.teacherId && !liveTeachers.has(e.teacherId)) return;
    if (e.evaluatorType === 'supervisor') sef++; else set++;
  });
  return { set, sef, total: set + sef };
}

function renderDashboard() {
  const students = getData('students', []).filter(s => !s.deleted);
  const teachers = getData('teachers', []).filter(t => !t.deleted);
  const subjects = getData('subjects', []);
  const evals = getData('evaluations', []);
  const _dashEvalCounts = _countDashEvals();
  const period = getData('evalPeriod', {});
  const sy = getActiveSY();

  document.getElementById('dashStats').innerHTML = `
    <div class="stat-card stat-card-clickable" onclick="showPage('students')" title="View all students"><div class="stat-icon" style="background:#eff
6ff;"><svg width="20" height="20" fill="none" stroke="#2563eb" stroke-width="2" viewBox="0 0 24 24"><path d="M17 21v-2a4 4 0 00-4-4H5a4 4 0 00-4 4v
2"/><circle cx="9" cy="7" r="4"/></svg></div><div class="value">${students.length}</div><div class="label">Total Students</div></div>
    <div class="stat-card stat-card-clickable" onclick="showPage('teachers')" title="View all teachers"><div class="stat-icon" style="background:#f0f
df4;"><svg width="20" height="20" fill="none" stroke="#16a34a" stroke-width="2" viewBox="0 0 24 24"><path d="M20 21v-2a4 4 0 00-4-4H8a4 4 0 00-4 4v
2"/><circle cx="12" cy="7" r="4"/></svg></div><div class="value">${teachers.length}</div><div class="label">Total Teachers</div></div>
    <div class="stat-card stat-card-clickable" onclick="showPage('subjects')" title="View all subjects"><div class="stat-icon" style="background:#fdf
4ff;"><svg width="20" height="20" fill="none" stroke="#9333ea" stroke-width="2" viewBox="0 0 24 24"><path d="M4 19.5A2.5 2.5 0 016.5 17H20"/><path d
="M6.5 2H20v20H6.5A2.5 2.5 0 014 19.5v-15A2.5 2.5 0 016.5 2z"/></svg></div><div class="value">${subjects.length}</div><div class="label">Total Subjec
ts</div></div>
    <div class="stat-card stat-card-clickable" onclick="showPage('reports')" title="View Reports & Analytics"><div class="stat-icon" style="backgroun
d:#fff7ed;"><svg width="20" height="20" fill="none" stroke="#ea580c" stroke-width="2" viewBox="0 0 24 24"><path d="M9 11l3 3L22 4"/></svg></div><div
class="value">${_dashEvalCounts.total}</div><div class="label">${_dashEvalCounts.set} SET &middot; ${_dashEvalCounts.sef} SEF this term</div></div>
  `;

  const deptMap = {};
  teachers.forEach(t => {
    if (!t.dept) return;
    if (!deptMap[t.dept]) deptMap[t.dept] = { setTotal: 0, sefTotal: 0, setCount: 0, sefCount: 0, faculty: 0 };
    deptMap[t.dept].faculty++;
```

```
    const set = parseFloat(calculateWeightedSETRating(t.id));
    const sef = getSEFForTeacher(t.id).average || 0;
    if (set > 0) { deptMap[t.dept].setTotal += set; deptMap[t.dept].setCount++; }
    if (sef > 0) { deptMap[t.dept].sefTotal += sef; deptMap[t.dept].sefCount++; }
  });

  const deptEntries = Object.entries(deptMap);
  const deptRatingsEl = document.getElementById('dashDeptRatings');
  if (deptRatingsEl && deptEntries.length > 0) {
    deptRatingsEl.innerHTML = `
      <div class="card" style="margin-bottom:24px;">
        <div class="card-header-bar"><h3>📊 Department Performance (SET | SEF)</h3></div>
        <div class="card-body">
          <p style="font-size:0.75rem;color:var(--muted);margin-bottom:14px;">SET and SEF displayed separately per CMO 19 Annex D — no combined scor
e.</p>
          <div style="display:grid;grid-template-columns:repeat(auto-fill,minmax(200px,1fr));gap:12px;">
            ${deptEntries.map(([dept, d]) => {
              const avgSET = d.setCount > 0 ? (d.setTotal / d.setCount).toFixed(2) : '—';
              const avgSEF = d.sefCount > 0 ? (d.sefTotal / d.sefCount).toFixed(2) : '—';
              return `<div style="border:1px solid var(--border);border-radius:10px;padding:14px;">
                <div style="font-weight:700;font-size:0.9rem;margin-bottom:8px;">${escapeHtml(dept)}</div>
                <div style="display:flex;justify-content:space-between;margin-bottom:4px;">
                  <span style="font-size:0.72rem;color:var(--muted);">SET Rating</span>
                  <strong style="color:#16a34a;">${avgSET}${avgSET !== '—' ? '%' : ''}</strong>
                </div>
                <div style="display:flex;justify-content:space-between;margin-bottom:4px;">
                  <span style="font-size:0.72rem;color:var(--muted);">SEF Rating</span>
                  <strong style="color:#d97706;">${avgSEF}${avgSEF !== '—' ? '%' : ''}</strong>
                </div>
                <div style="font-size:0.68rem;color:var(--muted);margin-top:6px;">${d.faculty} faculty member${d.faculty !== 1 ? 's' : ''}</div>
              </div>`;
            }).join('')}
          </div>
        </div>
      </div>
    `;
  } else if (deptRatingsEl) {
    deptRatingsEl.innerHTML = '';
  }

  document.getElementById('dashPeriodCard').innerHTML = `
    <h3 style="font-size:1rem;font-weight:700;margin-bottom:16px;">📅 Current Active Period</h3>
    <div class="info-row"><span class="info-label">School Year</span><span class="info-value">${sy ? sy.year : 'Not set'}</span></div>
    <div class="info-row"><span class="info-label">Semester</span><span class="info-value">${sy ? sy.activeSem : 'Not set'}</span></div>
    <div class="info-row"><span class="info-label">Evaluation Status</span><span class="info-value"><span class="badge ${!period.open ? 'badge-dange
r' : (period.startDate && new Date().toLocaleDateString('en-CA') < period.startDate) ? 'badge-warning' : 'badge-success'}">${!period.open ? 'Closed'
: (period.startDate && new Date().toLocaleDateString('en-CA') < period.startDate) ? 'Scheduled' : 'Open'}</span></span></div>
    <div class="info-row"><span class="info-label">Start Date</span><span class="info-value">${period.startDate || 'Not set'}</span></div>
    <div class="info-row"><span class="info-label">Deadline</span><span class="info-value">${period.deadline || 'Not set'}</span></div>
  `;
}

function renderSchoolYear() {
  const syl = getData('schoolYears', []);
  const el = document.getElementById('syList');
  if (!syl.length) { el.innerHTML = '<div class="empty-state"><p>No school years added yet.</p></div>'; return; }
  el.innerHTML = syl.map(sy => `
    <div class="card" style="margin-bottom:16px;">
      <div class="card-header-bar">
        <div style="display:flex;align-items:center;gap:10px;">
          <svg width="16" height="16" fill="none" stroke="var(--primary)" stroke-width="2" viewBox="0 0 24 24"><rect x="3" y="4" width="18" height="1
8" rx="2"/><path d="M16 2v4M8 2v4M3 10h18"/></svg>
          <div><div style="font-weight:700;">${sy.year}</div><div style="font-size:0.72rem;color:var(--muted);">${sy.semesters.length} semester(s)</d
iv></div>
        </div>
        <div style="display:flex;gap:8px;">
          <button class="btn btn-ghost btn-sm" onclick="addSemester('${sy.id}','1st Semester')">+ 1st Sem</button>
          <button class="btn btn-ghost btn-sm" onclick="addSemester('${sy.id}','2nd Semester')">+ 2nd Sem</button>
        </div>
      </div>
      <div class="card-body">
        ${sy.semesters.map(sem => `
          <div class="sem-card ${sem.active ? 'active-sem' : ''}">
            <div style="display:flex;align-items:center;gap:10px;">
              <div><div style="font-weight:600;font-size:0.85rem;">${sem.label}</div><div style="font-size:0.72rem;color:var(--muted);">${sem.active
? 'Active' : 'Inactive'}</div></div>
            </div>
            ${!sem.active ? `<button class="btn btn-success btn-sm" onclick="setActiveSem('${sy.id}','${sem.id}')">Set Active</button>` : '<span clas
s="badge badge-success">✓ Active</span>'}
          </div>
        `).join('')}
        ${!sy.semesters.length ? '<p style="color:var(--muted);font-size:0.82rem;text-align:center;padding:12px;">No semesters added yet.</p>' : ''}
      </div>
    </div>
  `).join('');
}

function addSchoolYear() {
  const year = document.getElementById('syYearInput').value.trim();
  if (!year) { showToast('Please enter a school year.', 'error'); return; }
  const syl = getData('schoolYears', []);
  syl.push({ id: 'sy' + Date.now(), year, semesters: [] });
  setData('schoolYears', syl);
  closeModal('addSYModal');
  document.getElementById('syYearInput').value = '';
  renderSchoolYear();
  addAudit('Add School Year', `Added: ${year}`);
  showToast('School year added!', 'success');
}

function addSemester(syId, label) {
  const syl = getData('schoolYears', []);
  const sy = syl.find(s => s.id === syId);
  if (sy.semesters.find(s => s.label === label)) { showToast('Semester already exists.', 'error'); return; }
  sy.semesters.push({ id: 'sem' + Date.now(), label, active: false });
```

```
  setData('schoolYears', syl);
  renderSchoolYear();
  addAudit('Add Semester', `Added ${label} to ${sy.year}`);
  showToast('Semester added!', 'success');
}

function setActiveSem(syId, semId) {
  const syl = getData('schoolYears', []);
  syl.forEach(sy => sy.semesters.forEach(s => s.active = false));
  const sy = syl.find(s => s.id === syId);
  if (sy) { const sem = sy.semesters.find(s => s.id === semId); if (sem) sem.active = true; }
  setData('schoolYears', syl);
  renderSchoolYear();
  addAudit('Set Active Semester', `Activated semester`);
  showToast('Active semester updated!', 'success');
}
```

admin/js/core/admin-router.js

34 lines

```
// admin-router.js - keeps the current page in the URL so reload and Back work.
// MUST load last: it wraps the final showPage.

(function () {
  const KNOWN = ['dashboard', 'schoolYear', 'students', 'teachers', 'subjects',
                 'evalControl', 'reports', 'auditLog', 'supervisor', 'feedback',
                 'dept', 'deptManage'];

  let applyingFromHash = false;   // stops showPage -> hashchange -> showPage loops

  const inner = window.showPage;
  window.showPage = function (page) {
    inner(page);
    if (!applyingFromHash && KNOWN.indexOf(page) !== -1) {
      try { history.replaceState(null, '', '#' + page); } catch (e) { }
    }
  };

  function pageFromHash() {
    const h = (location.hash || '').replace('#', '').trim();
    return KNOWN.indexOf(h) !== -1 ? h : null;
  }

  window.addEventListener('load', function () {
    const p = pageFromHash();
    if (p && p !== 'dashboard') setTimeout(() => window.showPage(p), 0);
  });

  window.addEventListener('hashchange', function () {
    const p = pageFromHash();
    if (!p) return;
    applyingFromHash = true;
    try { window.showPage(p); } finally { applyingFromHash = false; }
  });
})();
```

admin/js/core/admin-scoring.js

394 lines

```
// admin-scoring.js - all CMO 19 rating computation.

// Institutional bands, not defined in CMO 19.
window.getRemarks = function(percentage) {
    if (percentage >= 90) return 'Outstanding';
    if (percentage >= 75) return 'Very Satisfactory';
    if (percentage >= 60) return 'Satisfactory';
    if (percentage >= 50) return 'Fair';
    return 'Unsatisfactory';
};

window.getRemarksColor = function(percentage) {
    if (percentage >= 90) return '#16a34a';
    if (percentage >= 75) return '#2563eb';
    if (percentage >= 60) return '#d97706';
    if (percentage >= 50) return '#ea580c';
    return '#dc2626';
};

window.getActiveSY = function() {
    const syl = getData('schoolYears', []);
    for (const sy of syl) {
        const activeSem = sy.semesters.find(s => s.active);
        if (activeSem) {
            return { year: sy.year, activeSem: activeSem.label };
        }
    }
    return null;
};

// Annex C column (3): 'respondents' (who actually submitted) or 'enrolled'
// (class roll). 'respondents' assumes nothing about non-respondents.
const SET_WEIGHT_BY = 'respondents';

// Year/Section label, e.g. BSIT 4A.
function _sectionLabel(st) {
    if (!st) return '\u2014';
    const course = (typeof courseShorthand === 'function')
        ? courseShorthand(st.course) : String(st.course || '');
    const yr  = String(st.year || '').match(/\d+/);
    const sec = String(st.section || '').trim().toUpperCase();
    return (course + ' ' + (yr ? yr[0] : '') + sec).trim() || '\u2014';
}

// One Annex C row per section. Rows always sum to the class total.
// MULTIPLE TEACHERS PER SUBJECT: sub.teachers = [{ id, sections }].
// With sections, a teacher is rated only by those sections; with none,
// by every student. Legacy `teacherId` is treated as one teacher for all
// and kept in step with the first teacher on save.
window.subjectTeachers = function (sub) {
    // Deleted teachers are dropped here, for every screen and report.
    let live = null;
    try {
        if (typeof getData === 'function') {
            live = new Set(getData('teachers', []).filter(t => !t.deleted).map(t => t.id));
        }
    } catch (e) { live = null; }          // can't tell? then keep everyone
    const alive = id => !live || live.has(id);

    if (sub && Array.isArray(sub.teachers) && sub.teachers.length) {
        const seen = new Set();
        const list = sub.teachers
            .filter(t => t && t.id && alive(t.id) && !seen.has(t.id) && seen.add(t.id))
            .map(t => ({ id: t.id, sections: Array.isArray(t.sections) ? t.sections.filter(Boolean) : [] }));
        if (list.length) return list;
        // All listed teachers gone: fall back to teacherId.
    }
    return (sub && sub.teacherId && alive(sub.teacherId)) ? [{ id: sub.teacherId, sections: [] }] : [];
};

window.subjectTeacherIds = function (sub) {
    return subjectTeachers(sub).map(t => t.id);
};

// Is this faculty member one of the subject's teachers?
window.subjectHasTeacher = function (sub, teacherId) {
    return !!teacherId && subjectTeacherIds(sub).indexOf(teacherId) !== -1;
};

// Every teacher's name for a subject, for tables and lists.
window.subjectTeacherNames = function (sub, teachers) {
    const all = teachers || getData('teachers', []);
    return subjectTeacherIds(sub)
        .map(id => (all.find(t => t.id === id) || {}).name)
        .filter(Boolean);
};

// Teacher an evaluation belongs to; old ratings without one go to the
// FIRST teacher only, so they're never counted twice.
window.evalBelongsTo = function (e, sub, teacherId) {
    if (e.teacherId) return e.teacherId === teacherId;
    return !!sub && (subjectTeacherIds(sub)[0] === teacherId);
};

// Does this teacher teach this student in this subject?
window.teacherTeachesStudent = function (sub, teacherId, student) {
    const t = subjectTeachers(sub).find(x => x.id === teacherId);
    if (!t) return false;
    if (!t.sections.length) return true;                 // teaches everyone
    return !!student && t.sections.indexOf(_sectionLabel(student)) !== -1;
```

```
};

// Enrolled students no teacher covers once sections are split.
// `teachers` = [{ id, sections }]. Returns { keep: [ids], removed: [students] }.
window.splitUncoveredStudents = function (sub, teachers) {
    const ids = (sub && sub.enrolledIds) || [];
    const list = Array.isArray(teachers) ? teachers : [];
    if (!list.length || list.some(t => !t.sections || !t.sections.length)) {
        return { keep: ids.slice(), removed: [] };
    }
    const covered = new Set();
    list.forEach(t => t.sections.forEach(l => covered.add(l)));

    const byId = {};
    getData('students', []).forEach(s => { byId[s.id] = s; });

    const keep = [], removed = [];
    ids.forEach(id => {
        const st = byId[id];
        // Unknown/deleted records are left alone - not ours to judge here.
        if (!st || st.deleted || covered.has(_sectionLabel(st))) keep.push(id);
        else removed.push(st);
    });
    return { keep, removed };
};

function _setSectionRows(sub, classEvals, students) {
    const byId = {};
    students.forEach(s => { byId[s.id] = s; });
    const byEnrolled = (SET_WEIGHT_BY === 'enrolled');

    const scores = {};
    classEvals.forEach(e => {
        // Skip evaluations whose student is deleted or left this class.
        const st = byId[e.studentId];
        if (!st) return;
        const label = _sectionLabel(st);
        (scores[label] = scores[label] || []).push(parseFloat(e.totalScore) || 0);
    });

    const roll = {};
    (sub.enrolledIds || []).forEach(id => {
        const st = byId[id];
        if (!st) return;
        if (typeof isStudentExempted === 'function' && isStudentExempted(id, sub.id)) return;  // §8.2
        const label = _sectionLabel(st);
        roll[label] = (roll[label] || 0) + 1;
    });

    // One row per section on the roll, plus any section that answered but
    // is no longer on it.
    const labels = Array.from(new Set(
        Object.keys(roll).concat(Object.keys(scores))
    )).sort();

    if (!labels.length) {
        return [{ yearSection: '\u2014', count: 0, avgScore: '0.00',
                  weightedScore: '0.00', rated: false }];
    }

    return labels.map(label => {
        const given = scores[label] || [];
        // formula sa pag rate (SET): section average = sum of student scores / no. of responses
        const avg   = given.length ? given.reduce((a, b) => a + b, 0) / given.length : 0;
        const count = byEnrolled ? (roll[label] || 0) : given.length;
        return {
            yearSection:   label,
            count:         count,
            avgScore:      avg.toFixed(2),
            // Unanswered sections are listed but add nothing to the TOTAL.
            // formula sa pag rate (SET): section weighted score = average x no. of students
            weightedScore: given.length ? (avg * count).toFixed(2) : '0.00',
            rated:         given.length > 0
        };
    });
}

// CMO 19 8.3 - the single weighted-SET computation used by every report.
// Skips unrated classes; subtracts exempted students (8.2).
// formula sa pag rate (SET): overall SET = total weighted score / total students (max 100)
window.computeWeightedSET = function(facultyId, termFilter) {
    const inTerm = (typeof termFilter === 'function') ? termFilter
                 : ((typeof annexEvalInTerm === 'function') ? annexEvalInTerm : () => true);

    const students = getData('students', []).filter(s => !s.deleted);
    const byStudent = {};
    students.forEach(s => { byStudent[s.id] = s; });
    const subjects = getData('subjects', []).filter(s =>
        subjectHasTeacher(s, facultyId) &&  // any of the subject's teachers
        s.loadType !== 'Overload' &&        // §4.3 — overload excluded
        !s.isLabSchool                      // §8.5 — lab school excluded
    );

    const evals = getData('evaluations', []).filter(e =>
        e.evaluatorType !== 'supervisor' &&
        inTerm(e)
    );

    const classes = subjects.map((fullSub, idx) => {
        // Only the students THIS teacher teaches in the subject.
        const sub = Object.assign({}, fullSub, {
            enrolledIds: (fullSub.enrolledIds || []).filter(id =>
                teacherTeachesStudent(fullSub, facultyId, byStudent[id]))
        });
        const classEvals = evals.filter(e =>
```

```
            e.subjectId === fullSub.id && evalBelongsTo(e, fullSub, facultyId));
        const enrolledCount = (sub.enrolledIds || [])
            .filter(id => students.some(s => s.id === id))
            .filter(id => (typeof isStudentExempted === 'function')
                          ? !isStudentExempted(id, sub.id) : true)
            .length;
        const avgScore = classEvals.length
            ? classEvals.reduce((a, b) => a + (parseFloat(b.totalScore) || 0), 0) / classEvals.length
            : 0;

        const sections = _setSectionRows(sub, classEvals, students);

        // Only sections with responses are weighted.
        const weight   = sections.reduce((n, r) => n + (r.rated ? r.count : 0), 0);
        const weighted = sections.reduce((n, r) => n + (r.rated ? parseFloat(r.weightedScore) : 0), 0);

        return {
            seq:           idx + 1,
            sub,
            subjectId:     sub.id,
            subjectCode:   sub.code,
            subjectName:   sub.name,
            enrolledCount,                      // roll size, for the response rate
            evalCount:     classEvals.length,   // how many actually responded
            weight,                             // what column (3) prints
            rated:         classEvals.length > 0,
            avgScore:      avgScore.toFixed(2),
            percentage:    Math.min(100, avgScore).toFixed(2),
            weightedScore: weighted.toFixed(2),
            sections:      sections
        };
    });

    const rated         = classes.filter(c => c.rated);
    const totalStudents = rated.reduce((n, c) => n + c.weight, 0);
    const totalWeighted = rated.reduce((n, c) => n + parseFloat(c.weightedScore), 0);

    return {
        classes, rated, totalStudents, totalWeighted,
        overallSET: totalStudents > 0
            ? Math.min(100, totalWeighted / totalStudents).toFixed(2)
            : '0.00'
    };
};

// Wrapper kept for existing call sites.
window.calculateWeightedSETRating = function(facultyId, termFilter) {
    const r = computeWeightedSET(facultyId, termFilter);
    return r.totalStudents > 0 ? r.overallSET : 0;
};

window.printField = function(value, minWidth, placeholder) {
    const v = escapeHtml(value || '');
    return '<span contenteditable="true" spellcheck="false" data-print-field="1" '
      + 'data-placeholder="' + escapeHtml(placeholder || '') + '" '
      + 'style="display:inline-block;min-width:' + (minWidth || '60%')
      + ';padding:0 4px;outline:none;border-bottom:1px solid #333;'
      + 'line-height:1.4;min-height:1.15em;vertical-align:bottom;">'
      + (v || ' ') + '</span>';
};

window.printDate = function(idSuffix) {
    const today = new Date().toISOString().slice(0, 10);
    return '<input type="date" id="printDate_' + idSuffix + '" value="' + today + '" '
      + 'style="border:none;border-bottom:1px solid #333;font-family:inherit;'
      + 'font-size:0.75rem;padding:1px 3px;background:transparent;outline:none;"/>';
};

window.getSignatories = function() {
    const d = getData('reportSignatories', {});
    return {
        preparedName: d.preparedName || '',
        preparedRole: d.preparedRole || '',
        reviewedName: d.reviewedName || '',
        reviewedRole: d.reviewedRole || '',
        // "Approved by" on the Faculty Performance / Supervisors PDF lists.
        approvedName: d.approvedName || '',
        approvedRole: d.approvedRole || ''
    };
};

window.openSignatoriesModal = function() {
    const sig = getSignatories();
    let ov = document.getElementById('sigModal');
    if (ov) ov.remove();
    ov = document.createElement('div');
    ov.id = 'sigModal';
    ov.className = 'modal-overlay open';
    ov.innerHTML = `
      <div class="modal" style="max-width:520px;">
        <div class="modal-header">
          <h2 class="modal-title">Report Signatories</h2>
          <button class="modal-close" onclick="closeModal('sigModal')">&#10005;</button>
        </div>
        <div class="modal-body" style="padding:20px;">
          <p style="font-size:0.78rem;color:var(--muted);margin:0 0 16px;line-height:1.5;">
            Filled in automatically on Annex C, Annex D and the FER, and "Prepared by" /
            "Approved by" on the Faculty Performance list. You can still type over them on the
            Annex and FER pages before printing without changing what is saved here.
          </p>
          <div class="form-group"><label class="form-label">Prepared by &mdash; Name of Staff</label>
            <input class="form-control" id="sigPrepName" value="${escapeHtml(sig.preparedName)}" placeholder="Name of staff"/></div>
          <div class="form-group"><label class="form-label">Prepared by &mdash; Position</label>
            <input class="form-control" id="sigPrepRole" value="${escapeHtml(sig.preparedRole)}" placeholder="Admin Officer II / QA I"/></div>
          <div class="form-group"><label class="form-label">Reviewed by &mdash; Authorized Official</label>
```

```
            <input class="form-control" id="sigRevName" value="${escapeHtml(sig.reviewedName)}" placeholder="Name of official"/></div>
          <div class="form-group"><label class="form-label">Reviewed by &mdash; Position</label>
            <input class="form-control" id="sigRevRole" value="${escapeHtml(sig.reviewedRole)}" placeholder="Admin Officer V / QA Director"/></div>
          <div class="form-group"><label class="form-label">Approved by &mdash; Name</label>
            <input class="form-control" id="sigAppName" value="${escapeHtml(sig.approvedName)}" placeholder="Name of approving official"/></div>
          <div class="form-group"><label class="form-label">Approved by &mdash; Position</label>
            <input class="form-control" id="sigAppRole" value="${escapeHtml(sig.approvedRole)}" placeholder="e.g. Vice President for Academic Affair
s"/></div>
        </div>
        <div class="modal-footer" style="padding:14px 20px;display:flex;gap:8px;justify-content:flex-end;">
          <button class="btn btn-ghost" onclick="closeModal('sigModal')">Cancel</button>
          <button class="btn btn-primary" onclick="saveSignatories()">Save</button>
        </div>
      </div>`;
    ov.addEventListener('click', e => { if (e.target === ov) closeModal('sigModal'); });
    document.body.appendChild(ov);
};

window.saveSignatories = function() {
    const v = id => (document.getElementById(id) || {}).value.trim() || '';
    setData('reportSignatories', {
        preparedName: v('sigPrepName'), preparedRole: v('sigPrepRole'),
        reviewedName: v('sigRevName'),  reviewedRole: v('sigRevRole'),
        approvedName: v('sigAppName'),  approvedRole: v('sigAppRole')
    });
    addAudit('Update Signatories', 'Report signatories updated');
    closeModal('sigModal');
    showToast('Signatories saved. They will appear on all printed reports.', 'success');
};

window.sigDate = function(which) {
    return '<div style="font-size:0.75rem;">Date: ' + printDate(which) + '</div>';
};

window.sigLine = function(label, value, width) {
    return '<div style="margin-bottom:6px;font-size:0.75rem;">' + label + ': '
      + printField(value, width, label) + '</div>';
};

// CMO 19 9.3 - mean of the supervisors' ratings.
// formula sa pag rate (SEF): overall SEF = sum of supervisor ratings / no. of supervisors
window.getSEFForTeacher = function(teacherId, termFilter) {
    const list = getData('evaluations', [])
        .filter(e => e.teacherId === teacherId && e.evaluatorType === 'supervisor')
        .filter(e => typeof termFilter === 'function' ? termFilter(e) : true);

    if (!list.length) return { count: 0, average: null, list: [] };

    const sum = list.reduce((a, e) => a + (parseFloat(e.totalScore) || 0), 0);
    return { count: list.length, average: sum / list.length, list };
};

// INSTITUTIONAL 60/40 composite. NOT a CMO 19 formula.
// The printed annexes report SET and SEF separately.
window.calculateFinalRating = function(teacherId, termFilter) {
    const inTerm = (typeof termFilter === 'function') ? termFilter
                 : ((typeof annexEvalInTerm === 'function') ? annexEvalInTerm : () => true);
    const studentPercentage = parseFloat(calculateWeightedSETRating(teacherId, inTerm));
    const sefAgg = getSEFForTeacher(teacherId, inTerm);
    const sefData = sefAgg.list;

    const hasSET = studentPercentage > 0;
    const hasSEF = sefData.length > 0;
    const supervisorScore = hasSEF ? sefAgg.average : null;   // mean of ALL supervisors

    // formula sa pag rate (SET + SEF): final = SET x 60% + SEF x 40%
    const finalPercentage = (hasSET && hasSEF)
        ? Math.min(100, (studentPercentage * 0.60) + (supervisorScore * 0.40))
        : null;

    return {
        hasSET,
        hasSEF,
        weightedSET:          hasSET ? studentPercentage.toFixed(2) : '0.00',
        studentPercentage:    hasSET ? studentPercentage.toFixed(2) : '—',
        supervisorPercentage: hasSEF ? supervisorScore.toFixed(2)   : '—',
        finalPercentage:      finalPercentage !== null ? finalPercentage.toFixed(2) : '—',
        remarks:              finalPercentage !== null ? getRemarks(finalPercentage)
                                : (hasSET ? 'Awaiting SEF' : 'No evaluations yet'),
        remarksColor:         finalPercentage !== null ? getRemarksColor(finalPercentage) : '#64748b'
    };
};

setInterval(function () {
    if (typeof window.checkAndAutoFinalize === 'function') window.checkAndAutoFinalize();
}, 3600000);
```

admin/js/modules/adminEval.js

523 lines

```
// ===== ENHANCED EVALUATION FEATURES (CMO 19 Compliant) =====

// ---- Rating-period scoping (CMO §4.3) ----
// Counts evaluations tagged with the active school year + semester; untagged
// (legacy) ones are always included. setReportTerm() can pick a past term or
// "ALL"; null = follow the active term.
window._selectedTerm = null; // null = active term; 'ALL' = every term; or { year, sem }

// 'ACTIVE' forces the active term (Annex opened outside Reports);
// null follows the Reports selection.
window._annexTermOverride = null;

// Core matcher: does `ev` belong to `selected` (null=active, 'ALL'=any, or {year,sem})?
function _evalMatchesTerm(ev, selected) {
    if (!ev) return false;
    if (selected === 'ALL') return true;
    let term = selected;
    if (!term) {
        const sy = (typeof getActiveSY === 'function') ? getActiveSY() : null;
        term = sy ? { year: sy.year, sem: sy.activeSem } : null;
    }
    if (!term) return true;
    // Untagged legacy records only appear when viewing the ACTIVE term.
    if (!ev.schoolYear && !ev.semester) return selected === null;
    // Match only the fields the record has, so half-tagged SEF records still count.
    const yearOk = !term.year || !ev.schoolYear || ev.schoolYear === term.year;
    const semOk  = !term.sem  || !ev.semester   || ev.semester   === term.sem;
    return yearOk && semOk;
}

window.evalInActiveTerm = function(ev) {
    return _evalMatchesTerm(ev, window._selectedTerm);
};

// Annex C/D term matcher: Reports selection only when opened from Reports.
window.annexEvalInTerm = function(ev) {
    const sel = (window._annexTermOverride === 'ACTIVE') ? null : window._selectedTerm;
    return _evalMatchesTerm(ev, sel);
};

// Every term with evaluation data, for the Reports history dropdown.
window.listEvaluationTerms = function() {
    const terms = new Map(); // key -> { year, sem, label }
    const add = (year, sem) => {
        if (!year && !sem) return;
        const key = `${year || ''}|${sem || ''}`;
        if (!terms.has(key)) terms.set(key, { key, year: year || '', sem: sem || '', label: `${year || '—'} · ${sem || '—'}` });
    };
    // From defined school years / semesters
    (getData('schoolYears', []) || []).forEach(sy =>
        (sy.semesters || []).forEach(s => add(sy.year, s.label)));
    // From any evaluations actually submitted
    (getData('evaluations', []) || []).forEach(e => { if (e.schoolYear || e.semester) add(e.schoolYear, e.semester); });
    return [...terms.values()];
};

// -- Reports header "Actions" menu --
// .open goes on the wrapper so the trigger can style itself too.
window.toggleReportsMenu = function(e) {
    // Stop the document listener from closing it on the same click.
    if (e) e.stopPropagation();
    const wrap = document.getElementById('reportsActions');
    if (!wrap) return;
    const btn = document.getElementById('reportsActionsBtn');
    const isOpen = wrap.classList.toggle('open');
    if (btn) btn.setAttribute('aria-expanded', isOpen ? 'true' : 'false');
};

window.closeReportsMenu = function() {
    const wrap = document.getElementById('reportsActions');
    if (!wrap) return;
    wrap.classList.remove('open');
    const btn = document.getElementById('reportsActionsBtn');
    if (btn) btn.setAttribute('aria-expanded', 'false');
};

// Click outside closes the menu; items close it themselves.
document.addEventListener('click', function(e) {
    const wrap = document.getElementById('reportsActions');
    if (!wrap || !wrap.classList.contains('open')) return;
    if (!wrap.contains(e.target)) closeReportsMenu();
});

document.addEventListener('keydown', function(e) {
    if (e.key !== 'Escape') return;
    const wrap = document.getElementById('reportsActions');
    if (!wrap || !wrap.classList.contains('open')) return;
    closeReportsMenu();
    const btn = document.getElementById('reportsActionsBtn');
    if (btn) btn.focus();   // Esc should not strand keyboard focus in a hidden panel.
});

// Fill / refresh the Reports term dropdown and keep the current choice selected.
window._refreshReportTermLabel = function() {
    const sel = document.getElementById('reportTermSelect');
    if (!sel) return;
    const sy = (typeof getActiveSY === 'function') ? getActiveSY() : null;
    const activeLabel = sy ? `Active term (${sy.year} · ${sy.activeSem})` : 'Active term';
    const terms = listEvaluationTerms();

```

```
    let cur = 'ACTIVE';
    if (window._selectedTerm === 'ALL') cur = 'ALL';
    else if (window._selectedTerm) cur = `${window._selectedTerm.year}|${window._selectedTerm.sem}`;

    sel.innerHTML =
        `<option value="ACTIVE">${activeLabel}</option>` +
        `<option value="ALL">All terms (full history)</option>` +
        terms.map(t => `<option value="${t.key}">${t.label}</option>`).join('');
    sel.value = cur;
};

// Set the Reports term: '' / 'ACTIVE' = active, 'ALL' = all, or "year|sem".
window.setReportTerm = function(value) {
    if (!value || value === 'ACTIVE') {
        window._selectedTerm = null;
    } else if (value === 'ALL') {
        window._selectedTerm = 'ALL';
    } else {
        const [year, sem] = value.split('|');
        window._selectedTerm = { year, sem };
    }

    // Human label for the loading message.
    let label;
    if (window._selectedTerm === 'ALL') {
        label = 'All terms (full history)';
    } else if (window._selectedTerm) {
        label = `${window._selectedTerm.year} · ${window._selectedTerm.sem}`;
    } else {
        const sy = (typeof getActiveSY === 'function') ? getActiveSY() : null;
        label = sy ? `${sy.year} · ${sy.activeSem}` : 'the active term';
    }

    // Show a brief loading state so the admin sees the history being loaded.
    const content = document.getElementById('reportsContent');
    if (content) {
        content.innerHTML = `
            <div class="reports-loading">
                <div class="reports-spinner"></div>
                <div class="reports-loading-text">Loading ${label}…</div>
                <div class="reports-loading-sub">Gathering evaluation records for this period</div>
            </div>`;
    }
    if (typeof _refreshReportTermLabel === 'function') _refreshReportTermLabel();

    // Render after a short delay so the spinner is perceptible.
    setTimeout(() => {
        if (typeof renderReports === 'function') renderReports();
    }, 550);
};

// Term the Reports page is showing; label is "Semester / Year".
window.getReportTermInfo = function() {
    const sel = window._selectedTerm;
    const sy = (typeof getActiveSY === 'function') ? getActiveSY() : null;
    if (sel === 'ALL') {
        return { year: '', sem: '', label: 'All terms (full history)', isHistory: true, all: true };
    }
    if (sel) {
        return { year: sel.year || '', sem: sel.sem || '', label: `${sel.sem || '—'} / ${sel.year || '—'}`, isHistory: true, all: false };
    }
    return { year: sy ? sy.year : '', sem: sy ? sy.activeSem : '', label: sy ? `${sy.activeSem} / ${sy.year}` : '—', isHistory: false, all: false };
};

// Annex term: active term unless opened from Reports.
window.annexTermInfo = function() {
    if (window._annexTermOverride === 'ACTIVE') {
        const sy = (typeof getActiveSY === 'function') ? getActiveSY() : null;
        return { year: sy ? sy.year : '', sem: sy ? sy.activeSem : '', label: sy ? `${sy.activeSem} / ${sy.year}` : '—', isHistory: false, all: false
};
    }
    return getReportTermInfo();
};

// ---- Evaluation exemptions (CMO §8.2) ----
// Stored as { studentId, subjectId }.
window.isStudentExempted = function(studentId, subjectId) {
    return getData('exemptions', []).some(x =>
        x.studentId === studentId && x.subjectId === subjectId);
};

window.setStudentExemption = function(studentId, subjectId, exempt, reason) {
    const ex = getData('exemptions', []);
    const idx = ex.findIndex(x => x.studentId === studentId && x.subjectId === subjectId);
    if (exempt) {
        if (idx === -1) ex.push({ studentId, subjectId, reason: reason || '', at: new Date().toISOString() });
    } else if (idx > -1) {
        ex.splice(idx, 1);
    }
    setData('exemptions', ex);
    if (typeof addAudit === 'function') {
        addAudit('Evaluation Exemption', `${exempt ? 'Granted' : 'Removed'} exemption (student ${studentId}, subject ${subjectId})`);
    }
    return ex;
};

// Students expected to evaluate = enrolled minus exempted (CMO §8.2).
window.getExpectedEvaluatorCount = function(subject) {
    const enrolled = (subject && subject.enrolledIds) ? subject.enrolledIds : [];
    return enrolled.filter(id => !isStudentExempted(id, subject.id)).length;
};

// Verify student can evaluate a subject (only officially enrolled)
window.canStudentEvaluate = function(studentId, subjectId) {
    const student = getData('students', []).find(s => s.id === studentId);
```

```
    const subject = getData('subjects', []).find(s => s.id === subjectId);

    if (!student || student.status !== 'active' || student.deleted) {
        return { allowed: false, reason: 'Student account is not active.' };
    }

    const isEnrolled = subject && subject.enrolledIds && subject.enrolledIds.includes(studentId);
    if (!isEnrolled) {
        return { allowed: false, reason: 'You are not officially enrolled in this subject.' };
    }

    // CMO §8.2 — exempted students are not required to evaluate.
    if (isStudentExempted(studentId, subjectId)) {
        return { allowed: false, exempted: true, reason: 'You have been exempted from evaluating this subject.' };
    }

    const period = getData('evalPeriod', {});
    if (!period.open) {
        return { allowed: false, reason: 'Evaluation period is currently closed.' };
    }
    if (period.startDate && new Date().toLocaleDateString('en-CA') < period.startDate) {
        return { allowed: false, reason: 'Evaluation starts on ' + period.startDate + '.' };
    }

    const existing = getData('evaluations', []).find(e =>
        e.studentId === studentId && e.subjectId === subjectId
    );
    if (existing) {
        return { allowed: false, reason: 'You have already evaluated this subject.' };
    }

    return { allowed: true, reason: '' };
};

// Get teacher's SET and SEF ratings separately (CMO 19 — no combined score)
window.getTeacherOverallRating = function(teacherId) {
    // §8.3 maths lives in computeWeightedSET (admin-scoring.js).
    const agg          = computeWeightedSET(teacherId, evalInActiveTerm);
    const classRatings = agg.classes;          // same field names the CSV export reads
    const overallSET   = agg.overallSET;
    const subjects     = getData('subjects', []).filter(s =>
        subjectHasTeacher(s, teacherId) && s.loadType !== 'Overload' && !s.isLabSchool);
    const evalCount    = agg.classes.reduce((n, c) => n + c.evalCount, 0);

    // SEF (0-100): AVERAGE of all supervisor ratings this term (CMO §9.3, §4.3).
    const supervisorEvals = getData('evaluations', []).filter(e =>
        e.teacherId === teacherId && e.evaluatorType === 'supervisor'
        && evalInActiveTerm(e)
    );
    // formula sa pag rate (SEF): sum of supervisor ratings / no. of supervisors
    const sefScore = supervisorEvals.length > 0
        ? (supervisorEvals.reduce((a, b) => a + b.totalScore, 0) / supervisorEvals.length).toFixed(2)
        : '—';

    // NOTE: CMO 19 (Annex D) displays SET and SEF side-by-side — NO combined score
    return {
        teacherId,
        classRatings,
        overallSET,                // Weighted SET percentage
        sefScore,                  // SEF percentage (average of supervisor SEFs, §9.3)
        sefCount: supervisorEvals.length,   // how many chairs rated this faculty
        remarks: getRemarks(parseFloat(overallSET)),
        totalClasses: subjects.length,
        totalEvaluations: evalCount
    };
};

// Export detailed report with class breakdown — SET and SEF as separate columns
window.exportEnhancedReport = function() {
    const teachers = getData('teachers', []).filter(t => !t.deleted);
    let csv = 'Teacher ID,Name,Department,Faculty Type,Class Code,Class Name,Enrolled,Evaluations,Class SET Avg,Class SET %,Overall Weighted SET,SEF
Rating\n';

    teachers.forEach(teacher => {
        const rating = getTeacherOverallRating(teacher.id);
        const facultyType = teacher.facultyType || 'regular';

        if (rating.classRatings.length > 0) {
            rating.classRatings.forEach(cr => {
                csv += `"${teacher.tid}","${teacher.name}","${teacher.dept || 'N/A'}","${facultyType}","${cr.subjectCode}","${cr.subjectName}",${cr.e
nrolledCount},${cr.evalCount},${cr.avgScore},${cr.percentage}%,${rating.overallSET}%,${rating.sefScore !== '—' ? rating.sefScore + '%' : 'N/A'}\n`;
            });
        } else {
            csv += `"${teacher.tid}","${teacher.name}","${teacher.dept || 'N/A'}","${facultyType}",N/A,N/A,0,0,0,0%,${rating.overallSET}%,${rating.se
fScore !== '—' ? rating.sefScore + '%' : 'N/A'}\n`;
        }
    });

    const a = Object.assign(document.createElement('a'), {
        href: URL.createObjectURL(new Blob([csv], { type: 'text/csv' })),
        download: `teacher_eval_detailed_${new Date().toISOString().split('T')[0]}.csv`
    });
    a.click();
    addAudit('Export Enhanced Report', 'Exported detailed CSV with class breakdown (SET & SEF separate)');
    showToast('Enhanced report exported!', 'success');
};

// ===== FACULTY EVALUATION REPORT (FER) — CMO §6.8 =====
// "Prepared by" / "Reviewed by" from Reports > Actions > Signatories;
// blank names print as an empty signing line.
window.preparedReviewedHtml = function(marginTop) {
    const sig = (typeof getSignatories === 'function') ? getSignatories() : {};
    const esc = (typeof escapeHtml === 'function') ? escapeHtml : (s => String(s == null ? '' : s));
    const box = (label, name, role) =>
        '<div style="flex:1;max-width:260px;">'
```

```
      + '<div style="font-size:11px;margin-bottom:30px;">' + label + '</div>'
      + '<div style="border-bottom:1px solid #111;min-height:16px;padding-bottom:2px;text-align:center;'
      +       'font-weight:700;font-size:11px;text-transform:uppercase;">' + (esc(name || '') || ' ') + '</div>'
      + '<div style="text-align:center;font-size:10px;color:#333;margin-top:3px;min-height:13px;">'
      +       (esc(role || '') || ' ') + '</div>'
      + '</div>';
    return '<div style="display:flex;justify-content:space-between;gap:40px;margin-top:' + (marginTop || 40) + 'px;'
         +      'page-break-inside:avoid;break-inside:avoid;">'
         + box('Prepared by:', sig.preparedName, sig.preparedRole)
         + box('Reviewed by:', sig.reviewedName, sig.reviewedRole)
         + '</div>';
};

window.generateFERReport = function() {
    const teachers = getData('teachers', []).filter(t => !t.deleted);
    const subjects = getData('subjects', []);
    const DEPT = (typeof getDepartments === 'function') ? getDepartments() : {};
    const esc = (typeof escapeHtml === 'function') ? escapeHtml : (s => String(s));

    // Use the term the Reports page is showing.
    const sel = window._selectedTerm;
    const sy = (typeof getActiveSY === 'function') ? getActiveSY() : null;
    let termLabel, isHistoryView;
    if (sel === 'ALL') {
        termLabel = 'All terms (full history)';
        isHistoryView = true;
    } else if (sel) {
        termLabel = `${sel.year} · ${sel.sem}`;
        isHistoryView = true;   // a specific PAST term selected
    } else {
        termLabel = sy ? `${sy.year} · ${sy.activeSem}` : 'Active term';
        isHistoryView = false;  // following the active term
    }

    // Per-faculty ratings (SET) for the active term.
    const rows = teachers.map(t => {
        const r = getTeacherOverallRating(t.id);
        const set = parseFloat(r.overallSET) || 0;
        return {
            name: t.name,
            dept: t.dept || '—',
            set,
            sef: r.sefScore,
            evals: r.totalEvaluations,
            hasData: r.totalEvaluations > 0
        };
    });

    const rated = rows.filter(r => r.hasData);
    const n = rated.length;
    const avg = arr => arr.length ? (arr.reduce((a, b) => a + b, 0) / arr.length) : 0;

    // Statistical trends
    const overallAvg = avg(rated.map(r => r.set)).toFixed(2);
    const totalEvals = rows.reduce((a, r) => a + r.evals, 0);

    // Response rate (§8.2-aware); N/A outside the active term.
    let responseRate = '—';
    if (!isHistoryView) {
        const expected = subjects.reduce((a, s) =>
            a + (typeof getExpectedEvaluatorCount === 'function' ? getExpectedEvaluatorCount(s) : (s.enrolledIds || []).length), 0);
        responseRate = expected > 0 ? Math.min(100, (totalEvals / expected) * 100).toFixed(1) : '—';
    }

    // Distribution by remark band
    const bands = {};
    rated.forEach(r => {
        const b = (typeof getRemarks === 'function') ? getRemarks(r.set) : '—';
        bands[b] = (bands[b] || 0) + 1;
    });

    // Per-department averages
    const byDept = {};
    rated.forEach(r => {
        (byDept[r.dept] = byDept[r.dept] || []).push(r.set);
    });
    const deptRows = Object.entries(byDept).map(([d, arr]) => {
        const label = (DEPT[d] && DEPT[d].name) ? DEPT[d].name : d;
        return { dept: label, avg: avg(arr).toFixed(2), count: arr.length };
    }).sort((a, b) => parseFloat(b.avg) - parseFloat(a.avg));

    // Notable patterns — top / bottom 5
    const ranked = [...rated].sort((a, b) => b.set - a.set);
    const top = ranked.slice(0, 5);
    const bottom = ranked.slice(-5).reverse();

    const now = new Date().toLocaleDateString('en-PH', { year: 'numeric', month: 'long', day: 'numeric' });
    const bandRows = Object.entries(bands).map(([b, c]) =>
        `<tr><td>${esc(b)}</td><td style="text-align:center;">${c}</td><td style="text-align:center;">${n ? ((c / n) * 100).toFixed(1) : 0}%</td></tr
>`).join('');
    const deptTable = deptRows.map(d =>
        `<tr><td>${esc(d.dept)}</td><td style="text-align:center;">${d.count}</td><td style="text-align:center;">${d.avg}%</td></tr>`).join('');
    const listRows = arr => arr.map((r, i) =>
        `<tr><td style="text-align:center;">${i + 1}</td><td>${esc(r.name)}</td><td>${esc(r.dept)}</td><td style="text-align:center;">${r.set.toFixed
(2)}%</td></tr>`).join('');

    const html = `<!DOCTYPE html><html><head><meta charset="utf-8"><title>Faculty Evaluation Report (FER)</title>
    <style>
        * { box-sizing: border-box; }
        body { font-family: 'Segoe UI', Arial, sans-serif; color: #1a1a1a; margin: 32px; font-size: 12px; }
        h1 { font-size: 19px; margin: 0 0 2px; }
        h2 { font-size: 14px; margin: 20px 0 8px; border-bottom: 2px solid #059669; padding-bottom: 3px; }
        .sub { color: #555; margin: 0 0 4px; }
        .cards { display: flex; gap: 16px; margin: 14px 0; flex-wrap: wrap; }
        .card { border: 1px solid #ccc; border-radius: 8px; padding: 12px 18px; text-align: center; min-width: 130px; }
```

```
        .card .n { font-size: 22px; font-weight: 800; color: #059669; }
        .card .l { font-size: 10px; color: #555; text-transform: uppercase; letter-spacing: .04em; }
        table { width: 100%; border-collapse: collapse; margin-bottom: 8px; }
        th, td { border: 1px solid #ddd; padding: 5px 8px; font-size: 11px; text-align: left; }
        thead th { background: #eef2f5; }
        .muted { color: #777; font-size: 10px; }
        @media print { body { margin: 12px; } .noprint { display: none; } h2 { page-break-after: avoid; } }
        .noprint { text-align: center; margin-bottom: 16px; }
        .noprint button { padding: 8px 20px; font-size: 13px; border: none; border-radius: 6px; background: #059669; color: #fff; cursor: pointer; }
    </style></head><body>
        <div class="noprint"><button onclick="window.print()">🖨️ Print / Save as PDF</button></div>
        <h1>Faculty Evaluation Report (FER)</h1>
        <p class="sub">Institutional summary for the President / VPAA — CMO No. 19, s. 2025 §6.8</p>
        <p class="sub"><strong>Rating Period:</strong> ${esc(termLabel)}  ·  <strong>Generated:</strong> ${now}</p>
        ${isHistoryView ? `<p class="sub" style="color:#b45309;background:#fffbeb;border:1px solid #fde68a;border-radius:6px;padding:6px 10px;displa
y:inline-block;">Historical view — figures reflect the selected period, not the current term.</p>` : ''}

        <div class="cards">
            <div class="card"><div class="n">${n}</div><div class="l">Faculty w/ ratings</div></div>
            <div class="card"><div class="n">${overallAvg}%</div><div class="l">Avg SET</div></div>
            <div class="card"><div class="n">${totalEvals}</div><div class="l">Evaluations</div></div>
            <div class="card"><div class="n">${responseRate === '—' ? 'N/A' : responseRate + '%'}</div><div class="l">Response rate${isHistoryView ?
' *' : ''}</div></div>
        </div>
        ${isHistoryView ? `<p class="muted">* Response rate is shown only for the active term — enrollment rosters are per-term, so it isn't computed
for past/history views.</p>` : ''}

        <h2>Statistical Trends — Rating Distribution</h2>
        <table><thead><tr><th>Remark</th><th style="text-align:center;">Faculty</th><th style="text-align:center;">Share</th></tr></thead>
        <tbody>${bandRows || '<tr><td colspan="3" class="muted">No rated faculty in this period.</td></tr>'}</tbody></table>

        <h2>Averages by Department / College</h2>
        <table><thead><tr><th>Department / College</th><th style="text-align:center;">Faculty</th><th style="text-align:center;">Avg SET</th></tr></t
head>
        <tbody>${deptTable || '<tr><td colspan="3" class="muted">—</td></tr>'}</tbody></table>

        <h2>Notable Patterns — Highest Rated</h2>
        <table><thead><tr><th style="text-align:center;">#</th><th>Faculty</th><th>Dept</th><th style="text-align:center;">SET</th></tr></thead>
        <tbody>${listRows(top) || '<tr><td colspan="4" class="muted">—</td></tr>'}</tbody></table>

        <h2>Notable Patterns — Needing Support</h2>
        <table><thead><tr><th style="text-align:center;">#</th><th>Faculty</th><th>Dept</th><th style="text-align:center;">SET</th></tr></thead>
        <tbody>${listRows(bottom) || '<tr><td colspan="4" class="muted">—</td></tr>'}</tbody></table>

        <p class="muted" style="margin-top:18px;">This report presents aggregated, de-identified results. Individual student responses remain anonymo
us per CMO §6.10. SET and SEF are reported separately (no combined score) per Annex D.</p>
        ${preparedReviewedHtml()}
    </body></html>`;

    const w = window.open('', '_blank');
    if (!w) { showToast('Please allow pop-ups to view the report.', 'warning'); return; }
    w.document.write(html);
    w.document.close();
    w.focus();
    if (typeof addAudit === 'function') addAudit('Generate FER', `Institutional Faculty Evaluation Report (${termLabel})`);
    showToast('Faculty Evaluation Report generated!', 'success');
};

// ===== FULL SYSTEM BACKUP & RESTORE (CMO §6.9) =====
// Every collection to/from one JSON file.
window._backupKeys = ['students','teachers','subjects','evaluations','schoolYears',
    'auditLog','evalPeriod','customDepartments','customCourses','finalReports',
    'developmentPlans','exemptions'];

window.exportSystemBackup = function() {
    const backup = { _meta: { app: 'Faculty Evaluation System', exportedAt: new Date().toISOString(), version: 1 } };
    window._backupKeys.forEach(k => {
        // Full data, including ratings from deleted people.
        const v = getData(k, null, { includeHidden: true });
        if (v !== null && v !== undefined) backup[k] = v;
    });
    const blob = new Blob([JSON.stringify(backup, null, 2)], { type: 'application/json' });
    const a = document.createElement('a');
    a.href = URL.createObjectURL(blob);
    a.download = `feval-backup-${new Date().toISOString().slice(0,10)}.json`;
    document.body.appendChild(a); a.click(); a.remove();
    URL.revokeObjectURL(a.href);
    if (typeof addAudit === 'function') addAudit('Backup', 'Exported full system backup');
    if (typeof showToast === 'function') showToast('Backup downloaded.', 'success');
};

window.triggerRestore = function() {
    const inp = document.getElementById('restoreFileInput');
    if (inp) inp.click();
};

window.importSystemBackup = function(inputEl) {
    const file = inputEl && inputEl.files && inputEl.files[0];
    if (!file) return;
    const reader = new FileReader();
    reader.onload = e => {
        let data;
        try { data = JSON.parse(e.target.result); } catch (_) {
            if (typeof showToast === 'function') showToast('Invalid backup file (not valid JSON).', 'error');
            inputEl.value = ''; return;
        }
        if (!data || typeof data !== 'object' || !data._meta) {
            if (typeof showToast === 'function') showToast('This does not look like a system backup.', 'error');
            inputEl.value = ''; return;
        }
        const when = data._meta.exportedAt ? new Date(data._meta.exportedAt).toLocaleString('en-PH') : 'unknown date';
        if (!confirm(`Restore backup from ${when}?\n\nThis OVERWRITES the current data on this device and syncs it to the cloud. This cannot be undon
e.`)) {
            inputEl.value = ''; return;
        }
```

```
        let restored = 0;
        Object.keys(data).forEach(k => {
            if (k === '_meta') return;
            if (data[k] !== null && data[k] !== undefined) {
                setData(k, data[k]);
                if (typeof syncCollectionToFirestore === 'function') syncCollectionToFirestore(k, data[k]);
                restored++;
            }
        });
        if (typeof addAudit === 'function') addAudit('Restore', `Imported backup (${restored} collections) from ${when}`);
        if (typeof showToast === 'function') showToast('Backup restored. Reloading…', 'success');
        inputEl.value = '';
        setTimeout(() => location.reload(), 900);
    };
    reader.readAsText(file);
};
```

admin/js/modules/adminQuestions.js

412 lines

```
// adminQuestions.js — Versioned evaluation instrument (SET / SEF questions).
// Published sets are IMMUTABLE: editing makes a draft, publishing creates
// version n+1. Evaluations record their version so old wording stays correct.
// Storage: 'questionSets' -> [{ id, instrument, version, status, questions,
//   publishedAt, retiredAt, note }], synced to Firestore.

// Seeds version 1 only when no set exists for an instrument.
// CMO 19 s.2025 ANNEX A, verbatim (§4.2 forbids modifying indicators),
// including the annex's own punctuation in items 10 and 15.
const SEED_SET_QUESTIONS = [
  { id: 'q1',  sec: 'A', text: 'Comes to class on time.' },
  { id: 'q2',  sec: 'A', text: 'Explains learning outcomes, expectations, grading system, and various requirements of the subject/course.' },
  { id: 'q3',  sec: 'A', text: 'Maximizes the allocated time/learning hours effectively.' },
  { id: 'q4',  sec: 'A', text: 'Facilitates students to think critically and creatively by providing appropriate learning activities.' },
  { id: 'q5',  sec: 'A', text: 'Guides students to learn on their own, reflect on new ideas and experiences, and make decisions in accomplishing give
n tasks.' },
  { id: 'q6',  sec: 'A', text: 'Communicates constructive feedback to students for their academic growth.' },
  { id: 'q7',  sec: 'B', text: 'Demonstrates extensive and broad knowledge of the subject/course.' },
  { id: 'q8',  sec: 'B', text: 'Simplifies complex ideas in the lesson for ease of understanding.' },
  { id: 'q9',  sec: 'B', text: 'Relates the subject matter to contemporary issues and developments in the discipline and/or daily life activities.'
},
  { id: 'q10', sec: 'B', text: 'Promotes active learning and student engagement by using appropriate teaching and learning resources including ICT to
ols and platforms' },
  { id: 'q11', sec: 'B', text: 'Uses appropriate assessments (projects, exams, quizzes, assignments, etc.) aligned with the learning outcomes.' },
  { id: 'q12', sec: 'C', text: 'Recognizes and values the unique diversity and individual differences among students.' },
  { id: 'q13', sec: 'C', text: 'Assists students with their learning challenges during consultation hours.' },
  { id: 'q14', sec: 'C', text: 'Provides immediate feedback on student outputs and performance.' },
  { id: 'q15', sec: 'C', text: 'Provides transparent and clear criteria in rating student\'s performance.' }
];

// CMO 19 s.2025 ANNEX B, verbatim ("Provide" in item 4 is CHED's typo).
const SEED_SEF_QUESTIONS = [
  { id: 'q1',  sec: 'A', text: 'Comes to class on time.' },
  { id: 'q2',  sec: 'A', text: 'Submits updated syllabus, grade sheets, and other required reports on time.' },
  { id: 'q3',  sec: 'A', text: 'Maximizes the allocated time/learning hours effectively.' },
  { id: 'q4',  sec: 'A', text: 'Provide appropriate learning activities that facilitate critical thinking and creativity of students.' },
  { id: 'q5',  sec: 'A', text: 'Guides students to learn on their own, reflect on new ideas and experiences, and make decisions in accomplishing give
n tasks.' },
  { id: 'q6',  sec: 'A', text: 'Communicates constructive feedback to students for their academic growth.' },
  { id: 'q7',  sec: 'B', text: 'Demonstrates extensive and broad knowledge of the subject/course.' },
  { id: 'q8',  sec: 'B', text: 'Simplifies complex ideas in the lesson for ease of understanding.' },
  { id: 'q9',  sec: 'B', text: 'Integrates contemporary issues and developments in the discipline and/or daily life activities in the syllabus.' },
  { id: 'q10', sec: 'B', text: 'Promotes active learning and student engagement by using appropriate teaching and learning resources including ICT to
ols and platforms.' },
  { id: 'q11', sec: 'B', text: 'Uses appropriate assessments (projects, exams, quizzes, assignments, etc.) aligned with the learning outcomes.' },
  { id: 'q12', sec: 'C', text: 'Recognizes and values the unique diversity and individual differences among students.' },
  { id: 'q13', sec: 'C', text: 'Assists students with their learning challenges during consultation hours.' },
  { id: 'q14', sec: 'C', text: 'Provides immediate feedback on student outputs and performance.' },
  { id: 'q15', sec: 'C', text: 'Provides transparent and clear criteria in rating student\'s performance.' }
];

// Verbatim section headings from Annexes A and B.
const SECTION_LABELS = {
  A: 'A — Management of Teaching and Learning',
  B: 'B — Content Knowledge, Pedagogy and Technology',
  C: 'C — Commitment and Transparency'
};

// ---- Store ------------------------------------------------------------

function _allQuestionSets() { return getData('questionSets', []); }

// Seed version 1 once from the shipped instrument.
window.ensureQuestionSets = function () {
  let sets = _allQuestionSets();
  let changed = false;

  [['SET', SEED_SET_QUESTIONS], ['SEF', SEED_SEF_QUESTIONS]].forEach(([instrument, seed]) => {
    if (!sets.some(s => s.instrument === instrument)) {
      sets.push({
        id: instrument.toLowerCase() + '_v1',
        instrument, version: 1, status: 'published',
        questions: seed.map(q => ({ ...q })),
        publishedAt: new Date().toISOString(),
        note: 'Initial version, matching the instrument shipped in the app.'
      });
      changed = true;
    }
  });

  if (changed) setData('questionSets', sets);
  return _allQuestionSets();
};

/** The set the app should serve right now. */
window.getPublishedQuestionSet = function (instrument) {
  ensureQuestionSets();
  return _allQuestionSets()
    .filter(s => s.instrument === instrument && s.status === 'published')
    .sort((a, b) => b.version - a.version)[0] || null;
};

window.getQuestionSetById = function (id) {
  return _allQuestionSets().find(s => s.id === id) || null;
};

/**
* Wording for an answered question, via the version it was answered against.
*/
window.questionTextFor = function (evaluation, questionId) {
```

```
  const instrument = evaluation && evaluation.evaluatorType === 'supervisor' ? 'SEF' : 'SET';
  const set = (evaluation && evaluation.questionSetId)
    ? getQuestionSetById(evaluation.questionSetId)
    : null;
  const fallback = getPublishedQuestionSet(instrument);
  const q = ((set || fallback || {}).questions || []).find(x => x.id === questionId);
  return q ? q.text : questionId;
};

/** Has anything been submitted against this set? If so it must never be edited. */
window.questionSetInUse = function (setId) {
  // Counts every stored answer, hidden or not.
  return getData('evaluations', [], { includeHidden: true }).some(e => e.questionSetId === setId);
};

// ---- Editor ------------------------------------------------------------

let _qsInstrument = 'SET';
let _qsDraft = null;            // working copy; null = showing the published set

window.openQuestionsModal = function (instrument) {
  ensureQuestionSets();
  _qsInstrument = instrument || 'SET';
  _qsDraft = null;
  let ov = document.getElementById('questionsModal');
  if (ov) ov.remove();
  ov = document.createElement('div');
  ov.id = 'questionsModal';
  ov.className = 'modal-overlay open';
  ov.innerHTML = `
    <div class="modal" style="max-width:820px;">
      <div class="modal-header">
        <h2 class="modal-title">Evaluation Questions</h2>
        <button class="modal-close" onclick="closeModal('questionsModal')">&#10005;</button>
      </div>
      <div class="modal-body" id="qsBody" style="padding:18px 20px;max-height:70vh;overflow-y:auto;"></div>
    </div>`;
  ov.addEventListener('click', e => { if (e.target === ov) closeModal('questionsModal'); });
  document.body.appendChild(ov);
  renderQuestionsEditor();
};

window.qsSwitchInstrument = function (instrument) {
  if (_qsDraft && !confirm('Discard the unsaved draft?')) return;
  _qsInstrument = instrument;
  _qsDraft = null;
  renderQuestionsEditor();
};

window.qsStartDraft = function () {
  const pub = getPublishedQuestionSet(_qsInstrument);
  _qsDraft = {
    questions: (pub ? pub.questions : []).map(q => ({ ...q })),
    note: ''
  };
  renderQuestionsEditor();
};

window.qsCancelDraft = function () {
  if (!confirm('Discard this draft? The published version is unchanged.')) return;
  _qsDraft = null;
  renderQuestionsEditor();
};

window.qsEditText = function (idx, value) { if (_qsDraft) _qsDraft.questions[idx].text = value; };
window.qsEditSec  = function (idx, value) { if (_qsDraft) { _qsDraft.questions[idx].sec = value; renderQuestionsEditor(); } };
window.qsEditNote = function (value)      { if (_qsDraft) _qsDraft.note = value; };

window.qsAddQuestion = function () {
  if (!_qsDraft) return;
  // New ids continue the sequence and are never reused.
  const used = new Set(_qsDraft.questions.map(q => q.id));
  let n = _qsDraft.questions.length + 1;
  while (used.has('q' + n)) n++;
  _qsDraft.questions.push({ id: 'q' + n, sec: 'A', text: '' });
  renderQuestionsEditor();
};

window.qsRemoveQuestion = function (idx) {
  if (!_qsDraft) return;
  _qsDraft.questions.splice(idx, 1);
  renderQuestionsEditor();
};

window.qsMove = function (idx, delta) {
  if (!_qsDraft) return;
  const to = idx + delta;
  if (to < 0 || to >= _qsDraft.questions.length) return;
  const [q] = _qsDraft.questions.splice(idx, 1);
  _qsDraft.questions.splice(to, 0, q);
  renderQuestionsEditor();
};

window.qsPublish = function () {
  if (!_qsDraft) return;
  const qs = _qsDraft.questions
    .map(q => ({ id: q.id, sec: q.sec, text: String(q.text || '').trim() }))
    .filter(q => q.text);

  if (!qs.length) { showToast('Add at least one question before publishing.', 'error'); return; }

  const pub = getPublishedQuestionSet(_qsInstrument);
  const nextVersion = pub ? pub.version + 1 : 1;

  if (!confirm(
```

```
      `Publish version ${nextVersion} of the ${_qsInstrument} instrument?\n\n` +
      `${qs.length} question(s). Evaluations submitted from now on use this version.\n` +
      `Everything already submitted stays on version ${pub ? pub.version : '-'} and is not affected.`)) return;

  const sets = _allQuestionSets();
  // Retire rather than delete: old evaluations still resolve their wording here.
  sets.forEach(s => {
    if (s.instrument === _qsInstrument && s.status === 'published') {
      s.status = 'retired';
      s.retiredAt = new Date().toISOString();
    }
  });
  sets.push({
    id: _qsInstrument.toLowerCase() + '_v' + nextVersion,
    instrument: _qsInstrument,
    version: nextVersion,
    status: 'published',
    questions: qs,
    publishedAt: new Date().toISOString(),
    note: String(_qsDraft.note || '').trim()
  });

  setData('questionSets', sets);
  addAudit('Publish Questions', `${_qsInstrument} instrument version ${nextVersion} published (${qs.length} items)`);
  showToast(`${_qsInstrument} version ${nextVersion} published.`, 'success');
  _qsDraft = null;
  renderQuestionsEditor();
};

/**
* Make an existing version the live one (e.g. roll back) without creating a
* new version. Evaluations keep their own questionSetId.
*/
window.qsActivateVersion = function (setId) {
  const sets = _allQuestionSets();
  const target = sets.find(s => s.id === setId);
  if (!target) return;
  if (target.status === 'published') return;   // already live, nothing to do

  const current = sets.find(s => s.instrument === target.instrument && s.status === 'published');
  if (!confirm(
      `Serve version ${target.version} of the ${target.instrument} instrument?\n\n` +
      `${target.questions.length} question(s). The app will show this version from now on.\n` +
      (current ? `Version ${current.version} goes back to retired.\n` : '') +
      `Evaluations already submitted are not affected.`)) return;

  sets.forEach(x => {
    if (x.instrument !== target.instrument) return;
    if (x.status === 'published') { x.status = 'retired'; x.retiredAt = new Date().toISOString(); }
  });
  target.status = 'published';
  target.publishedAt = new Date().toISOString();
  delete target.retiredAt;

  setData('questionSets', sets);
  addAudit('Activate Questions', `${target.instrument} instrument switched to version ${target.version}`);
  showToast(`${target.instrument} now serving version ${target.version}.`, 'success');
  renderQuestionsEditor();
};

/**
* Delete a version outright - refused if any evaluation used it (retire
* instead). Removes it from localStorage AND Firestore.
*/
window.qsDeleteVersion = async function (setId) {
  const sets = _allQuestionSets();
  const target = sets.find(s => s.id === setId);
  if (!target) return;

  if (questionSetInUse(target.id)) {
    alert(`Version ${target.version} cannot be deleted \u2014 evaluations were submitted against it.\n\n` +
          `Those reports resolve their question wording through this version. Use "Use this version" ` +
          `on another version instead; this one stays as history.`);
    return;
  }
  if (target.status === 'published') {
    alert(`Version ${target.version} is the one currently being served.\n\n` +
          `Switch to another version first, then delete this one.`);
    return;
  }
  if (!confirm(`Delete version ${target.version} of the ${target.instrument} instrument?\n\n` +
               `${target.questions.length} question(s). Nothing has been submitted against it. ` +
               `This cannot be undone.`)) return;

  setData('questionSets', sets.filter(s => s.id !== setId));

  try {
    await firebase.firestore().collection('questionSets').doc(setId).delete();
  } catch (e) {
    // localStorage is the source of truth; say so plainly.
    console.warn('Firestore delete failed:', e);
    showToast('Removed here, but the Firestore copy could not be deleted. Check your rules.', 'error');
    renderQuestionsEditor();
    return;
  }

  addAudit('Delete Questions', `${target.instrument} instrument version ${target.version} deleted (unused)`);
  showToast(`Version ${target.version} deleted.`, 'success');
  renderQuestionsEditor();
};

window.renderQuestionsEditor = function () {
  const body = document.getElementById('qsBody');
  if (!body) return;
```

```
  const sets = _allQuestionSets().filter(s => s.instrument === _qsInstrument)
                                 .sort((a, b) => b.version - a.version);
  const pub = sets.find(s => s.status === 'published');
  const editing = !!_qsDraft;
  const questions = editing ? _qsDraft.questions : (pub ? pub.questions : []);

  const tab = (label, value) =>
    `<button type="button" class="student-dept-filter-btn${_qsInstrument === value ? ' active' : ''}"
             onclick="qsSwitchInstrument('${value}')">${label}</button>`;

  const secSelect = (idx, sec) =>
    `<select class="form-control" style="width:auto;font-size:0.72rem;padding:3px 6px;"
             onchange="qsEditSec(${idx}, this.value)">
       ${['A', 'B', 'C'].map(v => `<option value="${v}"${v === sec ? ' selected' : ''}>${v}</option>`).join('')}
     </select>`;

  const rows = questions.map((q, i) => editing
    ? `<div style="display:flex;gap:8px;align-items:flex-start;padding:8px;border-bottom:1px solid var(--border,#e2e8f0);">
         <span style="font-size:0.72rem;color:var(--muted);min-width:34px;padding-top:7px;">${escapeHtml(q.id)}</span>
         ${secSelect(i, q.sec)}
         <textarea rows="2" class="form-control" style="flex:1;font-size:0.8rem;"
                   oninput="qsEditText(${i}, this.value)">${escapeHtml(q.text)}</textarea>
         <div style="display:flex;flex-direction:column;gap:2px;">
           <button class="btn btn-ghost btn-sm" style="padding:1px 6px;" onclick="qsMove(${i},-1)" title="Move up">&#9650;</button>
           <button class="btn btn-ghost btn-sm" style="padding:1px 6px;" onclick="qsMove(${i},1)" title="Move down">&#9660;</button>
         </div>
         <button class="btn btn-ghost btn-sm" style="color:var(--danger,#dc2626);" onclick="qsRemoveQuestion(${i})" title="Remove">&#10005;</button>
       </div>`
    : `<div style="display:flex;gap:10px;padding:7px 8px;border-bottom:1px solid var(--border,#e2e8f0);">
         <span style="font-size:0.72rem;color:var(--muted);min-width:34px;">${escapeHtml(q.id)}</span>
         <span class="badge badge-primary" style="font-size:0.62rem;height:fit-content;">${escapeHtml(q.sec)}</span>
         <span style="font-size:0.82rem;flex:1;">${escapeHtml(q.text)}</span>
       </div>`).join('');

  // Every version, so it's clear which is live.
  const history = sets.map(s => {
    const live = s.status === 'published';
    const used = questionSetInUse(s.id);
    return `<div style="display:flex;align-items:center;gap:10px;padding:6px 8px;border-bottom:1px solid var(--border,#e2e8f0);">
       <span style="font-size:0.78rem;font-weight:${live ? '700' : '500'};min-width:82px;">Version ${s.version}</span>
       ${live
         ? '<span class="badge badge-success" style="font-size:0.62rem;">In use</span>'
         : '<span class="badge" style="font-size:0.62rem;background:var(--surface2,#f1f5f9);color:var(--muted,#64748b);">Retired</span>'}
       <span style="font-size:0.72rem;color:var(--muted);flex:1;">
         ${s.questions.length} items &middot;
         ${live ? 'published' : 'retired'} ${new Date((live ? s.publishedAt : s.retiredAt) || s.publishedAt).toLocaleDateString('en-PH')}
         ${used ? ' &middot; <strong>has submitted evaluations</strong>' : ''}
         ${s.note ? ' &middot; ' + escapeHtml(s.note) : ''}
       </span>
       ${live || editing ? '' :
         `<button class="btn btn-ghost btn-sm" style="font-size:0.72rem;"
                  onclick="qsActivateVersion('${escapeHtml(s.id)}')">Use this version</button>
          ${used ? '' : `<button class="btn btn-ghost btn-sm" style="font-size:0.72rem;color:var(--danger,#dc2626);"
                  onclick="qsDeleteVersion('${escapeHtml(s.id)}')" title="Nothing was submitted against this version">Delete</button>`}`}
     </div>`;
  }).join('');

  body.innerHTML = `
    <div style="display:flex;gap:6px;margin-bottom:14px;">
      ${tab('Student (SET)', 'SET')}${tab('Supervisor (SEF)', 'SEF')}
    </div>

    <div style="display:flex;align-items:center;gap:10px;flex-wrap:wrap;margin-bottom:12px;">
      <strong style="font-size:0.9rem;">
        ${editing ? 'Draft \u2014 version ' + ((pub ? pub.version : 0) + 1) : 'Published version ' + (pub ? pub.version : '\u2014')}
      </strong>
      <span style="font-size:0.74rem;color:var(--muted);">${questions.length} question(s)</span>
      ${editing
        ? `<span style="margin-left:auto;display:flex;gap:8px;">
             <button class="btn btn-ghost btn-sm" onclick="qsCancelDraft()">Cancel</button>
             <button class="btn btn-primary btn-sm" onclick="qsPublish()">Publish new version</button>
           </span>`
        : `<span style="margin-left:auto;">
             <button class="btn btn-primary btn-sm" onclick="qsStartDraft()">Edit questions</button>
           </span>`}
    </div>

    <p style="font-size:0.76rem;color:var(--muted);line-height:1.55;margin:0 0 12px;">
      ${editing
        ? 'Editing creates a NEW version when published. Evaluations already submitted keep the wording they were answered under, so past reports sta
y accurate.'
        : 'A published version cannot be edited in place \u2014 evaluations store answers by question id, so changing wording after the fact would mi
sreport what students were actually asked. Editing creates the next version instead.'}
    </p>

    ${editing ? `
      <div class="form-group">
        <label class="form-label">What changed (optional)</label>
        <input class="form-control" placeholder="e.g. Reworded Q4 per CHED memo 2027-03"
               value="${escapeHtml(_qsDraft.note || '')}" oninput="qsEditNote(this.value)"/>
      </div>` : ''}

    <div style="border:1px solid var(--border,#e2e8f0);border-radius:8px;overflow:hidden;">
      ${rows || '<div style="padding:20px;text-align:center;color:var(--muted);font-size:0.82rem;">No questions.</div>'}
    </div>

    ${editing ? `<button class="btn btn-ghost btn-sm" style="margin-top:10px;" onclick="qsAddQuestion()">+ Add question</button>` : ''}

    ${history ? `<div style="margin-top:18px;">
      <div style="font-size:0.78rem;font-weight:700;margin-bottom:6px;">Versions</div>
      <div style="border:1px solid var(--border,#e2e8f0);border-radius:8px;overflow:hidden;">${history}</div>
      <p style="font-size:0.72rem;color:var(--muted);margin:8px 0 0;">
        The app serves whichever version is marked <strong>In use</strong>. Switching does not
        create a new version and does not change evaluations already submitted.
      </p>
```

```
    </div>` : ''}`;
};

// Seed on load so the app always has something published to read.
try { ensureQuestionSets(); } catch (e) { /* getData not ready yet; the modal seeds it */ }
```

admin/js/modules/admindept.js

1029 lines

```
// ===== DEPARTMENTS =====
// No seed departments; all are added from the admin panel (Firestore + localStorage).
window.getDepartments = function() {
    const stored = getData('departments', null);
    if (stored && typeof stored === 'object') {
        return stored;
    }
    return {};
};

// Make DEPT_CONFIG available globally (for backward compatibility)
window.DEPT_CONFIG = getDepartments();

// Refresh DEPT_CONFIG from localStorage and re-render sidebar
window.refreshDeptConfig = function() {
    window.DEPT_CONFIG = getDepartments();
    regenerateSidebar();
};

// Add new department — saved directly into the departments store → syncs to Firestore
window.addDepartment = function(code, name, short, desc, icon, color) {
    code = code.toUpperCase();

    // Creating a code clears its tombstone (local and Firestore).
    try {
        const tomb = JSON.parse(localStorage.getItem('deletedDepts') || '[]');
        const kept = tomb.filter(function (c) { return c !== code; });
        if (kept.length !== tomb.length) localStorage.setItem('deletedDepts', JSON.stringify(kept));
    } catch (e) {}
    if (typeof db !== 'undefined') {
        db.collection('deletedDepts').doc(code).delete()
          .catch(function () { /* nothing recorded for this code - fine */ });
    }

    const depts = getDepartments();
    const newCfg = {
        name: name,
        short: short || code,
        desc: desc || name,
        icon: icon || null,
        color: color || '#2563eb',
        colorClass: 'dept-custom',
        isCustom: true
    };
    depts[code] = newCfg;
    // FIX Bug #2: base64 images can exceed the localStorage quota; catch it so
    // the Firestore save still runs.
    try {
        localStorage.setItem('departments', JSON.stringify(depts));
    } catch (e) {
        if (e.name === 'QuotaExceededError' || e.code === 22) {
            showToast('Image is too large to store locally. Try a smaller image or use an emoji icon instead.', 'error');
            return;
        }
        throw e;
    }
    // Write directly to Firestore
    if (typeof db !== 'undefined') {
        db.collection('departments').doc(code).set({ code, ...newCfg })
          .then(function() { console.log('Dept saved to Firestore:', code); })
          .catch(function(e) { console.error('Firestore dept save error:', e); });
    }
    refreshDeptConfig();
    // Also refresh the main dept cards page if currently visible
    if (typeof renderDeptManagePage === 'function') renderDeptManagePage();
    addAudit('Add Department', `Added: ${name} (${code})`);
    showToast('Department added successfully!', 'success');
};

// Remove department — only custom ones (no isCustom flag = protected)
window.removeDepartment = function(code) {
    code = code.toUpperCase();
    const depts = getDepartments();
    if (!depts[code]) { showToast('Department not found.', 'error'); return; }
    delete depts[code];
    setData('departments', depts);

    // Remove its courses too (localStorage customCourses and Firestore courses/<code>).
    try {
        const cc = getData('customCourses', {});
        if (cc && cc[code]) { delete cc[code]; setData('customCourses', cc); }
    } catch (e) { /* the Firestore delete below still stands */ }
    if (typeof db !== 'undefined') {
        db.collection('courses').doc(code).delete()
          .catch(function () { /* no courses recorded for this code - fine */ });
    }

    // Local deletion marker, so loadFromFirebase() doesn't re-upload it.
    try {
        const tomb = JSON.parse(localStorage.getItem('deletedDepts') || '[]');
        if (tomb.indexOf(code) === -1) tomb.push(code);
        localStorage.setItem('deletedDepts', JSON.stringify(tomb));
    } catch (e) { /* the Firestore delete below still stands */ }

    if (typeof db !== 'undefined') {
        db.collection('departments').doc(code).delete()
          .then(function() { console.log('Dept deleted from Firestore:', code); })
          .catch(function(e) { console.error('Firestore dept delete error:', e); });

        // Firestore tombstone, so other devices don't re-upload it either.
```

```
        db.collection('deletedDepts').doc(code).set({
            code: code,
            deletedAt: new Date().toISOString()
        }).catch(function(e) { console.warn('Could not record the deletion:', e.message); });
    }
    refreshDeptConfig();
    // Also refresh the main dept cards page if visible
    if (typeof renderDeptManagePage === 'function') renderDeptManagePage();
    addAudit('Remove Department', `Removed department: ${code}`);
    showToast('Department removed successfully!', 'success');
};

// Safe delete: warns if teachers/subjects/students still use this department.
window.deleteDeptSafe = function(code) {
    code = (code || '').toUpperCase();
    const depts = getDepartments();
    const cfg = depts[code];
    if (!cfg) { showToast('Department not found.', 'error'); return; }

    const teachers = getData('teachers', []).filter(t => !t.deleted && (t.dept || '').toUpperCase() === code);
    const subjects = getData('subjects', []).filter(s => (s.dept || '').toUpperCase() === code);
    // Count students too.
    const students = getData('students', []).filter(s => !s.deleted && (s.dept || '').toUpperCase() === code);

    let msg = 'Delete the department "' + cfg.name + '" (' + code + ')?';
    if (teachers.length || subjects.length || students.length) {
        msg += ' Warning: ' + students.length + ' student(s), ' + teachers.length + ' teacher(s) and ' + subjects.length +
               ' subject(s) are still assigned to it. They will not be deleted, but will show as ' +
               'unassigned until you move them to another department.';
    }
    msg += ' This cannot be undone.';

    showConfirm('Delete Department', msg, function() {
        removeDepartment(code);
        if (typeof closeModal === 'function') closeModal('deptEditModal');
        if (window.currentDept === code && typeof showPage === 'function') showPage('dashboard');
    });
};

// Toggle dept list visibility
window.toggleDeptNav = function() {
    const list = document.getElementById('deptNavList');
    const chevron = document.getElementById('deptNavChevron');
    if (!list) return;
    const isOpen = list.style.display !== 'none';
    list.style.display = isOpen ? 'none' : 'block';
    if (chevron) chevron.style.transform = isOpen ? 'rotate(-90deg)' : 'rotate(0deg)';
};

// Regenerate department navigation
window.regenerateSidebar = function() {
    const depts = getDepartments();
    const container = document.getElementById('deptNavSection');
    if (!container) return;

    const wasHidden = document.getElementById('deptNavList')?.style.display === 'none';

    let listHtml = '';
    for (const [code, config] of Object.entries(depts)) {
        let iconHtml;
        if (config.icon && (config.icon.includes('.jpg') || config.icon.includes('.png') || config.icon.includes('.jpeg') || config.icon.startsWith
('data:'))) {
            iconHtml = `<img src="${config.icon}" width="20" height="20" style="object-fit:cover;border-radius:4px;flex-shrink:0;">`;
        } else {
            iconHtml = `<svg width="20" height="20" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><rect x="3" y="3" width="1
8" height="18" rx="2"/></svg>`;
        }
        listHtml += `<div class="nav-item dept-nav-item" onclick="showDeptPage('${code}')" id="dept-${code}">${iconHtml}<span>${config.short}</span><
span class="dept-tag">${config.short}</span></div>`;
    }

    const chevronRot = wasHidden ? 'rotate(-90deg)' : 'rotate(0deg)';
    const listDisplay = wasHidden ? 'none' : 'block';

    container.innerHTML = `
        <div class="nav-section-label dept-section-toggle" onclick="toggleDeptNav()" style="cursor:pointer;display:flex;align-items:center;justify-co
ntent:space-between;user-select:none;">
            <span>Departments</span>
            <svg id="deptNavChevron" width="12" height="12" fill="none" stroke="currentColor" stroke-width="2.5" viewBox="0 0 24 24" style="transitio
n:transform 0.2s;margin-right:4px;transform:${chevronRot};"><polyline points="6 9 12 15 18 9"/></svg>
        </div>
        <div id="deptNavList" style="display:${listDisplay};">${listHtml}</div>`;
};

let currentDept = null;

function showDeptPage(deptCode) {
  document.querySelectorAll('.page').forEach(p => p.classList.remove('active'));
  document.querySelectorAll('.nav-item').forEach(n => n.classList.remove('active'));
  document.getElementById('page-dept').classList.add('active');
  const navEl = document.getElementById('dept-' + deptCode);
  if (navEl) navEl.classList.add('active');
  currentDept = deptCode;
  window.currentDept = deptCode;
  renderDeptPage(deptCode);
  closeSidebar();
}

/**
* Distinct, non-deleted students enrolled in a department's subjects.
* Single source of truth for the department student count.
*/
window.getDeptEnrolledStudentIds = function (deptCode) {
  const living = new Set(getData('students', []).filter(s => !s.deleted).map(s => s.id));
  const ids = new Set();
```

```
  getData('subjects', [])
    .filter(sub => sub.dept === deptCode)
    .forEach(sub => (sub.enrolledIds || []).forEach(eid => {
      if (living.has(eid)) ids.add(eid);
    }));
  return ids;
};

function renderDeptPage(deptCode) {
  const DEPT_CONFIG = getDepartments();
  const cfg = DEPT_CONFIG[deptCode];
  if (!cfg) return;

  document.getElementById('deptPageTitle').textContent = cfg.short + ' — ' + cfg.name;
  document.getElementById('deptPageSubtitle').textContent = cfg.desc;

  const banner = document.getElementById('deptBanner');
  banner.className = 'dept-banner ' + (cfg.colorClass || '');
  // For custom depts (no built-in CSS class), apply the saved hex color directly
  if (cfg.color && (cfg.isCustom || !cfg.colorClass || cfg.colorClass === 'dept-custom')) {
      const darken = (hex, amt) => {
          let c = hex.replace('#','');
          if (c.length === 3) c = c.split('').map(x=>x+x).join('');
          const num = parseInt(c, 16);
          const r = Math.max(0, (num >> 16) - amt);
          const g = Math.max(0, ((num >> 8) & 0xff) - amt);
          const b = Math.max(0, (num & 0xff) - amt);
          return '#' + [r,g,b].map(v=>v.toString(16).padStart(2,'0')).join('');
      };
      const darkStop = darken(cfg.color, 50);
      banner.style.background = `linear-gradient(135deg, ${darkStop} 0%, ${cfg.color} 100%)`;
      banner.style.borderColor = cfg.color;
  } else {
      banner.style.background = '';
      banner.style.borderColor = '';
  }

  // Check if icon is an image path or base64 data URL
if (cfg.icon && (cfg.icon.startsWith('data:') || cfg.icon.includes('.jpg') || cfg.icon.includes('.png') || cfg.icon.includes('.jpeg'))) {
    document.getElementById('deptBannerIcon').innerHTML = `<img src="${cfg.icon}" width="80" height="80" style="border-radius: 8px; object-fit: cove
r;">`;
} else if (cfg.icon) {
    document.getElementById('deptBannerIcon').textContent = cfg.icon;
} else {
    document.getElementById('deptBannerIcon').innerHTML = `<svg width="48" height="48" fill="none" stroke="currentColor" stroke-width="1.5" viewBox
="0 0 24 24" style="opacity:.5;"><rect x="3" y="3" width="18" height="18" rx="2"/><circle cx="8.5" cy="8.5" r="1.5"/><polyline points="21 15 16 10 5
21"/></svg>`;
}

  document.getElementById('deptBannerName').textContent = cfg.name;
  document.getElementById('deptBannerDesc').textContent = cfg.desc;

  const everyTeacher = getData('teachers', []).filter(t => !t.deleted);

  // Faculty of this department, incl. supervisors homed here who supervise
  // elsewhere (see teachesInDept in admin-teachers.js).
  const regularTeachers = everyTeacher.filter(t => teachesInDept(t, deptCode)).sort(byName);

  // Supervisors: anyone whose supervisedDepts includes this department.
  const deptSupervisors = everyTeacher.filter(t =>
    t.facultyType === 'supervisor' &&
    (typeof getSupervisedDepts === 'function'
      ? getSupervisedDepts(t).some(a => a.dept === deptCode)
      : t.dept === deptCode)).sort(byName);

  const allTeachers = regularTeachers.concat(deptSupervisors);
  const allSubjects = getData('subjects', []).filter(s => s.dept === deptCode);
  const allEvals    = getData('evaluations', []);
  const allStudents = getData('students', []).filter(s => !s.deleted);

  // Distinct students, not seats summed across subjects.
  const totalEnrolled = getDeptEnrolledStudentIds(deptCode).size;
  const deptEvals = allEvals.filter(e => allSubjects.some(s => s.id === e.subjectId));


///future me kung malimot ka style="background:#fff7ed sudsadi ang pag edit sa bg san icon
  document.getElementById('deptStatsRow').innerHTML = `
    <div class="dept-stat"><div class="dept-stat-icon" style="background:;"><img src ="../icons/teacher.png" width="28" height="28" alt="Teachers"></
div><div><div class="dept-stat-val">${regularTeachers.length}</div><div class="dept-stat-label">Teachers</div></div></div>
    <div class="dept-stat"><div class="dept-stat-icon" style="background:;"><img src ="../icons/subject.png" width="28" height="28" alt="Subjects"></
div><div><div class="dept-stat-val">${allSubjects.length}</div><div class="dept-stat-label">Subjects</div></div></div>
    <div class="dept-stat"><div class="dept-stat-icon" style="background:;"><img src ="../icons/enrolled.png" width="28" height="28" alt="Enrolled">
</div><div><div class="dept-stat-val">${totalEnrolled}</div><div class="dept-stat-label">Enrolled Students</div></div></div>
    <div class="dept-stat" style="cursor:pointer;" onclick="goToDeptFeedback('${deptCode}')" title="View evaluation feedback for this department"><di
v class="dept-stat-icon" style="background:;"><img src ="../icons/evaluate.png" width="28" height="28" alt="Evalaute"></div><div><div class="dept-sta
t-val">${deptEvals.length}</div><div class="dept-stat-label">Evaluations</div></div></div>
  `;

  // Preview cards: at most DEPT_PREVIEW_ROWS rows; "View all" shows the rest.
  const DEPT_PREVIEW_ROWS = 5;

  document.getElementById('deptTeacherCount').textContent = regularTeachers.length;
  document.getElementById('deptTeachersTbody').innerHTML = regularTeachers.length
    ? regularTeachers.slice(0, DEPT_PREVIEW_ROWS).map(t => {
        const tSubs = getData('subjects', []).filter(s => subjectHasTeacher(s, t.id));
        const tEvals = allEvals.filter(e => tSubs.some(s => s.id === e.subjectId));
        const setSc = calculateWeightedSETRating(t.id);
        const sefEvs = getData('evaluations', []).filter(e => e.teacherId === t.id && e.evaluatorType === 'supervisor');
        // Mean of every supervisor rating — see getSEFForTeacher() in admin.js.
        const sefAggD = getSEFForTeacher(t.id);
        const sefSc = sefAggD.count ? sefAggD.average.toFixed(2) : null;
        return `<tr onclick="showAnnexReports('${t.id}')" title="Click to view Annex C/D" style="cursor:pointer;">
          <td><span style="font-family:\'JetBrains Mono\',monospace;font-size:0.78rem;">${escapeHtml(t.tid)}</span></td>
          <td><strong style="font-size:0.82rem;">${escapeHtml(listName(t))}</strong>${t.facultyType === 'supervisor'
              ? `<div style="font-size:0.68rem;color:var(--muted);">also ${escapeHtml(supervisorRoleSummary(t))}</div>` : ''}</td>
```

```
          <td><span class="badge ${t.status==='active'?'badge-success':'badge-danger'}" style="font-size:0.68rem;">${t.status}</span></td>
          <td><strong style="font-size:0.82rem;">SET: ${Number(setSc) > 0 ? setSc + '%' : 'N/A'} | SEF: ${sefSc ? sefSc + '%' : 'N/A'}</strong></td>
        </tr>`;
      }).join('')
    : `<tr><td colspan="4" style="text-align:center;padding:24px;">No teachers found.</td></tr>`;

  // ===== DEPARTMENT SUPERVISORS SECTION =====
  const deptSupervisorEl = document.getElementById('deptSupervisorsSection');
  if (deptSupervisorEl) {
    // Always visible, so a missing supervisor is noticed (CMO 9.1 needs one for SEF).
    deptSupervisorEl.style.display = '';
    document.getElementById('deptSupervisorCount').textContent = deptSupervisors.length;
    {
      document.getElementById('deptSupervisorsTbody').innerHTML = deptSupervisors.length
        ? deptSupervisors.map(t => {
        // Role held in THIS department (t.deptRole is only the first one).
        const here = (typeof getSupervisedDepts === 'function')
          ? (getSupervisedDepts(t).find(a => a.dept === deptCode) || {})
          : { role: t.deptRole };
        const roleKey = here.role || t.deptRole || 'supervisor';
        const roleLabel = roleKey === 'dean' ? '🎓 Dean' : roleKey === 'chairperson' ? '🪑 Chairperson' : '👤 Supervisor';
        const roleColor = roleKey === 'dean' ? '#7c3aed' : roleKey === 'chairperson' ? '#0369a1' : '#374151';
        // Make it obvious when someone's home department is elsewhere.
        const visiting = t.dept && t.dept !== deptCode
          ? `<div style="font-size:0.68rem;color:var(--muted);">home: ${escapeHtml(t.dept)}</div>`
          : '';
        const sefEvs = getData('evaluations', []).filter(e => e.teacherId === t.id && e.evaluatorType === 'supervisor');
        const sefCount = sefEvs.length;
        return `<tr onclick="showAnnexReports('${t.id}')" title="Click to view Annex C/D" style="cursor:pointer;">
          <td><span style="font-family:\'JetBrains Mono\',monospace;font-size:0.78rem;">${escapeHtml(t.tid)}</span></td>
          <td><strong style="font-size:0.82rem;">${escapeHtml(listName(t))}</strong></td>
          <td><span style="font-size:0.75rem;font-weight:600;color:${roleColor};">${roleLabel}</span>${visiting}</td>
          <td><span class="badge ${t.status==='active'?'badge-success':'badge-danger'}" style="font-size:0.68rem;">${t.status}</span></td>
          <td><span style="font-size:0.78rem;">${sefCount} SEF rating${sefCount !== 1 ? 's' : ''} given</span></td>
        </tr>`;
        }).join('')
        : `<tr><td colspan="5" style="text-align:center;padding:24px;color:var(--muted);">
             No supervisor assigned to this department yet.
             <div style="font-size:0.72rem;margin-top:4px;">
               Faculty here cannot receive a SEF rating until a Program Chair or Dean
               is assigned (CMO 19 &sect;9.1, &sect;9.2). Open Teachers &rarr; Supervisors to assign one.
             </div>
           </td></tr>`;
    }
  }

  document.getElementById('deptSubjectCount').textContent = allSubjects.length;
  document.getElementById('deptSubjectsTbody').innerHTML = allSubjects.length
    ? allSubjects.slice(0, DEPT_PREVIEW_ROWS).map(sub => {
        const enrolled = (sub.enrolledIds || []).filter(eid => allStudents.find(s => s.id===eid)).length;
        const liveTeachers = getData('teachers', []).filter(t => !t.deleted);
        const names = subjectTeacherNames(sub, liveTeachers);
        // At most two names; the rest become a count.
        const teacherName = names.length
            ? names.slice(0, 2).map(n => escapeHtml(n)).join('<br>')
              + (names.length > 2
                  ? `<span style="color:var(--muted);font-size:0.72rem;">\u2026 +${names.length - 2} more</span>`
                  : '')
            : '<span style="color:var(--danger); font-style:italic;">Unassigned</span>';
        return `<tr>
          <td><span style="font-family:\'JetBrains Mono\',monospace;font-weight:700;font-size:0.78rem;">${escapeHtml(sub.code)}</span></td>
          <td style="font-size:0.82rem;">${escapeHtml(sub.name)}</td>
          <td style="font-size:0.82rem;">
            ${sub.category
              ? `<span class="badge" style="background:var(--primary-light,#eff6ff);color:var(--primary,#2563eb);font-size:0.68rem;">${escapeHtml(su
b.category)}</span>`
              : '<span style="color:var(--muted);font-size:0.72rem;">\u2014</span>'}
          </td>
          <td style="font-size:0.82rem;" title="${names.length ? escapeHtml(names.join(', ')) : 'No teacher assigned'}">${teacherName}</td>
          <td><span class="badge badge-primary" style="font-size:0.68rem;">${enrolled}</span></td>
        </tr>`;
      }).join('')
    : `<tr><td colspan="5" style="text-align:center;padding:24px;">No subjects found.</td></tr>`;
}

// ===== DEPT MANAGEMENT PAGE — layout preference =====
let _deptMgmtLayout = localStorage.getItem('deptMgmtLayout') || 'grid';

// ===== INJECT DEPT MANAGEMENT STYLES (once) =====
(function injectDeptMgmtStyles() {
    if (document.getElementById('deptMgmtStyles')) return;
    const s = document.createElement('style');
    s.id = 'deptMgmtStyles';
    s.textContent = `
    /* ── Dept Management Modal overrides ── */
    #deptMgmtModal .modal { max-width:860px; width:96vw; }
    #deptMgmtModal .modal-body { padding:0; overflow:hidden; }

    .dm-shell { display:flex; flex-direction:column; height:100%; }

    /* toolbar */
    .dm-toolbar {
        display:flex; align-items:center; gap:10px; flex-wrap:wrap;
        padding:14px 20px 10px; border-bottom:1px solid var(--border);
        background:var(--bg);
    }
    .dm-toolbar h3 { font-size:1rem; font-weight:700; margin:0; flex:1; }
    .dm-layout-btn {
        display:inline-flex; align-items:center; justify-content:center;
        width:32px; height:32px; border-radius:6px; border:1px solid var(--border);
        background:transparent; cursor:pointer; color:var(--muted-foreground);
        transition:background .15s, color .15s;
    }
    .dm-layout-btn.active, .dm-layout-btn:hover { background:var(--primary); color:#fff; border-color:var(--primary); }
    .dm-add-btn {
```

```
        display:inline-flex; align-items:center; gap:6px; padding:6px 14px;
        background:var(--primary); color:#fff; border:none; border-radius:6px;
        font-size:0.8rem; font-weight:600; cursor:pointer; transition:opacity .15s;
    }
    .dm-add-btn:hover { opacity:.88; }

    /* add form accordion */
    .dm-add-panel {
        border-bottom:1px solid var(--border);
        overflow:hidden; max-height:0; transition:max-height .3s ease;
    }
    .dm-add-panel.open { max-height:420px; }
    .dm-add-inner { padding:16px 20px; background:#f8fafc; }
    .dm-form-row { display:grid; grid-template-columns:1fr 1fr; gap:12px; margin-bottom:10px; }
    .dm-form-row.three { grid-template-columns:1fr 1fr 1fr; }
    .dm-form-group { display:flex; flex-direction:column; gap:4px; }
    .dm-form-group label { font-size:0.72rem; font-weight:600; color:var(--muted-foreground); text-transform:uppercase; letter-spacing:.04em; }
    .dm-form-group input, .dm-form-group textarea {
        padding:7px 10px; border:1px solid var(--border); border-radius:6px;
        font-size:0.82rem; background:#fff; color:var(--foreground);
        outline:none; transition:border-color .15s;
    }
    .dm-form-group input:focus, .dm-form-group textarea:focus { border-color:var(--primary); }

    /* icon picker area */
    .dm-icon-area { display:flex; align-items:center; gap:10px; }
    .dm-icon-preview {
        width:40px; height:40px; border-radius:8px; background:var(--primary-light);
        display:flex; align-items:center; justify-content:center; font-size:1.4rem;
        border:1px dashed var(--border); flex-shrink:0; overflow:hidden;
    }
    .dm-icon-preview img { width:100%; height:100%; object-fit:cover; border-radius:7px; }
    .dm-emoji-picker { display:flex; flex-wrap:wrap; gap:4px; margin-top:6px; }
    .dm-emoji-opt {
        width:28px; height:28px; display:flex; align-items:center; justify-content:center;
        font-size:1rem; border-radius:5px; cursor:pointer; border:1px solid transparent;
        transition:background .12s;
    }
    .dm-emoji-opt:hover, .dm-emoji-opt.sel { background:var(--primary-light); border-color:var(--primary); }
    .dm-upload-icon-btn {
        display:inline-flex; align-items:center; gap:5px; padding:5px 10px;
        border:1px solid var(--border); border-radius:6px; font-size:0.75rem;
        cursor:pointer; background:#fff; color:var(--foreground); transition:background .15s;
    }
    .dm-upload-icon-btn:hover { background:var(--primary-light); }
    .dm-form-actions { display:flex; gap:8px; justify-content:flex-end; margin-top:12px; }
    .dm-cancel-btn {
        padding:6px 16px; border:1px solid var(--border); border-radius:6px;
        background:transparent; font-size:0.8rem; cursor:pointer; color:var(--foreground);
    }
    .dm-save-btn {
        padding:6px 16px; background:var(--primary); color:#fff; border:none;
        border-radius:6px; font-size:0.8rem; font-weight:600; cursor:pointer;
    }

    /* dept list body */
    .dm-body { padding:16px 20px; overflow-y:auto; flex:1; max-height:420px; }

    /* grid layout */
    .dm-grid { display:grid; grid-template-columns:repeat(auto-fill,minmax(220px,1fr)); gap:14px; }
    .dm-card {
        border:1px solid var(--border); border-radius:10px; padding:14px;
        background:#fff; position:relative; transition:box-shadow .18s;
    }
    .dm-card:hover { box-shadow:0 4px 18px rgba(0,0,0,.09); }
    .dm-card-icon {
        width:44px; height:44px; border-radius:9px; display:flex; align-items:center;
        justify-content:center; font-size:1.5rem; margin-bottom:10px; overflow:hidden;
        background:var(--primary-light);
    }
    .dm-card-icon img { width:100%; height:100%; object-fit:cover; border-radius:8px; }
    .dm-card-code { font-family:'JetBrains Mono',monospace; font-size:0.68rem; font-weight:700; color:var(--primary); letter-spacing:.05em; }
    .dm-card-name { font-size:0.85rem; font-weight:700; margin:2px 0 4px; line-height:1.25; }
    .dm-card-desc { font-size:0.72rem; color:var(--muted-foreground); line-height:1.4; display:-webkit-box; -webkit-line-clamp:2; -webkit-box-orient:
vertical; overflow:hidden; }
    .dm-card-badge { display:inline-block; font-size:0.62rem; font-weight:700; padding:2px 7px; border-radius:20px; background:#e5e7eb; color:#6b728
0; margin-top:6px; }
    .dm-card-badge.custom { background:#dbeafe; color:#1d4ed8; }
    .dm-card-actions { position:absolute; top:10px; right:10px; display:flex; gap:5px; opacity:0; transition:opacity .15s; }
    .dm-card:hover .dm-card-actions { opacity:1; }
    .dm-icon-action-btn {
        width:28px; height:28px; border-radius:6px; display:flex; align-items:center;
        justify-content:center; border:1px solid transparent; cursor:pointer;
        transition:background .15s, border-color .15s, box-shadow .15s;
    }
    .dm-icon-action-btn.edit {
        background:#fff; border-color:#d1d5db; color:#374151;
        box-shadow:0 1px 2px rgba(0,0,0,.06);
    }
    .dm-icon-action-btn.edit:hover { background:#f0f9ff; border-color:#2563eb; color:#2563eb; }
    .dm-icon-action-btn.view {
        background:#fff; border-color:#d1d5db; color:#374151;
        box-shadow:0 1px 2px rgba(0,0,0,.06);
    }
    .dm-icon-action-btn.view:hover { background:#f0fdf4; border-color:#059669; color:#059669; }
    .dm-icon-action-btn.del { background:#fef2f2; color:#dc2626; border-color:#fecaca; }
    .dm-icon-action-btn.del:hover { background:#fee2e2; }
    /* Color picker */
    .dm-color-picker-row { display:flex; align-items:center; gap:10px; margin-top:2px; }
    .dm-color-input { width:36px; height:36px; border-radius:7px; border:1px solid var(--border); cursor:pointer; padding:2px; background:#fff; }
    .dm-color-swatches { display:flex; flex-wrap:wrap; gap:5px; }
    .dm-color-swatch { width:22px; height:22px; border-radius:5px; cursor:pointer; border:2px solid transparent; transition:transform .12s, border-co
lor .12s; }
    .dm-color-swatch:hover { transform:scale(1.18); border-color:rgba(0,0,0,.25); }
    .dm-color-swatch.sel { border-color:rgba(0,0,0,.4); transform:scale(1.12); }
```

```
    /* list layout */
    .dm-list { display:flex; flex-direction:column; gap:0; }
    .dm-list-row {
        display:flex; align-items:center; gap:12px; padding:10px 12px;
        border-bottom:1px solid var(--border); background:#fff;
        transition:background .12s;
    }
    .dm-list-row:hover { background:#f8fafc; }
    .dm-list-icon {
        width:36px; height:36px; flex-shrink:0; border-radius:8px;
        display:flex; align-items:center; justify-content:center; font-size:1.1rem;
        background:var(--primary-light); overflow:hidden;
    }
    .dm-list-icon img { width:100%; height:100%; object-fit:cover; border-radius:7px; }
    .dm-list-info { flex:1; min-width:0; }
    .dm-list-title { font-size:0.83rem; font-weight:700; }
    .dm-list-sub { font-size:0.72rem; color:var(--muted-foreground); white-space:nowrap; overflow:hidden; text-overflow:ellipsis; }
    .dm-list-code { font-family:'JetBrains Mono',monospace; font-size:0.7rem; color:var(--primary); font-weight:700; }
    .dm-list-actions { display:flex; gap:5px; flex-shrink:0; }

    /* delete confirmation accordion */
    .dm-delete-confirm {
        overflow:hidden; max-height:0; transition:max-height .25s ease;
        background:#fef2f2; border-top:1px solid #fecaca;
    }
    .dm-delete-confirm.open { max-height:80px; }
    .dm-delete-confirm-inner {
        padding:10px 14px; display:flex; align-items:center; gap:10px;
        font-size:0.78rem; color:#991b1b;
    }
    .dm-delete-confirm-inner strong { flex:1; }
    .dm-confirm-yes {
        padding:4px 12px; background:#dc2626; color:#fff; border:none;
        border-radius:5px; font-size:0.75rem; font-weight:600; cursor:pointer;
    }
    .dm-confirm-no {
        padding:4px 12px; background:#fff; color:#374151; border:1px solid #d1d5db;
        border-radius:5px; font-size:0.75rem; cursor:pointer;
    }

    /* edit modal */
    #deptEditModal .modal { max-width:520px; width:94vw; }
    .dem-icon-row { display:flex; align-items:flex-start; gap:14px; margin-bottom:14px; }
    .dem-icon-preview {
        width:56px; height:56px; border-radius:10px; background:var(--primary-light);
        display:flex; align-items:center; justify-content:center; font-size:2rem;
        border:2px dashed var(--border); flex-shrink:0; overflow:hidden;
    }
    .dem-icon-preview img { width:100%; height:100%; object-fit:cover; border-radius:9px; }
    .dem-emoji-grid { display:flex; flex-wrap:wrap; gap:4px; }
    .dem-emoji-btn {
        width:30px; height:30px; border-radius:6px; font-size:1rem;
        display:flex; align-items:center; justify-content:center;
        cursor:pointer; border:1px solid transparent; transition:background .12s;
    }
    .dem-emoji-btn:hover, .dem-emoji-btn.sel { background:var(--primary-light); border-color:var(--primary); }
    .dem-upload-btn {
        display:inline-flex; align-items:center; gap:5px; padding:5px 11px;
        border:1px solid var(--border); border-radius:6px; font-size:0.75rem;
        cursor:pointer; background:#fff; transition:background .15s; margin-top:6px;
    }
    .dem-upload-btn:hover { background:var(--primary-light); }

    @media (max-width:600px) {
        .dm-form-row { grid-template-columns:1fr; }
        .dm-form-row.three { grid-template-columns:1fr; }
        .dm-grid { grid-template-columns:1fr 1fr; }
    }
    `;
    document.head.appendChild(s);
})();

// ── emoji palette shared ──
const DEPT_EMOJIS = ['🎓','📚','🏛️','💻','⚕️','⚖️','🏗️','🌾','🔬','🎨','🎭','📐','🧬','🏥','👩‍💼','📊','🛠️','🌍','✈️','🏋️'];

// ── render icon HTML (shared) ──
function _deptIconHtml(icon, cls, size = 44) {
    const isImg = icon && (icon.startsWith('data:') || icon.includes('.jpg') || icon.includes('.png') || icon.includes('.jpeg'));
    return `<div class="${cls}" style="width:${size}px;height:${size}px;overflow:hidden;border-radius:9px;display:flex;align-items:center;justify-con
tent:center;">
        ${isImg ? `<img src="${icon}" alt="" style="width:100%;height:100%;object-fit:cover;border-radius:inherit;">` : `<span style="font-size:${Mat
h.round(size*0.55)}px;">${icon || '🏛️'}</span>`}
    </div>`;
}

// ── open manage departments modal ──
window.openManageDeptsModal = function() {
    let overlay = document.getElementById('deptMgmtModal');
    if (!overlay) {
        overlay = document.createElement('div');
        overlay.id = 'deptMgmtModal';
        overlay.className = 'modal-overlay';
        overlay.innerHTML = `
        <div class="modal" style="padding:0;overflow:hidden;display:flex;flex-direction:column;max-height:92vh;">
            <div class="modal-header" style="padding:14px 18px 12px;border-bottom:1px solid var(--border);display:flex;align-items:center;">
                <h2 class="modal-title" style="font-size:1rem;font-weight:700;flex:1;">Manage Courses</h2>
                <button class="modal-close" onclick="closeModal('deptMgmtModal')">×</button>
            </div>
            <div id="deptMgmtBody" class="modal-body" style="padding:0;overflow:hidden;flex:1;display:flex;flex-direction:column;"></div>
        </div>`;
        overlay.addEventListener('click', e => { if (e.target === overlay) closeModal('deptMgmtModal'); });
        document.body.appendChild(overlay);
    }
```

```
    _renderCoursesMgmtBody();
    openModal('deptMgmtModal');
};

function _renderDeptMgmtBody() {
    const depts = getDepartments();
    const layout = _deptMgmtLayout;
    const body = document.getElementById('deptMgmtBody');
    if (!body) return;

    body.innerHTML = `
    <div class="dm-shell">
        <!-- Toolbar -->
        <div class="dm-toolbar">
            <h3>All Departments <span style="font-size:0.72rem;font-weight:400;color:var(--muted-foreground);">${Object.keys(depts).length} total</span></h3>
            <button class="dm-layout-btn ${layout === 'grid' ? 'active' : ''}" title="Grid view" onclick="_setDeptLayout('grid')">
                <svg width="14" height="14" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><rect x="3" y="3" width="7" height="7"/><rect x="14" y="3" width="7" height="7"/><rect x="3" y="14" width="7" height="7"/><rect x="14" y="14" width="7" height="7"/></svg>
            </button>
            <button class="dm-layout-btn ${layout === 'list' ? 'active' : ''}" title="List view" onclick="_setDeptLayout('list')">
                <svg width="14" height="14" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><line x1="3" y1="6" x2="21" y2="6"/><line x1="3" y1="12" x2="21" y2="12"/><line x1="3" y1="18" x2="21" y2="18"/></svg>
            </button>
            <button class="dm-add-btn" onclick="_toggleAddDeptPanel()">
                <svg width="12" height="12" fill="none" stroke="currentColor" stroke-width="2.5" viewBox="0 0 24 24"><line x1="12" y1="5" x2="12" y2="19"/><line x1="5" y1="12" x2="19" y2="12"/></svg>
                Add Department
            </button>
        </div>

        <!-- Add form accordion -->
        <div class="dm-add-panel" id="dmAddPanel">
            <div class="dm-add-inner">
                <div class="dm-form-row">
                    <div class="dm-form-group">
                        <label>Short Name *</label>
                        <input id="newDeptShort" class="form-control" placeholder="e.g. GS">
                    </div>
                    <div class="dm-form-group">
                        <label>Full Name *</label>
                        <input id="newDeptName" class="form-control" placeholder="e.g. Graduate School">
                    </div>
                </div>
                <div class="dm-form-row">
                    <div class="dm-form-group">
                        <label>Description</label>
                        <input id="newDeptDesc" class="form-control" placeholder="Brief description">
                    </div>
                    <div class="dm-form-group">
                        <label>Department Color</label>
                        <div class="dm-color-picker-row">
                            <input type="color" id="newDeptColor" value="#2563eb" class="dm-color-input">
                            <div class="dm-color-swatches">
                                ${['#2563eb','#7c3aed','#0891b2','#059669','#dc2626','#d97706','#db2777','#4f46e5','#0f766e','#b45309'].map(c => `<span class="dm-color-swatch" style="background:${c};" onclick="_pickNewDeptColor('${c}')" title="${c}"></span>`).join('')}
                            </div>
                        </div>
                    </div>
                </div>
                <div class="dm-form-group" style="margin-bottom:10px;">
                    <label>Icon / Image</label>
                    <div class="dm-icon-area">
                        <div class="dm-icon-preview" id="newDeptIconPreview">
                            <svg width="22" height="22" fill="none" stroke="currentColor" stroke-width="1.5" viewBox="0 0 24 24" style="color:#94a3b8;"><rect x="3" y="3" width="18" height="18" rx="2"/><circle cx="8.5" cy="8.5" r="1.5"/><polyline points="21 15 16 10 5 21"/></svg>
                        </div>
                        <label class="dm-upload-icon-btn">
                            <svg width="12" height="12" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M21 15v4a2 2 0 01-2 2H5a2 2 0 01-2-2v-4"/><polyline points="17 8 12 3 7 8"/><line x1="12" y1="3" x2="12" y2="15"/></svg>
                            Upload Image
                            <input type="file" accept="image/*" style="display:none;" onchange="_handleNewDeptImageUpload(event)">
                        </label>
                    </div>
                </div>
                <input type="hidden" id="newDeptIconValue" value="">
                <input type="hidden" id="newDeptColorValue" value="#2563eb">
                <div class="dm-form-actions">
                    <button class="dm-cancel-btn" onclick="_toggleAddDeptPanel()">Cancel</button>
                    <button class="dm-save-btn" onclick="addDeptFromModal()">
                        <svg width="12" height="12" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><polyline points="20 6 9 17 4 12"/></svg>
                        Save Department
                    </button>
                </div>
            </div>
        </div>

        <!-- Dept list body -->
        <div class="dm-body" id="dmDeptListBody">
            ${_buildDeptListHtml(depts, layout)}
        </div>
    </div>`;
}

function _buildDeptListHtml(depts, layout) {
    const entries = Object.entries(depts);
    if (!entries.length) return `<p style="text-align:center;color:var(--muted-foreground);padding:32px;">No departments found.</p>`;

    if (layout === 'grid') {
        return `<div class="dm-grid">
        ${entries.map(([code, cfg]) => {
            const isImg = cfg.icon && (cfg.icon.startsWith('data:') || cfg.icon.includes('.jpg') || cfg.icon.includes('.png') || cfg.icon.includes('.jpeg'));
            const iconBg = cfg.color ? cfg.color + '22' : 'var(--primary-light)';
```

```
            const iconBorder = cfg.color ? cfg.color + '44' : 'var(--border)';
            const accentBar = cfg.color || 'var(--primary)';
            const iconInner = isImg
                ? `<img src="${cfg.icon}" alt="" style="width:100%;height:100%;object-fit:cover;border-radius:8px;">`
                : `<svg width="22" height="22" fill="none" stroke="${cfg.color || 'var(--primary)'}" stroke-width="1.5" viewBox="0 0 24 24" style="op
acity:.6;"><rect x="3" y="3" width="18" height="18" rx="2"/><circle cx="8.5" cy="8.5" r="1.5"/><polyline points="21 15 16 10 5 21"/></svg>`;
            return `
            <div class="dm-card" id="dmcard-${code}" style="border-top:3px solid ${accentBar};">
                <div class="dm-card-actions">
                    <button class="dm-icon-action-btn view" onclick="event.stopPropagation();showDeptPage('${code}')" title="View">
                        <svg width="13" height="13" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M1 12s4-8 11-8 11
8 11 8-4 8-11 8-11-8-11-8z"/><circle cx="12" cy="12" r="3"/></svg>
                    </button>
                    <button class="dm-icon-action-btn edit" onclick="event.stopPropagation();openEditDeptModal('${code}')" title="Edit">
                        <svg width="13" height="13" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M11 4H4a2 2 0 00-
2 2v14a2 2 0 002 2h14a2 2 0 002-2v-7"/><path d="M18.5 2.5a2.121 2.121 0 013 3L12 15l-4 1 1-4 9.5-9.5z"/></svg>
                    </button>
                    <button class="dm-icon-action-btn del" onclick="event.stopPropagation();deleteDeptSafe('${code}')" title="Delete">
                        <svg width="13" height="13" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><polyline points="3 6 5 6
21 6"/><path d="M19 6l-1 14a2 2 0 01-2 2H8a2 2 0 01-2-2L5 6m3 0V4a2 2 0 012-2h4a2 2 0 012 2v2"/><line x1="10" y1="11" x2="10" y2="17"/><line x1="14"
y1="11" x2="14" y2="17"/></svg>
                    </button>
                </div>
                <div class="dm-card-icon" style="background:${iconBg};border:1px solid ${iconBorder};overflow:hidden;">${iconInner}</div>
                <div class="dm-card-code">${escapeHtml(code)}</div>
                <div class="dm-card-name">${escapeHtml(cfg.name)}</div>
                <div class="dm-card-desc">${escapeHtml(cfg.desc || '')}</div>
                ${cfg.isCustom ? `<span class="dm-card-badge custom">Custom</span>` : `<span class="dm-card-badge">Default</span>`}
            </div>`;
        }).join('')}
        </div>`;
    }

    // List layout
    return `<div class="dm-list">
    ${entries.map(([code, cfg]) => {
        const isImg = cfg.icon && (cfg.icon.startsWith('data:') || cfg.icon.includes('.jpg') || cfg.icon.includes('.png') || cfg.icon.includes('.jpe
g'));
        const iconBg2 = cfg.color ? cfg.color + '22' : 'var(--primary-light)';
        const iconInner = isImg
            ? `<img src="${cfg.icon}" alt="" style="width:100%;height:100%;object-fit:cover;border-radius:7px;">`
            : `<svg width="18" height="18" fill="none" stroke="${cfg.color || 'var(--primary)'}" stroke-width="1.5" viewBox="0 0 24 24" style="opacit
y:.6;"><rect x="3" y="3" width="18" height="18" rx="2"/><circle cx="8.5" cy="8.5" r="1.5"/><polyline points="21 15 16 10 5 21"/></svg>`;
        return `
        <div>
            <div class="dm-list-row" id="dmrow-${code}" style="border-left:3px solid ${cfg.color || 'var(--primary)'};padding-left:13px;">
                <div class="dm-list-icon" style="background:${iconBg2};overflow:hidden;">${iconInner}</div>
                <div class="dm-list-info">
                    <div class="dm-list-title">${escapeHtml(cfg.name)}</div>
                    <div class="dm-list-sub"><span class="dm-list-code">${escapeHtml(code)}</span> — ${escapeHtml(cfg.desc || '')}</div>
                </div>
                <div class="dm-list-actions">
                    <button class="dm-icon-action-btn view" onclick="showDeptPage('${code}')" title="View">
                        <svg width="13" height="13" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M1 12s4-8 11-8 11
8 11 8-4 8-11 8-11-8-11-8z"/><circle cx="12" cy="12" r="3"/></svg>
                    </button>
                    <button class="dm-icon-action-btn edit" onclick="openEditDeptModal('${code}')" title="Edit department">
                        <svg width="13" height="13" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M11 4H4a2 2 0 00-
2 2v14a2 2 0 002 2h14a2 2 0 002-2v-7"/><path d="M18.5 2.5a2.121 2.121 0 013 3L12 15l-4 1 1-4 9.5-9.5z"/></svg>
                    </button>
                    <button class="dm-icon-action-btn del" onclick="deleteDeptSafe('${code}')" title="Delete department">
                        <svg width="13" height="13" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><polyline points="3 6 5 6
21 6"/><path d="M19 6l-1 14a2 2 0 01-2 2H8a2 2 0 01-2-2L5 6m3 0V4a2 2 0 012-2h4a2 2 0 012 2v2"/><line x1="10" y1="11" x2="10" y2="17"/><line x1="14"
y1="11" x2="14" y2="17"/></svg>
                    </button>
                </div>
            </div>
        </div>`;
    }).join('')}
    </div>`;
}

window._setDeptLayout = function(layout) {
    _deptMgmtLayout = layout;
    localStorage.setItem('deptMgmtLayout', layout);
    _renderDeptMgmtBody();
};

window._toggleAddDeptPanel = function() {
    const panel = document.getElementById('dmAddPanel');
    if (!panel) return;
    panel.classList.toggle('open');
};

window._pickNewDeptColor = function(color) {
    document.getElementById('newDeptColorValue').value = color;
    const inp = document.getElementById('newDeptColor');
    if (inp) inp.value = color;
    document.querySelectorAll('#dmAddPanel .dm-color-swatch').forEach(el => {
        el.classList.toggle('sel', el.getAttribute('onclick') === "_pickNewDeptColor('" + color + "')");
    });
    // Live-update the icon preview background so the chosen color is immediately visible
    const prev = document.getElementById('newDeptIconPreview');
    if (prev) {
        prev.style.background = color + '22';
        prev.style.borderColor = color + '88';
        // If no image uploaded yet, tint the placeholder SVG stroke too
        const svg = prev.querySelector('svg');
        if (svg) svg.style.color = color;
    }
};

// Keep native color input in sync with hidden value + live preview
document.addEventListener('change', function(e) {
    if (e.target && e.target.id === 'newDeptColor') {
```

```
        const color = e.target.value;
        document.getElementById('newDeptColorValue').value = color;
        document.querySelectorAll('#dmAddPanel .dm-color-swatch').forEach(el => el.classList.remove('sel'));
        // Also update the preview
        const prev = document.getElementById('newDeptIconPreview');
        if (prev) {
            prev.style.background = color + '22';
            prev.style.borderColor = color + '88';
            const svg = prev.querySelector('svg');
            if (svg) svg.style.color = color;
        }
    }
    // Keep edit modal native color input in sync with hidden field + live preview
    if (e.target && e.target.id === 'editDeptColor') {
        const color = e.target.value;
        document.getElementById('editDeptColorValue').value = color;
        document.querySelectorAll('#editDeptModalBody .dm-color-swatch').forEach(el => el.classList.remove('sel'));
        const prev = document.getElementById('editDeptIconPreview');
        if (prev) {
            prev.style.background = color + '22';
            prev.style.borderColor = color + '88';
            const svg = prev.querySelector('svg');
            if (svg) svg.setAttribute('stroke', color);
        }
    }
});

window._handleNewDeptImageUpload = function(e) {
    const file = e.target.files[0];
    if (!file) return;
    const reader = new FileReader();
    reader.onload = function(ev) {
        const data = ev.target.result;
        document.getElementById('newDeptIconValue').value = data;
        const prev = document.getElementById('newDeptIconPreview');
        if (prev) prev.innerHTML = `<img src="${data}" style="width:100%;height:100%;object-fit:cover;border-radius:7px;">`;
        document.querySelectorAll('.dm-emoji-opt').forEach(el => el.classList.remove('sel'));
    };
    reader.readAsDataURL(file);
};

window._toggleDeleteConfirm = function(code, layout) {
    const el = document.getElementById(`dmdelconfirm-${code}`);
    if (!el) return;
    el.classList.toggle('open');
};

window._confirmDeleteDept = function(code) {
    removeDepartment(code);
    _renderDeptMgmtBody();
};

// ── Add dept from modal (new) ──
window.addDeptFromModal = function() {
    const name  = (document.getElementById('newDeptName')?.value  || '').trim();
    const short = (document.getElementById('newDeptShort')?.value || '').trim();
    const icon  = (document.getElementById('newDeptIconValue')?.value || '').trim();
    const desc  = (document.getElementById('newDeptDesc')?.value  || '').trim() || name;
    const color = (document.getElementById('newDeptColorValue')?.value || '#2563eb').trim();

    if (!name || !short) { showToast('Please fill in Short Name and Full Name.', 'error'); return; }

    let code = short.toUpperCase().replace(/[^A-Z0-9]/g, '').substring(0, 8);
    if (!code) code = 'DEPT';

    // FIX Bug #1: block saving a duplicate department code.
    const existing = getDepartments();
    if (existing[code]) {
        showToast(`Department code "${code}" already exists. Use a different short name.`, 'error');
        return;
    }

    addDepartment(code, name, short, desc, icon, color);
    _renderDeptMgmtBody();
};

// REMOVED (dead code): openEditDeptModal - defined in adminfeatures.js.

window._pickEditDeptColor = function(color) {
    document.getElementById('editDeptColorValue').value = color;
    const inp = document.getElementById('editDeptColor');
    if (inp) inp.value = color;
    document.querySelectorAll('#editDeptModalBody .dm-color-swatch').forEach(el => {
        el.classList.toggle('sel', el.getAttribute('onclick') === "_pickEditDeptColor('" + color + "')");
    });
    // Live-update the icon preview background so the chosen color is immediately visible
    const prev = document.getElementById('editDeptIconPreview');
    if (prev) {
        prev.style.background = color + '22';
        prev.style.borderColor = color + '88';
        const svg = prev.querySelector('svg');
        if (svg) svg.setAttribute('stroke', color);
    }
};

window._handleEditDeptImageUpload = function(e) {
    const file = e.target.files[0];
    if (!file) return;
    const reader = new FileReader();
    reader.onload = function(ev) {
        const data = ev.target.result;
        document.getElementById('editDeptIconValue').value = data;
        const prev = document.getElementById('editDeptIconPreview');
        if (prev) prev.innerHTML = `<img src="${data}" style="width:100%;height:100%;object-fit:cover;border-radius:9px;">`;

```

```
    };
    reader.readAsDataURL(file);
};

// REMOVED (dead code): saveEditDept - defined in adminfeatures.js.
// ===== MANAGE COURSES PANEL (inside deptMgmtModal, Courses tab) =====

(function injectCoursesMgmtStyles() {
    if (document.getElementById('coursesMgmtStyles')) return;
    const s = document.createElement('style');
    s.id = 'coursesMgmtStyles';
    s.textContent = `
    .cm-shell { display:flex; flex-direction:column; height:100%; }
    .cm-toolbar {
        display:flex; align-items:center; gap:10px; flex-wrap:wrap;
        padding:12px 20px 10px; border-bottom:1px solid var(--border);
        background:var(--bg,#fff);
    }
    .cm-toolbar h3 { font-size:0.95rem; font-weight:700; margin:0; flex:1; }
    .cm-dept-sel { padding:6px 10px; border:1px solid var(--border); border-radius:6px; font-size:0.82rem; min-width:200px; }
    .cm-body { padding:14px 20px; overflow-y:auto; flex:1; max-height:430px; }
    .cm-crse-row {
        display:flex; align-items:center; gap:6px; padding:5px 10px;
        border-radius:6px; background:var(--surface2,#f8fafc);
        margin-bottom:5px; border:1px solid var(--border,#e2e8f0);
    }
    .cm-crse-label { flex:1; font-size:0.8rem; line-height:1.4; word-break:break-word; }
    .cm-crse-input { flex:1; font-size:0.8rem; padding:3px 7px; display:none; }
    .cm-action-btn { padding:2px 8px; font-size:0.72rem; border:none; background:transparent; cursor:pointer; border-radius:4px; }
    .cm-action-btn:hover { background:var(--surface3,#e2e8f0); }
    .cm-action-btn.del { color:var(--danger,#e74c3c); }
    .cm-action-btn.save { color:var(--primary,#2563eb); font-weight:700; }
    .cm-add-row { display:flex; gap:8px; align-items:center; padding:12px 20px; border-top:1px solid var(--border); background:var(--bg,#fff); }
    .cm-add-row input { flex:1; font-size:0.82rem; }
    .cm-bulk-bar {
        display:flex; align-items:center; gap:8px; flex-wrap:wrap;
        padding:8px 20px; border-top:1px solid var(--border);
        background:var(--surface2,#f8fafc); font-size:0.78rem; color:var(--muted-foreground);
    }
    .cm-bulk-label { display:inline-flex; align-items:center; gap:5px; padding:5px 12px;
        border:1px solid var(--border); border-radius:6px; font-size:0.78rem; cursor:pointer;
        background:#fff; color:var(--foreground); transition:background .15s; font-weight:600; }
    .cm-bulk-label:hover { background:var(--primary-light,#eff6ff); }
    .cm-tmpl-btn { display:inline-flex; align-items:center; gap:5px; padding:5px 12px;
        border:1px solid var(--primary,#2563eb); border-radius:6px; font-size:0.78rem; cursor:pointer;
        background:transparent; color:var(--primary,#2563eb); font-weight:600; transition:background .15s; }
    .cm-tmpl-btn:hover { background:var(--primary-light,#eff6ff); }
    .cm-empty { color:var(--muted-foreground); font-size:0.82rem; padding:18px 0 4px; }
    `;
    document.head.appendChild(s);
})();

function _renderCoursesMgmtBody() {
    const body = document.getElementById('deptMgmtBody');
    if (!body) return;
    const depts = getDepartments();
    const deptOptions = Object.entries(depts)
        .map(([code, cfg]) => `<option value="${code}">${code} — ${escapeHtml(cfg.name)}</option>`)
        .join('');

    body.innerHTML = `
    <div class="cm-shell">
        <!-- Toolbar: dept selector + totals -->
        <div class="cm-toolbar">
            <h3>Courses</h3>
            <select class="cm-dept-sel" id="mcCrseDept" onchange="mcLoadDept(this.value)">
                <option value="">— Select Department —</option>
                ${deptOptions}
            </select>
            <span id="mcCrseCount" style="font-size:0.78rem;color:var(--muted-foreground);"></span>
        </div>

        <!-- Course list -->
        <div class="cm-body" id="mcCrseList">
            <p class="cm-empty">Select a department above to see its courses.</p>
        </div>

        <!-- Add single course row -->
        <div class="cm-add-row">
            <input class="form-control" id="mcNewCrseInput"
                placeholder="e.g. Bachelor of Science in Marine Transportation (BSMT)"
                onkeydown="if(event.key==='Enter'){mcAddCourseFromPanel()}" />
            <button class="btn btn-primary" style="white-space:nowrap;font-size:0.8rem;"
                onclick="mcAddCourseFromPanel()">+ Add Course</button>
        </div>

        <!-- Bulk upload bar -->
        <div class="cm-bulk-bar">
            <span>Bulk upload:</span>
            <label class="cm-bulk-label">
                <svg width="13" height="13" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M21 15v4a2 2 0 01-2 2H5a2
2 0 01-2-2v-4"/><polyline points="17 8 12 3 7 8"/><line x1="12" y1="3" x2="12" y2="15"/></svg>
                Upload Excel (.xlsx)
                <input type="file" accept=".xlsx,.xls" style="display:none;" onchange="mcHandleBulkUpload(event)" />
            </label>
            <button class="cm-tmpl-btn" onclick="mcDownloadTemplate()">
                <svg width="13" height="13" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M21 15v4a2 2 0 01-2 2H5a2
2 0 01-2-2v-4"/><polyline points="7 10 12 15 17 10"/><line x1="12" y1="3" x2="12" y2="15"/></svg>
                Download Template
            </button>
            <span style="color:var(--muted-foreground);font-size:0.72rem;margin-left:auto;">Columns: A = Dept Code, B = Course Name</span>
        </div>
    </div>`;
}
```

```
// Called from the courses panel add button (reads from mcCrseDept)
window.mcAddCourseFromPanel = function() {
    const deptCode = (document.getElementById('mcCrseDept') || {}).value || '';
    mcAddCourse(deptCode);
    _mcUpdateCount(deptCode);
};

// Update the count badge next to h3
function _mcUpdateCount(deptCode) {
    const el = document.getElementById('mcCrseCount');
    if (!el || !deptCode) return;
    reloadCoursesByDept();
    const n = (COURSES_BY_DEPT[deptCode] || []).length;
    el.textContent = `${n} course${n !== 1 ? 's' : ''}`;
}

// Override mcLoadDept to also update count badge
const _origMcLoadDept = window.mcLoadDept;
window.mcLoadDept = function(deptCode) {
    if (typeof _origMcLoadDept === 'function') _origMcLoadDept(deptCode);
    _mcUpdateCount(deptCode);
};


// Build the sidebar now from stored data; loadFromFirebase() refreshes it.
try { if (typeof refreshDeptConfig === 'function') refreshDeptConfig(); } catch (e) {}
```

admin/js/modules/adminfeatures.js

1978 lines

```
// adminfeatures.js - department drill-downs, feedback page, shared table view.
// Loads late: redefines renderStudents and wraps showPage.

window.showDeptEnrolledStudents = function(deptCode) {
    const DEPT_CONFIG = getDepartments();
    const cfg = DEPT_CONFIG[deptCode] || { name: deptCode, short: deptCode };

    const allStudents = getData('students', []).filter(s => !s.deleted);
    const allSubjects = getData('subjects', []).filter(s => s.dept === deptCode);

    const enrolledIds = getDeptEnrolledStudentIds(deptCode);
    const enrolledStudents = allStudents.filter(s => enrolledIds.has(s.id)).sort(byName);

    const teachers = getData('teachers', []);
    const inTerm = (typeof annexEvalInTerm === 'function') ? annexEvalInTerm : function () { return true; };
    const evals = getData('evaluations', []).filter(e => e.evaluatorType !== 'supervisor' && inTerm(e));

    const subjectMap = {};
    const stById = {};
    getData('students', []).forEach(s => { stById[s.id] = s; });
    allSubjects.forEach(sub => {
        (sub.enrolledIds || []).forEach(eid => {
            if (!subjectMap[eid]) subjectMap[eid] = [];
            // One entry per teacher of this student in the subject, each with its evaluated flag.
            const mine = subjectTeacherIds(sub).filter(tid => teacherTeachesStudent(sub, tid, stById[eid]));
            if (!mine.length) {
                subjectMap[eid].push({ code: sub.code, name: sub.name,
                                       teacher: 'No teacher assigned', evaluated: false });
                return;
            }
            mine.forEach(tid => {
                const teacher = teachers.find(t => t.id === tid);
                subjectMap[eid].push({
                    code: sub.code,
                    name: sub.name,
                    teacher: teacher ? teacher.name : 'No teacher assigned',
                    evaluated: evals.some(e => e.studentId === eid && e.subjectId === sub.id
                                          && evalBelongsTo(e, sub, tid))
                });
            });
        });
    });

    document.getElementById('deptEnrolledTitle').textContent = `${cfg.name} \u2014 Enrolled Students`;
    document.getElementById('deptEnrolledCount').textContent =
        `${enrolledStudents.length} student${enrolledStudents.length !== 1 ? 's' : ''} \u00b7 click a row to see their subjects`;

    window._deptEnrolledStudents = enrolledStudents;
    window._deptEnrolledSubjectMap = subjectMap;

    _deptEnrolledRender(enrolledStudents, 'No enrolled students found for this department.');
    openModal('deptEnrolledModal');
};

window._deptEnrolledRender = function(list, emptyMsg) {
    const tbody = document.getElementById('deptEnrolledTbody');
    if (!tbody) return;
    if (!list.length) {
        tbody.innerHTML = `<tr><td colspan="6" style="text-align:center;padding:32px;color:var(--muted);">${escapeHtml(emptyMsg)}</td></tr>`;
        return;
    }
    tbody.innerHTML = list.map(_deptEnrolledRow).join('');
};

window._deptEnrolledRow = function(s) {
    const subs = (window._deptEnrolledSubjectMap || {})[s.id] || [];
    const done = subs.filter(x => x.evaluated).length;

    const detail = subs.length ? subs.map(x => `
        <div style="display:flex;align-items:center;gap:10px;padding:6px 4px;border-bottom:1px solid var(--border,#e2e8f0);">
            <span class="badge badge-primary" style="font-size:0.65rem;min-width:74px;text-align:center;">${escapeHtml(x.code)}</span>
            <span style="flex:1;font-size:0.78rem;">${escapeHtml(x.name)}</span>
            <span style="font-size:0.72rem;color:var(--muted,#64748b);">${escapeHtml(x.teacher)}</span>
            <span class="badge ${x.evaluated ? 'badge-success' : ''}" style="font-size:0.62rem;${x.evaluated ? '' : 'background:var(--surface2,#f1f5f
9);color:var(--muted,#64748b);'}">
                ${x.evaluated ? 'Evaluated' : 'Pending'}
            </span>
        </div>`).join('')
      : '<div style="padding:8px 4px;color:var(--muted);font-size:0.78rem;">No subjects in this department.</div>';

    return `<tr onclick="toggleDeptEnrolledDetail('${escapeHtml(s.id)}')" style="cursor:pointer;">
        <td><span style="font-family:'JetBrains Mono',monospace;font-weight:600;font-size:0.8rem;">${escapeHtml(s.sid)}</span></td>
        <td><strong style="font-size:0.85rem;">${escapeHtml(listName(s))}</strong></td>
        <td style="font-size:0.8rem;">${escapeHtml(s.year)}</td>
        <td style="font-size:0.8rem;">Sec ${escapeHtml(s.section)}</td>
        <td style="font-size:0.78rem;white-space:nowrap;">
            <svg id="dechev-${escapeHtml(s.id)}" width="12" height="12" fill="none" stroke="currentColor" stroke-width="2.5" viewBox="0 0 24 24" styl
e="vertical-align:-1px;margin-right:5px;transition:transform 0.15s;"><polyline points="9 18 15 12 9 6"/></svg>
            ${subs.length} subject${subs.length !== 1 ? 's' : ''} &middot; ${done}/${subs.length} evaluated
        </td>
        <td><span class="badge ${s.status === 'active' ? 'badge-success' : 'badge-danger'}" style="font-size:0.68rem;">${escapeHtml(s.status)}</span>
</td>
    </tr>
    <tr id="dedet-${escapeHtml(s.id)}" style="display:none;">
        <td colspan="6" style="background:var(--surface2,#f8fafc);padding:10px 16px;">${detail}</td>
    </tr>`;
};

window.toggleDeptEnrolledDetail = function(studentId) {
    const row = document.getElementById('dedet-' + studentId);
```

```
    if (!row) return;
    const open = row.style.display !== 'table-row';
    row.style.display = open ? 'table-row' : 'none';
    const chev = document.getElementById('dechev-' + studentId);
    if (chev) chev.style.transform = open ? 'rotate(90deg)' : 'rotate(0deg)';
};

window.filterDeptEnrolledStudents = function(query) {
    const students = window._deptEnrolledStudents || [];
    const q = String(query || '').toLowerCase();
    const filtered = students.filter(s =>
        (s.name || '').toLowerCase().includes(q) ||
        (s.sid || '').toLowerCase().includes(q) ||
        (s.section || '').toLowerCase().includes(q) ||
        (s.year || '').toLowerCase().includes(q)
    );
    _deptEnrolledRender(filtered, 'No students match your search.');
};

(function patchDeptStatCards() {
    const origRenderDeptPage = window.renderDeptPage;
    if (!origRenderDeptPage) {
        document.addEventListener('DOMContentLoaded', patchDeptStatCards);
        return;
    }
    window.renderDeptPage = function(deptCode) {
        origRenderDeptPage(deptCode);
        const statsRow = document.getElementById('deptStatsRow');
        if (!statsRow) return;

        const wire = (idx, opts) => {
            const card = statsRow.children[idx];
            if (!card) return;
            card.style.cursor = 'pointer';
            card.title = opts.title;
            card.style.transition = 'box-shadow 0.2s, transform 0.15s';
            card.onmouseenter = () => { card.style.boxShadow = '0 4px 16px rgba(5,150,105,0.18)'; card.style.transform = 'translateY(-2px)'; };
            card.onmouseleave = () => { card.style.boxShadow = ''; card.style.transform = ''; };
            card.onclick = opts.onClick;
        };

        const scrollToEl = (el) => {
            if (!el) return;
            const box = el.closest('.card') || el.closest('.dept-table-card') || el.parentElement || el;
            box.scrollIntoView({ behavior: 'smooth', block: 'center' });
            const prev = box.style.boxShadow;
            box.style.transition = 'box-shadow 0.3s';
            box.style.boxShadow = '0 0 0 3px rgba(5,150,105,0.45)';
            setTimeout(() => { box.style.boxShadow = prev || ''; }, 1200);
        };

        wire(0, { title: "View all faculty in this department", arrow: 'View All',
            onClick: () => showDeptFullList(deptCode, 'teachers') });

        wire(1, { title: "View all subjects in this department", arrow: 'View All',
            onClick: () => showDeptFullList(deptCode, 'subjects') });

        wire(2, { title: 'View all enrolled students', arrow: 'View All',
            onClick: () => showDeptFullList(deptCode, 'enrolled') });

        wire(3, { title: 'View this department\'s evaluation feedback', arrow: 'View Feedback',
            onClick: () => { if (typeof goToDeptFeedback === 'function') goToDeptFeedback(deptCode); } });
    };
})();

let _studentSearchQuery = '';
let _studentDeptFilter = '';

// STUDENTS TABLE ORDER: by section, then last name (byName), then ID.
function _stuSectionLabel(st) {
    if (typeof _sectionLabel === 'function') return _sectionLabel(st);
    return ((st.course || '') + ' ' + (st.year || '') + ' ' + (st.section || '')).trim();
}
window.studentListCompare = function(a, b) {
    const opts = { sensitivity: 'base', numeric: true };
    return _stuSectionLabel(a).localeCompare(_stuSectionLabel(b), undefined, opts)
        || byName(a, b)
        || String(a.sid || '').localeCompare(String(b.sid || ''), undefined, opts);
};

// "NO DEPT" FILTER: blank or unknown department codes. Used by every pill
// count and table filter so they agree.
window.deptFilterKey = function(dept) {
    const cfg = (typeof getDepartments === 'function') ? getDepartments() : {};
    return (dept && cfg[dept]) ? dept : 'UNASSIGNED';
};

window.renderStudents = function(search) {
    if (search !== undefined) _studentSearchQuery = search;
    // Rebuild the department pills on every redraw so counts stay current.
    buildStudentDeptFilterBar();
    const query = _studentSearchQuery;
    const deptFilter = _studentDeptFilter;

    const allStudents = getData('students', []).filter(s => !s.deleted);
    const DEPT_CONFIG = getDepartments();

    // Search rules: parseStudentSearch / studentMatchesTerm (admin-students.js).
    const terms = parseStudentSearch(query || '');
    const filtered = allStudents.filter(s => {
        const matchSearch = terms.every(t => studentMatchesTerm(s, t));
        const matchDept = !deptFilter || deptFilterKey(s.dept) === deptFilter;
        return matchSearch && matchDept;
    });
    // By section, then last name A to Z - see studentListCompare above.
```

```
    filtered.sort(studentListCompare);

    const tbody = document.getElementById('studentsTbody');
    if (!tbody) return;

    if (!filtered.length) {
        tbody.innerHTML = `<tr><td colspan="8" style="text-align:center;padding:32px;color:var(--muted);">No students found.</td></tr>`;
        return;
    }

    const groups = {};
    filtered.forEach(s => {
        const dept = s.dept || 'UNASSIGNED';
        if (!groups[dept]) groups[dept] = [];
        groups[dept].push(s);
    });

    const deptOrder = Object.keys(DEPT_CONFIG);
    const sortedDepts = [
        ...deptOrder.filter(d => groups[d]),
        ...Object.keys(groups).filter(d => !deptOrder.includes(d))
    ];

    let html = '';
    sortedDepts.forEach(dept => {
        const students = groups[dept];
        const cfg = DEPT_CONFIG[dept];
        const deptLabel = cfg ? `${cfg.short} — ${cfg.name}` : (dept === 'UNASSIGNED' ? 'No Department Assigned' : dept);
        const colorStyle = dept === 'UNASSIGNED' ? 'background:#f1f5f9;color:#475569;' : '';

        html += `
        <tr class="dept-group-header-row">
            <td colspan="8">
                <div class="dept-group-header" style="${colorStyle}">
                    <svg width="13" height="13" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M17 21v-2a4 4 0 00-4-
4H5a4 4 0 00-4 4v2"/><circle cx="9" cy="7" r="4"/></svg>
                    <span>${escapeHtml(deptLabel)}</span>
                    <span class="dept-group-count">${students.length} student${students.length !== 1 ? 's' : ''}</span>
                </div>
            </td>
        </tr>`;

        students.forEach(s => {
            html += `<tr class="dept-group-student-row">
                <td><span style="font-family:'JetBrains Mono',monospace;font-weight:600;">${escapeHtml(s.sid)}</span></td>
                <td><strong>${escapeHtml(listName(s))}</strong></td>
                <td style="font-size:0.78rem;max-width:160px;white-space:normal;line-height:1.3;">${escapeHtml(s.course || '—')}</td>
                <td>${escapeHtml(s.year)}</td>
                <td>Sec ${escapeHtml(s.section)}</td>
                <td><span class="dept-tag-inline">${escapeHtml(s.dept || '—')}</span></td>
                <td><span class="badge ${s.status === 'active' ? 'badge-success' : 'badge-danger'}">${s.status}</span></td>
                <td><div class="td-actions">
                    <button class="btn btn-ghost btn-icon btn-sm" title="Edit" onclick="openEditStudentModal('${s.id}')"><svg width="14" height="14"
fill="none" stroke="var(--primary)" stroke-width="2" viewBox="0 0 24 24"><path d="M11 4H4a2 2 0 00-2 2v14a2 2 0 002 2h14a2 2 0 002-2v-7"/><path d="M1
8.5 2.5a2.121 2.121 0 013 3L12 15l-4 1 1-4 9.5-9.5z"/></svg></button>
                    <button class="btn btn-ghost btn-icon btn-sm" title="Reset Password" onclick="openResetPass('${s.id}')"><svg width="14" height="1
4" fill="none" stroke="var(--warning)" stroke-width="2" viewBox="0 0 24 24"><rect x="3" y="11" width="18" height="11" rx="2"/><path d="M7 11V7a5 5 0
0110 0v4"/></svg></button>
                    <button class="btn btn-ghost btn-icon btn-sm" title="Toggle Status" onclick="toggleStudentStatus('${s.id}')"><svg width="14" heig
ht="14" fill="none" stroke="${s.status === 'active' ? 'var(--muted)' : 'var(--success)'}" stroke-width="2" viewBox="0 0 24 24"><path d="M18.36 6.64a9
9 0 11-12.73 0"/><line x1="12" y1="2" x2="12" y2="12"/></svg></button>
                    <button class="btn btn-ghost btn-icon btn-sm" title="Delete" onclick="deleteStudent('${s.id}')"><svg width="14" height="14" fill
="none" stroke="var(--danger)" stroke-width="2" viewBox="0 0 24 24"><polyline points="3 6 5 6 21 6"/><path d="M19 6l-1 14a2 2 0 01-2 2H8a2 2 0 01-2-2
L5 6"/></svg></button>
                </div></td>
            </tr>`;
        });
    });

    tbody.innerHTML = html;
};

window.filterStudentsByDept = function(deptCode) {
    _studentDeptFilter = deptCode;
    renderStudents();

    document.querySelectorAll('.student-dept-filter-btn').forEach(btn => {
        btn.classList.toggle('active', btn.dataset.dept === deptCode);
    });
};

window.buildStudentDeptFilterBar = function() {
    const container = document.getElementById('studentDeptFilterBar');
    if (!container) return;
    const DEPT_CONFIG = getDepartments();
    const allStudents = getData('students', []).filter(s => !s.deleted);

    const counts = {};
    allStudents.forEach(s => { const k = deptFilterKey(s.dept); counts[k] = (counts[k] || 0) + 1; });

    // Viewed department lost its last student: fall back to All.
    if (_studentDeptFilter && !counts[_studentDeptFilter]) _studentDeptFilter = '';
    // Highlight the pill matching the current filter.
    const on = code => (_studentDeptFilter === code ? ' active' : '');

    let html = `<button class="student-dept-filter-btn${on('')}" data-dept="" onclick="filterStudentsByDept('')">
        All <span class="dept-filter-count">${allStudents.length}</span>
    </button>`;

    Object.entries(DEPT_CONFIG).forEach(([code, cfg]) => {
        if (counts[code]) {
            html += `<button class="student-dept-filter-btn${on(code)}" data-dept="${code}" onclick="filterStudentsByDept('${code}')">
                ${escapeHtml(cfg.short)} <span class="dept-filter-count">${counts[code]}</span>
            </button>`;
        }
```

```
    });

    if (counts['UNASSIGNED']) {
        html += `<button class="student-dept-filter-btn${on('UNASSIGNED')}" data-dept="UNASSIGNED" onclick="filterStudentsByDept('UNASSIGNED')">
            No Dept <span class="dept-filter-count">${counts['UNASSIGNED']}</span>
        </button>`;
    }

    container.innerHTML = html;
};

const _origShowPage = window.showPage;
window.showPage = function(page) {
    _origShowPage(page);
    if (page === 'students') {
        _studentSearchQuery = '';
        _studentDeptFilter = '';
        setTimeout(() => {
            buildStudentDeptFilterBar();
            renderStudents();
        }, 0);
    }
    if (page === 'teachers') {
        _teacherDeptFilter = '';
        _supervisorDeptFilter = '';
        setTimeout(() => {
            buildTeacherDeptFilterBar();
            buildSupervisorDeptFilterBar();
            const input = document.getElementById('teacherSearchInput');
            if (input) input.value = '';
            const supInput = document.getElementById('supervisorSearchInput');
            if (supInput) supInput.value = '';
            renderTeachers('');
        }, 0);
    }
    if (page === 'subjects') {
        if (typeof _subjectDeptFilter !== 'undefined') _subjectDeptFilter = '';
        window._subjectDeptFilter = '';
        const searchInput = document.querySelector('#page-subjects input[type="text"]');
        if (searchInput) searchInput.value = '';
        setTimeout(() => {
            if (typeof buildSubjectDeptPills === 'function') buildSubjectDeptPills();
            if (typeof renderSubjects === 'function') renderSubjects('');
        }, 0);
    }
};

const DM_EMOJIS = ['🎓','📚','🏛️','💻','⚕️','⚖️','🏗️','🌾','🔬','🎨','🎭','📐','🧬','🏥','👩‍💼','📊','🛠️','🌍','✈️','🏋️'];

let _dmLayout = localStorage.getItem('deptManageLayout') || 'grid';

(function injectDMStyles() {
    if (document.getElementById('_dmStyles')) return;
    const s = document.createElement('style');
    s.id = '_dmStyles';
    s.textContent = `
    .dm-layout-toggle { border: 1px solid var(--border) !important; padding: 5px 8px !important; }
    .dm-layout-toggle.active { background: var(--primary) !important; color: #fff !important; border-color: var(--primary) !important; }

    .dmc-del-confirm {
        position: absolute; bottom: 0; left: 0; right: 0;
        overflow: hidden; max-height: 0; transition: max-height .28s cubic-bezier(.4,0,.2,1);
        background: rgba(20,5,5,0.92); border-radius: 0 0 14px 14px; z-index: 10;
        display: flex; flex-direction: column; align-items: center; justify-content: center;
        gap: 8px; padding: 0 16px;
    }
    .dmc-del-confirm.open { max-height: 100px; padding: 12px 16px; }
    .dmc-del-msg { font-size: 0.75rem; color: #fca5a5; text-align: center; margin: 0; line-height: 1.4; }
    .dmc-del-msg strong { color: #fff; }
    .dmc-del-msg span { font-size: 0.68rem; opacity: 0.75; }
    .dmc-del-actions { display: flex; gap: 8px; }
    .dmc-del-cancel { padding: 4px 14px; background: rgba(255,255,255,0.12); color: #fff; border: 1px solid rgba(255,255,255,0.25); border-radius: 6p
x; font-size: 0.72rem; cursor: pointer; transition: background .12s; font-family: inherit; }
    .dmc-del-cancel:hover { background: rgba(255,255,255,0.22); }
    .dmc-del-confirm-btn { padding: 4px 14px; background: #dc2626; color: #fff; border: none; border-radius: 6px; font-size: 0.72rem; font-weight: 70
0; cursor: pointer; transition: background .12s; font-family: inherit; }
    .dmc-del-confirm-btn:hover { background: #b91c1c; }

    .dml-del-wrap { border-top: 1px solid #fecaca; background: #fef2f2; }

    .dept-manage-list { display: flex; flex-direction: column; gap: 0; border: 1px solid var(--border); border-radius: 10px; overflow: hidden; backgr
ound: #fff; }
    .dml-row {
        display: flex; align-items: center; gap: 12px; padding: 12px 16px;
        border-bottom: 1px solid var(--border); position: relative; transition: background .12s;
    }
    .dml-row:last-child { border-bottom: none; }
    .dml-row:hover { background: #f8fafc; }
    .dml-icon {
        width: 40px; height: 40px; flex-shrink: 0; border-radius: 9px;
        display: flex; align-items: center; justify-content: center; font-size: 1.15rem;
        background: var(--primary-light); overflow: hidden;
    }
    .dml-icon img { width: 100%; height: 100%; object-fit: cover; border-radius: 8px; }
    .dml-info { flex: 1; min-width: 0; }
    .dml-name { font-size: 0.85rem; font-weight: 700; }
    .dml-meta { font-size: 0.72rem; color: var(--muted-foreground); display: flex; gap: 10px; flex-wrap: wrap; margin-top: 2px; }
    .dml-code { font-family: 'JetBrains Mono', monospace; font-weight: 700; color: var(--primary); }
    .dml-badge { display: inline-block; font-size: 0.62rem; font-weight: 700; padding: 2px 7px; border-radius: 20px; background: #e5e7eb; color: #6b7
280; }
    .dml-badge.custom { background: #dbeafe; color: #1d4ed8; }
    .dml-stats { display: flex; gap: 16px; }
    .dml-stat { text-align: center; }
    .dml-stat-val { font-size: 0.85rem; font-weight: 700; display: block; }
    .dml-stat-label { font-size: 0.65rem; color: var(--muted-foreground); }
```

```
    .dml-actions { display: flex; gap: 6px; flex-shrink: 0; }
    .dml-del-wrap { border-top: 1px solid #fecaca; background: #fef2f2; }

    .edm-icon-prev {
        width: 52px; height: 52px; border-radius: 10px; background: var(--primary-light);
        display: flex; align-items: center; justify-content: center; font-size: 1.8rem;
        border: 2px dashed var(--border); flex-shrink: 0; overflow: hidden;
    }
    .edm-icon-prev img { width: 100%; height: 100%; object-fit: cover; border-radius: 9px; }
    .edm-emoji-btn {
        width: 28px; height: 28px; border-radius: 5px; font-size: 0.9rem;
        display: flex; align-items: center; justify-content: center;
        cursor: pointer; border: 1px solid transparent; transition: background .1s;
    }
    .edm-emoji-btn:hover, .edm-emoji-btn.sel { background: var(--primary-light); border-color: var(--primary); }
    .edm-upload-lbl {
        display: inline-flex; align-items: center; gap: 5px; padding: 4px 10px;
        border: 1px solid var(--border); border-radius: 6px; font-size: 0.73rem;
        cursor: pointer; background: #fff; margin-top: 6px; transition: background .15s;
    }
    .edm-upload-lbl:hover { background: var(--primary-light); }

    .dm-color-swatch {
        width: 22px; height: 22px; border-radius: 50%; cursor: pointer;
        border: 2px solid transparent; transition: transform .1s, border-color .1s;
        flex-shrink: 0;
    }
    .dm-color-swatch:hover { transform: scale(1.18); }
    .dm-color-swatch.sel { border-color: #1e293b; transform: scale(1.18); box-shadow: 0 0 0 2px #fff inset; }
    `;
    document.head.appendChild(s);
})();

const DM_COLORS = [
    { hex: '#3b82f6', label: 'Blue' },
    { hex: '#8b5cf6', label: 'Violet' },
    { hex: '#ec4899', label: 'Pink' },
    { hex: '#10b981', label: 'Emerald' },
    { hex: '#f59e0b', label: 'Amber' },
    { hex: '#ef4444', label: 'Red' },
    { hex: '#06b6d4', label: 'Cyan' },
    { hex: '#f97316', label: 'Orange' },
    { hex: '#6366f1', label: 'Indigo' },
    { hex: '#14b8a6', label: 'Teal' },
    { hex: '#84cc16', label: 'Lime' },
    { hex: '#64748b', label: 'Slate' },
    { hex: '#9ca3af', label: 'Steel (CCJS)' },
    { hex: '#eab308', label: 'Gold (CCIS)' },
    { hex: '#059669', label: 'Green (CON)' },
    { hex: '#ea580c', label: 'Burnt Orange (CEA)' },
    { hex: '#0ea5e9', label: 'Sky (COM)' },
    { hex: '#be123c', label: 'Crimson (CAT)' },
];

let _dmSelectedColor = DM_COLORS[0].hex;

function _initDMColorPicker() {
    const picker = document.getElementById('dmColorPicker');
    if (!picker || picker.dataset.init) return;
    picker.dataset.init = '1';
    picker.innerHTML = DM_COLORS.map((c, i) =>
        `<span class="dm-color-swatch${i === 0 ? ' sel' : ''}" style="background:${c.hex};" title="${c.label}" onclick="_dmPickColor('${c.hex}', thi
s)"></span>`
    ).join('');
    _dmSelectedColor = DM_COLORS[0].hex;
    const hiddenInput = document.getElementById('dmColor');
    if (hiddenInput) hiddenInput.value = _dmSelectedColor;
}

window._dmPickColor = function(hex, el) {
    _dmSelectedColor = hex;
    document.querySelectorAll('#dmColorPicker .dm-color-swatch').forEach(b => b.classList.remove('sel'));
    if (el) el.classList.add('sel');
    const hiddenInput = document.getElementById('dmColor');
    if (hiddenInput) hiddenInput.value = hex;
};

window._resizeDeptImage = function(file, cb) {
    const reader = new FileReader();
    reader.onload = function(ev) {
        const img = new Image();
        img.onload = function() {
            const MAX = 128;
            let w = img.width, h = img.height;
            if (w >= h) { if (w > MAX) { h = Math.round(h * MAX / w); w = MAX; } }
            else        { if (h > MAX) { w = Math.round(w * MAX / h); h = MAX; } }
            try {
                const canvas = document.createElement('canvas');
                canvas.width = w; canvas.height = h;
                canvas.getContext('2d').drawImage(img, 0, 0, w, h);
                let out;
                if (file.type === 'image/png') {
                    out = canvas.toDataURL('image/png');
                    if (out.length > 150000) out = canvas.toDataURL('image/jpeg', 0.85);
                } else {
                    out = canvas.toDataURL('image/jpeg', 0.85);
                }
                cb(out);
            } catch (e) { cb(ev.target.result); }
        };
        img.onerror = function() { cb(ev.target.result); };
        img.src = ev.target.result;
    };
    reader.readAsDataURL(file);
};
```

```

window.dmHandleImageUpload = function(e) {
    const file = e.target.files[0];
    if (!file) return;
    _resizeDeptImage(file, function(data) {
        const iv = document.getElementById('dmIcon');
        if (iv) iv.value = data;
        const prev = document.getElementById('dmIconPreview');
        if (prev) prev.innerHTML = `<img src="${data}" style="width:100%;height:100%;object-fit:cover;border-radius:7px;">`;
        document.querySelectorAll('#dmEmojiPicker .dm-emoji-btn').forEach(b => b.classList.remove('sel'));
    });
};

window.setDeptManageLayout = function(layout) {
    _dmLayout = layout;
    localStorage.setItem('deptManageLayout', layout);
    document.getElementById('dmLayoutGrid')?.classList.toggle('active', layout === 'grid');
    document.getElementById('dmLayoutList')?.classList.toggle('active', layout === 'list');
    renderDeptManagePage();
};

window.showDeptManagePage = function() {
    document.querySelectorAll('.page').forEach(p => p.classList.remove('active'));
    document.querySelectorAll('.nav-item').forEach(n => n.classList.remove('active'));
    const pageEl = document.getElementById('page-deptManage');
    if (pageEl) pageEl.classList.add('active');
    const navEl = document.getElementById('nav-deptManage');
    if (navEl) navEl.classList.add('active');
    document.getElementById('dmLayoutGrid')?.classList.toggle('active', _dmLayout === 'grid');
    document.getElementById('dmLayoutList')?.classList.toggle('active', _dmLayout === 'list');
    _initDMColorPicker();
    renderDeptManagePage();
    closeSidebar();
};

window.renderDeptManagePage = function() {
    const DEPT_CONFIG = getDepartments();
    const allStudents = getData('students', []).filter(s => !s.deleted);
    const allTeachers = getData('teachers', []).filter(t => !t.deleted);
    const allSubjects = getData('subjects', []);

    const deptStats = {};
    Object.keys(DEPT_CONFIG).forEach(code => {
        deptStats[code] = {
            students: allStudents.filter(s => s.dept === code).length,
            teachers: allTeachers.filter(t => t.dept === code).length,
            subjects: allSubjects.filter(s => s.dept === code).length
        };
    });

    const grid = document.getElementById('deptManageGrid');
    if (!grid) return;

    if (_dmLayout === 'list') {
        _renderDeptList(grid, DEPT_CONFIG, deptStats);
    } else {
        _renderDeptGrid(grid, DEPT_CONFIG, deptStats);
    }
};

function _deptIconHtmlDM(icon, size, cls) {
    const isImg = icon && (icon.includes('.jpg') || icon.includes('.png') || icon.includes('.jpeg') || icon.startsWith('data:'));
    if (isImg) return `<div class="${cls}" style="width:${size}px;height:${size}px;"><img src="${icon}" style="width:100%;height:100%;object-fit:cove
r;border-radius:${size/4}px;"></div>`;
    return `<div class="${cls}" style="width:${size}px;height:${size}px;font-size:${Math.round(size*0.52)}px;">${icon || '🏛️'}</div>`;
}

function _renderDeptGrid(grid, DEPT_CONFIG, deptStats) {
    grid.className = 'dept-manage-grid';
    grid.innerHTML = Object.entries(DEPT_CONFIG).map(([code, cfg]) => {
        const stats = deptStats[code] || { students: 0, teachers: 0, subjects: 0 };
        const isDefault = !cfg.isCustom;
        const isImg = cfg.icon && (cfg.icon.includes('.jpg') || cfg.icon.includes('.png') || cfg.icon.includes('.jpeg') || cfg.icon.startsWith('dat
a:'));
        const iconHtml = isImg
            ? `<img src="${cfg.icon}" width="32" height="32" style="border-radius:6px;object-fit:cover;">`
            : `<span style="font-size:1.4rem;">${cfg.icon || '🏛️'}</span>`;
        const _darken = (hex, amt) => {
            let c = hex.replace('#','');
            if (c.length===3) c=c.split('').map(x=>x+x).join('');
            const n=parseInt(c,16),r=Math.max(0,(n>>16)-amt),g=Math.max(0,((n>>8)&0xff)-amt),b=Math.max(0,(n&0xff)-amt);
            return '#'+[r,g,b].map(v=>v.toString(16).padStart(2,'0')).join('');
        };
        const _customBg = (cfg.isCustom && cfg.color)
            ? `background:linear-gradient(135deg,${_darken(cfg.color,50)} 0%,${cfg.color} 100%);`
            : '';
        return `
        <div class="dept-manage-card ${cfg.colorClass}" style="position:relative;overflow:hidden;${_customBg}">
            <div class="dept-manage-card-header">
                <div class="dept-manage-icon">${iconHtml}</div>
                <div class="dept-manage-labels">
                    <div class="dept-manage-name">${escapeHtml(cfg.name)}</div>
                </div>
            </div>
            <div class="dept-manage-desc">${escapeHtml(cfg.desc || '')}</div>
            <div class="dept-manage-stats">
                <div class="dept-manage-stat"><span class="dept-ms-val">${stats.teachers}</span><span class="dept-ms-label">Teachers</span></div>
                <div class="dept-manage-stat"><span class="dept-ms-val">${stats.students}</span><span class="dept-ms-label">Students</span></div>
                <div class="dept-manage-stat"><span class="dept-ms-val">${stats.subjects}</span><span class="dept-ms-label">Subjects</span></div>
            </div>
            <!-- Action bar: always visible, consistent across all cards -->
            <div class="dmc-action-bar">
                <button class="dmc-action-btn dmc-view-btn" onclick="showDeptPage('${code}')">
                    <svg width="12" height="12" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M1 12s4-8 11-8 11 8 1
1 8-4 8-11 8-11-8-11-8z"/><circle cx="12" cy="12" r="3"/></svg>
```

```
                    View
                </button>
                <button class="dmc-action-btn dmc-edit-btn2" onclick="openEditDeptModal('${code}')">
                    <svg width="12" height="12" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M11 4H4a2 2 0 00-2 2v
14a2 2 0 002 2h14a2 2 0 002-2v-7"/><path d="M18.5 2.5a2.121 2.121 0 013L12 15l-4 1 1-4 9.5-9.5z"/></svg>
                    Edit
                </button>
                <button class="dmc-action-btn dmc-del-btn" onclick="toggleDMDeleteConfirm('${code}')">
                    <svg width="12" height="12" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><polyline points="3 6 5 6 21
6"/><path d="M19 6l-1 14a2 2 0 01-2 2H8a2 2 0 01-2-2L5 6"/><path d="M10 11v6"/><path d="M14 11v6"/></svg>
                    Delete
                </button>
            </div>
            <div id="dmdelconfirm-${code}" class="dmc-del-confirm">
                <p class="dmc-del-msg">Delete <strong>${escapeHtml(cfg.name)}</strong>?<br><span>This cannot be undone.</span></p>
                <div class="dmc-del-actions">
                    <button class="dmc-del-cancel" onclick="toggleDMDeleteConfirm('${code}')">Cancel</button>
                    <button class="dmc-del-confirm-btn" onclick="executeDMDelete('${code}')">Yes, Delete</button>
                </div>
            </div>
        </div>`;
    }).join('');
}

function _renderDeptList(grid, DEPT_CONFIG, deptStats) {
    grid.className = 'dept-manage-list';
    grid.innerHTML = Object.entries(DEPT_CONFIG).map(([code, cfg]) => {
        const stats = deptStats[code] || { students: 0, teachers: 0, subjects: 0 };
        const isDefault = !cfg.isCustom;
        const isImg = cfg.icon && (cfg.icon.includes('.jpg') || cfg.icon.includes('.png') || cfg.icon.includes('.jpeg') || cfg.icon.startsWith('dat
a:'));
        const iconInner = isImg
            ? `<img src="${cfg.icon}" alt="">`
            : `<span style="font-size:1.15rem;">${cfg.icon || '🏛️'}</span>`;
        return `
        <div>
            <div class="dml-row">
                <div class="dml-icon">${iconInner}</div>
                <div class="dml-info">
                    <div class="dml-name">
                        ${escapeHtml(cfg.name)}
                        <span class="dml-badge${!isDefault ? ' custom' : ''}" style="margin-left:6px;">${isDefault ? 'Default' : 'Custom'}</span>
                    </div>
                    <div class="dml-meta">
                        <span class="dml-code">${escapeHtml(code)}</span>
                        <span>${escapeHtml(cfg.desc || '')}</span>
                    </div>
                </div>
                <div class="dml-stats">
                    <div class="dml-stat"><span class="dml-stat-val">${stats.teachers}</span><span class="dml-stat-label">Teachers</span></div>
                    <div class="dml-stat"><span class="dml-stat-val">${stats.students}</span><span class="dml-stat-label">Students</span></div>
                    <div class="dml-stat"><span class="dml-stat-val">${stats.subjects}</span><span class="dml-stat-label">Subjects</span></div>
                </div>
                <div class="dml-actions">
                    <button class="dml-action-btn dml-view" onclick="showDeptPage('${code}')" title="View">
                        <svg width="12" height="12" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M1 12s4-8 11-8 11
8 11 8-4 8-11 8-11-8-11-8z"/><circle cx="12" cy="12" r="3"/></svg>
                        View
                    </button>
                    <button class="dml-action-btn dml-edit" onclick="openEditDeptModal('${code}')" title="Edit">
                        <svg width="12" height="12" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M11 4H4a2 2 0 00-
2 2v14a2 2 0 002 2h14a2 2 0 002-2v-7"/><path d="M18.5 2.5a2.121 2.121 0 013L12 15l-4 1 1-4 9.5-9.5z"/></svg>
                        Edit
                    </button>
                    <button class="dml-action-btn dml-del" onclick="toggleDMDeleteConfirm('${code}')" title="Delete">
                        <svg width="12" height="12" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><polyline points="3 6 5 6
21 6"/><path d="M19 6l-1 14a2 2 0 01-2 2H8a2 2 0 01-2-2L5 6"/><path d="M10 11v6"/><path d="M14 11v6"/></svg>
                        Delete
                    </button>
                </div>
            </div>
            <div id="dmdelconfirm-${code}" style="display:none;padding:10px 16px;background:#fef2f2;border-top:1px solid #fecaca;border-radius:0 0 8p
x 8px;flex-direction:row;align-items:center;gap:10px;">
                <span style="font-size:0.8rem;color:#b91c1c;flex:1;">Delete <strong>${escapeHtml(cfg.name)}</strong>? This cannot be undone.</span>
                <button onclick="toggleDMDeleteConfirm('${code}')" style="padding:4px 12px;border-radius:5px;border:1px solid #fca5a5;background:tran
sparent;color:#b91c1c;cursor:pointer;font-size:0.78rem;">Cancel</button>
                <button onclick="executeDMDelete('${code}')" style="padding:4px 12px;border-radius:5px;border:none;background:#ef4444;color:#fff;curs
or:pointer;font-size:0.78rem;font-weight:600;">Delete</button>
            </div>
        </div>`;
    }).join('');
}

window.toggleDMDeleteConfirm = function(code) {
    const el = document.getElementById('dmdelconfirm-' + code);
    if (!el) return;
    el.classList.toggle('open');
};

window.executeDMDelete = function(code) {
    removeDepartment(code);   // has built-in guard for default depts + now syncs Firestore
    renderDeptManagePage();
    if (typeof buildStudentDeptFilterBar === 'function') buildStudentDeptFilterBar();
    if (typeof buildTeacherDeptFilterBar === 'function') buildTeacherDeptFilterBar();
    if (typeof buildSubjectDeptPills === 'function') buildSubjectDeptPills();
};

window.confirmRemoveDept = window.executeDMDelete; // backward compat alias

window.toggleDMCardMenu = function(code, e) {
    e.stopPropagation();
    const dd = document.getElementById('dmc-dd-' + code);
    if (!dd) return;
    const isOpen = dd.classList.contains('open');
    closeDMCardMenus();
```

```
    if (!isOpen) dd.classList.add('open');
};

window.closeDMCardMenus = function() {
    document.querySelectorAll('.dmc-dropdown.open').forEach(d => d.classList.remove('open'));
};

document.addEventListener('click', function() { closeDMCardMenus(); });

window.saveDeptFromManagePage = function() {
    const name  = (document.getElementById('dmName')?.value  || '').trim();
    const short = (document.getElementById('dmShort')?.value || '').trim() || name;
    const icon  = (document.getElementById('dmIcon')?.value  || '').trim();
    const desc  = (document.getElementById('dmDesc')?.value  || '').trim() || name;
    const color = (document.getElementById('dmColor')?.value || _dmSelectedColor || '#3b82f6').trim();

    const baseCode = short.toUpperCase().replace(/[^A-Z0-9]/g, '').substring(0, 8);

    if (!name) { showToast('Please fill in the Full Name.', 'error'); return; }
    if (!baseCode) { showToast('Short Name must contain letters or numbers.', 'error'); return; }

    const existing = getDepartments();
    let code = baseCode;
    let suffix = 2;
    while (existing[code]) { code = baseCode.substring(0, 7) + suffix; suffix++; }

    addDepartment(code, name, short, desc, icon, color);

    ['dmName','dmShort','dmDesc'].forEach(id => { const el = document.getElementById(id); if (el) el.value = ''; });
    document.getElementById('dmIcon').value = '';
    const prev = document.getElementById('dmIconPreview');
    if (prev) prev.innerHTML = `<svg width="18" height="18" fill="none" stroke="var(--muted)" stroke-width="1.5" viewBox="0 0 24 24"><rect x="3" y
="3" width="18" height="18" rx="3"/><circle cx="8.5" cy="8.5" r="1.5"/><polyline points="21 15 16 10 5 21"/></svg>`;
    _dmSelectedColor = DM_COLORS[0].hex;
    document.querySelectorAll('#dmColorPicker .dm-color-swatch').forEach((b, i) => b.classList.toggle('sel', i === 0));
    const colorInput = document.getElementById('dmColor');
    if (colorInput) colorInput.value = _dmSelectedColor;

    renderDeptManagePage();
    buildStudentDeptFilterBar();
};

window.openEditDeptModal = function(code) {
    const depts = getDepartments();
    const cfg = depts[code];
    if (!cfg) return;

    const isDefault = !cfg.isCustom;
    const isImg = cfg.icon && (cfg.icon.includes('.jpg') || cfg.icon.includes('.png') || cfg.icon.includes('.jpeg') || cfg.icon.startsWith('data:'));
    const previewHtml = isImg
        ? `<img src="${cfg.icon}" style="width:100%;height:100%;object-fit:cover;border-radius:9px;">`
        : `<svg width="22" height="22" fill="none" stroke="var(--muted)" stroke-width="1.5" viewBox="0 0 24 24"><rect x="3" y="3" width="18" height
="18" rx="3"/><circle cx="8.5" cy="8.5" r="1.5"/><polyline points="21 15 16 10 5 21"/></svg>`;

    const currentColor = cfg.color || '#3b82f6';

    document.getElementById('editDeptModalTitle').textContent = `Edit — ${cfg.short}`;
    document.getElementById('editDeptModalBody').innerHTML = `
        <input type="hidden" id="edm_code" value="${escapeHtml(code)}">
        <input type="hidden" id="edm_icon" value="${escapeHtml(cfg.icon || '')}">
        <input type="hidden" id="edm_color" value="${escapeHtml(currentColor)}">

        <!-- Icon / Image upload only -->
        <div style="display:flex;align-items:flex-start;gap:14px;margin-bottom:16px;">
            <div class="edm-icon-prev" id="edm_iconPreview">${previewHtml}</div>
            <div style="flex:1;">
                <div style="font-size:0.7rem;font-weight:700;text-transform:uppercase;letter-spacing:.04em;color:var(--muted-foreground);margin-botto
m:8px;">Department Icon</div>
                <label class="edm-upload-lbl">
                    <svg width="11" height="11" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M21 15v4a2 2 0 01-2 2
H5a2 2 0 01-2-2v-4"/><polyline points="17 8 12 3 7 8"/><line x1="12" y1="3" x2="12" y2="15"/></svg>
                    Upload Image
                    <input type="file" accept="image/*" style="display:none;" onchange="edmHandleImage(event)">
                </label>
                ${isImg ? `<button onclick="edmClearImage()" style="display:inline-flex;align-items:center;gap:4px;margin-left:8px;padding:4px 10px;b
order:1px solid var(--border);border-radius:6px;font-size:0.72rem;cursor:pointer;background:#fff;color:var(--danger);">
                    <svg width="10" height="10" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><line x1="18" y1="6" x2="6" y2
="18"/><line x1="6" y1="6" x2="18" y2="18"/></svg> Remove
                </button>` : ''}
            </div>
        </div>

        <!-- Color picker -->
        <div style="margin-bottom:14px;">
            <div style="font-size:0.7rem;font-weight:700;text-transform:uppercase;letter-spacing:.04em;color:var(--muted-foreground);margin-bottom:8p
x;">Department Color</div>
            <div style="display:flex;flex-wrap:wrap;gap:7px;" id="edm_colorPicker">
                ${DM_COLORS.map(c => `<span class="dm-color-swatch${currentColor === c.hex ? ' sel' : ''}" style="background:${c.hex};" title="${c.la
bel}" onclick="edmPickColor('${c.hex}', this)"></span>`).join('')}
            </div>
        </div>

        <!-- Fields -->
        <div class="form-row" style="margin-bottom:10px;">
            <div class="form-group">
                <label class="form-label">Code</label>
                <input class="form-control" value="${escapeHtml(code)}" ${isDefault ? 'disabled style="opacity:.5;"' : 'id="edm_newCode"'}>
                ${isDefault ? `<input type="hidden" id="edm_newCode" value="${escapeHtml(code)}"><span style="font-size:0.67rem;color:var(--muted-for
eground);">Default codes are locked.</span>` : ''}
            </div>
            <div class="form-group">
                <label class="form-label">Short Name <span style="color:var(--danger)">*</span></label>
                <input class="form-control" id="edm_short" value="${escapeHtml(cfg.short || code)}">
            </div>
        </div>
```

```
        <div class="form-group" style="margin-bottom:10px;">
            <label class="form-label">Full Name <span style="color:var(--danger)">*</span></label>
            <input class="form-control" id="edm_name" value="${escapeHtml(cfg.name)}">
        </div>
        <div class="form-group" style="margin-bottom:16px;">
            <label class="form-label">Description</label>
            <input class="form-control" id="edm_desc" value="${escapeHtml(cfg.desc || '')}">
        </div>
        <div style="display:flex;gap:8px;justify-content:flex-end;">
            <button class="btn btn-ghost" onclick="closeModal('editDeptModal')">Cancel</button>
            <button class="btn btn-primary" onclick="saveEditDept()">
                <svg width="12" height="12" fill="none" stroke="currentColor" stroke-width="2.5" viewBox="0 0 24 24" style="margin-right:3px;"><polyl
ine points="20 6 9 17 4 12"/></svg>
                Save Changes
            </button>
        </div>`;

    openModal('editDeptModal');
};

window.edmPickColor = function(hex, el) {
    document.getElementById('edm_color').value = hex;
    document.querySelectorAll('#edm_colorPicker .dm-color-swatch').forEach(b => b.classList.remove('sel'));
    if (el) el.classList.add('sel');
};

window.edmClearImage = function() {
    document.getElementById('edm_icon').value = '';
    const prev = document.getElementById('edm_iconPreview');
    if (prev) prev.innerHTML = `<svg width="22" height="22" fill="none" stroke="var(--muted)" stroke-width="1.5" viewBox="0 0 24 24"><rect x="3" y
="3" width="18" height="18" rx="3"/><circle cx="8.5" cy="8.5" r="1.5"/><polyline points="21 15 16 10 5 21"/></svg>`;
};

window.edmHandleImage = function(e) {
    const file = e.target.files[0];
    if (!file) return;
    _resizeDeptImage(file, function(data) {
        const iv = document.getElementById('edm_icon');
        if (iv) iv.value = data;
        const prev = document.getElementById('edm_iconPreview');
        if (prev) prev.innerHTML = `<img src="${data}" style="width:100%;height:100%;object-fit:cover;border-radius:9px;">`;
        document.querySelectorAll('.edm-emoji-btn').forEach(b => b.classList.remove('sel'));
    });
};

window.saveEditDept = function() {
    const originalCode = document.getElementById('edm_code').value;
    const name  = (document.getElementById('edm_name')?.value  || '').trim();
    const short = (document.getElementById('edm_short')?.value || '').trim();
    const desc  = (document.getElementById('edm_desc')?.value  || '').trim();
    const icon  = (document.getElementById('edm_icon')?.value  || '').trim();
    const color = (document.getElementById('edm_color')?.value || '#3b82f6').trim();

    if (!name || !short) { showToast('Name and Short Name are required.', 'error'); return; }

    const depts = getDepartments();
    const cfg   = depts[originalCode];
    if (!cfg) { showToast('Department not found.', 'error'); return; }

    const isDefault = !cfg.isCustom;

    const changes = [];
    if (cfg.name  !== name)  changes.push(`name → "${name}"`);
    if (cfg.short !== short) changes.push(`short → "${short}"`);
    if ((cfg.desc || '') !== desc) changes.push('desc updated');
    if (cfg.icon  !== icon)  changes.push('icon updated');
    if ((cfg.color || '') !== color) changes.push('color updated');

    const allDepts = getDepartments();
    allDepts[originalCode] = {
        ...allDepts[originalCode],
        name, short, desc, icon, color,
        isCustom: cfg.isCustom ? true : false
    };
    try {
        setData('departments', allDepts);
    } catch (e) {
        if (e && (e.name === 'QuotaExceededError' || e.code === 22)) {
            showToast('Image is too large to store. Try a smaller image.', 'error');
            return;
        }
        throw e;
    }

    refreshDeptConfig();
    addAudit('Edit Department', `${originalCode}: ${changes.length ? changes.join('; ') : 'no changes'}`);
    showToast(`"${short}" updated successfully!`, 'success');
    closeModal('editDeptModal');
    renderDeptManagePage();

    if (typeof currentDept !== 'undefined' && currentDept === originalCode) {
        renderDeptPage(originalCode);
    }
};

window.showFeedbackPage = function(deptFilter) {
    document.querySelectorAll('.page').forEach(p => p.classList.remove('active'));
    document.querySelectorAll('.nav-item').forEach(n => n.classList.remove('active'));
    const pageEl = document.getElementById('page-feedback');
    if (pageEl) pageEl.classList.add('active');
    const navEl = document.getElementById('nav-feedback');
    if (navEl) navEl.classList.add('active');
    _feedbackDeptFilter = deptFilter || '';
    _feedbackSearchQuery = '';
    const searchInput = document.querySelector('#page-feedback input[type="text"]');
```

```
    if (searchInput) searchInput.value = '';
    renderFeedbackPage();
    closeSidebar();
};

let _feedbackDeptFilter = '';
let _feedbackSearchQuery = '';

window._liveFeedbackEvals = function() {
    const teachers = getData('teachers', []);
    const subjects = getData('subjects', []);
    return getData('evaluations', []).filter(function (e) {
        if (e.evaluatorType === 'supervisor') return false;
        const sub = subjects.find(function (x) { return x.id === e.subjectId; });
        const tid = e.teacherId || (sub ? sub.teacherId : null);   // the rating's own teacher first
        const t = teachers.find(function (x) { return x.id === tid; });
        return !!t && !t.deleted;
    });
};

window._orphanFeedbackEvals = function() {
    const live = new Set(_liveFeedbackEvals().map(function (e) { return e.id; }));
    return getData('evaluations', []).filter(function (e) {
        return e.evaluatorType !== 'supervisor' && !live.has(e.id);
    });
};

window.deleteFeedbackEntry = async function(evalId) {
    if (!confirm('Delete this evaluation permanently?\n\nThe rating and comment are removed and the faculty average is recomputed without them. This
cannot be undone.')) return;
    const evals = getData('evaluations', [], { includeHidden: true });   // written back below
    const idx = evals.findIndex(function (e) { return e.id === evalId; });
    if (idx === -1) { showToast('Already removed.', 'info'); return; }
    evals.splice(idx, 1);
    // Local copy only; the document is deleted just below.
    setLocalData('evaluations', evals);
    try { await deleteDocFromFirestore('evaluations', evalId); }
    catch (e) { console.warn('Firestore delete failed:', e && e.message); }
    addAudit('Delete Evaluation', 'Evaluation ' + evalId + ' removed');
    showToast('Evaluation deleted.', 'success');
    renderFeedbackPage();
};

window.purgeOrphanFeedback = async function() {
    const orphans = _orphanFeedbackEvals();
    if (!orphans.length) { showToast('Nothing to clean up.', 'info'); return; }
    if (!confirm('Delete ' + orphans.length + ' evaluation' + (orphans.length !== 1 ? 's' : '')
        + ' belonging to removed faculty?\n\nThis cannot be undone. Print any Annex C you still need first.')) return;

    const ids = new Set(orphans.map(function (e) { return e.id; }));
    // Local copy only - each document is deleted individually below.
    setLocalData('evaluations', getData('evaluations', [], { includeHidden: true }).filter(function (e) { return !ids.has(e.id); }));
    for (const id of ids) {
        try { await deleteDocFromFirestore('evaluations', id); }
        catch (e) { console.warn('Firestore delete failed for', id, e && e.message); }
    }
    addAudit('Purge Evaluations', orphans.length + ' evaluation(s) of removed faculty deleted');
    showToast(orphans.length + ' evaluation(s) deleted.', 'success');
    renderFeedbackPage();
};

window.renderFeedbackPage = function() {
    const allEvals = _liveFeedbackEvals();
    const allTeachers = getData('teachers', []);
    const allStudents = getData('students', []);
    const allSubjects = getData('subjects', []);
    const DEPT_CONFIG = getDepartments();

    const feedbackDeptBar = document.getElementById('feedbackDeptBar');
    if (feedbackDeptBar) {
        const deptCounts = {};
        allEvals.forEach(ev => {
            const sub = allSubjects.find(s => s.id === ev.subjectId);
            const dept = sub ? sub.dept : null;
            if (dept) deptCounts[dept] = (deptCounts[dept] || 0) + 1;
        });

        let pillHtml = `<button class="student-dept-filter-btn ${!_feedbackDeptFilter ? 'active' : ''}" onclick="filterFeedback('','')">
            All <span class="dept-filter-count">${allEvals.length}</span>
        </button>`;
        Object.entries(DEPT_CONFIG).forEach(([code, cfg]) => {
            if (deptCounts[code]) {
                pillHtml += `<button class="student-dept-filter-btn ${_feedbackDeptFilter === code ? 'active' : ''}" data-dept="${code}" onclick="fil
terFeedback('${code}','')">
                    ${escapeHtml(cfg.short)} <span class="dept-filter-count">${deptCounts[code]}</span>
                </button>`;
            }
        });
        feedbackDeptBar.innerHTML = pillHtml;
    }

    const purgeBtn = document.getElementById('feedbackPurgeBtn');
    if (purgeBtn) {
        const n = _orphanFeedbackEvals().length;
        purgeBtn.style.display = n ? '' : 'none';
        purgeBtn.textContent = 'Clean up ' + n + ' from removed faculty';
        purgeBtn.title = n + ' evaluation(s) belong to faculty who have been deleted. '
            + 'They are hidden from this page. Click to delete them permanently.';
    }

    _renderFeedbackList();
};

window._renderFeedbackList = function() {
    const allEvals = _liveFeedbackEvals();
```

```
    const allTeachers = getData('teachers', []);
    const allStudents = getData('students', []);
    const allSubjects = getData('subjects', []);
    const DEPT_CONFIG = getDepartments();

    const q = _feedbackSearchQuery.toLowerCase();

    const filtered = allEvals.filter(ev => {
        const sub = allSubjects.find(s => s.id === ev.subjectId);
        const teacher = allTeachers.find(t => t.id === (ev.teacherId || (sub ? sub.teacherId : null)));
        const dept = sub ? sub.dept : '';

        const matchDept = !_feedbackDeptFilter || (dept || '').toUpperCase() === _feedbackDeptFilter.toUpperCase();
        const matchSearch = !q ||
            (teacher && teacher.name.toLowerCase().includes(q)) ||
            (sub && (sub.name.toLowerCase().includes(q) || sub.code.toLowerCase().includes(q))) ||
            (ev.comment || '').toLowerCase().includes(q);

        return matchDept && matchSearch;
    });

    const container = document.getElementById('feedbackList');
    if (!container) return;

    const statsEl = document.getElementById('feedbackStats');
    if (statsEl) {
        const withComments = filtered.filter(ev => ev.comment && ev.comment.trim()).length;
        statsEl.innerHTML = `
            <div class="feedback-stat-chip"><strong>${filtered.length}</strong> evaluation${filtered.length !== 1 ? 's' : ''} · ${withComments} with
comment${withComments !== 1 ? 's' : ''}</div>
        `;
    }

    if (!filtered.length) {
        container.innerHTML = `<div class="empty-state" style="padding:48px;text-align:center;">
            <svg width="40" height="40" fill="none" stroke="var(--muted)" stroke-width="1.5" viewBox="0 0 24 24" style="margin:0 auto 12px;display:bl
ock;opacity:0.4;"><path d="M21 15a2 2 0 01-2 2H7l-4 4V5a2 2 0 012-2h14a2 2 0 012 2z"/></svg>
            <p style="color:var(--muted);font-size:0.85rem;">No evaluations found${_feedbackDeptFilter ? ' for this department' : ''}.</p>
        </div>`;
        return;
    }

    const byTeacher = {};
    filtered.forEach(ev => {
        const sub = allSubjects.find(s => s.id === ev.subjectId);
        const teacherId = ev.teacherId || (sub ? sub.teacherId : null);
        const teacher = allTeachers.find(t => t.id === teacherId);
        const dept = sub ? sub.dept : 'Unknown';
        const key = teacherId || 'unknown';
        if (!byTeacher[key]) {
            byTeacher[key] = {
                teacher: teacher,
                dept: dept,
                comments: []
            };
        }
        byTeacher[key].comments.push({ ev, sub });
    });

    container.innerHTML = Object.entries(byTeacher).map(([tid, group]) => {
        const teacher = group.teacher;
        const cfg = DEPT_CONFIG[group.dept];
        const deptLabel = cfg ? cfg.short : group.dept;
        const colorClass = cfg ? cfg.colorClass : 'dept-custom';

        const anonMap = _anonRespondentMap(tid);

        const bySubject = {};
        group.comments.forEach(({ ev, sub }) => {
            const subKey = sub ? sub.id : 'unknown';
            if (!bySubject[subKey]) bySubject[subKey] = { sub, items: [] };
            bySubject[subKey].items.push({ ev, sub });
        });

        const subjectSections = Object.entries(bySubject).map(([subKey, sg]) => {
            const sub = sg.sub;
            const subLabel = sub ? `${escapeHtml(sub.code)} — ${escapeHtml(sub.name)}` : 'Unknown Subject';
            const scored = sg.items.filter(({ ev }) => typeof ev.totalScore === 'number');
            const subAvg = scored.length
                ? (scored.reduce((a, { ev }) => a + ev.totalScore, 0) / scored.length).toFixed(2)
                : null;

            const itemHtmls = sg.items.map(({ ev, sub }, idx) => {
                const ts = ev.timestamp ? new Date(ev.timestamp).toLocaleDateString('en-PH', { year: 'numeric', month: 'short', day: 'numeric' }) :
'';
                const ratingId = `fbrate-${tid}-${subKey}-${ev.id || idx}`;
                const scoreVal = (typeof ev.totalScore === 'number') ? ev.totalScore.toFixed(2) : null;
                const breakdownHtml = _buildFeedbackRatingBreakdown(ev);
                const respondent = `Respondent ${anonMap[ev.id] || '—'}`;
                return `<div class="feedback-comment-item">
                    <div class="feedback-comment-meta">
                        ${ts ? `<span class="feedback-timestamp">${ts}</span>` : ''}
                    </div>
                    ${ev.comment && ev.comment.trim()
                        ? `<div class="feedback-comment-text">"${escapeHtml(ev.comment)}"</div>`
                        : `<div class="feedback-comment-text feedback-no-comment">No written comment — rating only</div>`}
                    <div class="feedback-comment-from">
                        — ${respondent} <span class="feedback-anon-tag">anonymous</span>
                    </div>
                    <div class="feedback-item-actions">
                        ${scoreVal !== null ? `<button class="feedback-view-rating-btn" id="${ratingId}-btn" onclick="toggleFeedbackRating('${ratingI
d}', this)">View Rating</button>` : ''}
                        <button class="feedback-printone-btn" onclick="printOneEvaluation('${ev.id}')" title="Print this comment & rating">
                            <svg width="12" height="12" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><polyline points="6 9
6 2 18 2 18 9"/><path d="M6 18H4a2 2 0 01-2-2v-5a2 2 0 012-2h16a2 2 0 012 2v5a2 2 0 01-2 2h-2"/><rect x="6" y="14" width="12" height="8"/></svg>
```

```
                        Print
                    </button>
                    <button class="feedback-printone-btn" onclick="deleteFeedbackEntry('${ev.id}')" title="Delete this evaluation permanently" st
yle="color:var(--danger,#dc2626);border-color:#fecaca;">
                        <svg width="12" height="12" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><polyline points="3 6
5 6 21 6"/><path d="M19 6l-1 14a2 2 0 01-2 2H8a2 2 0 01-2-2L5 6"/><path d="M10 11v6M14 11v6"/><path d="M9 6V4a1 1 0 011-1h4a1 1 0 011 1v2"/></svg>
                        Delete
                    </button>
                </div>
                ${scoreVal !== null ? `
                <div class="feedback-rating-panel" id="${ratingId}" style="display:none;">
                    ${breakdownHtml}
                    <div class="feedback-rating-total">Overall Rating: <strong>${scoreVal}%</strong></div>
                </div>` : ''}
            </div>`;
        });

        const groupId = `fbsub-${tid}-${subKey}`;
        const firstHtml = itemHtmls[0] || '';
        const restHtml = itemHtmls.slice(1).join('');
        const hasMore = itemHtmls.length > 1;
        const listHtml = `
            ${firstHtml}
            ${hasMore ? `<div class="feedback-more" id="${groupId}-more" style="display:none;">${restHtml}</div>` : ''}`;

        return `
        <div class="feedback-subject-group">
            <div class="feedback-subject-header">
                <span class="feedback-subject-badge">${subLabel}</span>
                <span class="feedback-subject-stats">
                    ${sg.items.length} evaluation${sg.items.length !== 1 ? 's' : ''}${subAvg !== null ? ` · avg ${subAvg}%` : ''}
                </span>
            </div>
            <div class="feedback-comments-list">${listHtml}</div>
        </div>`;
    }).join('');

    const numSubjects = Object.keys(bySubject).length;
    const hasHidden = group.comments.length > numSubjects;

    return `
    <div class="feedback-teacher-card" id="fbteacher-${tid}">
        <div class="feedback-teacher-header">
            <div class="feedback-teacher-avatar ${colorClass}">
                ${teacher ? teacher.name.charAt(0).toUpperCase() : '?'}
            </div>
            <div class="feedback-teacher-info">
                <div class="feedback-teacher-name">${teacher ? escapeHtml(listName(teacher)) : 'Unknown Teacher'}</div>
                <div class="feedback-teacher-meta">
                    <span class="dept-tag-inline">${escapeHtml(deptLabel)}</span>
                    <span style="color:var(--muted);font-size:0.72rem;margin-left:6px;">${numSubjects} subject${numSubjects !== 1 ? 's' : ''} ·
${group.comments.length} evaluation${group.comments.length !== 1 ? 's' : ''}</span>
                </div>
            </div>
            <div class="feedback-header-actions">
                ${hasHidden ? `<button class="feedback-viewall-btn" id="fbteacher-${tid}-btn" onclick="toggleTeacherMore('${tid}', this, ${group.
comments.length})">View all ${group.comments.length} ratings</button>` : ''}
                <button class="feedback-print-btn" onclick="printTeacherFeedback('${tid}')" title="Print all ratings for this faculty">
                    <svg width="14" height="14" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><polyline points="6 9 6 2
18 2 18 9"/><path d="M6 18H4a2 2 0 01-2-2v-5a2 2 0 012-2h16a2 2 0 012 2v5a2 2 0 01-2 2h-2"/><rect x="6" y="14" width="12" height="8"/></svg>
                    Print All Ratings
                </button>
            </div>
        </div>
        ${subjectSections}
    </div>`;
}).join('');
};

function _teacherStudentEvals(teacherId) {
    const allSubjects = getData('subjects', []);
    return getData('evaluations', []).filter(e =>
        e.evaluatorType !== 'supervisor' &&
        ((allSubjects.find(s => s.id === e.subjectId) || {}).teacherId === teacherId || e.teacherId === teacherId)
    );
}

function _anonRespondentMap(teacherId) {
    const evals = _teacherStudentEvals(teacherId).slice().sort((a, b) =>
        String(a.timestamp || '').localeCompare(String(b.timestamp || '')) ||
        String(a.id || '').localeCompare(String(b.id || ''))
    );
    const map = {};
    evals.forEach((e, i) => { map[e.id] = i + 1; });
    return map;
}

window.printTeacherFeedback = function(teacherId) {
    const evals = _teacherStudentEvals(teacherId);
    if (!evals.length) {
        showToast('No ratings to print for this faculty yet.', 'warning');
        return;
    }

    if (evals.length <= 1) {
        _doPrintTeacherFeedback(teacherId, 'all');
        return;
    }

    const teacher = getData('teachers', []).find(t => t.id === teacherId);
    const allSubjects = getData('subjects', []);
    const anonMap = _anonRespondentMap(teacherId);
    const esc = (typeof escapeHtml === 'function') ? escapeHtml : (s => String(s));

    const options = evals
```

```
        .slice()
        .sort((a, b) => (anonMap[a.id] || 0) - (anonMap[b.id] || 0))
        .map(e => {
            const sub = allSubjects.find(s => s.id === e.subjectId);
            const subTxt = sub ? ` — ${esc(sub.code)}` : '';
            return `<option value="${esc(e.id)}">Respondent ${anonMap[e.id] || '—'}${subTxt}</option>`;
        }).join('');

    const overlay = document.createElement('div');
    overlay.className = 'fb-print-overlay';
    overlay.innerHTML = `
        <div class="fb-print-dialog">
            <h3>Print Ratings — ${esc(teacher ? teacher.name : 'Faculty')}</h3>
            <p>Choose what to print (responses are anonymous):</p>
            <select id="fbPrintScope" class="fb-print-select">
                <option value="all">All responses (${evals.length} evaluations)</option>
                ${options}
            </select>
            <div class="fb-print-actions">
                <button class="fb-print-cancel">Cancel</button>
                <button class="fb-print-go">Print</button>
            </div>
        </div>`;
    document.body.appendChild(overlay);

    const close = () => overlay.remove();
    overlay.addEventListener('click', (e) => { if (e.target === overlay) close(); });
    overlay.querySelector('.fb-print-cancel').onclick = close;
    overlay.querySelector('.fb-print-go').onclick = () => {
        const scope = overlay.querySelector('#fbPrintScope').value;
        close();
        _doPrintTeacherFeedback(teacherId, scope);
    };
};

window._doPrintTeacherFeedback = function(teacherId, scope) {
    const teacher = getData('teachers', []).find(t => t.id === teacherId);
    const allSubjects = getData('subjects', []);
    const anonMap = _anonRespondentMap(teacherId);
    let evals = _teacherStudentEvals(teacherId);
    if (scope && scope !== 'all') {
        evals = evals.filter(e => e.id === scope);
    }

    if (!evals.length) {
        showToast('No ratings to print for this selection.', 'warning');
        return;
    }

    const singleRespondent = (scope && scope !== 'all') ? (anonMap[scope] || null) : null;

    const cfg = (typeof getDepartments === 'function' ? getDepartments() : {})[teacher ? teacher.dept : ''] || {};
    const deptName = cfg.name || (teacher ? teacher.dept : '') || 'N/A';
    const now = new Date().toLocaleDateString('en-PH', { year: 'numeric', month: 'long', day: 'numeric' });

    const scored = evals.filter(e => typeof e.totalScore === 'number');
    const avgSET = scored.length ? (scored.reduce((a, b) => a + b.totalScore, 0) / scored.length).toFixed(2) : '—';

    const esc = (typeof escapeHtml === 'function') ? escapeHtml : (s => String(s));

    const bySubjectPrint = {};
    evals.forEach(ev => {
        const sub = allSubjects.find(s => s.id === ev.subjectId);
        const k = sub ? sub.id : 'unknown';
        if (!bySubjectPrint[k]) bySubjectPrint[k] = { sub, items: [] };
        bySubjectPrint[k].items.push(ev);
    });

    const oneEvalBlock = (ev, i) => {
        const sub = allSubjects.find(s => s.id === ev.subjectId);
        const ts = ev.timestamp ? new Date(ev.timestamp).toLocaleDateString('en-PH', { year: 'numeric', month: 'short', day: 'numeric' }) : '—';
        const rows = _extractRatingRows(ev);
        const scoreVal = (typeof ev.totalScore === 'number') ? ev.totalScore.toFixed(2) + '%' : '—';
        const commentHtml = (ev.comment && ev.comment.trim())
            ? '"' + esc(ev.comment) + '"'
            : '<em style="color:#777;">No written comment — rating only</em>';
        const questionRows = rows.length
            ? rows.map(r => `<tr><td>${esc(String(r.label))}</td><td style="text-align:center;font-weight:600;">${esc(String(r.score))}</td></tr>`).j
oin('')
            : `<tr><td colspan="2" style="color:#777;font-style:italic;">Per-question breakdown wasn't recorded — overall rating only.</td></tr>`;
        return `
        <div class="ev-block">
            <div class="ev-head">
                <span class="ev-num">#${i + 1}</span>
                <span class="ev-subj">Respondent ${anonMap[ev.id] || '—'} <em style="font-weight:400;color:#777;">(anonymous)</em></span>
                <span class="ev-date">${ts}</span>
            </div>
            <table class="q-table">
                <thead><tr><th>Question / Criterion</th><th style="width:90px;text-align:center;">Score</th></tr></thead>
                <tbody>${questionRows}</tbody>
                <tfoot><tr><td style="text-align:right;font-weight:700;">Overall Rating</td><td style="text-align:center;font-weight:700;">${scoreVa
l}</td></tr></tfoot>
            </table>
            <div class="ev-comment"><strong>Comment:</strong> ${commentHtml}</div>
            ${typeof preparedReviewedHtml === 'function' ? preparedReviewedHtml(24) : ''}
        </div>`;
    };

    const evalBlocks = Object.values(bySubjectPrint).map(sg => {
        const sub = sg.sub;
        const subLabel = sub ? esc(sub.code) + ' — ' + esc(sub.name) : 'Unknown Subject';
        const scoredS = sg.items.filter(e => typeof e.totalScore === 'number');
        const subAvg = scoredS.length ? (scoredS.reduce((a, b) => a + b.totalScore, 0) / scoredS.length).toFixed(2) + '%' : '—';
        const blocks = sg.items.map((ev, i) => oneEvalBlock(ev, i)).join('');
        return `
```

```
        <div class="subj-section">
            <div class="subj-head">
                <span class="subj-name">${subLabel}</span>
                <span class="subj-meta">${sg.items.length} evaluation${sg.items.length !== 1 ? 's' : ''} · avg ${subAvg}</span>
            </div>
            ${blocks}
        </div>`;
    }).join('');

    const html = `<!DOCTYPE html><html><head><meta charset="utf-8"><title>Ratings — ${esc(teacher ? teacher.name : 'Faculty')}</title>
    <style>
        * { box-sizing: border-box; }
        body { font-family: 'Segoe UI', Arial, sans-serif; color: #1a1a1a; margin: 32px; font-size: 12px; }
        h1 { font-size: 18px; margin: 0 0 4px; }
        .sub { color: #555; font-size: 12px; margin: 0 0 16px; }
        .info { border: 1px solid #ccc; border-radius: 6px; padding: 12px 14px; margin-bottom: 14px; background: #f8f9fa; }
        .info div { margin: 2px 0; }
        .info strong { display: inline-block; min-width: 150px; }
        .summary { display: flex; gap: 24px; margin: 10px 0 20px; }
        .summary .box { border: 1px solid #ccc; border-radius: 6px; padding: 10px 16px; text-align: center; }
        .summary .box .n { font-size: 22px; font-weight: 800; }
        .summary .box .l { font-size: 10px; color: #555; text-transform: uppercase; }
        .ev-block { border: 1px solid #ddd; border-radius: 6px; padding: 12px; margin-bottom: 14px; page-break-inside: avoid; }
        .subj-section { margin-bottom: 18px; }
        .subj-head { display: flex; justify-content: space-between; align-items: baseline; gap: 10px; background: #eef2f5; border-left: 4px solid #05
9669; padding: 8px 12px; margin-bottom: 10px; border-radius: 4px; }
        .subj-name { font-weight: 800; font-size: 13px; }
        .subj-meta { font-size: 11px; color: #555; }
        .ev-head { display: flex; gap: 10px; align-items: baseline; margin-bottom: 8px; border-bottom: 1px solid #eee; padding-bottom: 6px; }
        .ev-num { font-weight: 800; color: #059669; }
        .ev-subj { font-weight: 700; flex: 1; }
        .ev-date { color: #777; font-size: 11px; }
        .q-table { width: 100%; border-collapse: collapse; margin-bottom: 8px; }
        .q-table th, .q-table td { border: 1px solid #ddd; padding: 5px 8px; font-size: 11px; text-align: left; }
        .q-table thead th { background: #eef2f5; }
        .q-table tfoot td { background: #f8f9fa; }
        .ev-comment { font-size: 11px; margin: 6px 0 2px; font-style: italic; }
        .ev-from { font-size: 11px; color: #555; text-align: right; }
        @media print { body { margin: 12px; } .noprint { display: none; } }
        .noprint { text-align: center; margin-bottom: 16px; }
        .noprint button { padding: 8px 20px; font-size: 13px; border: none; border-radius: 6px; background: #059669; color: #fff; cursor: pointer; }
    </style></head><body>
        <div class="noprint"><button onclick="window.print()">🖨️ Print</button></div>
        <h1>Faculty Evaluation Ratings</h1>
        <p class="sub">Student Evaluation of Teachers (SET) — generated ${now}</p>
        <div class="info">
            <div><strong>Faculty Name:</strong> ${esc(teacher ? teacher.name : 'Unknown')}</div>
            <div><strong>Faculty ID:</strong> ${esc(teacher ? teacher.tid : '—')}</div>
            <div><strong>Department / College:</strong> ${esc(deptName)}</div>
            <div><strong>Faculty Type:</strong> ${esc(teacher ? (teacher.facultyType || 'regular') : '—')}</div>
            <div><strong>Scope:</strong> ${singleRespondent ? 'Single respondent — Respondent ' + singleRespondent + ' (anonymous)' : 'All respondent
s'}</div>
        </div>
        <div class="summary">
            <div class="box"><div class="n">${evals.length}</div><div class="l">Evaluations</div></div>
            <div class="box"><div class="n">${avgSET}${avgSET !== '—' ? '%' : ''}</div><div class="l">Average SET</div></div>
        </div>
        ${evalBlocks}
    </body></html>`;

    const w = window.open('', '_blank');
    if (!w) { showToast('Please allow pop-ups to print.', 'warning'); return; }
    w.document.write(html);
    w.document.close();
    w.focus();
    setTimeout(() => { try { w.print(); } catch (e) {} }, 350);

    if (typeof addAudit === 'function') addAudit('Print Ratings', `Printed ${singleRespondent ? 'one anonymous respondent' : 'all ratings'} for ${tea
cher ? teacher.name : teacherId}`);
};

window.printOneEvaluation = function(evalId) {
    const ev = getData('evaluations', []).find(e => e.id === evalId);
    if (!ev) { showToast('Could not find that evaluation.', 'warning'); return; }

    const allSubjects = getData('subjects', []);
    const sub = allSubjects.find(s => s.id === ev.subjectId);
    const teacher = getData('teachers', []).find(t => t.id === (ev.teacherId || (sub ? sub.teacherId : null)));
    const esc = (typeof escapeHtml === 'function') ? escapeHtml : (s => String(s));
    const respondentNo = _anonRespondentMap(teacher ? teacher.id : ev.teacherId)[ev.id] || '—';

    const cfg = (typeof getDepartments === 'function' ? getDepartments() : {})[teacher ? teacher.dept : ''] || {};
    const deptName = cfg.name || (teacher ? teacher.dept : '') || 'N/A';
    const now = new Date().toLocaleDateString('en-PH', { year: 'numeric', month: 'long', day: 'numeric' });
    const ts = ev.timestamp ? new Date(ev.timestamp).toLocaleDateString('en-PH', { year: 'numeric', month: 'short', day: 'numeric' }) : '—';

    const rows = _extractRatingRows(ev);
    const scoreVal = (typeof ev.totalScore === 'number') ? ev.totalScore.toFixed(2) + '%' : '—';
    const commentHtml = (ev.comment && ev.comment.trim())
        ? '"' + esc(ev.comment) + '"'
        : '<em style="color:#777;">No written comment — rating only</em>';
    const questionRows = rows.length
        ? rows.map(r => `<tr><td>${esc(String(r.label))}</td><td style="text-align:center;font-weight:600;">${esc(String(r.score))}</td></tr>`).join
('')
        : `<tr><td colspan="2" style="color:#777;font-style:italic;">Per-question breakdown wasn't recorded — overall rating only.</td></tr>`;

    const html = `<!DOCTYPE html><html><head><meta charset="utf-8"><title>Rating — Respondent ${respondentNo}</title>
    <style>
        * { box-sizing: border-box; }
        body { font-family: 'Segoe UI', Arial, sans-serif; color: #1a1a1a; margin: 32px; font-size: 12px; }
        h1 { font-size: 18px; margin: 0 0 4px; }
        .sub { color: #555; font-size: 12px; margin: 0 0 16px; }
        .info { border: 1px solid #ccc; border-radius: 6px; padding: 12px 14px; margin-bottom: 16px; background: #f8f9fa; }
        .info div { margin: 2px 0; }
        .info strong { display: inline-block; min-width: 150px; }
```

```
        .q-table { width: 100%; border-collapse: collapse; margin-bottom: 10px; }
        .q-table th, .q-table td { border: 1px solid #ddd; padding: 5px 8px; font-size: 11px; text-align: left; }
        .q-table thead th { background: #eef2f5; }
        .q-table tfoot td { background: #f8f9fa; }
        .cbox { border: 1px solid #ddd; border-radius: 6px; padding: 10px 12px; font-style: italic; font-size: 12px; }
        @media print { body { margin: 12px; } .noprint { display: none; } }
        .noprint { text-align: center; margin-bottom: 16px; }
        .noprint button { padding: 8px 20px; font-size: 13px; border: none; border-radius: 6px; background: #059669; color: #fff; cursor: pointer; }
    </style></head><body>
        <div class="noprint"><button onclick="window.print()">🖨️ Print</button></div>
        <h1>Student Evaluation — Single Record</h1>
        <p class="sub">Student Evaluation of Teachers (SET) — generated ${now}</p>
        <div class="info">
            <div><strong>Faculty:</strong> ${esc(teacher ? teacher.name : 'Unknown')} (${esc(teacher ? teacher.tid : '—')})</div>
            <div><strong>Department / College:</strong> ${esc(deptName)}</div>
            <div><strong>Subject:</strong> ${sub ? esc(sub.code) + ' — ' + esc(sub.name) : 'Unknown Subject'}</div>
            <div><strong>Respondent:</strong> Respondent ${respondentNo} <em style="color:#777;">(anonymous — CMO No. 19 §6.10)</em></div>
            <div><strong>Date:</strong> ${ts}</div>
        </div>
        <table class="q-table">
            <thead><tr><th>Question / Criterion</th><th style="width:90px;text-align:center;">Score</th></tr></thead>
            <tbody>${questionRows}</tbody>
            <tfoot><tr><td style="text-align:right;font-weight:700;">Overall Rating</td><td style="text-align:center;font-weight:700;">${scoreVal}</t
d></tr></tfoot>
        </table>
        <div class="cbox"><strong style="font-style:normal;">Comment:</strong> ${commentHtml}</div>
        ${typeof preparedReviewedHtml === 'function' ? preparedReviewedHtml() : ''}
    </body></html>`;

    const w = window.open('', '_blank');
    if (!w) { showToast('Please allow pop-ups to print.', 'warning'); return; }
    w.document.write(html);
    w.document.close();
    w.focus();
    setTimeout(() => { try { w.print(); } catch (e) {} }, 350);

    if (typeof addAudit === 'function') addAudit('Print Rating', `Printed one anonymous rating (Respondent ${respondentNo} → ${teacher ? teacher.name
: 'faculty'})`);
};

window.SET_QUESTIONS = [
    { id: 'q1',  sec: 'A', text: 'Comes to class on time.' },
    { id: 'q2',  sec: 'A', text: 'Explains learning outcomes and the grading system at the start of the course.' },
    { id: 'q3',  sec: 'A', text: 'Maximizes the allocated time/learning hours effectively.' },
    { id: 'q4',  sec: 'A', text: 'Facilitates students to think critically and creatively.' },
    { id: 'q5',  sec: 'A', text: 'Guides students to learn independently and make informed decisions.' },
    { id: 'q6',  sec: 'A', text: 'Communicates constructive feedback to promote student growth.' },
    { id: 'q7',  sec: 'B', text: 'Demonstrates extensive and up-to-date knowledge of the subject.' },
    { id: 'q8',  sec: 'B', text: 'Simplifies complex ideas and concepts for ease of understanding.' },
    { id: 'q9',  sec: 'B', text: 'Relates subject matter to contemporary issues and real-world scenarios.' },
    { id: 'q10', sec: 'B', text: 'Promotes active learning through the use of ICT tools and digital platforms.' },
    { id: 'q11', sec: 'B', text: 'Uses assessments that are aligned with stated learning outcomes.' },
    { id: 'q12', sec: 'C', text: 'Recognizes, respects, and values diversity among students.' },
    { id: 'q13', sec: 'C', text: 'Makes themselves available and assists students during consultation hours.' },
    { id: 'q14', sec: 'C', text: 'Provides immediate and timely feedback on student outputs.' },
    { id: 'q15', sec: 'C', text: 'Provides transparent and fair criteria in rating student performance.' }
];

function _feedbackQuestionLabel(itemId) {
    const q = (window.SET_QUESTIONS || []).find(x => x.id === itemId);
    if (q) return q.text;
    const known = (typeof CATEGORIES !== 'undefined' ? CATEGORIES : [])
        .concat(typeof SUPERVISOR_CATEGORIES !== 'undefined' ? SUPERVISOR_CATEGORIES : []);
    const match = known.find(c => c.id === itemId || c.label === itemId);
    if (match) return match.label;
    return String(itemId).replace(/^sef/i, 'Item ').replace(/[_-]/g, ' ').replace(/\b\w/g, c => c.toUpperCase());
}

function _extractRatingRows(ev) {
    if (!ev || typeof ev !== 'object') return [];
    const candidateKeys = ['ratings', 'scores', 'answers', 'responses',
        'categoryScores', 'criteria', 'categories', 'items', 'questions', 'perQuestion'];

    let data = null;
    for (const k of candidateKeys) {
        const v = ev[k];
        if (v && typeof v === 'object' && Object.keys(v).length) { data = v; break; }
    }
    if (!data) return [];

    const rows = [];
    if (Array.isArray(data)) {
        data.forEach((item, i) => {
            if (item && typeof item === 'object') {
                const label = item.label || item.question || item.name || item.category || item.id || `Item ${i + 1}`;
                const score = (item.score != null ? item.score
                    : item.value != null ? item.value
                    : item.rating != null ? item.rating
                    : item.points != null ? item.points : '—');
                rows.push({ label: _feedbackQuestionLabel(label), score });
            } else {
                rows.push({ label: `Item ${i + 1}`, score: item });
            }
        });
    } else {
        const order = (window.SET_QUESTIONS || []).map(q => q.id);
        const entries = Object.entries(data).sort((a, b) => {
            const ia = order.indexOf(a[0]);
            const ib = order.indexOf(b[0]);
            if (ia === -1 && ib === -1) return 0;
            if (ia === -1) return 1;
            if (ib === -1) return -1;
            return ia - ib;
        });
        entries.forEach(([key, val]) => {
```

```
            const score = (val != null && typeof val === 'object')
                ? (val.score != null ? val.score : val.value != null ? val.value : val.rating != null ? val.rating : '—')
                : val;
            rows.push({ label: _feedbackQuestionLabel(key), score });
        });
    }
    return rows;
}

function _buildFeedbackRatingBreakdown(ev) {
    const rows = _extractRatingRows(ev);
    if (!rows.length) {
        return `<p style="font-size:0.72rem;color:var(--muted);margin:0 0 8px;">Per-question breakdown wasn't recorded for this evaluation — only the
overall rating is available.</p>`;
    }
    return `<div class="feedback-rating-items">
        ${rows.map(r => `
            <div class="feedback-rating-item">
                <span class="feedback-rating-item-label">${escapeHtml(String(r.label))}</span>
                <span class="feedback-rating-item-score">${escapeHtml(String(r.score))} pts</span>
            </div>`).join('')}
    </div>`;
}

window.goToDeptFeedback = function(deptCode) {
    showFeedbackPage(deptCode || '');
};

window.toggleFeedbackRating = function(ratingId, btnEl) {
    const el = document.getElementById(ratingId);
    if (!el) return;
    const showing = el.style.display !== 'none';
    el.style.display = showing ? 'none' : 'block';
    if (btnEl) btnEl.textContent = showing ? 'View Rating' : 'Hide Rating';
};

window.toggleTeacherMore = function(teacherId, btnEl, total) {
    const card = document.getElementById(`fbteacher-${teacherId}`);
    if (!card) return;
    const mores = card.querySelectorAll('.feedback-more');
    const anyHidden = Array.from(mores).some(m => m.style.display === 'none');
    mores.forEach(m => { m.style.display = anyHidden ? 'block' : 'none'; });
    if (btnEl) btnEl.textContent = anyHidden ? 'Show less' : `View all ${total} ratings`;
};

window.filterFeedback = function(dept, search) {
    if (dept !== undefined && dept !== null) _feedbackDeptFilter = dept;
    if (search !== undefined && search !== null) _feedbackSearchQuery = search;

    document.querySelectorAll('#feedbackDeptBar .student-dept-filter-btn').forEach(btn => {
        btn.classList.toggle('active', btn.dataset.dept === _feedbackDeptFilter ||
            (!btn.dataset.dept && !_feedbackDeptFilter));
    });

    _renderFeedbackList();
};

let _teacherDeptFilter = '';

window.buildTeacherDeptFilterBar = function() {
    const container = document.getElementById('teacherDeptFilterBar');
    if (!container) return;
    const DEPT_CONFIG = getDepartments();
    const allFaculty = getData('teachers', []).filter(t => !t.deleted && t.facultyType !== 'supervisor');

    const counts = {};
    allFaculty.forEach(t => { const k = deptFilterKey(t.dept); counts[k] = (counts[k] || 0) + 1; });

    let html = `<button class="student-dept-filter-btn ${!_teacherDeptFilter ? 'active' : ''}" data-dept="" onclick="filterTeachersByDept('')">
        All <span class="dept-filter-count">${allFaculty.length}</span>
    </button>`;

    const deptOrder = Object.keys(DEPT_CONFIG);
    deptOrder.forEach(code => {
        if (!counts[code]) return;
        const cfg = DEPT_CONFIG[code];
        html += `<button class="student-dept-filter-btn ${_teacherDeptFilter === code ? 'active' : ''}" data-dept="${code}" onclick="filterTeachersBy
Dept('${code}')">
            ${escapeHtml(cfg.short)} <span class="dept-filter-count">${counts[code]}</span>
        </button>`;
    });

    if (counts['UNASSIGNED']) {
        html += `<button class="student-dept-filter-btn ${_teacherDeptFilter === 'UNASSIGNED' ? 'active' : ''}" data-dept="UNASSIGNED" onclick="filte
rTeachersByDept('UNASSIGNED')">
            No Dept <span class="dept-filter-count">${counts['UNASSIGNED']}</span>
        </button>`;
    }

    container.innerHTML = html;
};

window.filterTeachersByDept = function(deptCode) {
    _teacherDeptFilter = deptCode;
    document.querySelectorAll('#teacherDeptFilterBar .student-dept-filter-btn').forEach(btn => {
        btn.classList.toggle('active', btn.dataset.dept === deptCode ||
            (!btn.dataset.dept && !deptCode));
    });
    const search = document.getElementById('teacherSearchInput')?.value || '';
    renderTeachers(search);
};

let _supervisorDeptFilter = '';

window.buildSupervisorDeptFilterBar = function() {
```

```
    const container = document.getElementById('supervisorDeptFilterBar');
    if (!container) return;
    const DEPT_CONFIG = getDepartments();
    const allSupervisors = getData('teachers', []).filter(t => !t.deleted && t.facultyType === 'supervisor');

    const counts = {};
    allSupervisors.forEach(t => {
        const list = (typeof getSupervisedDepts === 'function') ? getSupervisedDepts(t) : [];
        if (!list.length) { counts['UNASSIGNED'] = (counts['UNASSIGNED'] || 0) + 1; return; }
        list.forEach(a => { counts[a.dept] = (counts[a.dept] || 0) + 1; });
    });

    let html = `<button class="student-dept-filter-btn ${!_supervisorDeptFilter ? 'active' : ''}" data-dept="" onclick="filterSupervisorsByDept('')">
        All <span class="dept-filter-count">${allSupervisors.length}</span>
    </button>`;

    Object.entries(DEPT_CONFIG).forEach(([code, cfg]) => {
        if (!counts[code]) return;
        html += `<button class="student-dept-filter-btn ${_supervisorDeptFilter === code ? 'active' : ''}" data-dept="${code}" onclick="filterSupervi
sorsByDept('${code}')">
            ${escapeHtml(cfg.short)} <span class="dept-filter-count">${counts[code]}</span>
        </button>`;
    });

    if (counts['UNASSIGNED']) {
        html += `<button class="student-dept-filter-btn ${_supervisorDeptFilter === 'UNASSIGNED' ? 'active' : ''}" data-dept="UNASSIGNED" onclick="fi
lterSupervisorsByDept('UNASSIGNED')">
            No Dept <span class="dept-filter-count">${counts['UNASSIGNED']}</span>
        </button>`;
    }

    container.innerHTML = html;
};

window.filterSupervisorsByDept = function(deptCode) {
    _supervisorDeptFilter = deptCode;
    document.querySelectorAll('#supervisorDeptFilterBar .student-dept-filter-btn').forEach(btn => {
        btn.classList.toggle('active', btn.dataset.dept === deptCode ||
            (!btn.dataset.dept && !deptCode));
    });
    const search = document.getElementById('supervisorSearchInput')?.value || '';
    renderSupervisorTable(search);
};

window._dflToggle = function(studentId) {
    const row = document.querySelector('tr._dfl-detail[data-for="' + studentId + '"]');
    if (!row) return;
    const open = row.style.display !== 'table-row';
    row.style.display = open ? 'table-row' : 'none';
    const prev = row.previousElementSibling;
    const chev = prev && prev.querySelector('._dfl-chev');
    if (chev) chev.style.transform = open ? 'rotate(90deg)' : '';
};

window.showDeptFullList = function(deptCode, type) {
  const esc = (typeof escapeHtml === 'function') ? escapeHtml : (x => String(x == null ? '' : x));
  const DEPT = (typeof getDepartments === 'function') ? getDepartments() : {};
  const cfg = DEPT[deptCode] || { name: deptCode, short: deptCode };

  const allStudents = getData('students', []).filter(s => !s.deleted);
  const allTeachers = getData('teachers', []).filter(t => !t.deleted);
  const allSubjects = getData('subjects', []).filter(s => s.dept === deptCode);
  const allEvals    = getData('evaluations', []);

  const mono   = v => '<span style="font-family:\'JetBrains Mono\',monospace;font-size:0.8rem;">' + esc(v) + '</span>';
  const strong = v => '<strong>' + esc(v) + '</strong>';
  const pill   = (txt, color) => '<span class="badge" style="background:' + color + '22;color:' + color + ';font-size:0.68rem;">' + esc(txt) + '</spa
n>';

  let columns, rows;

  if (type === 'teachers') {
    // Faculty of this department (teachesInDept) plus its supervisors.
    const list = allTeachers.filter(t => {
        if (teachesInDept(t, deptCode)) return true;
        return (t.facultyType || 'regular') === 'supervisor'
            && getSupervisedDepts(t).some(a => a.dept === deptCode);
    }).sort(byName);
    // Role column: Faculty, Dean/Chairperson of this dept, or
    // "Faculty · also Dean of X" when supervising elsewhere.
    const roleLabel = r => r === 'dean' ? 'Dean' : r === 'chairperson' ? 'Chairperson' : 'Supervisor';
    const roleColor = r => r === 'dean' ? '#7c3aed' : r === 'chairperson' ? '#2563eb' : '#0891b2';
    const roleOf = t => {
      if ((t.facultyType || 'regular') !== 'supervisor') return { text: 'Faculty', html: pill('Faculty', '#475569') };
      const sup  = getSupervisedDepts(t).filter(a => a.dept);
      const here = sup.find(a => a.dept === deptCode);
      if (here) return { text: roleLabel(here.role), html: pill(roleLabel(here.role), roleColor(here.role)) };
      const elsewhere = sup.map(a => roleLabel(a.role) + ' of ' + a.dept).join(', ');
      return {
        text: 'Faculty \u00b7 also ' + elsewhere,
        html: pill('Faculty', '#475569') +
              '<div style="font-size:0.72rem;color:var(--muted);margin-top:3px;">also ' + esc(elsewhere) + '</div>'
      };
    };
    columns = ['ID', 'Name', 'Role', 'Status', 'SET %', 'SEF %'];
    rows = list.map(t => {
      const role = roleOf(t);
      // 0 means "no SET ratings yet"; escapeHtml(0) returns ''.
      const setRaw = (typeof calculateWeightedSETRating === 'function') ? calculateWeightedSETRating(t.id) : 0;
      const setSc  = Number(setRaw) > 0 ? setRaw + '%' : 'N/A';
      const sefAgg = (typeof getSEFForTeacher === 'function')
        ? getSEFForTeacher(t.id)
        : (() => {
            const l = allEvals.filter(e => e.teacherId === t.id && e.evaluatorType === 'supervisor');
            return { count: l.length, average: l.length ? l.reduce((a,b)=>a+b.totalScore,0)/l.length : null };
```

```
        })();
      const sefSc = sefAgg.count ? sefAgg.average.toFixed(2) + '%' : 'N/A';
      return {
        // Role is searchable too: typing "dean" finds the deans.
        s: ((t.tid || '') + ' ' + (t.name || '') + ' ' + role.text).toLowerCase(),
        onclick: "showAnnexReports('" + t.id + "')",
        cells: [mono(t.tid), strong(listName(t)), role.html, pill(t.status || 'active', (t.status === 'active' ? '#059669' : '#dc2626')), esc(setSc),
esc(sefSc)],
        plain: [t.tid || '', listName(t), role.text, t.status || 'active', setSc, sefSc]
      };
    });
  } else if (type === 'subjects') {
    columns = ['Code', 'Name', 'Teacher', 'Enrolled'];
    rows = allSubjects.map(sub => {
      const enrolled = (sub.enrolledIds || []).filter(eid => allStudents.find(s => s.id === eid)).length;
      const list  = (typeof subjectTeachers === 'function') ? subjectTeachers(sub) : [];
      const names = subjectTeacherNames(sub, allTeachers);
      const tName = names.length ? names.join(', ') : 'Unassigned';

      // Several teachers: show MANY with a drop-down list.
      const chev = '<svg class="_dfl-chev" width="11" height="11" fill="none" stroke="currentColor" '
        + 'stroke-width="2.5" viewBox="0 0 24 24" style="vertical-align:-1px;margin-right:7px;'
        + 'color:var(--muted,#64748b);transition:transform 0.15s;"><polyline points="9 18 15 12 9 6"/></svg>';

      let teacherCell, detail = null;
      if (names.length > 1) {
        teacherCell = chev + '<span class="badge" style="background:#ecfdf5;color:#047857;'
          + 'font-size:0.64rem;font-weight:800;letter-spacing:0.05em;">MANY</span>'
          + '<span style="color:var(--muted,#64748b);font-size:0.74rem;margin-left:6px;">'
          + names.length + ' teachers</span>';
        detail = list.map(t => {
          const info = allTeachers.find(x => x.id === t.id) || {};
          const secs = t.sections.length
            ? t.sections.map(x => '<span class="badge" style="background:#ecfdf5;color:#047857;'
                + 'font-size:0.62rem;margin-right:4px;">' + esc(x) + '</span>').join('')
            : '<span style="color:var(--muted,#64748b);font-size:0.72rem;">All sections</span>';
          // How many students this teacher actually rates in this subject.
          const roll = (sub.enrolledIds || []).filter(eid => {
            const st = allStudents.find(x => x.id === eid);
            return st && (typeof teacherTeachesStudent === 'function'
              ? teacherTeachesStudent(sub, t.id, st) : true);
          }).length;
          return '<div style="display:flex;gap:12px;align-items:center;padding:5px 0;'
            + 'border-bottom:1px solid var(--border,#e2e8f0);">'
            + '<span style="flex:1;font-size:0.8rem;font-weight:600;">' + esc(info.name || '(removed teacher)') + '</span>'
            + '<span style="min-width:170px;">' + secs + '</span>'
            + '<span style="font-size:0.74rem;color:var(--muted,#64748b);min-width:74px;text-align:right;">'
            + roll + ' student' + (roll === 1 ? '' : 's') + '</span>'
            + '</div>';
        }).join('');
      } else {
        teacherCell = names.length ? esc(names[0])
                    : '<em style="color:var(--danger,#dc2626);">Unassigned</em>';
      }

      return {
        id: sub.id,
        s: ((sub.code || '') + ' ' + (sub.name || '') + ' ' + tName).toLowerCase(),
        cells: [mono(sub.code), esc(sub.name), teacherCell, pill(enrolled + '', '#2563eb')],
        plain: [sub.code || '', sub.name || '', tName, enrolled],
        detail: detail
      };
    });
  } else { // enrolled
    columns = ['Student ID', 'Name', 'Course', 'Year & Section', 'Subjects'];
    const deptIds = getDeptEnrolledStudentIds(deptCode);
    const inTerm = (typeof annexEvalInTerm === 'function') ? annexEvalInTerm : function () { return true; };
    const termEvals = allEvals.filter(e => e.evaluatorType !== 'supervisor' && inTerm(e));
    const map = {};
    allSubjects.forEach(sub => {
      (sub.enrolledIds || []).forEach(eid => {
        if (!deptIds.has(eid)) return;
        const st = allStudents.find(s => s.id === eid);
        if (!st) return;
        if (!map[eid]) map[eid] = { st: st, subs: [] };
        const mine = subjectTeacherIds(sub).filter(tid => teacherTeachesStudent(sub, tid, st));
        (mine.length ? mine : ['']).forEach(tid => {
          const tch = allTeachers.find(t => t.id === tid);
          const ev = tid && termEvals.find(e => e.studentId === eid && e.subjectId === sub.id
                                            && evalBelongsTo(e, sub, tid));
          map[eid].subs.push({
            code: sub.code, name: sub.name || '',
            teacher: tch ? tch.name : 'Unassigned',
            done: !!ev
          });
        });
      });
    });
    const orderedKeys = Object.keys(map).sort((ka, kb) => byCourseThenYear(map[ka].st, map[kb].st));
    rows = orderedKeys.map(k => {
      const st = map[k].st, subs = map[k].subs;
      const ys = (st.year || '') + (st.section ? ' - ' + st.section : '');
      const courseFull  = String(st.course || '');
      const courseShort = courseFull
        ? ((typeof courseShorthand === 'function') ? courseShorthand(courseFull) : courseFull)
        : '\u2014';
      const chev = '<svg class="_dfl-chev" width="11" height="11" fill="none" stroke="currentColor" '
        + 'stroke-width="2.5" viewBox="0 0 24 24" style="vertical-align:-1px;margin-right:7px;'
        + 'color:var(--muted,#64748b);transition:transform 0.15s;"><polyline points="9 18 15 12 9 6"/></svg>';
      const summary = chev + '<span style="color:var(--muted,#64748b);">'
        + subs.length + ' subject' + (subs.length !== 1 ? 's' : '') + '</span>';

      const detail = subs.map(x => '<div style="display:flex;gap:12px;align-items:center;padding:5px 0;'
          + 'border-bottom:1px solid var(--border,#e2e8f0);">'
          + '<span style="font-family:\'JetBrains Mono\',monospace;font-size:0.76rem;min-width:88px;">' + esc(x.code) + '</span>'
```

```
          + '<span style="flex:1;font-size:0.8rem;">' + esc(x.name) + '</span>'
          + '<span style="font-size:0.74rem;color:var(--muted,#64748b);min-width:150px;">' + esc(x.teacher) + '</span>'
          + (x.done
              ? '<span class="badge badge-success" style="font-size:0.62rem;">Evaluated</span>'
              : '<span class="badge" style="font-size:0.62rem;background:#fef3c7;color:#92400e;">Pending</span>')
        + '</div>').join('');
      return {
        id: st.id,
        student: st,
        s: ((st.sid || '') + ' ' + (st.name || '') + ' ' + courseFull + ' ' + courseShort
            + ' ' + subs.map(x => x.code).join(' ')).toLowerCase(),
        cells: [mono(st.sid), strong(listName(st)), esc(courseShort), esc(ys), summary],
        plain: [st.sid || '', listName(st), courseFull, ys, subs.map(x => x.code).join(', ')],
        detail: detail || '<div style="font-size:0.8rem;color:var(--muted);">No subjects.</div>'
      };
    });
  }

  const titleMap = { teachers: 'Teachers', subjects: 'Subjects', enrolled: 'Enrolled Students' };
  const deptLabel = esc(cfg.short || deptCode);
  const title = (titleMap[type] || 'List') + ' \u2014 ' + esc(cfg.name);
  const sub = rows.length + ' ' + (rows.length === 1 ? 'record' : 'records');

  let pageEl = document.getElementById('page-deptFullList');
  if (!pageEl) {
    pageEl = document.createElement('div');
    pageEl.className = 'page';
    pageEl.id = 'page-deptFullList';
    const anchor = document.getElementById('page-dept');
    (anchor && anchor.parentNode ? anchor.parentNode : document.body).appendChild(pageEl);
  }

  const headHtml = '<tr>' + columns.map(c => '<th>' + esc(c) + '</th>').join('') + '</tr>';
  const bodyHtml = rows.length
    ? rows.map(r => {
        // The tooltip follows the list being shown, not always students.
        const openTip = (type === 'subjects') ? 'Show each teacher and the sections they take'
                                              : 'Show this student\'s subjects and evaluation status';
        const open = r.detail ? ' onclick="_dflToggle(\'' + r.id + '\')" style="cursor:pointer;" title="' + openTip + '"' : '';
        const click = r.onclick ? ' onclick="' + r.onclick + '" style="cursor:pointer;" title="Open report"' : open;
        return '<tr data-s="' + r.s + '"' + click + '>'
          + r.cells.map(c => '<td>' + c + '</td>').join('') + '</tr>'
          + (r.detail
              ? '<tr class="_dfl-detail" data-for="' + r.id + '" style="display:none;">'
                + '<td colspan="' + columns.length + '" style="background:var(--surface2,#f8fafc);padding:10px 18px;">'
                + r.detail + '</td></tr>'
              : '');
      }).join('')
    : '<tr><td colspan="' + columns.length + '" style="text-align:center;padding:40px;color:var(--muted);">Nothing here yet for ' + deptLabel + '.</td></tr>';

  pageEl.innerHTML =
    '<div class="page-header" style="display:flex;align-items:flex-start;justify-content:space-between;gap:12px;flex-wrap:wrap;">'
      + '<div class="page-title"><h1>' + title + '</h1><p id="_dflSub">' + sub + '</p></div>'
      + '<div style="display:flex;gap:8px;align-items:center;flex-wrap:wrap;">'
        + '<input id="_dflSearch" type="text" placeholder="Search\u2026" style="padding:8px 12px;border:1px solid var(--border,#cbd5e1);border-radius:8px;font-size:0.85rem;outline:none;">'
        + '<button class="btn btn-ghost btn-sm" onclick="_exportDeptListPDF()" title="Opens the print dialog - choose Save as PDF"><svg width="13" height="13" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24" style="margin-right:4px;"><path d="M14 2H6a2 2 0 00-2 2v16a2 2 0 002 2h12a2 2 0 002-2V8z"/><polyline points="14 2 14 8 20 8"/><line x1="8" y1="13" x2="16" y2="13"/><line x1="8" y1="17" x2="13" y2="17"/></svg>Export PDF</button>'
        + '<button class="btn btn-ghost btn-sm" onclick="showDeptPage(\'' + deptCode + '\')"><svg width="13" height="13" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24" style="margin-right:4px;"><polyline points="15 18 9 12 15 6"/></svg>Back to ' + deptLabel + '</button>'
      + '</div>'
    + '</div>'
    + '<div class="card"><div class="table-wrap"><table class="data-table"><thead>' + headHtml + '</thead><tbody id="_dflBody">' + bodyHtml + '</tbody></table></div></div>';

  document.querySelectorAll('.page').forEach(p => p.classList.remove('active'));
  pageEl.classList.add('active');
  if (typeof closeSidebar === 'function') closeSidebar();
  window.scrollTo(0, 0);

  window._dflExport = { title: title, columns: columns, rows: rows, dept: cfg.name || deptCode, type: type };

  const searchEl = document.getElementById('_dflSearch');
  const bodyEl   = document.getElementById('_dflBody');
  const subEl    = document.getElementById('_dflSub');
  if (searchEl) {
    searchEl.addEventListener('input', function() {
      const q = this.value.trim().toLowerCase();
      // Same search as the Students page; rows match window._dflExport.rows.
      const terms = (type === 'enrolled' && typeof parseStudentSearch === 'function') ? parseStudentSearch(q) : null;
      let shown = 0;
      Array.prototype.forEach.call(bodyEl.querySelectorAll('tr[data-s]'), (tr, i) => {
        const st = rows[i] && rows[i].student;
        const hay = tr.getAttribute('data-s');
        const hit = !q || (terms && st
          ? terms.every(t => t.year ? studentMatchesTerm(st, t) : hay.indexOf(t.text) > -1)
          : hay.indexOf(q) > -1);
        tr.style.display = hit ? '' : 'none';
        if (hit) shown++;
      });
      Array.prototype.forEach.call(bodyEl.querySelectorAll('tr._dfl-detail'), tr => { tr.style.display = 'none'; });
      Array.prototype.forEach.call(bodyEl.querySelectorAll('._dfl-chev'), c => { c.style.transform = ''; });
      if (subEl) subEl.textContent = shown + ' ' + (shown === 1 ? 'record' : 'records');
    });
  }
};

// EXPORT PDF (department "View all" lists): opens a print-ready tab of the
// rows currently shown (search filter included); "Save as PDF" to export.
window._exportDeptListPDF = function() {
  const d = window._dflExport;
  if (!d || !d.rows.length) { if (typeof showToast === 'function') showToast('Nothing to export.', 'info'); return; }
```

```
  const esc = (typeof escapeHtml === 'function') ? escapeHtml : (x => String(x == null ? '' : x));
  const plainEsc = v => esc(String(v == null ? '' : v));

  // Only the rows the search box left visible.
  const bodyEl = document.getElementById('_dflBody');
  const search = (document.getElementById('_dflSearch') || {}).value || '';
  let rows = d.rows;
  if (bodyEl && search.trim()) {
    const shown = Array.prototype.map.call(bodyEl.querySelectorAll('tr[data-s]'), tr => tr.style.display !== 'none');
    rows = d.rows.filter((r, i) => shown[i]);
  }
  if (!rows.length) { if (typeof showToast === 'function') showToast('No rows match the search.', 'info'); return; }

  const titleText = String(d.title).replace(/&amp;/g, '&');   // d.title was escaped for the page
  const now = new Date().toLocaleDateString('en-PH', { year: 'numeric', month: 'long', day: 'numeric' });
  const logo = new URL('../icons/NWSSU-LOGO1.png', location.href).href;
  const numeric = i => /enrolled|%/i.test(d.columns[i]);   // right-align counts and scores

  const head = '<tr><th class="n">#</th>' + d.columns.map((c, i) =>
      '<th' + (numeric(i) ? ' class="r"' : '') + '>' + esc(c) + '</th>').join('') + '</tr>';
  const body = rows.map((r, k) => '<tr><td class="n">' + (k + 1) + '</td>' + r.plain.map((v, i) =>
      '<td' + (numeric(i) ? ' class="r"' : '') + '>' + plainEsc(v) + '</td>').join('') + '</tr>').join('');

  const html = '<!DOCTYPE html><html><head><meta charset="utf-8"><title>' + esc(titleText) + '</title><style>'
    + '*{box-sizing:border-box}'
    + 'body{font-family:"Segoe UI",Arial,sans-serif;color:#1a1a1a;margin:32px;font-size:12px}'
    + '.hd{display:flex;align-items:center;gap:14px;border-bottom:2px solid #059669;padding-bottom:10px;margin-bottom:14px}'
    + '.hd img{width:54px;height:54px}'
    + '.u{font-size:11px;letter-spacing:.08em;text-transform:uppercase;color:#065f46;font-weight:700}'
    + 'h1{font-size:17px;margin:2px 0 0}'
    + '.sub{color:#555;margin:0 0 12px}'
    + 'table{width:100%;border-collapse:collapse}'
    + 'th,td{border:1px solid #d9dee3;padding:6px 8px;text-align:left;vertical-align:top}'
    + 'thead th{background:#064e3b;color:#fff;font-weight:700}'
    + 'tbody tr:nth-child(even) td{background:#f3faf6}'
    + '.n{width:34px;text-align:center;color:#666}.r{text-align:right}'
    + 'thead{display:table-header-group}tr{page-break-inside:avoid}'
    + '.ft{margin-top:14px;color:#777;font-size:10px}'
    + '.noprint{text-align:center;margin-bottom:16px}'
    + '.noprint button{padding:8px 20px;font-size:13px;border:none;border-radius:6px;background:#059669;color:#fff;cursor:pointer}'
    + '@media print{body{margin:12px}.noprint{display:none}thead th{-webkit-print-color-adjust:exact;print-color-adjust:exact}'
    + 'tbody tr:nth-child(even) td{-webkit-print-color-adjust:exact;print-color-adjust:exact}}'
    + '</style></head><body>'
    + '<div class="noprint"><button onclick="window.print()">Print / Save as PDF</button></div>'
    + '<div class="hd"><img src="' + logo + '" alt="" onerror="this.style.display=\'none\'">'
    +   '<div><div class="u">Northwest Samar State University</div><h1>' + esc(titleText) + '</h1></div></div>'
    + '<p class="sub"><strong>' + rows.length + '</strong> ' + (rows.length === 1 ? 'record' : 'records')
    +   (search.trim() ? ' &middot; filtered by &ldquo;' + esc(search.trim()) + '&rdquo;' : '')
    +   ' &middot; Generated ' + esc(now) + '</p>'
    + '<table><thead>' + head + '</thead><tbody>' + body + '</tbody></table>'
    + '<p class="ft">Faculty Evaluation System &mdash; ' + esc(d.dept) + '</p>'
    // Print once the logo has loaded (or failed), so it is on the page.
    + '<script>window.onload=function(){setTimeout(function(){window.print();},250);};<\/script>'
    + '</body></html>';

  const w = window.open('', '_blank');
  if (!w) { if (typeof showToast === 'function') showToast('Please allow pop-ups to export the PDF.', 'warning'); return; }
  w.document.write(html);
  w.document.close();
  w.focus();
  if (typeof addAudit === 'function') addAudit('Export PDF', titleText + ' (' + rows.length + ' rows)');
};
```

admin/js/modules/adminrate.js

964 lines

```
// ===== FIREBASE SYNC & FIXES =====

// getData() lives in admin.js.

// addAudit() lives in admin.js.

// showToast() lives in admin.js.

// Add animation styles if not present
if (!document.querySelector('#toastStyles')) {
    const style = document.createElement('style');
    style.id = 'toastStyles';
    style.textContent = `
        @keyframes slideIn {
            from { transform: translateX(100%); opacity: 0; }
            to { transform: translateX(0); opacity: 1; }
        }
    `;
    document.head.appendChild(style);
}

// ===== CMO COMPLIANT SCORING SYSTEM =====
window.CATEGORIES = [
    { id: 'teaching', label: 'Teaching Methodology', max: 4, weight: 0.60 },
    { id: 'knowledge', label: 'Subject Matter Expertise', max: 4, weight: 0.60 },
    { id: 'communication', label: 'Communication Skills', max: 4, weight: 0.60 },
    { id: 'attitude', label: 'Professionalism & Attitude', max: 4, weight: 0.60 },
    { id: 'assessment', label: 'Assessment & Feedback', max: 4, weight: 0.60 }
];

// Supervisor Categories (40%)
window.SUPERVISOR_CATEGORIES = [
    { id: 'sef_teaching', label: 'Teaching Effectiveness', max: 5, weight: 0.40 },
    { id: 'sef_professional', label: 'Professionalism', max: 5, weight: 0.40 },
    { id: 'sef_content', label: 'Content Knowledge', max: 5, weight: 0.40 },
    { id: 'sef_assessment', label: 'Assessment Practices', max: 5, weight: 0.40 }
];

// getRemarks() lives in admin.js.

// REMOVED duplicate getRemarksColor() — byte-identical to the copy in admin.js.

// Final 60/40 composite (not CMO 19) lives in admin-scoring.js.


// Check and auto-finalize evaluation period
window.checkAndAutoFinalize = function() {
    const period = getData('evalPeriod', {});
    if (!period.open) return;

    if (period.deadline) {
        const today = new Date().toISOString().split('T')[0];
        if (today >= period.deadline) {
            period.open = false;
            setData('evalPeriod', period);
            addAudit('Auto-Finalize', 'Evaluation period automatically closed on due date');
            generateFinalReports();
            showToast('Evaluation period has been automatically finalized per due date.', 'info');
        }
    }
};

// Generate final reports for all faculty (CMO 19: SET and SEF displayed separately)
window.generateFinalReports = function() {
    const teachers = getData('teachers', []).filter(t => !t.deleted);
    const finalReports = teachers.map(teacher => {
        const setSc = calculateWeightedSETRating(teacher.id);
        // Mean of every supervisor rating — see getSEFForTeacher() in admin.js.
        const sefAggR = getSEFForTeacher(teacher.id);
        const sefSc = sefAggR.count ? sefAggR.average.toFixed(2) : null;
        return {
            teacherId: teacher.id,
            teacherName: teacher.name,
            teacherDept: teacher.dept,
            setSET: setSc,
            sefSEF: sefSc,
            generatedAt: new Date().toISOString()
        };
    });
    setData('finalReports', finalReports);
    addAudit('Generate Final Reports', `Generated reports for ${finalReports.length} faculty`);
    return finalReports;
};

// Auto-finalize check every hour (late-bound).
setInterval(function () {
    if (typeof window.checkAndAutoFinalize === 'function') window.checkAndAutoFinalize();
}, 3600000);

// getActiveSY() lives in admin.js.

// ===== TEACHER PICKER =====
// Search box, department pills and a scrollable list. The value still lives
// in hidden input #subTeacher.

let _tpSearch = '';
let _tpDept   = '';          // '' = all departments

function _tpTeachers() {
    return getData('teachers', []).filter(t => !t.deleted && t.status !== 'archived');
```

```
}

// Teachers chosen for the subject being edited: [{ id, sections: [] }].
window._tpChosen = [];

// Accepts a subject, a single teacher id, or nothing (new subject).
window.populateTeacherSelect = function(arg) {
    // Default to the form's own picker unless the standalone modal set its own.
    if (!window._tpHost) window._tpHost = 'teacherPicker';
    if (arg && typeof arg === 'object') {
        _tpChosen = (typeof subjectTeachers === 'function') ? subjectTeachers(arg) : [];
    } else if (arg) {
        _tpChosen = [{ id: arg, sections: [] }];
    } else {
        _tpChosen = [];
    }
    _tpSyncHidden();
    // Start on "All"; the subject's department is sorted to the top.
    _tpDept   = '';
    _tpSearch = '';
    renderTeacherPicker();
};

window.tpSetDept = function(dept) { _tpDept = dept; renderTeacherPicker(); };
window.tpSetSearch = function(q)  { _tpSearch = q;  renderTeacherPicker(true); };

// Hidden field holds the first teacher id; saveSubject() reads _tpChosen.
function _tpSyncHidden() {
    const hidden = document.getElementById('subTeacher');
    if (hidden) hidden.value = _tpChosen.length ? _tpChosen[0].id : '';
}

// Tapping a teacher adds them, or removes them if already chosen.
window.tpSelect = function(id) {
    if (!id) { _tpChosen = []; }
    else if (_tpChosen.some(t => t.id === id)) { _tpChosen = _tpChosen.filter(t => t.id !== id); }
    else { _tpChosen.push({ id: id, sections: [] }); }
    _tpSyncHidden();
    renderTeacherPicker();
};

window.tpRemove = function(id) {
    _tpChosen = _tpChosen.filter(t => t.id !== id);
    _tpSyncHidden();
    renderTeacherPicker();
};

// Toggle a section for a teacher; none ticked = rated by every student.
window.tpToggleSection = function(id, label) {
    const t = _tpChosen.find(x => x.id === id);
    if (!t) return;
    const i = t.sections.indexOf(label);
    if (i === -1) t.sections.push(label); else t.sections.splice(i, 1);
    renderTeacherPicker();
};

// "All students": clear this teacher's sections.
window.tpAllSections = function(id) {
    const t = _tpChosen.find(x => x.id === id);
    if (!t) return;
    t.sections = [];
    renderTeacherPicker();
};

// Sections present among the subject's enrolled students.
function _tpSubjectSections() {
    const editingId = window._tpSubjectId
        || ((typeof editSubjectId !== 'undefined') ? editSubjectId : null);
    if (!editingId || typeof _sectionLabel !== 'function') return [];
    const sub = getData('subjects', []).find(x => x.id === editingId);
    if (!sub) return [];
    const byId = {};
    getData('students', []).filter(x => !x.deleted).forEach(x => { byId[x.id] = x; });
    const set = new Set();
    (sub.enrolledIds || []).forEach(id => { if (byId[id]) set.add(_sectionLabel(byId[id])); });
    return Array.from(set).filter(l => l && l !== '\u2014').sort();
}

window.renderTeacherPicker = function(keepFocus) {
    // Host: the Subject form or the Teachers & Sections modal (_tpHost).
    const host = document.getElementById(window._tpHost || 'teacherPicker');
    if (!host) return;

    const chosenIds  = _tpChosen.map(t => t.id);
    // From the form if open, else from the subject itself.
    const subDept    = window._tpSubDept !== undefined
        ? window._tpSubDept
        : ((document.getElementById('subDept') || {}).value || '');
    const all        = _tpTeachers();

    // Counts drive the pills, so an empty department is never offered.
    const counts = {};
    all.forEach(t => { const d = t.dept || 'No dept'; counts[d] = (counts[d] || 0) + 1; });

    // Classes each faculty carries this term; free faculty are listed first.
    const subjects = getData('subjects', []).filter(x => !x.deleted);
    const editingId = (typeof editSubjectId !== 'undefined') ? editSubjectId : null;
    const loadOf = {};
    subjects.forEach(x => {
        // Don't count the subject being edited.
        const skipId = window._tpSubjectId
            || ((typeof editSubjectId !== 'undefined') ? editSubjectId : null);
        if (skipId && x.id === skipId) return;
        const ids = (typeof subjectTeacherIds === 'function') ? subjectTeacherIds(x)
                  : (x.teacherId ? [x.teacherId] : []);
```

```
        ids.forEach(id => { loadOf[id] = (loadOf[id] || 0) + 1; });
    });

    const q = _tpSearch.trim().toLowerCase();
    // Other-department faculty are listed after the department's own, tagged
    // "from <DEPT>".
    const isHome = t => !subDept || (t.dept || '') === subDept;
    const list = all
        .filter(t => !_tpDept || (t.dept || 'No dept') === _tpDept)
        .filter(t => !q || (t.name + ' ' + t.tid + ' ' + (t.dept || '')).toLowerCase().includes(q))
        .sort((a, b) => {
            const ha = isHome(a), hb = isHome(b);
            if (ha !== hb) return ha ? -1 : 1;                  // this department first
            const la = loadOf[a.id] || 0, lb = loadOf[b.id] || 0;
            if (la !== lb) return la - lb;                      // then lightest load
            return String(a.name).localeCompare(String(b.name)); // then alphabetical
        });

    const sections = _tpSubjectSections();
    const chosenRows = _tpChosen.map(c => {
        const t = all.find(x => x.id === c.id) || { id: c.id, name: '(removed teacher)', tid: '', dept: '' };
        // Who rates this teacher, then per-teacher section chips.
        const chosenSecs = c.sections.filter(l => sections.indexOf(l) !== -1);
        const summary = chosenSecs.length
            ? 'Rated by ' + chosenSecs.map(escapeHtml).join(', ')
            : 'Rated by all students in this subject';
        const chip = (label, on, onclick) =>
            `<button type="button" class="tp-sec${on ? ' is-on' : ''}" onclick="${onclick}">`
            + `${escapeHtml(label)}</button>`;
        const secHtml = sections.length
            ? `<div class="tp-secs">
                 <span class="tp-secs-label">${summary}</span>
                 <span class="tp-sec-chips">
                   ${chip('All students', chosenSecs.length === 0, `tpAllSections('${t.id}')`)}
                   ${sections.map(l => chip(l, c.sections.indexOf(l) !== -1,
                       `tpToggleSection('${t.id}', '${escapeHtml(l).replace(/'/g, "&#39;")}')`)).join('')}
                 </span>
               </div>`
            : `<div class="tp-secs"><span class="tp-secs-label">Rated by all students
                 \u2014 enrol students first to split this subject by section.</span></div>`;
        return `<div class="tp-chosen${window._tpFocusTeacher === t.id ? ' is-focus' : ''}">
            <div>
              <strong>${escapeHtml(listName(t))}</strong>
              <span class="tp-tid">${escapeHtml(t.tid || '')}</span>
              ${t.dept ? `<span class="dept-tag-inline">${escapeHtml(t.dept)}</span>` : ''}
              ${t.dept && subDept && t.dept !== subDept ? '<span class="tp-outside">outside this department</span>' : ''}
              ${secHtml}
            </div>
            <button type="button" class="tp-clear" onclick="tpRemove('${t.id}')" title="Remove">&#10005;</button>
          </div>`;
    }).join('');

    // Reuses the Teachers page filter-bar classes so both look the same.
    const pill = (label, value, n, isActive) =>
        `<button type="button" class="student-dept-filter-btn${isActive ? ' active' : ''}" onclick="tpSetDept('${value}')">`
        + escapeHtml(label) + (n != null ? ` <span class="dept-filter-count">${n}</span>` : '') + `</button>`;

    const deptKeys = Object.keys(counts).sort((a, b) => {
        if (a === subDept) return -1;          // subject's own department first
        if (b === subDept) return 1;
        return a.localeCompare(b);
    });

    host.innerHTML = `
      <div class="tp-selected">
        ${chosenRows || '<div class="tp-empty">No teacher assigned yet \u2014 pick one or more below.</div>'}
      </div>

      <input type="text" id="tpSearchInput" class="form-control tp-search"
             placeholder="Search by name, ID or department\u2026"
             value="${escapeHtml(_tpSearch)}" oninput="tpSetSearch(this.value)"/>

      <div class="tp-pills">
        ${pill('All', '', all.length, _tpDept === '')}
        ${deptKeys.map(d => pill(d, d, counts[d], _tpDept === d)).join('')}
      </div>

      <div class="tp-list">
        ${list.length ? list.map(t => `
          <button type="button" class="tp-item${chosenIds.indexOf(t.id) !== -1 ? ' selected' : ''}"
                  onclick="tpSelect('${t.id}')">
            <span class="tp-avatar">${escapeHtml((t.name || '?').charAt(0).toUpperCase())}</span>
            <span class="tp-info">
              <span class="tp-name">${escapeHtml(listName(t))}${
                isHome(t) ? '' : `<span class="tp-foreign">from ${escapeHtml(t.dept || 'no dept')}</span>`}</span>
              <span class="tp-meta">${escapeHtml(t.tid)}${t.dept ? ' \u00b7 ' + escapeHtml(t.dept) : ''}${
                t.facultyType === 'supervisor' ? ' \u00b7 Supervisor' : ''}</span>
            </span>
            ${(loadOf[t.id] || 0) === 0
                ? '<span class="tp-load free">No subjects yet</span>'
                : `<span class="tp-load">${loadOf[t.id]} class${loadOf[t.id] === 1 ? '' : 'es'}</span>`}
            ${chosenIds.indexOf(t.id) !== -1 ? '<span class="tp-check">\u2713</span>' : ''}
          </button>`).join('')
        : '<div class="tp-none">No faculty match that search.</div>'}
      </div>`;

    // Typing re-renders the list, so focus and caret have to be restored.
    if (keepFocus) {
        const el = document.getElementById('tpSearchInput');
        if (el) { el.focus(); el.setSelectionRange(el.value.length, el.value.length); }
    }
};

// Changing department re-sorts pills but keeps the chosen teacher.
document.addEventListener('DOMContentLoaded', function() {
```

```
    const subDeptSelect = document.getElementById('subDept');
    if (subDeptSelect) {
        subDeptSelect.addEventListener('change', function() {
            _tpDept = this.value || '';
            renderTeacherPicker();
        });
    }
});

// REMOVED: stale saveTeacher override; admin.js's is the only definition.

// TEACHERS & SECTIONS - assign sections per teacher after enrolment,
// without reopening the subject form.
window.openSubjectTeachersModal = function (subId) {
    const sub = getData('subjects', []).find(s => s.id === subId);
    if (!sub) { showToast('Subject not found.', 'error'); return; }

    window._tpSubjectId = subId;              // which subject the picker is for
    if (window._tpFocusTeacher === undefined) window._tpFocusTeacher = null;
    window._tpHost      = 'teacherPicker2';   // render into this modal
    window._tpSubDept   = sub.dept || '';

    const label = document.getElementById('stSubjectName');
    if (label) label.textContent = (sub.code || '') + ' \u2014 ' + (sub.name || '');

    populateTeacherSelect(sub);
    openModal('subjectTeachersModal');
};

// WHO TEACHES THIS SUBJECT - list behind the MULTIPLE tag, with each
// teacher's sections and enrolment.
window.openSubjectTeacherList = function (subId) {
    const sub = getData('subjects', []).find(s => s.id === subId);
    if (!sub) { showToast('Subject not found.', 'error'); return; }
    window._tlSubjectId = subId;

    const label = document.getElementById('tlSubjectName');
    if (label) label.textContent = (sub.code || '') + ' \u2014 ' + (sub.name || '');
    renderSubjectTeacherList();
    openModal('subjectTeacherListModal');
};

function renderSubjectTeacherList() {
    const host = document.getElementById('teacherListBody');
    if (!host) return;
    const subId = window._tlSubjectId;
    const sub = getData('subjects', []).find(s => s.id === subId);
    if (!sub) { host.innerHTML = ''; return; }

    const all      = getData('teachers', []).filter(t => !t.deleted);
    const students = getData('students', []).filter(s => !s.deleted);
    const list     = (typeof subjectTeachers === 'function') ? subjectTeachers(sub) : [];

    // How many students each teacher actually rates - the point of the sections.
    const roll = (t) => (sub.enrolledIds || []).filter(id => {
        const st = students.find(x => x.id === id);
        return st && (typeof teacherTeachesStudent === 'function'
            ? teacherTeachesStudent(sub, t.id, st) : true);
    }).length;

    const enrolled = (sub.enrolledIds || []).filter(id => students.find(x => x.id === id)).length;

    host.innerHTML = `
      <div class="tl-summary">
        <span><strong>${list.length}</strong> teacher${list.length === 1 ? '' : 's'}</span>
        <span><strong>${enrolled}</strong> student${enrolled === 1 ? '' : 's'} enrolled</span>
      </div>
      ${list.length ? list.map(t => {
        const info = all.find(x => x.id === t.id) || { name: '(removed teacher)', tid: '', dept: '' };
        const secs = t.sections.length
            ? t.sections.map(x => `<span class="tl-sec">${escapeHtml(x)}</span>`).join('')
            : '<span class="tl-sec tl-sec-all">All sections</span>';
        const n = roll(t);
        return `
          <button type="button" class="tl-card" onclick="tlEditTeacher('${t.id}')">
            <span class="tl-avatar">${escapeHtml((info.name || '?').trim().charAt(0).toUpperCase())}</span>
            <span class="tl-main">
              <span class="tl-name">${escapeHtml(info.name)}</span>
              <span class="tl-meta">${escapeHtml(info.tid || '')}${info.dept ? ' \u00b7 ' + escapeHtml(info.dept) : ''}</span>
              <span class="tl-secs">${secs}</span>
            </span>
            <span class="tl-right">
              <span class="tl-roll">${n} student${n === 1 ? '' : 's'}</span>
              <span class="tl-go">Edit sections \u203a</span>
            </span>
          </button>`;
      }).join('') : '<div class="tl-empty">No teachers assigned to this subject yet.</div>'}
    `;
}

// Straight to the section chips, with this teacher first.
window.tlEditTeacher = function (teacherId) {
    const subId = window._tlSubjectId;
    closeModal('subjectTeacherListModal');
    openSubjectTeachersModal(subId);
    window._tpFocusTeacher = teacherId;
    renderTeacherPicker();
};

// Enrol students for the subject being viewed.
window.tlEnroll = function () {
    const subId = window._tlSubjectId;
    closeModal('subjectTeacherListModal');
    if (typeof openEnrollModal === 'function') openEnrollModal(subId);
};
```

```

// Assign or remove teachers for the subject being viewed.
window.tlManageTeachers = function () {
    const subId = window._tlSubjectId;
    closeModal('subjectTeacherListModal');
    openSubjectTeachersModal(subId);
};

window.saveSubjectTeachers = function () {
    const subId = window._tpSubjectId;
    const subjects = getData('subjects', []);
    const idx = subjects.findIndex(s => s.id === subId);
    if (idx === -1) { showToast('Subject not found.', 'error'); return; }

    const teachers = _tpTeachersToSave();
    const { keep, removed } = splitUncoveredStudents(subjects[idx], teachers);

    const commit = () => {
        subjects[idx].teachers  = teachers;
        // Kept in step with the first teacher, for any code reading a single id.
        subjects[idx].teacherId = teachers.length ? teachers[0].id : '';
        // Students outside every chosen section are unenrolled - nobody rates them.
        subjects[idx].enrolledIds = keep;
        setData('subjects', subjects);

        addAudit('Edit Subject', `Teachers updated on ${subjects[idx].code}: ${teachers.length} assigned`
            + (removed.length ? `, ${removed.length} student(s) outside the chosen sections unenrolled` : ''));
        showToast(teachers.length
            ? 'Teachers saved.' + (removed.length ? ` ${removed.length} student(s) unenrolled.` : '')
            : 'All teachers removed from this subject.', 'success');

        closeModal('subjectTeachersModal');
        _tpResetHost();
        renderSubjects();
        // Came from the teacher list: show it again with the change applied.
        if (window._tlSubjectId === subId) { renderSubjectTeacherList(); openModal('subjectTeacherListModal'); }
    };

    if (removed.length) _tpConfirmUnenrol(removed, commit);
    else commit();
};

// Chosen teachers as saved; sections no longer in the subject are dropped.
function _tpTeachersToSave() {
    const present = _tpSubjectSections();
    return _tpChosen.map(t => ({
        id: t.id,
        sections: present.length ? t.sections.filter(l => present.indexOf(l) !== -1) : t.sections.slice()
    }));
}

// Ask before unenrolling students that no teacher's sections cover.
function _tpConfirmUnenrol(removed, onYes) {
    const secs = Array.from(new Set(removed.map(st => _sectionLabel(st)))).sort();
    showConfirm('Unenrol students outside these sections?',
        `${removed.length} enrolled student${removed.length === 1 ? ' is' : 's are'} not in any teacher's section `
        + `(${secs.join(', ')}) and will be removed from this subject. `
        + `To keep them, tick their section for a teacher or set a teacher to "All students".`,
        onYes);
}

// Back to the Add/Edit Subject form, so opening it next still works.
function _tpResetHost() {
    window._tpFocusTeacher = null;
    window._tpSubjectId = null;
    window._tpHost      = 'teacherPicker';
    window._tpSubDept   = undefined;
}

// Cancel or the X: same reset, without saving.
document.addEventListener('click', function (e) {
    const t = e.target;
    if (!t || !t.closest) return;
    if (t.closest('#subjectTeachersModal [onclick*="closeModal"]')) _tpResetHost();
});

// Patch saveSubject to include dept + category fields
window.saveSubject = function() {
    const code = document.getElementById('subCode').value.trim();
    const name = document.getElementById('subName').value.trim();
    // Every chosen teacher, each with the sections they take (none = everyone).
    const teachers  = _tpTeachersToSave();
    // The first teacher, kept for any code that still reads a single teacherId.
    const teacherId = teachers.length ? teachers[0].id : '';
    const dept = document.getElementById('subDept').value;
    const loadType = document.getElementById('subLoad').value;
    const isLabSchool = document.getElementById('subIsLab').checked;
    // BUG FIX: category was never read, causing it to silently reset to '' on every save
    const category = (document.getElementById('subCategory') ? document.getElementById('subCategory').value : '') || '';
    // Curriculum slot. Empty courses = open to every course (GE/PE/NSTP).
    const courses = (typeof getSubCourses === 'function') ? getSubCourses() : [];
    const yearLevel = (document.getElementById('subYear') || {}).value || '';

    if (!code || !name) { showToast('Fill all fields.', 'error'); return; }

    const subjects = getData('subjects', []);

    if (typeof editSubjectId !== 'undefined' && editSubjectId) {
        const idx = subjects.findIndex(s => s.id === editSubjectId);
        // Students outside every teacher's sections are unenrolled, after asking.
        const { keep, removed } = splitUncoveredStudents(subjects[idx], teachers);
        if (removed.length && !window._subUnenrolOk) {
            _tpConfirmUnenrol(removed, () => { window._subUnenrolOk = true; window.saveSubject(); });
            return;
        }
```

```
        window._subUnenrolOk = false;
        subjects[idx].enrolledIds = keep;
        // Ratings are kept per teacher, so changing teachers keeps existing ratings.
        Object.assign(subjects[idx], { code, name, teacherId, teachers, dept, loadType, isLabSchool,
                                       category, courses, yearLevel });
        addAudit('Edit Subject', `Updated: ${name} (${code})`);
        showToast('Subject updated!', 'success');
    } else {
        subjects.push({
            id: 'sub' + Date.now(),
            code, name, teacherId, teachers, dept,
            loadType, isLabSchool, category,
            courses, yearLevel,
            enrolledIds: []
        });
        addAudit('Add Subject', `Added: ${name} (${code})`);
        showToast('Subject added!', 'success');
    }

    setData('subjects', subjects);
    closeModal('addSubjectModal');
    renderSubjects();
};

// REMOVED: redundant openEditTeacherModal / openEditSubjectModal wrappers.

// Ensure Firestore sync works properly
window.syncCollectionToFirestore = async function(key, value) {
    if (typeof firebase === 'undefined' || !firebase.firestore) return;

    const db = firebase.firestore();

    // departments — each dept is its own doc
    if (key === 'departments') {
        try {
            const batch = db.batch();
            const keep  = new Set(Object.keys(value || {}).map(String));
            Object.entries(value || {}).forEach(([code, cfg]) => {
                batch.set(db.collection('departments').doc(code), { code, ...cfg });
            });

            // Delete removed departments too, or they come back on reload.
            const existing = await db.collection('departments').get();
            let removed = 0;
            existing.forEach(doc => {
                if (!keep.has(doc.id)) { batch.delete(doc.ref); removed++; }
            });

            await batch.commit();
            if (removed) console.log('🗑 Removed', removed, 'deleted department(s)');
            console.log('✅ Synced departments to Firestore');
        } catch(e) { console.warn('Firestore sync error (departments):', e.message); }
        return;
    }

    // customCourses: one doc per dept in 'courses' (courses/{DEPT_CODE}).
    if (key === 'customCourses') {
        try {
            const coursesMap = value || {};
            const entries = Object.entries(coursesMap);

            // No early return on an empty map, so clearing the last one still saves.
            const batch = db.batch();
            const keep  = new Set(entries.map(([deptCode]) => String(deptCode)));
            entries.forEach(([deptCode, list]) => {
                batch.set(db.collection('courses').doc(deptCode), {
                    dept: deptCode,
                    courses: Array.isArray(list) ? list : []
                });
            });

            // Delete departments dropped from the map.
            const existing = await db.collection('courses').get();
            let removed = 0;
            existing.forEach(doc => {
                if (!keep.has(doc.id)) { batch.delete(doc.ref); removed++; }
            });

            await batch.commit();
            if (removed) console.log('🗑 Removed', removed, 'department(s) from the courses collection');
            console.log('✅ Synced customCourses to Firestore courses collection:', Object.keys(coursesMap));
        } catch(e) { console.warn('Firestore sync error (customCourses):', e.message); }
        return;
    }

    const MAP = {
        students: 'students',
        teachers: 'teachers',
        subjects: 'subjects',
        evaluations: 'evaluations',
        schoolYears: 'schoolYears',
        auditLog: 'auditLog',
        evalPeriod: 'settings',
        finalReports: 'finalReports',
        customDepartments: 'customDepartments',
        // questionSets must reach Firestore - the Android app reads it.
        questionSets: 'questionSets',
        developmentPlans: 'developmentPlans',
        exemptions: 'exemptions',
        reportSignatories: 'settings'
    };
    const col = MAP[key];
    if (!col) return;

    try {
```

```
        if (Array.isArray(value)) {
            const batch = db.batch();
            const keep  = new Set();
            value.forEach(item => {
                if (item && item.id) {
                    keep.add(String(item.id));
                    // Merge students/teachers so a copy without a password doesn't erase it.
                    if (key === 'students' || key === 'teachers') {
                        batch.set(db.collection(col).doc(item.id), item, { merge: true });
                    } else {
                        batch.set(db.collection(col).doc(item.id), item);
                    }
                }
            });

            // DELETE documents no longer in the local copy, except append-only
            // collections (auditLog, evaluations).
            const MANAGED_AS_A_WHOLE = ['students', 'teachers', 'subjects',
                                        'schoolYears', 'customDepartments',
                                        'questionSets', 'exemptions'];
            if (MANAGED_AS_A_WHOLE.indexOf(key) !== -1) {
                const existing = await db.collection(col).get();
                let removed = 0;
                existing.forEach(doc => {
                    if (!keep.has(doc.id)) { batch.delete(doc.ref); removed++; }
                });
                if (removed) console.log(`🗑 Removing ${removed} deleted item(s) from ${col}`);
            }

            await batch.commit();
            console.log(`✅ Synced ${value.length} items to ${col}`);
        } else if (value && typeof value === 'object') {
            // Keyed objects (customDepartments): one doc per key, delete missing ones.
            const byId = Object.keys(value).every(k => value[k] && typeof value[k] === 'object');
            if (byId && key === 'customDepartments') {
                const batch = db.batch();
                const keep  = new Set(Object.keys(value).map(String));
                Object.keys(value).forEach(k => batch.set(db.collection(col).doc(k), value[k]));

                const existing = await db.collection(col).get();
                let removed = 0;
                existing.forEach(doc => {
                    if (!keep.has(doc.id)) { batch.delete(doc.ref); removed++; }
                });
                await batch.commit();
                if (removed) console.log(`🗑 Removed ${removed} deleted entry(ies) from ${col}`);
                console.log(`✅ Synced ${Object.keys(value).length} entries to ${col}`);
            } else {
                await db.collection(col).doc(key).set(value);
                console.log(`✅ Synced ${key} to ${col}`);
            }
        }
    } catch(e) {
        console.warn('Firestore sync error:', e.message);
    }
};

// Force a full sync of all data to Firebase on page load
window.fullSyncToFirebase = async function() {
    console.log('🔄 Performing full sync to Firebase...');
    // Collections retried by full sync. questionSets is read by the Android app.
    // students/teachers use per-record saves (commitRecords) and evaluations
    // arrive live, so they're excluded.
    const keys = ['subjects', 'schoolYears', 'auditLog', 'evalPeriod', 'finalReports', 'customDepartments', 'customCourses', 'questionSets', 'develop
mentPlans', 'exemptions', 'reportSignatories'];

    for (const key of keys) {
        const data = localStorage.getItem(key);
        if (data) {
            try {
                await syncCollectionToFirestore(key, JSON.parse(data));
            } catch(e) {
                console.warn(`Failed to sync ${key}:`, e);
            }
        }
    }
    console.log('✅ Full sync complete');
};

// Override setData to sync with Firebase
window.originalSetData = window.setData;
// REMOVED duplicate setData() — byte-identical to the copy in admin.js.

// Full sync ONLY after a clean load from Firestore (_firestoreLoadClean),
// since it also deletes documents missing locally.
setTimeout(() => {
    if (typeof firebase === 'undefined' || !firebase.firestore) return;
    if (window._firestoreLoadClean !== true) {
        console.warn('Full sync skipped: the load from Firestore was incomplete, '
                   + 'so the local copy cannot be trusted as the newer one.');
        return;
    }
    fullSyncToFirebase();
}, 2000);

// REMOVED: stale saveStudent override; admin.js's is used.

// REMOVED: openEditStudentModal wrapper that discarded admin.js's
// dept-from-course fallback.

// REMOVED dead escapeHtml(); admin.js defines it.

const DEPT_NAMES = {
    COED: 'College of Education',
    CCJS: 'Criminal Justice & Safety',
```

```
    CCIS: 'Computing & Information Sciences',
    CON: 'College of Nursing',
    CEA: 'Engineering & Architecture',
    COM: 'College of Management',
    CAT: 'Agriculture & Technology',
    GS: 'Graduate School'
};

// Fix renderStudents
if (typeof window.renderStudents === 'function') {
  window.renderStudents = function(search = '') {
    const students = getData('students', []).filter(s => !s.deleted);
    const filtered = students.filter(s => s.name.toLowerCase().includes(search.toLowerCase()) || s.sid.includes(search));
    const tbody = document.getElementById('studentsTbody');
    if (!filtered.length) {
      tbody.innerHTML = '<tr><td colspan="7" style="text-align:center;padding:32px;">No students found.</td></tr>';
      return;
    }

    tbody.innerHTML = filtered.map(s => {
      const deptDisplay = s.dept && DEPT_NAMES[s.dept] ? DEPT_NAMES[s.dept] : (s.dept || '—');
      return `<tr>
        <td><span style="font-family:monospace;">${escapeHtml(s.sid)}</span></td>
        <td><strong>${escapeHtml(listName(s))}</strong></td>
        <td>${escapeHtml(s.year)}</td>
        <td>Sec ${escapeHtml(s.section)}</td>
        <td>${escapeHtml(deptDisplay)}</td>
        <td><span class="badge ${s.status === 'active' ? 'badge-success' : 'badge-danger'}">${s.status}</span></td>
        <td><div class="td-actions" style="display:flex;gap:6px;">
          <button class="btn btn-ghost btn-sm" onclick="openEditStudentModal('${s.id}')">✏️</button>
          <button class="btn btn-ghost btn-sm" onclick="toggleStudentStatus('${s.id}')">🔄</button>
          <button class="btn btn-ghost btn-sm" onclick="deleteStudent('${s.id}')">🗑️</button>
        </div></td>
      </tr>`;
    }).join('');
  };
}

// renderTeachers: regular faculty grouped by department, no supervisors.
if (typeof window.renderTeachers === 'function') {
    window.renderTeachers = function(search = '') {
        // 1. Sync search input (same pattern as admin.js)
        const searchInput = document.getElementById('teacherSearchInput');
        if (searchInput && search && searchInput.value !== search) searchInput.value = search;
        const liveSearch = searchInput ? searchInput.value : search;

        // 2. Rebuild dept filter pills (delegates to adminfeatures.js)
        if (typeof buildTeacherDeptPills === 'function') buildTeacherDeptPills();

        // 3. Active dept filter: _teacherDeptFilter, else the bar's dataset.
        const deptActive = (typeof _teacherDeptFilter !== 'undefined' ? _teacherDeptFilter : null)
            ?? (document.getElementById('teacherDeptFilterBar')?.dataset.active || '');

        const teachers = getData('teachers', []).filter(t => !t.deleted);
        const subjects = getData('subjects', []);

        // 4. CRITICAL: Filter out supervisors so they don't appear in the Faculty table
        const regularFaculty = teachers.filter(t => t.facultyType !== 'supervisor');

        // 5. Apply Search + Dept Filter
        const q = liveSearch.toLowerCase();
        const filtered = regularFaculty.filter(t => {
            const matchesSearch = !q ||
                t.name.toLowerCase().includes(q) ||
                t.tid.toLowerCase().includes(q) ||
                (t.dept || '').toLowerCase().includes(q);
            // Via deptFilterKey (adminfeatures.js) so "No Dept" matches a blank department.
            const matchesDept = !deptActive || deptFilterKey(t.dept) === deptActive;
            return matchesSearch && matchesDept;
        });
        // Order from the ID / Name headers (teacherListCompare, admin-teachers.js).
        filtered.sort(typeof teacherListCompare === 'function' ? teacherListCompare : byName);
        if (typeof paintTeacherSortArrows === 'function') paintTeacherSortArrows();

        const tbody = document.getElementById('teachersTbody');
        if (!tbody) return;

        if (!filtered.length) {
            tbody.innerHTML = '<tr><td colspan="6" style="text-align:center;padding:32px;color:var(--muted);">No regular faculty found.</td></tr>';
            if (typeof renderSupervisorTable === 'function') renderSupervisorTable(liveSearch);
            return;
        }

        // 6. Group Regular Faculty by Department
        const groupedByDept = {};
        filtered.forEach(t => {
            const deptKey = t.dept || 'UNASSIGNED';
            if (!groupedByDept[deptKey]) groupedByDept[deptKey] = [];
            groupedByDept[deptKey].push(t);
        });

        // 4. Generate HTML with Department Category Headers
        let html = '';
        const DEPT_NAMES = {
            COED: 'College of Education',
            CCJS: 'Criminal Justice & Safety',
            CCIS: 'Computing & Information Sciences',
            CON: 'College of Nursing',
            CEA: 'Engineering & Architecture',
            COM: 'College of Management',
            CAT: 'Agriculture & Technology',
            GS: 'Graduate School',
            UNASSIGNED: 'Unassigned Department'
        };
```

```
        // Sort depts in known order
        const deptOrder = ['COED','CCJS','CCIS','CON','CEA','COM','CAT','GS','UNASSIGNED'];
        const sortedDepts = [
            ...deptOrder.filter(d => groupedByDept[d]),
            ...Object.keys(groupedByDept).filter(d => !deptOrder.includes(d))
        ];

        sortedDepts.forEach(deptCode => {
            const facultyList = groupedByDept[deptCode];
            const deptLabel = DEPT_NAMES[deptCode] || deptCode;

            // Department group header row — matches student grouping style
            html += `
            <tr class="dept-group-header-row">
                <td colspan="6">
                    <div class="dept-group-header">
                        <svg width="13" height="13" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M20 21v-2a4 4 0 0
0-4-4H8a4 4 0 00-4 4v2"/><circle cx="12" cy="7" r="4"/></svg>
                        <span>${escapeHtml(deptLabel)}</span>
                        <span class="dept-group-count">${facultyList.length} teacher${facultyList.length !== 1 ? 's' : ''}</span>
                    </div>
                </td>
            </tr>`;

            facultyList.forEach(t => {
                const teacherSubs = subjects.filter(s => subjectHasTeacher(s, t.id));
                html += `
                <tr class="dept-group-student-row teacher-row" onclick="showAnnexReports('${t.id}')" title="Click to view Annex C & D" style="cursor:
pointer;">
                    <td><span style="font-family:'JetBrains Mono',monospace;font-weight:600;">${escapeHtml(t.tid)}</span></td>
                    <td><strong>${escapeHtml(listName(t))}</strong></td>
                    <td>${deptCode === 'UNASSIGNED'
                        ? '<span style="color:var(--muted);">\u2014</span>'
                        : `<span class="dept-tag-inline">${escapeHtml(deptCode)}</span>`}</td>
                    <td><span class="badge badge-primary">${teacherSubs.length} Subject${teacherSubs.length !== 1 ? 's' : ''}</span></td>
                    <td><span class="badge ${t.status === 'active' ? 'badge-success' : 'badge-danger'}">${t.status}</span></td>
                    <td>
                        <div class="td-actions" onclick="event.stopPropagation()">
                            <button class="btn btn-ghost btn-icon btn-sm" onclick="openEditTeacherModal('${t.id}')" title="Edit"><svg width="14" heig
ht="14" fill="none" stroke="var(--primary)" stroke-width="2" viewBox="0 0 24 24"><path d="M11 4H4a2 2 0 00-2 2v14a2 2 0 002 2h14a2 2 0 002-2v-7"/><pa
th d="M18.5 2.5a2.121 2.121 0 013 3L12 15l-4 1 1-4 9.5-9.5z"/></svg></button>
                            <button class="btn btn-ghost btn-icon btn-sm" onclick="toggleTeacherStatus('${t.id}')" title="Toggle Status"><svg width
="14" height="14" fill="none" stroke="${t.status === 'active' ? 'var(--muted)' : 'var(--success)'}" stroke-width="2" viewBox="0 0 24 24"><path d="M1
8.36 6.64a9 9 0 11-12.73 0"/><line x1="12" y1="2" x2="12" y2="12"/></svg></button>
                            <button class="btn btn-ghost btn-icon btn-sm" onclick="deleteTeacher('${t.id}')" title="Delete"><svg width="14" height="1
4" fill="none" stroke="var(--danger)" stroke-width="2" viewBox="0 0 24 24"><polyline points="3 6 5 6 21 6"/><path d="M19 6l-1 14a2 2 0 01-2 2H8a2 2 0
01-2-2L5 6"/></svg></button>
                        </div>
                    </td>
                </tr>`;
            });
        });

        tbody.innerHTML = html;

        // 7. Always trigger the supervisor table update alongside this one
        if (typeof window.renderSupervisorTable === 'function') {
            window.renderSupervisorTable(liveSearch);
        }
    };
}

// renderSubjects with dept filter pills and grouped table layout.
if (typeof window.renderSubjects === 'function') {
    window.renderSubjects = function(search = '') {
        // Rebuild dept filter pill bar with correct active state
        if (typeof buildSubjectDeptPills === 'function') buildSubjectDeptPills();

        const subjects = getData('subjects', []);
        const teachers = getData('teachers', []);
        const students = getData('students', []);

        // Apply BOTH the dept filter pill and the search query
        const deptFilter = (window._subjectDeptFilter || '');
        const filtered = subjects.filter(s =>
            (s.name.toLowerCase().includes(search.toLowerCase()) || s.code.toLowerCase().includes(search.toLowerCase())) &&
            (!deptFilter || s.dept === deptFilter)
        );

        const tbody = document.getElementById('subjectsTbody');
        if (!tbody) return;

        if (!filtered.length) {
            tbody.innerHTML = '<tr><td colspan="7" style="text-align:center;padding:32px;color:var(--muted);">No subjects found.</td></tr>';
            return;
        }

        // Group by department (matches admin.js dept-grouped layout)
        const groups = {};
        filtered.forEach(sub => {
            const dept = sub.dept || 'Unassigned';
            if (!groups[dept]) groups[dept] = [];
            groups[dept].push(sub);
        });

        let html = '';
        Object.entries(groups).forEach(([dept, subs]) => {
            html += `<tr class="dept-group-header-row"><td colspan="7"><div class="dept-group-header">
              <svg width="12" height="12" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M4 19.5A2.5 2.5 0 016.5 17H
20"/><path d="M6.5 2H20v20H6.5A2.5 2.5 0 014 19.5v-15A2.5 2.5 0 016.5 2z"/></svg>
              ${escapeHtml(dept)}<span class="dept-group-count">${subs.length} subject${subs.length !== 1 ? 's' : ''}</span>
            </div></td></tr>`;
            subs.forEach(sub => {
                // Teacher cell opens Teachers & Sections.
                const tIds = (typeof subjectTeacherIds === 'function') ? subjectTeacherIds(sub)
```

```
                        : (sub.teacherId ? [sub.teacherId] : []);
                const names = tIds.map(id => (teachers.find(t => t.id === id) || {}).name).filter(Boolean);
                // Several teachers show as one MULTIPLE tag.
                const teacherCell = names.length > 1
                    ? `<span class="tcell-multi">MULTIPLE</span>`
                      + `<span class="tcell-count">${names.length} teachers</span>`
                    : names.length === 1
                      ? `<span class="tcell-names">${escapeHtml(names[0])}</span>`
                      : '<span class="tcell-none">Not assigned \u2014 click to add</span>';
                const enrolled = (sub.enrolledIds || []).filter(eid => students.find(s => s.id === eid && !s.deleted)).length;
                html += `<tr class="dept-group-student-row">
                    <td><span style="font-family:'JetBrains Mono',monospace;font-weight:700;">${escapeHtml(sub.code)}</span></td>
                    <td><strong>${escapeHtml(sub.name)}</strong></td>
                    <td>${escapeHtml(sub.dept || '—')}</td>
                    <td>${escapeHtml(sub.category || '—')}</td>
                    <td class="tcell" role="button" tabindex="0"
                        title="${names.length ? escapeHtml(names.join(', ')) + (names.length > 1 ? ' \u2014 click to see each teacher' : ' \u2014 click to manage') : 'Click to assign teachers'}"
                        onclick="${names.length > 1 ? `openSubjectTeacherList('${sub.id}')` : `openSubjectTeachersModal('${sub.id}')`}"
                        onkeydown="if(event.key==='Enter'||event.key===' '){event.preventDefault();${names.length > 1 ? `openSubjectTeacherList('${sub.id}')` : `openSubjectTeachersModal('${sub.id}')`};}"
                    >${teacherCell}</td>
                    <!-- nowrap: the pill used to break into "30 / students" on rows
                         where a long subject name squeezed this column. -->
                    <td><span class="badge badge-primary" style="white-space:nowrap;">${enrolled} student${enrolled !== 1 ? 's' : ''}</span></td>
                    <td><div class="td-actions">
                        <button class="btn btn-ghost btn-icon btn-sm" onclick="openEditSubjectModal('${sub.id}')"><svg width="14" height="14" fill="none" stroke="var(--primary)" stroke-width="2" viewBox="0 0 24 24"><path d="M11 4H4a2 2 0 00-2 2v14a2 2 0 002 2h14a2 2 0 002-2v-7"/><path d="M18.5 2.5a2.121 2.121 0 013 3L12 15l-4 1 1-4 9.5-9.5z"/></svg></button>
                        <button class="btn btn-ghost btn-icon btn-sm" onclick="openSubjectTeachersModal('${sub.id}')" title="Teachers &amp; sections"><svg width="14" height="14" fill="none" stroke="var(--primary)" stroke-width="2" viewBox="0 0 24 24"><path d="M12 14l9-5-9-5-9 5 9 5z"/><path d="M12 14l6.16-3.42a12 12 0 01.84 4.42 12 12 0 01-7 1 12 12 0 01-7-1 12 12 0 01.84-4.42L12 14z"/></svg></button>
                        <button class="btn btn-ghost btn-icon btn-sm" onclick="openEnrollModal('${sub.id}')"><svg width="14" height="14" fill="none" stroke="var(--success)" stroke-width="2" viewBox="0 0 24 24"><path d="M17 21v-2a4 4 0 00-4-4H5a4 4 0 00-4 4v2"/><circle cx="9" cy="7" r="4"/><path d="M23 21v-2a4 4 0 00-3-3.87M16 3.13a4 4 0 010 7.75"/></svg></button>
                        <button class="btn btn-ghost btn-icon btn-sm" onclick="deleteSubject('${sub.id}')"><svg width="14" height="14" fill="none" stroke="var(--danger)" stroke-width="2" viewBox="0 0 24 24"><polyline points="3 6 5 6 21 6"/><path d="M19 6l-1 14a2 2 0 01-2 2H8a2 2 0 01-2-2L5 6"/></svg></button>
                    </div></td>
                </tr>`;
            });
        });
        tbody.innerHTML = html;
    };
}
```

admin/js/records/admin-bulk.js

325 lines

```
// admin-bulk.js - CSV bulk upload for student rosters and subjects.
// Load order matters (see dashboard.html): the last definition wins.

// ===== BULK UPLOAD =====
window.previewBulkStudents = function(input) {
  const file = input.files[0];
  if (!file) return;
  const reader = new FileReader();
  reader.onload = function(e) {
    // Strip Excel's UTF-8 byte-order mark.
    const lines = e.target.result.replace(/^\uFEFF/, '').split('\n').map(l => l.trim()).filter(Boolean);
    const parsed = [];
    lines.forEach(line => {
      const cols = line.split(',').map(c => c.trim().replace(/^"|"$/g,''));
      if (cols.length < 4) return;
      // CSV format: ID, Name, Year, Section, Course, Department, Subject Codes
      parsed.push({
        sid: cols[0], name: cols[1], year: cols[2] || '1st Year',
        section: cols[3], course: cols[4] || '', dept: cols[5] || '',
        subjectCodes: cols[6] ? cols[6].split(';').map(s=>s.trim()).filter(Boolean) : []
      });
    });
    window._bulkStudentData = parsed;
    const preview = document.getElementById('bulkStudentPreview');
    const btn = document.getElementById('bulkStudentImportBtn');
    if (!parsed.length) { preview.innerHTML = '<p style="color:var(--danger);font-size:0.8rem;">No valid rows found.</p>'; btn.style.display='none';
return; }
    preview.innerHTML = `<p style="font-size:0.8rem;color:var(--muted);margin-bottom:8px;">Preview: ${parsed.length} student(s) to import</p>
      <div style="max-height:200px;overflow-y:auto;border:1px solid var(--border);border-radius:6px;">
        <table style="width:100%;border-collapse:collapse;font-size:0.78rem;">
          <thead><tr style="background:var(--bg);">
            <th style="padding:6px 8px;text-align:left;">ID</th>
            <th style="padding:6px 8px;text-align:left;">Name</th>
            <th style="padding:6px 8px;text-align:left;">Year</th>
            <th style="padding:6px 8px;text-align:left;">Sec</th>
            <th style="padding:6px 8px;text-align:left;">Course</th>
            <th style="padding:6px 8px;text-align:left;">Dept</th>
            <th style="padding:6px 8px;text-align:left;">Subjects</th>
          </tr></thead>
          <tbody>${parsed.map(r=>`<tr style="border-bottom:1px solid var(--border);">
            <td style="padding:6px 8px;">${escapeHtml(r.sid)}</td>
            <td style="padding:6px 8px;">${escapeHtml(r.name)}</td>
            <td style="padding:6px 8px;">${escapeHtml(r.year)}</td>
            <td style="padding:6px 8px;">${escapeHtml(r.section)}</td>
            <td style="padding:6px 8px;font-size:0.72rem;">${escapeHtml(r.course||'—')}</td>
            <td style="padding:6px 8px;">${escapeHtml(r.dept)}</td>
            <td style="padding:6px 8px;">${r.subjectCodes.join(', ')||'—'}</td>
          </tr>`).join('')}</tbody>
        </table>
      </div>`;
    btn.style.display = '';
  };
  reader.readAsText(file);
};

// Guards against double-click importing twice.
let _importingStudents = false;

window.importBulkStudents = async function() {
  const rows = window._bulkStudentData || [];
  if (!rows.length || _importingStudents) return;
  _importingStudents = true;
  const btn = document.getElementById('bulkStudentImportBtn');
  const btnText = btn ? btn.textContent : '';
  if (btn) { btn.disabled = true; btn.textContent = 'Importing\u2026'; }
  try {
    const students = getData('students', []);
    const subjects = getData('subjects', []);
    // Normalize IDs so the duplicate check catches them.
    rows.forEach(r => { if (r.sid) r.sid = normalizeStudentId(r.sid); });

    // Check IDs against the DATABASE; deleted records are ignored.
    let inDb;
    try {
      inDb = await findRosterRecordsMany('students', 'sid', rows.map(r => r.sid));
    } catch (e) {
      console.error('Bulk duplicate check failed:', e);
      showToast('Could not check the database for these IDs. Check your connection and try again.', 'error');
      return;
    }

    let added = 0, skipped = 0;
    const changes = [];
    const seen = new Set();                       // the same ID twice in one CSV
    rows.forEach(r => {
      if (!r.sid || !r.name || seen.has(r.sid)) { skipped++; return; }
      const found = inDb[r.sid] || [];
      if (students.find(s => s.sid === r.sid && !s.deleted) || found.some(m => !m.deleted)) { skipped++; return; }
      if (!STUDENT_ID_RE.test(r.sid)) { skipped++; return; }
      seen.add(r.sid);

      // forceReset TRUE - see admin-students.js.
      const fresh = { sid: r.sid, name: r.name, course: normalizeCourseName(r.course), year: r.year,
                      section: String(r.section || '').trim().toUpperCase(), dept: r.dept,
                      password: r.sid, status: 'active', forceReset: true, deleted: false };
      const recId = 'stu' + Date.now() + Math.random().toString(36).slice(2,6);
      const rec = Object.assign({ id: recId }, fresh);
      students.push(rec);
      changes.push({ before: null, after: rec });
      added++;
```

```
      r.subjectCodes.forEach(code => {
        const sub = subjects.find(s => s.code.toLowerCase() === code.toLowerCase());
        if (sub) { if (!sub.enrolledIds) sub.enrolledIds = []; if (!sub.enrolledIds.includes(recId)) sub.enrolledIds.push(recId); }
      });
    });

    const ok = await commitRecordsBatch('students', students, changes);
    if (!ok) return;
    setData('subjects', subjects);
    addAudit('Bulk Upload Students', `Imported ${added} students, skipped ${skipped}`);
    closeModal('bulkUploadStudentModal');
    renderStudents();
    showToast(`Imported ${added} students! ${skipped ? skipped + ' skipped.' : ''}`, 'success');
  } finally {
    _importingStudents = false;
    if (btn) { btn.disabled = false; btn.textContent = btnText; }
  }
};

window.previewBulkTeachers = function(input) {
  const file = input.files[0];
  if (!file) return;
  const reader = new FileReader();
  reader.onload = function(e) {
    // Strip Excel's UTF-8 byte-order mark.
    const lines = e.target.result.replace(/^\uFEFF/, '').split('\n').map(l => l.trim()).filter(Boolean);
    const parsed = [];
    lines.forEach(line => {
      const cols = line.split(',').map(c => c.trim().replace(/^"|"$/g,''));
      if (cols.length < 2) return;
      parsed.push({
        tid: cols[0], name: cols[1], dept: cols[2] || '',
        facultyType: (cols[3] || 'regular').toLowerCase().includes('super') ? 'supervisor' : 'regular',
        subjectCodes: cols[4] ? cols[4].split(';').map(s=>s.trim()).filter(Boolean) : []
      });
    });
    window._bulkTeacherData = parsed;
    const preview = document.getElementById('bulkTeacherPreview');
    const btn = document.getElementById('bulkTeacherImportBtn');
    if (!parsed.length) { preview.innerHTML = '<p style="color:var(--danger);font-size:0.8rem;">No valid rows found.</p>'; btn.style.display='none';
return; }
    preview.innerHTML = `<p style="font-size:0.8rem;color:var(--muted);margin-bottom:8px;">Preview: ${parsed.length} teacher(s) to import</p>
      <div style="max-height:200px;overflow-y:auto;border:1px solid var(--border);border-radius:6px;">
        <table style="width:100%;border-collapse:collapse;font-size:0.78rem;">
          <thead><tr style="background:var(--bg);"><th style="padding:6px 8px;text-align:left;">ID</th><th style="padding:6px 8px;text-align:left;">N
ame</th><th style="padding:6px 8px;text-align:left;">Dept</th><th style="padding:6px 8px;text-align:left;">Type</th><th style="padding:6px 8px;text-a
lign:left;">Subjects</th></tr></thead>
          <tbody>${parsed.map(r=>`<tr style="border-bottom:1px solid var(--border);"><td style="padding:6px 8px;">${escapeHtml(r.tid)}</td><td style
="padding:6px 8px;">${escapeHtml(r.name)}</td><td style="padding:6px 8px;">${escapeHtml(r.dept)}</td><td style="padding:6px 8px;">${r.facultyType}</t
d><td style="padding:6px 8px;">${r.subjectCodes.join(', ')||'—'}</td></tr>`).join('')}</tbody>
        </table>
      </div>`;
    btn.style.display = '';
  };
  reader.readAsText(file);
};

let _importingTeachers = false;

window.importBulkTeachers = async function() {
  const rows = window._bulkTeacherData || [];
  if (!rows.length || _importingTeachers) return;
  _importingTeachers = true;
  const btn = document.getElementById('bulkTeacherImportBtn');
  const btnText = btn ? btn.textContent : '';
  if (btn) { btn.disabled = true; btn.textContent = 'Importing\u2026'; }
  try {
    const teachers = getData('teachers', []);
    const subjects = getData('subjects', []);

    // Same database check as the student import.
    let inDb;
    try {
      inDb = await findRosterRecordsMany('teachers', 'tid', rows.map(r => r.tid));
    } catch (e) {
      console.error('Bulk duplicate check failed:', e);
      showToast('Could not check the database for these IDs. Check your connection and try again.', 'error');
      return;
    }

    let added = 0, skipped = 0;
    const changes = [];
    const seen = new Set();
    rows.forEach(r => {
      if (!r.tid || !r.name || seen.has(r.tid)) { skipped++; return; }
      const found = inDb[r.tid] || [];
      if (teachers.find(t => t.tid === r.tid && !t.deleted) || found.some(m => !m.deleted)) { skipped++; return; }
      seen.add(r.tid);

      // Always a new record - a deleted teacher with this ID stays deleted.
      const recId = 'tch' + Date.now() + Math.random().toString(36).slice(2,6);
      const rec = { id: recId, tid: r.tid, name: r.name, dept: r.dept, facultyType: r.facultyType, status: 'active', deleted: false };
      teachers.push(rec);
      changes.push({ before: null, after: rec });
      added++;
      r.subjectCodes.forEach(code => {
        const sub = subjects.find(s => s.code.toLowerCase() === code.toLowerCase());
        // Add this teacher to the subject's teachers - a subject can have several.
        if (sub) {
          const list = subjectTeachers(sub);
          if (!list.some(t => t.id === recId)) list.push({ id: recId, sections: [] });
          sub.teachers = list;
          if (!sub.teacherId) sub.teacherId = recId;
        }
      });
```

```
    });

    const ok = await commitRecordsBatch('teachers', teachers, changes);
    if (!ok) return;
    setData('subjects', subjects);
    addAudit('Bulk Upload Teachers', `Imported ${added} teachers, skipped ${skipped}`);
    closeModal('bulkUploadTeacherModal');
    renderTeachers();
    showToast(`Imported ${added} teachers! ${skipped ? skipped + ' skipped.' : ''}`, 'success');
  } finally {
    _importingTeachers = false;
    if (btn) { btn.disabled = false; btn.textContent = btnText; }
  }
};

// ===== BULK UPLOAD SUBJECTS =====
// CSV: Subject Code, Subject Name, Department, Category, Load Type, Teacher ID (TID), Student IDs (semicolon-separated SIDs)
window.previewBulkSubjects = function(input) {
  const file = input.files[0];
  if (!file) return;
  const reader = new FileReader();
  reader.onload = function(e) {
    // Strip Excel's UTF-8 byte-order mark.
    const lines = e.target.result.replace(/^\uFEFF/, '').split('\n').map(l => l.trim()).filter(Boolean);
    const parsed = [];
    lines.forEach((line, i) => {
      if (i === 0 && line.toLowerCase().startsWith('subject')) return;
      const cols = line.split(',').map(c => c.trim().replace(/^"|"$/g, ''));
      if (cols.length < 2) return;
      parsed.push({
        code:        cols[0] || '',
        name:        cols[1] || '',
        dept:        cols[2] || '',
        category:    cols[3] || '',
        loadType:    cols[4] || 'Regular',
        teacherTid:  cols[5] || '',
        studentSids: cols[6] ? cols[6].split(';').map(s => s.trim()).filter(Boolean) : []
      });
    });
    window._bulkSubjectData = parsed;
    const preview = document.getElementById('bulkSubjectPreview');
    const btn = document.getElementById('bulkSubjectImportBtn');
    if (!parsed.length) { preview.innerHTML = '<p style="color:var(--danger);font-size:0.8rem;">No valid rows found.</p>'; btn.style.display = 'non
e'; return; }

    const teachers = getData('teachers', []).filter(t => !t.deleted);
    const students = getData('students', []).filter(s => !s.deleted);
    const existing = new Set(getData('subjects', []).map(s => s.code.toLowerCase()));
    const newCount  = parsed.filter(r => r.code && !existing.has(r.code.toLowerCase())).length;
    const skipCount = parsed.length - newCount;

    preview.innerHTML = `
      <p style="font-size:0.8rem;color:var(--muted);margin-bottom:8px;">Preview: <strong>${parsed.length}</strong> row(s) — <span style="color:#16a34
a;font-weight:600;">${newCount} new</span>${skipCount ? `, <span style="color:#d97706;font-weight:600;">${skipCount} skipped (code exists)</span>` :
''}</p>
      <div style="max-height:200px;overflow-y:auto;border:1px solid var(--border);border-radius:6px;">
        <table style="width:100%;border-collapse:collapse;font-size:0.78rem;">
          <thead><tr style="background:var(--bg);">
            <th style="padding:6px 8px;text-align:left;">Code</th>
            <th style="padding:6px 8px;text-align:left;">Subject Name</th>
            <th style="padding:6px 8px;text-align:left;">Dept</th>
            <th style="padding:6px 8px;text-align:left;">Load</th>
            <th style="padding:6px 8px;text-align:left;">Teacher</th>
            <th style="padding:6px 8px;text-align:left;">Students</th>
            <th style="padding:6px 8px;text-align:left;">Status</th>
          </tr></thead>
          <tbody>${parsed.map(r => {
            const isDup = existing.has((r.code || '').toLowerCase());
            const teacher = r.teacherTid ? teachers.find(t => t.tid.toLowerCase() === r.teacherTid.toLowerCase()) : null;
            const tchLabel = r.teacherTid ? (teacher ? escapeHtml(teacher.name) : `\u26a0 "${escapeHtml(r.teacherTid)}" not found`) : '\u2014';
            const tchColor = r.teacherTid && !teacher ? 'color:#d97706;' : '';
            const stuResolved = r.studentSids.map(sid => { const s = students.find(st => st.sid.toLowerCase() === sid.toLowerCase()); return s ? s.na
me : `\u26a0 ${sid}`; });
            const stuLabel = stuResolved.length ? escapeHtml(stuResolved.slice(0,2).join(', ') + (stuResolved.length > 2 ? ` +${stuResolved.length-2}
more` : '')) : '\u2014';
            const rowBg = isDup ? 'background:#fff7ed;' : '';
            return `<tr style="border-bottom:1px solid var(--border);${rowBg}">
              <td style="padding:6px 8px;font-family:'JetBrains Mono',monospace;font-weight:700;">${escapeHtml(r.code)}</td>
              <td style="padding:6px 8px;">${escapeHtml(r.name)}</td>
              <td style="padding:6px 8px;">${escapeHtml(r.dept||'\u2014')}</td>
              <td style="padding:6px 8px;">${escapeHtml(r.loadType)}</td>
              <td style="padding:6px 8px;${tchColor}">${tchLabel}</td>
              <td style="padding:6px 8px;font-size:0.72rem;">${stuLabel}</td>
              <td style="padding:6px 8px;">${isDup ? '<span style="color:#d97706;font-weight:600;">\u26a0 Skip</span>' : '<span style="color:#16a34a;
font-weight:600;">\u2713 Add</span>'}</td>
            </tr>`;
          }).join('')}</tbody>
        </table>
      </div>`;
    btn.style.display = '';
  };
  reader.readAsText(file);
};

window.importBulkSubjects = function() {
  const rows = window._bulkSubjectData || [];
  if (!rows.length) return;
  const subjects = getData('subjects', []);
  const teachers = getData('teachers', []).filter(t => !t.deleted);
  const students = getData('students', []).filter(s => !s.deleted);
  const existing = new Set(subjects.map(s => s.code.toLowerCase()));
  let added = 0, skipped = 0, enrolled = 0;
  const badTeachers = [], badStudents = [];
  rows.forEach(r => {
    if (!r.code || !r.name) { skipped++; return; }
```

```
    if (existing.has(r.code.toLowerCase())) { skipped++; return; }
    let teacherId = '';
    if (r.teacherTid) {
      const t = teachers.find(t => t.tid.toLowerCase() === r.teacherTid.toLowerCase());
      if (t) teacherId = t.id; else badTeachers.push(r.teacherTid);
    }
    const enrolledIds = [];
    r.studentSids.forEach(sid => {
      const s = students.find(st => st.sid.toLowerCase() === sid.toLowerCase());
      if (s) { enrolledIds.push(s.id); enrolled++; } else badStudents.push(sid);
    });
    subjects.push({ id: 'sub'+Date.now()+Math.random().toString(36).slice(2,6), code: r.code, name: r.name, dept: r.dept, category: r.category, loadT
ype: r.loadType||'Regular', isLabSchool: false, teacherId, teachers: teacherId ? [{ id: teacherId, sections: [] }] : [], enrolledIds });
    existing.add(r.code.toLowerCase());
    added++;
  });
  setData('subjects', subjects);
  let detail = `Imported ${added} subjects, skipped ${skipped}. ${enrolled} student(s) enrolled.`;
  if (badTeachers.length) detail += ` Unmatched teachers: ${[...new Set(badTeachers)].join(', ')}.`;
  if (badStudents.length) detail += ` Unmatched students: ${[...new Set(badStudents)].join(', ')}.`;
  addAudit('Bulk Upload Subjects', detail);
  closeModal('bulkUploadSubjectModal');
  renderSubjects();
  let msg = `Imported ${added} subject${added!==1?'s':''}!`;
  if (skipped) msg += ` ${skipped} skipped.`;
  if (badTeachers.length) msg += ` \u26a0 ${[...new Set(badTeachers)].length} teacher ID(s) not found.`;
  if (badStudents.length) msg += ` \u26a0 ${[...new Set(badStudents)].length} student ID(s) not found.`;
  showToast(msg, added > 0 ? 'success' : 'warning');
};
```

admin/js/records/admin-courses.js

496 lines

```
// admin-courses.js - supervisor data migration, course-by-department map, page bootstrap.
// syncCollectionToFirestore lives in adminrate.js. Load order matters: the last definition wins.

// ===== SUPERVISOR DATA MIGRATION =====
// Only touches records where facultyType was never set; never overrides an admin's choice.
window.migrateSupervisorRecords = function() {
  const teachers = getData('teachers', []);
  // Writes only the records it changes.
  const changes = [];
  teachers.forEach(t => {
    if (t.deleted) return;
    // Skip any record that already has an explicit facultyType — admin set it intentionally.
    if (t.facultyType === 'regular' || t.facultyType === 'supervisor') return;
    // facultyType missing: derive it from deptRole.
    const hasSupervisorRole = t.deptRole === 'dean' || t.deptRole === 'chairperson' || t.deptRole === 'supervisor';
    const before = cloneRecord(t);
    t.facultyType = hasSupervisorRole ? 'supervisor' : 'regular';
    if (hasSupervisorRole && !t.password) t.password = t.tid;
    changes.push({ before, after: t });
  });
  if (changes.length) {
    // Before first load commitRecords refuses; this re-runs after load.
    commitRecords('teachers', teachers, changes);
    console.log('Migration: assigned facultyType to legacy records that lacked it.');
  }
};
// Run once on initial script load (handles localStorage data before Firebase arrives)
migrateSupervisorRecords();

// Initialize Dashboard
renderDashboard();

// REMOVED (dead code): syncCollectionToFirestore - defined in adminrate.js.
// ===== COURSE-BY-DEPARTMENT MAP =====
// Courses live in localStorage 'customCourses'; populate via the bulk-upload template.
let COURSES_BY_DEPT = getData('customCourses', {});

// Reload from localStorage (call after any save/delete)
window.reloadCoursesByDept = function() {
  COURSES_BY_DEPT = getData('customCourses', {});
};

let _stuCourseActiveDept = '';
let _stuPendingCourse    = '';   // course to re-select once the list loads (edit mode)

// Department dropdown -> its code and course list.
window.loadStuCourses = function(deptCode) {
  const sel  = document.getElementById('stuCourse');
  const chip = document.getElementById('stuCourseDeptChip');
  _stuCourseActiveDept = '';

  if (chip) { chip.textContent = ''; chip.style.display = 'none'; }
  if (!sel) return;

  if (!deptCode) {
    sel.innerHTML = '<option value="">— Select a department first —</option>';
    return;
  }

  if (chip) { chip.textContent = deptCode; chip.style.display = ''; }

  const courses = COURSES_BY_DEPT[deptCode];
  if (!courses || !courses.length) {
    // Keep the record's existing value, as in populateStuCourseDropdown.
    sel.innerHTML = '<option value="">— No courses set up for ' + deptCode + ' —</option>'
      + (_stuPendingCourse ? `<option value="${_stuPendingCourse}" selected>${_stuPendingCourse} (not registered)</option>` : '');
    _stuPendingCourse = '';
    return;
  }

  _stuCourseActiveDept = deptCode;
  populateStuCourseDropdown(deptCode, _stuPendingCourse);
  _stuPendingCourse = '';
};

// Kept so older call sites keep working.
window.syncStuDeptToCourseBar = function(deptCode) { loadStuCourses(deptCode); };

// Which department owns a course — used when editing a record with no dept set.
window.deptForCourse = function(course) {
  if (!course) return '';
  for (const [code, list] of Object.entries(COURSES_BY_DEPT)) {
    if (list.includes(course)) return code;
  }
  return '';
};

function populateStuCourseDropdown(deptCode, selected) {
  const sel = document.getElementById('stuCourse');
  if (!sel) return;
  const courses = (COURSES_BY_DEPT[deptCode] || []).slice();

  // Keep the student's current course even if not under this department,
  // or saving would wipe it.
  if (selected && courses.indexOf(selected) === -1) {
    sel.innerHTML = `<option value="">— Select Course —</option>`
      + `<option value="${selected}" selected>${selected} (not registered)</option>`
      + courses.map(c => `<option value="${c}">${c}</option>`).join('');
    return;
  }
```

```
  sel.innerHTML = `<option value="">— Select Course —</option>` +
    courses.map(c => `<option value="${c}"${c === selected ? ' selected' : ''}>${c}</option>`).join('');
}
// ===== MANAGE COURSES — helpers used by admindept.js panel =====

// Called from admindept.js Manage Courses panel when dept changes
window.mcLoadDept = function(deptCode) {
  reloadCoursesByDept();
  _mcRenderList(deptCode);
};

// Render course list rows for a given dept inside the panel
function _mcRenderList(deptCode) {
  const listEl = document.getElementById('mcCrseList');
  if (!listEl) return;
  if (!deptCode) {
    listEl.innerHTML = '<p style="color:var(--muted);font-size:0.82rem;padding:10px 0 4px;">Select a department above to see its courses.</p>';
    return;
  }
  const courses = (COURSES_BY_DEPT[deptCode] || []);
  if (courses.length === 0) {
    listEl.innerHTML = '<p style="color:var(--muted);font-size:0.82rem;padding:10px 0 4px;">No courses yet for this department. Add one below or use
Bulk Upload.</p>';
    return;
  }
  listEl.innerHTML = courses.map((c, idx) => `
    <div id="mcrow-${idx}" style="display:flex;align-items:center;gap:6px;padding:5px 8px;border-radius:6px;background:var(--surface2,#f8fafc);margin
-bottom:5px;border:1px solid var(--border);">
      <span class="mc-view-mode" id="mcspan-${idx}" style="flex:1;font-size:0.8rem;line-height:1.4;">${escapeHtml(c)}</span>
      <input class="form-control mc-edit-input" id="mcinput-${idx}" value="${escapeHtml(c)}" style="display:none;flex:1;font-size:0.8rem;padding:3px
7px;" />
      <button class="btn btn-ghost mc-edit-btn" style="padding:2px 8px;font-size:0.72rem;" onclick="mcStartEdit('${deptCode}',${idx})" title="Edit">
✏️</button>
      <button class="btn btn-ghost mc-save-btn" style="display:none;padding:2px 8px;font-size:0.72rem;color:var(--primary);" onclick="mcSaveEdit('${d
eptCode}',${idx})" title="Save">✔</button>
      <button class="btn btn-ghost mc-cancel-btn" style="display:none;padding:2px 8px;font-size:0.72rem;" onclick="mcCancelEdit(${idx})" title="Cance
l">✕</button>
      <button class="btn btn-ghost" style="padding:2px 8px;font-size:0.72rem;color:var(--danger,#e74c3c);" onclick="mcDeleteCourse('${deptCode}',${id
x})" title="Delete">🗑</button>
    </div>`).join('');
}

window.mcStartEdit = function(deptCode, idx) {
  document.getElementById(`mcspan-${idx}`).style.display = 'none';
  document.getElementById(`mcinput-${idx}`).style.display = '';
  document.getElementById(`mcinput-${idx}`).focus();
  document.querySelector(`#mcrow-${idx} .mc-edit-btn`).style.display = 'none';
  document.querySelector(`#mcrow-${idx} .mc-save-btn`).style.display = '';
  document.querySelector(`#mcrow-${idx} .mc-cancel-btn`).style.display = '';
};

window.mcCancelEdit = function(idx) {
  document.getElementById(`mcspan-${idx}`).style.display = '';
  document.getElementById(`mcinput-${idx}`).style.display = 'none';
  document.querySelector(`#mcrow-${idx} .mc-edit-btn`).style.display = '';
  document.querySelector(`#mcrow-${idx} .mc-save-btn`).style.display = 'none';
  document.querySelector(`#mcrow-${idx} .mc-cancel-btn`).style.display = 'none';
};

window.mcSaveEdit = function(deptCode, idx) {
  const input = document.getElementById(`mcinput-${idx}`);
  const newName = normalizeCourseName(input ? input.value : '');
  if (!newName) { showToast('Course name cannot be empty.', 'error'); return; }

  const custom = getData('customCourses', {});
  const list = custom[deptCode] || [];
  const oldName = list[idx];
  if (oldName === undefined) { showToast('Course not found.', 'error'); return; }
  // Check dup
  const dup = list.find((c, i) => i !== idx && courseKey(c) === courseKey(newName));
  if (dup) { showToast('A course with that name already exists.', 'error'); return; }

  list[idx] = newName;
  custom[deptCode] = list;
  setData('customCourses', custom);
  reloadCoursesByDept();
  addAudit('Edit Course', `"${oldName}" → "${newName}" in ${deptCode}`);
  showToast('Course updated!', 'success');
  _mcRenderList(deptCode);
};

window.mcDeleteCourse = function(deptCode, idx) {
  const custom = getData('customCourses', {});
  const list = custom[deptCode] || [];
  const removed = list.splice(idx, 1)[0];
  custom[deptCode] = list;
  setData('customCourses', custom);
  reloadCoursesByDept();
  addAudit('Delete Course', `Deleted "${removed}" from ${deptCode}`);
  showToast('Course deleted.', 'success');
  _mcRenderList(deptCode);
};

window.mcAddCourse = function(deptCode) {
  const input = document.getElementById('mcNewCrseInput');
  const name = normalizeCourseName(input ? input.value : '');
  if (!deptCode) { showToast('Select a department first.', 'error'); return; }
  if (!name) { showToast('Enter a course name.', 'error'); return; }

  const custom = getData('customCourses', {});
  if (!custom[deptCode]) custom[deptCode] = [];
  if (custom[deptCode].some(c => courseKey(c) === courseKey(name))) {
    showToast('This course already exists.', 'error'); return;
  }
```

```
  custom[deptCode].push(name);
  setData('customCourses', custom);
  reloadCoursesByDept();
  if (input) input.value = '';
  addAudit('Add Course', `Added "${name}" to ${deptCode}`);
  showToast('Course added!', 'success');
  _mcRenderList(deptCode);
};

// ── Bulk upload from Excel (reads col A=dept, col B=course via SheetJS) ──
window.mcHandleBulkUpload = function(event) {
  const file = event.target.files[0];
  if (!file) return;

  const reader = new FileReader();
  reader.onload = function(e) {
    try {
      // Use SheetJS if available (loaded via CDN in dashboard.html), else CSV fallback
      if (typeof XLSX !== 'undefined') {
        const wb = XLSX.read(e.target.result, { type: 'array' });
        const ws = wb.Sheets[wb.SheetNames[0]];
        const rows = XLSX.utils.sheet_to_json(ws, { header: 1, defval: '' });

        // Find the header row (contains "Department Code" and "Course Name")
        let dataStart = 0;
        for (let i = 0; i < rows.length; i++) {
          const r = rows[i];
          if (r[0] && String(r[0]).toLowerCase().includes('department')) { dataStart = i + 1; break; }
        }

        const custom = getData('customCourses', {});
        let added = 0, skipped = 0;

        for (let i = dataStart; i < rows.length; i++) {
          const dept = String(rows[i][0] || '').trim().toUpperCase();
          const course = normalizeCourseName(rows[i][1]);
          if (!dept || !course) { skipped++; continue; }
          // Skip dept-header separator rows (e.g. "── COED ──")
          if (dept.startsWith('─') || course === '') { skipped++; continue; }
          if (!custom[dept]) custom[dept] = [];
          if (!custom[dept].some(c => courseKey(c) === courseKey(course))) {
            custom[dept].push(course);
            added++;
          } else {
            skipped++;
          }
        }

        setData('customCourses', custom);
        reloadCoursesByDept();
        addAudit('Bulk Upload Courses', `Imported ${added} courses, ${skipped} skipped`);
        showToast(`Bulk upload complete: ${added} added, ${skipped} skipped.`, 'success');

        // Refresh the list view if a dept is selected
        const deptSel = document.getElementById('mcCrseDept');
        if (deptSel && deptSel.value) _mcRenderList(deptSel.value);
        // Reset file input
        event.target.value = '';
      } else {
        showToast('SheetJS not loaded. Please reload the page and try again.', 'error');
      }
    } catch(err) {
      console.error('Bulk upload error:', err);
      showToast('Upload failed: ' + err.message, 'error');
    }
  };
  reader.readAsArrayBuffer(file);
};

// ── Download the bulk-upload template ──
// Built with SheetJS. Sheet 1 "Courses" is what the upload reads (blank
// course rows are skipped); sheet 2 "Instructions" holds the examples.
window.mcDownloadTemplate = function() {
  if (typeof XLSX === 'undefined') {
    showToast('The spreadsheet tool has not loaded. Check your internet connection and reload the page.', 'error');
    return;
  }
  const depts = (typeof getDepartments === 'function') ? getDepartments() : {};
  const codes = Object.keys(depts).sort();

  const courses = [['Department Code', 'Course Name']]
    .concat(codes.map(code => [code, '']));
  const wsCourses = XLSX.utils.aoa_to_sheet(courses);
  wsCourses['!cols'] = [{ wch: 18 }, { wch: 70 }];

  const help = [
    ['How to use this template'],
    [''],
    ['1. Fill in sheet "Courses" only. Column A = Department Code, column B = Course Name.'],
    ['2. One course per row. For more courses in a department, add rows with the same code.'],
    ['3. Use the department codes already listed (they match the departments in the system).'],
    ['4. Write the course name with its short form in brackets, e.g.:'],
    ['      Bachelor of Science in Information Technology (BSIT)'],
    ['      Bachelor of Secondary Education major in English (BSEd-English)'],
    ['5. Rows with an empty Course Name are ignored, and courses that already exist are skipped.'],
    ['6. Keep the header row (Department Code | Course Name). Save as .xlsx and upload it in Manage Courses.'],
    [''],
    ['Department codes in the system:']
  ].concat(codes.map(code => ['      ' + code + ' \u2014 ' + ((depts[code] && depts[code].name) || '')]));
  const wsHelp = XLSX.utils.aoa_to_sheet(help);
  wsHelp['!cols'] = [{ wch: 95 }];

  const wb = XLSX.utils.book_new();
  XLSX.utils.book_append_sheet(wb, wsCourses, 'Courses');        // must stay first: the upload reads sheet 1
  XLSX.utils.book_append_sheet(wb, wsHelp, 'Instructions');
```

```
  XLSX.writeFile(wb, 'courses_bulk_upload_template.xlsx');
};
// SEF RECORDS AUDIT: list supervisor evaluations with their source so
// stray ones can be deleted (locally and in Firestore).
(function () {
  function teacherMap() {
    const m = {};
    getData('teachers', []).forEach(t => { m[t.id] = t; });
    return m;
  }

  function classifySef(rec, byId) {
    const faculty = byId[rec.teacherId];
    const facultyOk = !!faculty && !faculty.deleted;
    let source, supName = '\u2014', orphan = false;

    // App-submitted SEFs carry the supervisor id in `studentId`.
    const recSupId = rec.supervisorId
      || (rec.evaluatorType === 'supervisor' ? rec.studentId : '');
    if (recSupId) {
      const sup = byId[recSupId];
      if (sup && !sup.deleted && (sup.facultyType === 'supervisor')) {
        source = 'App supervisor';
        supName = sup.name || sup.tid || recSupId;
      } else {
        // Already not counted (countedEvaluations); listed so it can be deleted.
        source = 'Orphaned \u2014 supervisor removed (not counted)';
        orphan = true;
        supName = (sup && sup.name) ? (sup.name + ' (removed)') : (rec.supervisorTid || recSupId);
      }
    } else {
      source = 'Admin-entered';   // created via the admin SEF modal (saveSEFRating); no supervisor link
      supName = '\u2014';
    }

    if (!facultyOk) { source = 'Orphaned \u2014 faculty removed'; orphan = true; }
    return { faculty, facultyOk, source, supName, orphan };
  }

  function orphanIds() {
    const byId = teacherMap();
    return getData('evaluations', [], { includeHidden: true })   // the audit sees everything
      .filter(e => e.evaluatorType === 'supervisor')
      .filter(e => classifySef(e, byId).orphan)
      .map(e => e.id);
  }

  window.openSEFAudit = function () {
    const byId = teacherMap();
    const sef = getData('evaluations', [], { includeHidden: true }).filter(e => e.evaluatorType === 'supervisor');

    const rows = sef.map(rec => {
      const c = classifySef(rec, byId);
      return {
        id: rec.id,
        facultyName: c.faculty ? (c.faculty.name || c.faculty.tid || rec.teacherId) : ('Unknown (' + rec.teacherId + ')'),
        dept: c.faculty ? (c.faculty.dept || '\u2014') : '\u2014',
        score: (rec.totalScore !== undefined && rec.totalScore !== null) ? rec.totalScore : '\u2014',
        date: rec.timestamp ? new Date(rec.timestamp).toLocaleDateString() : '\u2014',
        source: c.source,
        supName: c.supName,
        orphan: c.orphan,
        admin: c.source === 'Admin-entered'
      };
    });

    // orphaned first, then admin-entered, then valid
    const rank = r => r.orphan ? 0 : (r.admin ? 1 : 2);
    rows.sort((a, b) => rank(a) - rank(b) || a.facultyName.localeCompare(b.facultyName));

    const total = rows.length;
    const valid = rows.filter(r => !r.orphan && !r.admin).length;
    const adminN = rows.filter(r => r.admin && !r.orphan).length;
    const orphN = rows.filter(r => r.orphan).length;

    const pill = (txt, color) => `<span style="display:inline-block;padding:2px 9px;border-radius:20px;font-size:0.7rem;font-weight:700;backgroun
d:${color}22;color:${color};white-space:nowrap;">${txt}</span>`;
    const srcColor = r => r.orphan ? '#dc2626' : (r.admin ? '#d97706' : '#059669');

    const body = total === 0
      ? `<div style="padding:40px;text-align:center;color:#64748b;">No supervisor (SEF) records found. Nothing to clean.</div>`
      : `
        <div style="display:flex;gap:10px;flex-wrap:wrap;margin-bottom:14px;">
          ${pill('Total: ' + total, '#475569')}
          ${pill('Valid: ' + valid, '#059669')}
          ${pill('Admin-entered: ' + adminN, '#d97706')}
          ${pill('Orphaned: ' + orphN, '#dc2626')}
        </div>
        <div style="overflow:auto;max-height:52vh;border:1px solid #e2e8f0;border-radius:10px;">
          <table style="width:100%;border-collapse:collapse;font-size:0.8rem;">
            <thead>
              <tr style="background:#f8fafc;text-align:left;position:sticky;top:0;">
                <th style="padding:9px 10px;">Faculty</th>
                <th style="padding:9px 10px;">Dept</th>
                <th style="padding:9px 10px;text-align:center;">SEF %</th>
                <th style="padding:9px 10px;">Date</th>
                <th style="padding:9px 10px;">Source</th>
                <th style="padding:9px 10px;">Supervisor</th>
                <th style="padding:9px 10px;text-align:center;">Action</th>
              </tr>
            </thead>
            <tbody>
              ${rows.map(r => `
                <tr style="border-top:1px solid #eef2f7;${r.orphan ? 'background:#fef2f2;' : ''}">
                  <td style="padding:9px 10px;font-weight:600;">${escapeHtml(r.facultyName)}</td>
```

```
                <td style="padding:9px 10px;color:#64748b;">${escapeHtml(r.dept)}</td>
                <td style="padding:9px 10px;text-align:center;font-weight:700;">${r.score}${r.score !== '\u2014' ? '%' : ''}</td>
                <td style="padding:9px 10px;color:#64748b;white-space:nowrap;">${escapeHtml(r.date)}</td>
                <td style="padding:9px 10px;">${pill(r.source, srcColor(r))}</td>
                <td style="padding:9px 10px;color:#64748b;">${escapeHtml(r.supName)}</td>
                <td style="padding:9px 10px;text-align:center;">
                  <button onclick="deleteSefRecord('${r.id}')" style="border:none;background:#fee2e2;color:#dc2626;font-weight:700;font-size:0.74rem;padding:5px 11px;border-radius:7px;cursor:pointer;">Delete</button>
                </td>
              </tr>`).join('')}
            </tbody>
          </table>
        </div>
        <div style="margin-top:14px;display:flex;gap:10px;flex-wrap:wrap;justify-content:space-between;align-items:center;">
          <div style="font-size:0.72rem;color:#64748b;line-height:1.5;max-width:60%;">
            <strong>Valid</strong> = submitted by a real supervisor in the app.
            <strong>Admin-entered</strong> = typed in via the admin SEF form (no supervisor link).
            <strong>Orphaned</strong> = the faculty or supervisor no longer exists.
          </div>
          ${orphN > 0 ? `<button onclick="deleteOrphanedSef()" style="border:none;background:#dc2626;color:#fff;font-weight:700;font-size:0.78rem;padding:9px 16px;border-radius:9px;cursor:pointer;">Delete all ${orphN} orphaned</button>` : ''}
        </div>`;

    let overlay = document.getElementById('sefAuditOverlay');
    if (overlay) overlay.remove();
    overlay = document.createElement('div');
    overlay.id = 'sefAuditOverlay';
    overlay.style.cssText = 'position:fixed;inset:0;z-index:9999;background:rgba(15,23,42,0.55);display:flex;align-items:center;justify-content:center;padding:18px;';
    overlay.onclick = e => { if (e.target === overlay) overlay.remove(); };
    overlay.innerHTML = `
      <div style="background:#fff;border-radius:16px;max-width:880px;width:100%;max-height:88vh;overflow:auto;padding:22px 22px 20px;box-shadow:0 20px 60px rgba(0,0,0,0.3);font-family:inherit;">
        <div style="display:flex;justify-content:space-between;align-items:center;margin-bottom:6px;">
          <h3 style="margin:0;font-size:1.05rem;">\uD83D\uDD0D SEF Records Audit</h3>
          <button onclick="document.getElementById('sefAuditOverlay').remove()" style="border:none;background:#f1f5f9;width:30px;height:30px;border-radius:8px;cursor:pointer;font-size:1rem;">\u2715</button>
        </div>
        <p style="margin:0 0 14px;font-size:0.78rem;color:#64748b;">Every supervisor (SEF) evaluation in the system, with its source. Remove stray or test records here.</p>
        ${body}
      </div>`;
    document.body.appendChild(overlay);
  };

  window.deleteSefRecord = function (id) {
    showConfirm('Delete SEF Record', 'Remove this supervisor evaluation permanently? This cannot be undone.', async () => {
      const evals = getData('evaluations', [], { includeHidden: true }).filter(e => e.id !== id);
      setLocalData('evaluations', evals);   // local only; the delete below does the database
      try {
        if (typeof firebase !== 'undefined' && firebase.firestore) {
          await firebase.firestore().collection('evaluations').doc(id).delete();
        }
      } catch (e) { console.warn('Firestore delete failed:', e); }
      addAudit('Delete SEF', 'Removed SEF record ' + id);
      showToast('SEF record deleted.', 'success');
      if (typeof renderReports === 'function') { try { renderReports(); } catch (e) {} }
      openSEFAudit();
    });
  };

  window.deleteOrphanedSef = function () {
    const ids = orphanIds();
    if (!ids.length) { showToast('No orphaned SEF records to delete.', 'info'); return; }
    showConfirm('Delete Orphaned SEF', `Remove ${ids.length} orphaned SEF record(s)? These point to faculty or supervisors that no longer exist. This cannot be undone.`, async () => {
      const evals = getData('evaluations', [], { includeHidden: true }).filter(e => !ids.includes(e.id));
      setLocalData('evaluations', evals);   // local only; each delete below does the database
      for (const id of ids) {
        try {
          if (typeof firebase !== 'undefined' && firebase.firestore) {
            await firebase.firestore().collection('evaluations').doc(id).delete();
          }
        } catch (e) { console.warn('Firestore delete failed:', e); }
      }
      addAudit('Delete Orphaned SEF', `Removed ${ids.length} orphaned SEF records`);
      showToast(`${ids.length} orphaned SEF record(s) deleted.`, 'success');
      if (typeof renderReports === 'function') { try { renderReports(); } catch (e) {} }
      openSEFAudit();
    });
  };

  // Inject a small launcher button (admin pages only; admin.js already guards login).
  function injectFab() {
    if (document.getElementById('sefAuditFab')) return;
    const fab = document.createElement('button');
    fab.id = 'sefAuditFab';
    fab.type = 'button';
    fab.textContent = 'SEF Audit';
    fab.title = 'Audit & clean supervisor (SEF) evaluation records';
    fab.onclick = () => openSEFAudit();
    fab.style.cssText = 'position:fixed;right:16px;bottom:16px;z-index:9990;background:#064e3b;color:#fff;border:none;border-radius:10px;padding:10px 15px;font-size:0.78rem;font-weight:700;font-family:inherit;cursor:pointer;box-shadow:0 6px 18px rgba(6,78,59,0.4);';
    document.body.appendChild(fab);
  }
  // SEF Audit opens from Reports (openSEFAudit); injectFab() is not auto-called.
  void injectFab;
})();
```

admin/js/records/admin-students.js

376 lines

```
// admin-students.js - student records, name helpers, and the YY-NNNNN ID format.
// Load order matters (see dashboard.html): the last definition wins.

// ===== STUDENTS DEPT PILLS =====
function renderStudentDeptFilterBar() {
  const bar = document.getElementById('studentDeptFilterBar');
  if (!bar) return;
  const depts = ['COED','CCJS','CCIS','CON','CEA','COM','CAT','GS'];
  const current = bar.dataset.active || '';
  const allStudents = getData('students', []).filter(s => !s.deleted);
  const totalCount = allStudents.length;
  const deptCounts = {};
  allStudents.forEach(s => { if (s.dept) deptCounts[s.dept] = (deptCounts[s.dept] || 0) + 1; });
  const activeDepts = depts.filter(d => deptCounts[d] > 0);
  bar.innerHTML =
    `<button class="dept-filter-pill ${current===''?'active':''}" onclick="setStudentDeptFilter('')">All <span class="dept-filter-count">${totalCount}</span></button>` +
    activeDepts.map(d =>
      `<button class="dept-filter-pill ${current===d?'active':''}" onclick="setStudentDeptFilter('${d}')">${d} <span class="dept-filter-count">${deptCounts[d]}</span></button>`
    ).join('');
}

window.setStudentDeptFilter = function(dept) {
  const bar = document.getElementById('studentDeptFilterBar');
  if (bar) bar.dataset.active = dept;
  renderStudentDeptFilterBar();
  renderStudents();
};

// ===== STUDENTS =====
// renderStudents is overridden in adminfeatures.js.
// Search: every term must match. A section letter needs a year level:
//   "2c", "2-c", "2nd c", "bscs 2ndyr c" -> year 2, Sec C
//   "2", "2nd", "2nd year"               -> year 2, any section
//   anything else -> name, ID, course (or code) or department
const YEAR_TERM_RE = /^([1-5])(?:st|nd|rd|th)?(?:-?(?:yr|year))?(?:-?([a-z]))?$/;
function parseStudentSearch(search) {
  const words = search.toLowerCase().split(/\s+/).filter(Boolean);
  const terms = [];
  for (let i = 0; i < words.length; i++) {
    const m = words[i].match(YEAR_TERM_RE);
    if (!m) { terms.push({ text: words[i] }); continue; }
    const t = { year: m[1], section: m[2] || '' };
    if (!t.section && /^(yr|year)$/.test(words[i + 1] || '')) i++;
    if (!t.section && /^[a-z]$/.test(words[i + 1] || '')) t.section = words[++i];
    terms.push(t);
  }
  return terms;
}
function studentMatchesTerm(s, t) {
  if (t.year) {
    const yearNum = (String(s.year || '').match(/\d/) || [''])[0];
    return yearNum === t.year && (!t.section || String(s.section || '').trim().toLowerCase() === t.section);
  }
  const code = typeof courseShorthand === 'function' ? courseShorthand(s.course) : '';
  return [s.name, s.sid, s.dept, s.course, code].some(v => String(v || '').toLowerCase().includes(t.text));
}

function renderStudents(search = '') {
  renderStudentDeptFilterBar();
  const deptActive = (document.getElementById('studentDeptFilterBar') || {}).dataset?.active || '';
  const students = getData('students', []).filter(s => !s.deleted);
  const terms = parseStudentSearch(search);
  const filtered = students.filter(s =>
    terms.every(t => studentMatchesTerm(s, t)) &&
    (!deptActive || s.dept === deptActive)
  ).sort(byName);
  const tbody = document.getElementById('studentsTbody');
  if (!filtered.length) { tbody.innerHTML = `<tr><td colspan="7" style="text-align:center;padding:32px;color:var(--muted);">No students found.</td></tr>`; return; }
  tbody.innerHTML = filtered.map(s => `
    <tr>
      <td><span style="font-family:'JetBrains Mono',monospace;font-weight:600;">${escapeHtml(s.sid)}</span></td>
      <td><strong>${escapeHtml(listName(s))}</strong></td>
      <td>${escapeHtml(s.course || '—')}</td>
      <td>${escapeHtml(s.year)}</td>
      <td>Sec ${escapeHtml(s.section)}</td>
      <td>${escapeHtml(s.dept || '—')}</td>
      <td><span class="badge ${s.status==='active'?'badge-success':'badge-danger'}">${s.status}</span></td>
      <td><div class="td-actions">
        <button class="btn btn-ghost btn-icon btn-sm" title="Edit" onclick="openEditStudentModal('${s.id}')"><svg width="14" height="14" fill="none" stroke="var(--primary)" stroke-width="2" viewBox="0 0 24 24"><path d="M11 4H4a2 2 0 00-2 2v14a2 2 0 002 2h14a2 2 0 002-2v-7"/><path d="M18.5 2.5a2.121 2.121 0 013 3L12 15l-4 1 1-4 9.5-9.5z"/></svg></button>
        <button class="btn btn-ghost btn-icon btn-sm" title="Reset Password" onclick="openResetPass('${s.id}')"><svg width="14" height="14" fill="none" stroke="var(--warning)" stroke-width="2" viewBox="0 0 24 24"><rect x="3" y="11" width="18" height="11" rx="2"/><path d="M7 11V7a5 5 0 0110 0v4"/></svg></button>
        <button class="btn btn-ghost btn-icon btn-sm" title="Toggle Status" onclick="toggleStudentStatus('${s.id}')"><svg width="14" height="14" fill="none" stroke="${s.status==='active'?'var(--muted)':'var(--success)'}" stroke-width="2" viewBox="0 0 24 24"><path d="M18.36 6.64a9 9 0 11-12.73 0"/><line x1="12" y1="2" x2="12" y2="12"/></svg></button>
        <button class="btn btn-ghost btn-icon btn-sm" title="Delete" onclick="deleteStudent('${s.id}')"><svg width="14" height="14" fill="none" stroke="var(--danger)" stroke-width="2" viewBox="0 0 24 24"><polyline points="3 6 5 6 21 6"/><path d="M19 6l-1 14a2 2 0 01-2 2H8a2 2 0 01-2-2L5 6"/></svg></button>
      </div></td>
    </tr>
  `).join('');
}

// ===== NAME HELPERS =====
// Display name in reading order: "Prof. Juan M. Santos Jr."
```

```
function buildName(parts) {
  return [parts.title, parts.first, parts.middle, parts.last, parts.suffix]
    .map(v => String(v || '').trim()).filter(Boolean).join(' ');
}

// Splits a legacy free-text name to pre-fill the edit modal.
function splitName(full) {
  const TITLES   = ['Prof.', 'Dr.', 'Engr.', 'Atty.', 'Rev.', 'Mr.', 'Ms.', 'Mrs.'];
  const SUFFIXES = ['Jr.', 'Jr', 'Sr.', 'Sr', 'II', 'III', 'IV', 'V'];
  const w = String(full || '').replace(/\s+/g, ' ').trim().split(' ').filter(Boolean);
  const out = { title: '', first: '', middle: '', last: '', suffix: '' };
  if (!w.length) return out;
  if (TITLES.includes(w[0])) out.title = w.shift();
  if (w.length && SUFFIXES.includes(w[w.length - 1])) out.suffix = w.pop();
  if (!w.length) return out;
  out.last  = w.length > 1 ? w.pop() : '';
  out.first = w.shift() || '';
  out.middle = w.join(' ');
  if (!out.last) { out.last = out.first; out.first = ''; }
  return out;
}

// Sort key must match personSortKey in admin-core.js (this byName replaces it).
function sortKey(p) {
  if (typeof personSortKey === 'function') return personSortKey(p);
  const last = (p && p.lastName) ? p.lastName : splitName(p && p.name).last;
  return String(last || (p && p.name) || '').trim();
}

// Alphabetical by surname, then by full name for ties.
function byName(a, b) {
  return sortKey(a).localeCompare(sortKey(b), 'en', { sensitivity: 'base' })
      || String(a.name || '').localeCompare(String(b.name || ''), 'en', { sensitivity: 'base' });
}

// ── Shared helper: populate any dept <select> from live getDepartments() ──
function populateDeptDropdown(selectId, selectedValue) {
  const sel = document.getElementById(selectId);
  if (!sel || typeof getDepartments !== 'function') return;
  const depts = getDepartments();
  sel.innerHTML = '<option value="">-- Select Department --</option>' +
    Object.entries(depts).map(([code, cfg]) =>
      `<option value="${code}">${code} — ${cfg.name}</option>`
    ).join('');
  if (selectedValue !== undefined) sel.value = selectedValue;
}

// Read / write the split name inputs on the student modal
function stuNameParts() {
  const v = id => (document.getElementById(id) || {}).value || '';
  return { first: v('stuFirst'), middle: v('stuMiddle'), last: v('stuLast'), suffix: v('stuSuffix') };
}
function fillStuNameFields(s) {
  const p = s && s.lastName
    ? { first: s.firstName, middle: s.middleName, last: s.lastName, suffix: s.suffix }
    : splitName(s && s.name);
  const set = (id, val) => { const el = document.getElementById(id); if (el) el.value = val || ''; };
  set('stuFirst', p.first); set('stuMiddle', p.middle);
  set('stuLast', p.last);   set('stuSuffix', p.suffix);
}

function openAddStudentModal() {
  editStudentId = null;
  _stuCourseActiveDept = '';
  document.getElementById('studentModalTitle').textContent = 'Add Student';
  document.getElementById('saveStudentBtn').textContent = 'Add Student';
  ['stuId','stuSection','stuPass'].forEach(id => { const el = document.getElementById(id); if(el) el.value=''; });
  fillStuNameFields(null);
  document.getElementById('stuYear').value = '1st Year';
  populateDeptDropdown('stuDept', '');
  loadStuCourses('');
  openModal('addStudentModal');
}

function openEditStudentModal(id) {
  const s = getData('students', []).find(s => s.id === id);
  editStudentId = id;
  _stuCourseActiveDept = '';
  document.getElementById('studentModalTitle').textContent = 'Edit Student';
  document.getElementById('saveStudentBtn').textContent = 'Save Changes';
  document.getElementById('stuId').value = s.sid;
  fillStuNameFields(s);
  document.getElementById('stuYear').value = s.year;
  document.getElementById('stuSection').value = s.section;
  const stuDeptCode = s.dept || deptForCourse(s.course);
  populateDeptDropdown('stuDept', stuDeptCode);
  document.getElementById('stuPass').value = '';
  _stuPendingCourse = s.course || '';
  loadStuCourses(stuDeptCode);
  openModal('addStudentModal');
}

// ===== STUDENT ID FORMAT =====
// Two accepted formats:
//   YY-NNNNN  e.g. 24-00001 (zero padding matters for duplicate checks)
//   YYYY-N-N  e.g. 2025-9902-1, 2024-652-1 (middle part NOT padded)
// Anything else is rejected; the ID becomes the sign-in (<id>@nwssu.app).
const STUDENT_ID_RE = /^(?:\d{2}-\d{5}|\d{4}-\d{1,5}-\d{1,2})$/;

// Repairs common near-misses only. Separator is required; bare digit
// strings are ambiguous and left to fail validation.
function normalizeStudentId(raw) {
  const v = String(raw || '').trim();
  const SEP = '\\s*[-\\u2013\\u2014_ ]\\s*';     // hyphen, en/em dash, underscore or space

```

```
  // YYYY-N-N checked first; only separators are repaired, no padding.
  const n = v.match(new RegExp('^(\\d{4})' + SEP + '(\\d{1,5})' + SEP + '(\\d{1,2})$'));
  if (n) return n[1] + '-' + n[2] + '-' + n[3];

  // YY-NNNNN, with the zero padding repaired ("22-1745" -> "22-01745").
  const m = v.match(new RegExp('^(\\d{2})' + SEP + '(\\d{1,5})$'));
  if (m) return m[1] + '-' + m[2].padStart(5, '0');

  return v;                                     // left alone, allowed to fail validation
}

// Guards against double-click creating the same student twice.
let _savingStudent = false;

async function saveStudent() {
  if (_savingStudent) return;
  // Captured now: the modal stays open while the database check runs.
  const editingId = editStudentId;
  const sid = normalizeStudentId(document.getElementById('stuId').value);
  // Write the normalized value back so the field shows what was actually saved.
  document.getElementById('stuId').value = sid;
  const nameParts = stuNameParts();
  const name = buildName(nameParts);
  // Saved in one standard spelling - see normalizeCourseName (admin-core.js).
  const course = normalizeCourseName(document.getElementById('stuCourse') ? document.getElementById('stuCourse').value : '');
  const year = document.getElementById('stuYear').value;
  // Uppercase the stored section so "A" and "a" don't split.
  const section = document.getElementById('stuSection').value.trim().toUpperCase();
  const dept = document.getElementById('stuDept').value;
  const pass = document.getElementById('stuPass').value;
  if (!sid || !nameParts.first.trim() || !nameParts.last.trim() || !section) {
    showToast('ID number, first name, last name and section are required.', 'error');
    return;
  }
  const nameFields = {
    firstName:  nameParts.first.trim(),
    middleName: nameParts.middle.trim(),
    lastName:   nameParts.last.trim(),
    suffix:     nameParts.suffix.trim()
  };
  const students = getData('students', []);

  // Enforced on NEW ids only; legacy ids keep working.
  const priorSid = editingId
    ? (students.find(s => s.id === editingId) || {}).sid
    : null;
  const sidUnchanged = priorSid != null && sid === priorSid;
  if (!sidUnchanged && !STUDENT_ID_RE.test(sid)) {
    showToast(`Student ID must look like 24-00001, 2025-9902-1 or 2024-652-1. Got "${sid}".`, 'error');
    return;
  }

  // Quick local check first.
  if (students.find(s => s.sid === sid && !s.deleted && s.id !== editingId)) {
    showToast(editingId ? 'Another student already uses that ID.' : 'ID already exists.', 'error');
    return;
  }

  _savingStudent = true;
  try {
    // Then check the DATABASE (skipped when the ID is unchanged).
    let matches = [];
    if (!sidUnchanged) {
      try {
        matches = await findRosterRecords('students', 'sid', sid);
      } catch (e) {
        console.error('Duplicate check failed:', e);
        showToast('Could not check the database for this ID. Check your connection and try again.', 'error');
        return;
      }
      if (matches.some(m => !m.deleted && m.id !== editingId)) {
        showToast('That ID is already in use (it may have just been added on another computer). ' +
                  'Refresh the list to see it.', 'error');
        return;
      }
    }

    let changes;
    if (editingId) {
      const idx = students.findIndex(s => s.id === editingId);
      if (idx === -1) { showToast('Student not found. Refresh and try again.', 'error'); return; }
      const before = cloneRecord(students[idx]);
      // Never let a blank course overwrite an existing one.
      const edits = { sid, name, year, section, dept };
      if (course) edits.course = course;
      Object.assign(students[idx], edits, nameFields);
      addAudit('Edit Student', `Updated: ${name} (${sid})`);
      // A password typed on the EDIT form goes through the proper reset path.
      if (pass) {
        await commitRecords('students', students, [{ before, after: students[idx] }]);
        showToast('Student updated. Starting password reset\u2026', 'info');
        closeModal('addStudentModal');
        await resetLoginPassword('student', students[idx], pass);
        if (typeof renderStudents === 'function') renderStudents(_studentSearchText());
        return;
      }
      changes = [{ before, after: students[idx] }];
      showToast('Student updated!', 'success');
    } else {
      // A deleted student with this ID stays deleted; this creates a NEW record.
      // forceReset TRUE: the default password is the student ID, so it must change.
      const rec = Object.assign({ id: 'stu' + Date.now(), sid, name, course, year, section, dept,
                                  password: pass || sid, status: 'active', forceReset: true, deleted: false },
                                nameFields);
      students.push(rec);
```

```
      changes = [{ before: null, after: rec }];
      const hadDeleted = matches.some(m => m.deleted);
      addAudit('Add Student', `Added: ${name} (${sid})` + (hadDeleted ? ' - new record; the deleted one with this ID stays archived' : ''));
      showToast(hadDeleted ? 'Student added as a new record. The deleted student with this ID stays archived.'
                           : 'Student added!', 'success');
    }
    await commitRecords('students', students, changes);
    closeModal('addStudentModal');
    renderStudents(_studentSearchText());
  } finally {
    _savingStudent = false;
  }
}

// What is typed in the Students search box, so a redraw keeps the filter.
function _studentSearchText() {
  const el = document.getElementById('studentSearchInput');
  return el ? el.value : '';
}

function toggleStudentStatus(id) {
  const students = getData('students', []);
  const idx = students.findIndex(s => s.id === id);
  if (idx === -1) return;
  const before = cloneRecord(students[idx]);
  students[idx].status = students[idx].status === 'active' ? 'inactive' : 'active';
  // Writes only `status` on only this student - see commitRecords (admin-core.js).
  commitRecords('students', students, [{ before, after: students[idx] }]);
  addAudit(students[idx].status==='active'?'Activate Student':'Deactivate Student', `${students[idx].name}`);
  renderStudents(_studentSearchText());
  showToast(`Student ${students[idx].status}!`, 'info');
}

function deleteStudent(id) {
  const s = getData('students', []).find(s => s.id === id);
  if (!s) return;
  // Their ratings stop counting (countedEvaluations) but stay in the database.
  const given = evaluationsSubmittedBy(id, 'student').length;
  const givenNote = given
      ? ` The ${given} evaluation${given === 1 ? '' : 's'} they submitted will stop counting toward` +
        ' faculty scores (kept in the database, not deleted).'
      : '';
  showConfirm('Delete Student', `Remove ${s.name}'s account?${givenNote}`, () => {
    const students = getData('students', []);
    const idx = students.findIndex(x => x.id === id);
    if (idx === -1) return;
    const before = cloneRecord(students[idx]);
    students[idx].deleted = true;
    commitRecords('students', students, [{ before, after: students[idx] }]);
    addAudit('Delete Student', `Deleted: ${s.name} (${s.sid})` +
      (given ? ` — ${given} evaluation(s) no longer counted` : '') + ' — records preserved');
    renderStudents(_studentSearchText());
    showToast('Student deleted.' + (given ? ` Their ${given} evaluation(s) no longer count.` : '') +
              ' Records preserved.', 'info');
  });
}

function openResetPass(id) {
  const s = getData('students', []).find(s => s.id === id);
  resetPassStudentId = id;
  document.getElementById('resetPassName').textContent = s.name;
  document.getElementById('resetPassInput').value = '';
  document.getElementById('forceResetCheck').checked = false;
  openModal('resetPassModal');
}

// Real reset via resetLoginPassword (admin-core.js); never fakes success.
async function saveResetPass() {
  const pass = document.getElementById('resetPassInput').value;
  const students = getData('students', []);
  const idx = students.findIndex(s => s.id === resetPassStudentId);
  if (idx === -1) { showToast('Student not found.', 'error'); return; }

  closeModal('resetPassModal');
  // Blank = back to their Student ID.
  await resetLoginPassword('student', students[idx], pass);
  if (typeof renderStudents === 'function') renderStudents();
}
```

admin/js/records/admin-subjects.js

526 lines

```
// admin-subjects.js - subjects, curriculum slot (courses[] + yearLevel) and enrolment.
// saveSubject, renderSubjects and populateTeacherSelect live in adminrate.js.
// Load order matters (see dashboard.html): the last definition wins.

// ===== SUBJECTS =====
function renderSubjects(search = '') {
  buildSubjectDeptPills();
  const subjects = getData('subjects', []);
  const teachers = getData('teachers', []);
  const students = getData('students', []);
  const deptFilter = (window._subjectDeptFilter || '');

  let filtered = subjects.filter(s =>
    (s.name.toLowerCase().includes(search.toLowerCase()) || s.code.toLowerCase().includes(search.toLowerCase())) &&
    (!deptFilter || s.dept === deptFilter)
  );

  const tbody = document.getElementById('subjectsTbody');
  if (!filtered.length) { tbody.innerHTML = `<tr><td colspan="6" style="text-align:center;padding:32px;color:var(--muted);">No subjects found.</td></
tr>`; return; }

  const groups = {};
  filtered.forEach(sub => {
    const dept = sub.dept || 'Unassigned';
    if (!groups[dept]) groups[dept] = [];
    groups[dept].push(sub);
  });

  let html = '';
  Object.entries(groups).forEach(([dept, subs]) => {
    html += `<tr class="dept-group-header-row"><td colspan="7"><div class="dept-group-header">
      <svg width="12" height="12" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><path d="M4 19.5A2.5 2.5 0 016.5 17H20"/><pa
th d="M6.5 2H20v20H6.5A2.5 2.5 0 014 19.5v-15A2.5 2.5 0 016.5 2z"/></svg>
      ${escapeHtml(dept)}<span class="dept-group-count">${subs.length} subject${subs.length !== 1 ? 's' : ''}</span>
    </div></td></tr>`;
    subs.forEach(sub => {
      const teacher = teachers.find(t => t.id === sub.teacherId);
      const enrolled = (sub.enrolledIds||[]).filter(eid => students.find(s => s.id===eid&&!s.deleted)).length;
      html += `<tr class="dept-group-student-row">
        <td><span style="font-family:'JetBrains Mono',monospace;font-weight:700;">${escapeHtml(sub.code)}</span></td>
        <td><strong>${escapeHtml(sub.name)}</strong></td>
        <td>${escapeHtml(sub.dept || '—')}</td>
        <td>${escapeHtml(sub.category || '—')}</td>
        <td>${teacher?escapeHtml(teacher.name):'<span style="color:var(--muted)">Not assigned</span>'}</td>
        <td><span class="badge badge-primary" style="white-space:nowrap;">${enrolled} student${enrolled!==1?'s':''}</span></td>
        <td><div class="td-actions">
          <button class="btn btn-ghost btn-icon btn-sm" onclick="openEditSubjectModal('${sub.id}')"><svg width="14" height="14" fill="none" stroke="v
ar(--primary)" stroke-width="2" viewBox="0 0 24 24"><path d="M11 4H4a2 2 0 00-2 2v14a2 2 0 002 2h14a2 2 0 002-2v-7"/><path d="M18.5 2.5a2.121 2.121 0
013 3L12 15l-4 1 1-4 9.5-9.5z"/></svg></button>
          <button class="btn btn-ghost btn-icon btn-sm" onclick="openEnrollModal('${sub.id}')"><svg width="14" height="14" fill="none" stroke="var(--
success)" stroke-width="2" viewBox="0 0 24 24"><path d="M17 21v-2a4 4 0 00-4-4H5a4 4 0 00-4 4v2"/><circle cx="9" cy="7" r="4"/><path d="M23 21v-2a4 4
0 00-3-3.87M16 3.13a4 4 0 010 7.75"/></svg></button>
          <button class="btn btn-ghost btn-icon btn-sm" onclick="deleteSubject('${sub.id}')"><svg width="14" height="14" fill="none" stroke="var(--da
nger)" stroke-width="2" viewBox="0 0 24 24"><polyline points="3 6 5 6 21 6"/><path d="M19 6l-1 14a2 2 0 01-2 2H8a2 2 0 01-2-2L5 6"/></svg></button>
        </div></td>
      </tr>`;
    });
  });
  tbody.innerHTML = html;
}

// ===== SUBJECT CURRICULUM SLOT (course + year level) =====
// `courses` is a LIST (shared subjects); empty = every course in the
// department. Used to pre-tick the cohort on enrolment.
let _subCourses = [];        // courses selected in the currently open modal

window.renderSubCourseChips = function() {
    const host    = document.getElementById('subCourseChips');
    const summary = document.getElementById('subCourseSummary');
    const actions = document.getElementById('subCourseActions');
    if (!host) return;

    const dept = (document.getElementById('subDept') || {}).value || '';
    if (typeof reloadCoursesByDept === 'function') reloadCoursesByDept();
    const courses = (typeof COURSES_BY_DEPT !== 'undefined' ? COURSES_BY_DEPT[dept] : null) || [];

    const setSummary = html => { if (summary) summary.innerHTML = html; };
    const setActions = html => { if (actions) actions.innerHTML = html; };

    if (!dept) {
        host.innerHTML = '<span class="sc-hint">Select a department first.</span>';
        setSummary(''); setActions('');
        return;
    }
    if (!courses.length) {
        host.innerHTML = '<span class="sc-hint">No courses set up for ' + escapeHtml(dept)
            + ' yet &mdash; this subject will be open to every student in the department.</span>';
        setSummary(''); setActions('');
        return;
    }

    // Drop any previously chosen course that does not belong to this department.
    _subCourses = _subCourses.filter(c => courses.includes(c));

    const n = _subCourses.length;
    const allSelected = n === courses.length;

    // Summary states the effect in plain words.
    setSummary(n === 0
        ? '<span class="sc-summary-all">All ' + courses.length + ' courses</span>'
```

```
        : '<span class="sc-summary-some">' + n + ' of ' + courses.length + ' selected</span>');

    setActions(
        (n === 0 || allSelected ? '' : '<button type="button" class="sc-link" onclick="setSubCoursesAll()">Select all</button>')
      + (n > 0 ? '<button type="button" class="sc-link" onclick="clearSubCourses()">Clear</button>' : '')
    );

    host.innerHTML = courses.map(c =>
        '<button type="button" class="student-dept-filter-btn' + (_subCourses.includes(c) ? ' active' : '') + '"'
        + ' onclick="toggleSubCourse(\'' + escapeHtml(c).replace(/'/g, "\\'") + '\')">' + escapeHtml(c) + '</button>'
    ).join('');
};

window.toggleSubCourse = function(course) {
    const i = _subCourses.indexOf(course);
    if (i > -1) _subCourses.splice(i, 1); else _subCourses.push(course);
    renderSubCourseChips();
};

window.setSubCoursesAll = function() {
    const dept = (document.getElementById('subDept') || {}).value || '';
    _subCourses = ((typeof COURSES_BY_DEPT !== 'undefined' ? COURSES_BY_DEPT[dept] : null) || []).slice();
    renderSubCourseChips();
};

window.clearSubCourses = function() { _subCourses = []; renderSubCourseChips(); };

window.setSubCourses = function(list) {
    _subCourses = Array.isArray(list) ? list.slice() : [];
    renderSubCourseChips();
};

window.getSubCourses = function() { return _subCourses.slice(); };

// Is this student in the subject's curriculum slot? Pre-tick suggestion only.
window.studentMatchesSubject = function(student, sub) {
    if (!sub) return false;
    const courses  = Array.isArray(sub.courses) ? sub.courses : [];
    const courseOk = courses.length === 0 ||
                     courses.some(c => courseKey(c) === courseKey(student.course));
    const yearOk   = !sub.yearLevel || student.year === sub.yearLevel;
    return courseOk && yearOk;
};

function openAddSubjectModal() {
  editSubjectId = null;
  document.getElementById('subjectModalTitle').textContent = 'Add Subject';
  document.getElementById('saveSubjectBtn').textContent = 'Add Subject';
  ['subCode','subName'].forEach(id => document.getElementById(id).value = '');
  populateDeptDropdown('subDept', '');
  const catEl = document.getElementById('subCategory'); if (catEl) catEl.value = '';
  const yrEl = document.getElementById('subYear'); if (yrEl) yrEl.value = '';
  setSubCourses([]);
  document.getElementById('subLoad').value = 'Regular';
  document.getElementById('subIsLab').checked = false;
  // The form's own picker, whatever the Teachers & Sections modal last used.
  window._tpHost = 'teacherPicker'; window._tpSubjectId = null; window._tpSubDept = undefined;
  populateTeacherSelect();
  openModal('addSubjectModal');
}

function openEditSubjectModal(id) {
  const sub = getData('subjects', []).find(s => s.id === id);
  editSubjectId = id;
  document.getElementById('subjectModalTitle').textContent = 'Edit Subject';
  document.getElementById('saveSubjectBtn').textContent = 'Save Changes';
  document.getElementById('subCode').value = sub.code;
  document.getElementById('subName').value = sub.name;
  populateDeptDropdown('subDept', sub.dept || '');
  const catEl2 = document.getElementById('subCategory'); if (catEl2) catEl2.value = sub.category || '';
  const yrEl2 = document.getElementById('subYear'); if (yrEl2) yrEl2.value = sub.yearLevel || '';
  setSubCourses(sub.courses || []);
  document.getElementById('subLoad').value = sub.loadType || 'Regular';
  document.getElementById('subIsLab').checked = sub.isLabSchool || false;
  // Pass the whole subject so every teacher and section loads.
  window._tpHost = 'teacherPicker'; window._tpSubjectId = null; window._tpSubDept = undefined;
  populateTeacherSelect(sub);
  openModal('addSubjectModal');
}

// REMOVED (dead code): populateTeacherSelect - defined in adminrate.js.

// REMOVED (dead code): saveSubject - defined in adminrate.js.

function deleteSubject(id) {
  const sub = getData('subjects', []).find(s => s.id === id);
  if (!sub) { showToast('That subject no longer exists.', 'error'); return; }
  showConfirm('Delete Subject', `Delete "${sub.name}"?`, () => {
    const subjects = getData('subjects', []);
    const idx = subjects.findIndex(s => s.id === id);
    // Guard -1: splice(-1, 1) would delete the wrong subject.
    if (idx === -1) { showToast('That subject no longer exists.', 'error'); return; }
    subjects.splice(idx, 1);
    setData('subjects', subjects);
    // Delete the Firestore document too.
    deleteDocFromFirestore('subjects', id);
    addAudit('Delete Subject', `Deleted: ${sub.name}`);
    renderSubjects();
    showToast('Subject deleted.', 'info');
  });
}

// Enrolment list is scoped to the subject's department, plus anyone
// already enrolled (so transfers can be unchecked).
function enrollCandidates(sub, showAll) {
```

```
  const all = getData('students', []).filter(s => !s.deleted && s.status === 'active');
  if (showAll || !sub || !sub.dept) return all;
  const enrolled = new Set(sub.enrolledIds || []);
  return all.filter(s => s.dept === sub.dept || enrolled.has(s.id));
}

function renderEnrollDeptNotice(sub) {
  const el = document.getElementById('enrollDeptFilter');
  if (!el) return;
  if (!sub || !sub.dept) {
    el.innerHTML = '<p style="font-size:0.76rem;color:var(--warning,#d97706);margin-bottom:10px;">'
      + 'This subject has no department set, so every active student is listed. '
      + 'Assign a department to the subject to narrow this down.</p>';
    return;
  }
  const showAll = !!window._enrollShowAll;
  const inDept = getData('students', []).filter(s => !s.deleted && s.status === 'active' && s.dept === sub.dept).length;
  el.innerHTML =
    '<div style="display:flex;align-items:center;gap:10px;flex-wrap:wrap;margin-bottom:10px;">'
    + '<span class="dept-tag-inline">' + escapeHtml(sub.dept) + '</span>'
    + '<span style="font-size:0.76rem;color:var(--muted);">'
    +   (showAll ? 'Showing all departments.' : inDept + ' student' + (inDept !== 1 ? 's' : '') + ' in this department.')
    + '</span>'
    + '<button class="btn btn-ghost btn-sm" style="margin-left:auto;" onclick="toggleEnrollShowAll()">'
    +   (showAll ? 'Show ' + escapeHtml(sub.dept) + ' only' : 'Show all departments')
    + '</button>'
    + '</div>'
    // Spell out the curriculum slot so it is obvious which cohort was pre-ticked.
    + (function () {
        const courses = Array.isArray(sub.courses) ? sub.courses : [];
        const slot = [courses.length ? courses.join(' / ') : 'All courses',
                      sub.yearLevel || 'All year levels'].join(' \u00b7 ');
        const cohortN = enrollCandidates(sub, false).filter(st => studentMatchesSubject(st, sub)).length;
        return '<div style="display:flex;align-items:center;gap:10px;flex-wrap:wrap;margin-bottom:10px;'
          + 'padding:8px 11px;border-radius:8px;background:var(--surface2,#f8fafc);border:1px solid var(--border,#e2e8f0);">'
          + '<span style="font-size:0.74rem;color:var(--muted);">Offered to</span>'
          + '<strong style="font-size:0.78rem;">' + escapeHtml(slot) + '</strong>'
          + '<span style="font-size:0.72rem;color:var(--muted);">' + cohortN + ' matching</span>'
          + (cohortN ? '<button class="btn btn-ghost btn-sm" style="margin-left:auto;" onclick="enrollSelectCohort()">Select cohort</button>' : '')
          + '</div>';
      })();
}

window.toggleEnrollShowAll = function() {
  window._enrollShowAll = !window._enrollShowAll;
  const sub = getData('subjects', []).find(s => s.id === enrollSubjectId);
  renderEnrollDeptNotice(sub);
  const search = (document.getElementById('enrollSearchInput') || {}).value || '';
  renderEnrollList(enrollCandidates(sub, window._enrollShowAll), sub.enrolledIds || [], search);
};

// MANAGE ENROLLMENT - Course -> Year -> Section drill-down.
// Ticks persist in window._enrollChecked across steps; the search box looks
// across the whole department for irregular students.

window._enrollChecked = new Set();
window._enrollStep    = { course: '', year: '' };

function openEnrollModal(subId) {
  enrollSubjectId = subId;
  window._enrollShowAll = false;
  const sub = getData('subjects', []).find(s => s.id === subId);
  document.getElementById('enrollSubjectName').textContent = `${sub.code} - ${sub.name}`;
  const searchEl = document.getElementById('enrollSearchInput');
  if (searchEl) searchEl.value = '';
  renderEnrollDeptNotice(sub);

  // Pre-tick the cohort only on first enrolment; afterwards the saved list wins.
  let preselected = sub.enrolledIds || [];
  if (!preselected.length) {
    const matches = enrollCandidates(sub, false).filter(st => studentMatchesSubject(st, sub));
    if (matches.length) preselected = matches.map(st => st.id);
  }

  window._enrollChecked = new Set(preselected);
  window._enrollStep = { course: '', year: '' };
  window._enrollOpenSecs = new Set();      // which sections are expanded (Step 3)
  renderEnrollPicker();
  openModal('enrollModal');
}

// Pool: the department roster plus anyone already enrolled.
function _enrollPool() {
  const sub = getData('subjects', []).find(s => s.id === enrollSubjectId);
  return enrollCandidates(sub, window._enrollShowAll);
}

// normalizeCourseName (admin-core.js): "(BSIS )" and "(BSIS)" are one course.
function _enrollCourseOf(st) { return normalizeCourseName(st.course) || '(no course)'; }
function _enrollYearOf(st)   { return (st.year   || '').trim() || '(no year)'; }
function _enrollSecOf(st)    { return (st.section|| '').trim() || '(no section)'; }

window.enrollPickCourse = function (course) {
  window._enrollStep = { course: course, year: '' };
  renderEnrollPicker();
};
window.enrollPickYear = function (year) {
  window._enrollStep.year = year;
  renderEnrollPicker();
};
window.enrollBack = function (to) {
  if (to === 'course') window._enrollStep = { course: '', year: '' };
  else window._enrollStep.year = '';
  renderEnrollPicker();
};
```

```
311
312 // Ticks live in the Set, not the DOM.
313 window.enrollToggle = function (id, on) {
314   if (on) window._enrollChecked.add(id); else window._enrollChecked.delete(id);
315   _enrollUpdateCount();
316 };
317
318 window.enrollBulkSection = function (course, year, section, on) {
319   _enrollPool()
320     .filter(st => _enrollCourseOf(st) === course && _enrollYearOf(st) === year && _enrollSecOf(st) === section)
321     .forEach(st => { if (on) window._enrollChecked.add(st.id); else window._enrollChecked.delete(st.id); });
322   renderEnrollPicker();
323 };
324
325 function _enrollUpdateCount() {
326   const el = document.getElementById('enrollCount');
327   if (el) el.textContent = window._enrollChecked.size;
328 }
329
330 function _enrollBtn(label, sub, onclick, badge) {
331   return `<button type="button" onclick="${onclick}"
332       style="display:flex;justify-content:space-between;align-items:center;gap:10px;width:100%;
333              text-align:left;padding:10px 12px;margin-bottom:6px;border:1px solid var(--border);
334              border-radius:8px;background:#fff;cursor:pointer;font-size:0.84rem;">
335       <span><span style="font-weight:600;">${escapeHtml(label)}</span>
336         ${sub ? `<span style="color:var(--muted);font-size:0.74rem;display:block;">${escapeHtml(sub)}</span>` : ''}</span>
337       <span style="font-size:0.72rem;color:var(--primary);font-weight:700;white-space:nowrap;">${badge || ''}</span>
338     </button>`;
339 }
340
341 function _enrollCrumb() {
342   const st = window._enrollStep;
343   if (!st.course) return '';
344   const parts = [`<a onclick="enrollBack('course')" style="cursor:pointer;color:var(--primary);">All courses</a>`];
345   if (st.year) {
346     parts.push(`<a onclick="enrollBack('year')" style="cursor:pointer;color:var(--primary);">${escapeHtml(st.course)}</a>`);
347     parts.push(`<span>${escapeHtml(st.year)}</span>`);
348   } else {
349     parts.push(`<span>${escapeHtml(st.course)}</span>`);
350   }
351   return `<div style="font-size:0.76rem;color:var(--muted);margin-bottom:10px;">${parts.join(' &rsaquo; ')}</div>`;
352 }
353
354 function renderEnrollPicker() {
355   const box = document.getElementById('enrollList');
356   if (!box) return;
357   const search = ((document.getElementById('enrollSearchInput') || {}).value || '').trim().toLowerCase();
358   const pool = _enrollPool();
359   const chosen = window._enrollChecked;
360
361   // Search overrides the drill-down and covers the whole pool.
362   _enrollUpdateCount();   // footer count was stuck at 0 until a box was ticked
363   if (search) {
364     const hits = pool.filter(st =>
365       (st.name || '').toLowerCase().includes(search) || (st.sid || '').toLowerCase().includes(search));
366     box.innerHTML = `<div style="font-size:0.76rem;color:var(--muted);margin-bottom:8px;">
367         ${hits.length} match${hits.length !== 1 ? 'es' : ''} across the whole department &mdash; tick anyone to enrol them, cohort or not.
368       </div>` + (hits.length ? hits.map(_enrollRow).join('') :
369         `<div style="padding:20px;text-align:center;color:var(--muted);font-size:0.82rem;">No student matches that search.</div>`);
370     return;
371   }
372
373   const st = window._enrollStep;
374
375   // STEP 1 - courses
376   if (!st.course) {
377     const byCourse = {};
378     pool.forEach(x => { (byCourse[_enrollCourseOf(x)] = byCourse[_enrollCourseOf(x)] || []).push(x); });
379     const names = Object.keys(byCourse).sort();
380     if (!names.length) {
381       box.innerHTML = `<div style="padding:20px;text-align:center;color:var(--muted);font-size:0.82rem;">No students in this department yet.</div>`;
382       return;
383     }
384     box.innerHTML = `<div style="font-size:0.76rem;color:var(--muted);margin-bottom:10px;">Step 1 &mdash; choose a course</div>`
385       + names.map(c => {
386           const list = byCourse[c];
387           const n = list.filter(x => chosen.has(x.id)).length;
388           return _enrollBtn(c, `${list.length} student${list.length !== 1 ? 's' : ''}`,
389                             `enrollPickCourse(&quot;${c.replace(/"/g,'')}&quot;)`,
390                             n ? `${n} enrolled` : '');
391         }).join('');
392     return;
393   }
394
395   // STEP 2 - year levels
396   if (!st.year) {
397     const inCourse = pool.filter(x => _enrollCourseOf(x) === st.course);
398     const byYear = {};
399     inCourse.forEach(x => { (byYear[_enrollYearOf(x)] = byYear[_enrollYearOf(x)] || []).push(x); });
400     const years = Object.keys(byYear).sort();
401     box.innerHTML = _enrollCrumb()
402       + `<div style="font-size:0.76rem;color:var(--muted);margin-bottom:10px;">Step 2 &mdash; choose a year level</div>`
403       + years.map(y => {
404           const list = byYear[y];
405           const n = list.filter(x => chosen.has(x.id)).length;
406           return _enrollBtn(y, `${list.length} student${list.length !== 1 ? 's' : ''}`,
407                             `enrollPickYear(&quot;${y.replace(/"/g,'')}&quot;)`,
408                             n ? `${n} enrolled` : '');
409         }).join('');
410     return;
411   }
412
413   // STEP 3 - sections, then the students in each
414   const inYear = pool.filter(x => _enrollCourseOf(x) === st.course && _enrollYearOf(x) === st.year);
415   const bySec = {};
```

```
  inYear.forEach(x => { (bySec[_enrollSecOf(x)] = bySec[_enrollSecOf(x)] || []).push(x); });
  const secs = Object.keys(bySec).sort();

  // Collapsible sections: all start closed when there are several;
  // open state is remembered while the window is open.
  if (!window._enrollOpenSecs) window._enrollOpenSecs = new Set();
  const openSet = window._enrollOpenSecs;

  // Sections no teacher of this subject takes; saveEnrollment() skips them.
  const subNow   = getData('subjects', []).find(s => s.id === enrollSubjectId);
  const tList    = subNow ? subjectTeachers(subNow) : [];
  const coverAll = !tList.length || tList.some(t => !t.sections.length);
  const covered  = new Set();
  tList.forEach(t => t.sections.forEach(l => covered.add(l)));
  const untaught = list => !coverAll && list.length && !covered.has(_sectionLabel(list[0]));

  box.innerHTML = _enrollCrumb()
    + `<div style="font-size:0.76rem;color:var(--muted);margin-bottom:10px;">Step 3 &mdash; enrol by section
        ${secs.length > 1 ? '<span style="margin-left:4px;">(click a section to show its students)</span>' : ''}</div>`
    + secs.map(sec => {
        const list = bySec[sec].slice().sort(byName);          // last name A to Z
        const n = list.filter(x => chosen.has(x.id)).length;
        const all = n === list.length;
        const c = st.course.replace(/"/g,''), y = st.year.replace(/"/g,''), sc = sec.replace(/"/g,'');
        const key  = st.course + '|' + st.year + '|' + sec;
        const open = secs.length === 1 || openSet.has(key);
        return `<div style="border:1px solid var(--border);border-radius:8px;margin-bottom:10px;overflow:hidden;">
            <div onclick="enrollToggleSectionEl(this)" data-key="${escapeHtml(key)}"
                 title="Show or hide this section's students"
                 style="display:flex;justify-content:space-between;align-items:center;gap:10px;
                        padding:9px 12px;background:var(--bg,#f2f7f4);cursor:pointer;user-select:none;">
              <div style="font-weight:600;font-size:0.84rem;display:flex;align-items:center;gap:8px;">
                <span class="enr-chev" style="display:inline-block;transition:transform .15s;font-size:0.7rem;
                      color:var(--muted);${open ? 'transform:rotate(90deg);' : ''}">&#9654;</span>
                <span>Section ${escapeHtml(sec)}
                <span style="font-weight:400;color:var(--muted);font-size:0.74rem;">
                  &middot; ${n} of ${list.length} enrolled</span>
                ${untaught(list) ? `<span style="font-weight:600;color:#b45309;font-size:0.72rem;margin-left:6px;"
                    title="Add this section to a teacher in Teachers &amp; Sections first">&middot; no teacher takes this section</span>` : ''}</span
></div>
              <button type="button" class="btn btn-ghost btn-sm"
                onclick="event.stopPropagation();enrollBulkSection(&quot;${c}&quot;,&quot;${y}&quot;,&quot;${sc}&quot;,${all ? 'false' : 'true'})">
                ${all ? 'Remove all' : 'Enrol all'}</button>
            </div>
            <div class="enr-sec-body" style="padding:4px 12px 8px;${open ? '' : 'display:none;'}">${list.map(_enrollRow).join('')}</div>
          </div>`;
      }).join('');
}

// Toggle the section in place (no redraw).
window.enrollToggleSectionEl = function (head) {
  const body = head && head.nextElementSibling;
  if (!body) return;
  const open = body.style.display === 'none';
  body.style.display = open ? '' : 'none';
  const chev = head.querySelector('.enr-chev');
  if (chev) chev.style.transform = open ? 'rotate(90deg)' : '';
  const key = head.getAttribute('data-key');
  if (!window._enrollOpenSecs) window._enrollOpenSecs = new Set();
  if (open) window._enrollOpenSecs.add(key); else window._enrollOpenSecs.delete(key);
};

function _enrollRow(s) {
  const on = window._enrollChecked.has(s.id);
  return `<label style="display:flex;align-items:center;gap:10px;cursor:pointer;font-size:0.82rem;
                        padding:6px 0;border-bottom:1px solid var(--border);">
      <input type="checkbox" ${on ? 'checked' : ''} style="accent-color:var(--primary);"
             onchange="enrollToggle('${s.id}', this.checked)"/>
      <div><div style="font-weight:600;">${escapeHtml(listName(s))}</div>
      <div style="font-size:0.72rem;color:var(--muted);">${escapeHtml(s.sid)} &middot; ${s.course ? escapeHtml(s.course) + ' &middot; ' : ''}${escape
Html(s.year || '')} Sec ${escapeHtml(s.section || '')} &middot; ${escapeHtml(s.dept || '—')}</div></div>
    </label>`;
}

// "Select cohort" adds the curriculum match on top of current ticks.
window.enrollSelectCohort = function() {
  const sub = getData('subjects', []).find(s => s.id === enrollSubjectId);
  if (!sub) return;
  enrollCandidates(sub, window._enrollShowAll)
    .filter(st => studentMatchesSubject(st, sub))
    .forEach(st => window._enrollChecked.add(st.id));
  renderEnrollPicker();
  _enrollUpdateCount();
  showToast('Cohort selected. Untick anyone who should not be enrolled.', 'info');
};

function filterEnrollList() { renderEnrollPicker(); }

function saveEnrollment() {
  // Read the Set, not the DOM (only one section is rendered).
  const checked = [...window._enrollChecked];
  const subjects = getData('subjects', []);
  const idx = subjects.findIndex(s => s.id === enrollSubjectId);
  if (idx === -1) { showToast('Subject not found.', 'error'); return; }

  // Skip students in sections no teacher takes.
  const teachers = subjectTeachers(subjects[idx]);
  const { keep, removed } = splitUncoveredStudents(
      Object.assign({}, subjects[idx], { enrolledIds: checked }), teachers);

  const commit = (ids, left) => {
    subjects[idx].enrolledIds = ids;
    setData('subjects', subjects);
    addAudit('Update Enrollment', `${subjects[idx].name} — ${ids.length} students`
      + (left ? `, ${left} left out (no teacher for their section)` : ''));
```

```
    closeModal('enrollModal');
    renderSubjects();
    showToast('Enrollment saved!' + (left ? ` ${left} student(s) left out.` : ''), 'success');
  };

  if (removed.length) {
    const secs = Array.from(new Set(removed.map(s => _sectionLabel(s)))).join(', ');
    showConfirm('Some students have no teacher',
      `${removed.length} student(s) from ${secs} are in a section that no teacher of this ` +
      `subject takes, so they would have nobody to rate. They will be left out. To enrol them, ` +
      `first add their section to a teacher in Teachers & Sections.`,
      () => commit(keep, removed.length));
    return;
  }
  commit(checked, 0);
}
```

admin/js/records/admin-teachers.js

939 lines

```
// admin-teachers.js - faculty and supervisor records, tabs, multi-department
// assignments (supervisedDepts: [{dept, role}]).
// Load order matters (see dashboard.html): the last definition wins.

// ===== TEACHER DEPT FILTER PILLS =====
// Pass-through stubs; adminfeatures.js builds the real bar.
function buildTeacherDeptPills() {
  if (typeof buildTeacherDeptFilterBar === 'function') buildTeacherDeptFilterBar();
  if (typeof buildSupervisorDeptFilterBar === 'function') buildSupervisorDeptFilterBar();
}

window.setTeacherDeptFilter = function(dept) {
  if (typeof filterTeachersByDept === 'function') { filterTeachersByDept(dept); return; }
  // Fallback if adminfeatures not loaded yet
  const bar = document.getElementById('teacherDeptFilterBar');
  if (bar) bar.dataset.active = dept;
  const currentSearch = document.getElementById('teacherSearchInput')?.value || '';
  renderTeachers(currentSearch);
};

// ===== TEACHER PAGE — TAB SWITCHING =====
window._teacherActiveTab = 'faculty'; // tracks current tab

window.switchTeacherTab = function(tab) {
  window._teacherActiveTab = tab;

  // Swap panel visibility
  const facultyPanel     = document.getElementById('teacherPanelFaculty');
  const supervisorsPanel = document.getElementById('teacherPanelSupervisors');
  if (facultyPanel)     facultyPanel.style.display     = tab === 'faculty'     ? '' : 'none';
  if (supervisorsPanel) supervisorsPanel.style.display = tab === 'supervisors' ? '' : 'none';

  // Swap tab active state
  document.getElementById('teacherTabFaculty')?.classList.toggle('teacher-tab-active',     tab === 'faculty');
  document.getElementById('teacherTabSupervisors')?.classList.toggle('teacher-tab-active', tab === 'supervisors');

  // Change header Add button label
  const addBtn = document.getElementById('teacherPageAddBtn');
  if (addBtn) {
    if (tab === 'supervisors') {
      addBtn.textContent = '';
      addBtn.innerHTML = '<svg width="14" height="14" fill="none" stroke="currentColor" stroke-width="2.5" viewBox="0 0 24 24"><line x1="12" y1="5" x
2="12" y2="19"/><line x1="5" y1="12" x2="19" y2="12"/></svg>Add Supervisor';
      addBtn.onclick = openAddSupervisorModal;
    } else {
      addBtn.textContent = '';
      addBtn.innerHTML = '<svg width="14" height="14" fill="none" stroke="currentColor" stroke-width="2.5" viewBox="0 0 24 24"><line x1="12" y1="5" x
2="12" y2="19"/><line x1="5" y1="12" x2="19" y2="12"/></svg>Add Teacher';
      addBtn.onclick = openAddTeacherModal;
    }
  }

  // Trigger a render so the visible table is populated
  if (tab === 'faculty') renderTeachers();
  else renderSupervisorTable();
};

// ===== TEACHER PAGE — COLLAPSIBLE SECTIONS =====
window._teacherSectionState = { faculty: true, supervisors: true }; // true = expanded

window.toggleTeacherSection = function(section) {
  const isOpen = window._teacherSectionState[section];
  window._teacherSectionState[section] = !isOpen;

  const bodyId = section === 'faculty' ? 'facultyCardBody' : 'supervisorsCardBody';
  const iconId = section === 'faculty' ? 'facultyCollapseIcon' : 'supervisorsCollapseIcon';
  const body   = document.getElementById(bodyId);
  const icon   = document.getElementById(iconId);

  if (body) {
    if (isOpen) {
      // Collapse: animate then hide
      body.style.maxHeight = body.scrollHeight + 'px';
      body.style.opacity   = '1';
      body.style.overflow  = 'hidden';
      body.style.transition = 'max-height 0.28s ease, opacity 0.22s ease';
      requestAnimationFrame(() => {
        body.style.maxHeight = '0';
        body.style.opacity   = '0';
      });
    } else {
      // Expand
      body.style.maxHeight = body.scrollHeight + 'px';
      body.style.opacity   = '1';
      body.style.overflow  = 'hidden';
      body.style.transition = 'max-height 0.28s ease, opacity 0.22s ease';
      setTimeout(() => {
        body.style.maxHeight = '';
        body.style.overflow  = '';
      }, 300);
    }
  }

  if (icon) icon.classList.toggle('collapsed', isOpen);
};

// ===== UPDATE TEACHER TAB COUNTS =====
window._updateTeacherTabCounts = function() {
  const teachers    = getData('teachers', []).filter(t => !t.deleted);
  const faculty     = teachers.filter(t => t.facultyType !== 'supervisor');
```

```
  const supervisors = teachers.filter(t => t.facultyType === 'supervisor');

  const fCount = document.getElementById('teacherTabFacultyCount');
  const sCount = document.getElementById('teacherTabSupervisorsCount');
  const fHeader = document.getElementById('facultyHeaderCount');
  const sHeader = document.getElementById('supervisorHeaderCount');

  if (fCount)  fCount.textContent  = faculty.length;
  if (sCount)  sCount.textContent  = supervisors.length;
  if (fHeader) fHeader.textContent = faculty.length;
  if (sHeader) sHeader.textContent = supervisors.length;
};

// ===== RENDER TEACHERS =====
window.renderTeachers = function(search = '') {
  const searchInput = document.getElementById('teacherSearchInput');
  // Sync a passed search string to the input, then read the live value.
  if (searchInput && search && searchInput.value !== search) {
    searchInput.value = search;
  }
  // Always use the live input value.
  const liveSearch = searchInput ? searchInput.value : search;

  // Rebuild dept filter bar (delegates to adminfeatures.js)
  buildTeacherDeptPills();

  // Use _teacherDeptFilter from adminfeatures.js if available, fallback to dataset
  const deptActive = (typeof _teacherDeptFilter !== 'undefined' ? _teacherDeptFilter : null) ??
    (document.getElementById('teacherDeptFilterBar')?.dataset.active || '');
  const teachers = getData('teachers', []).filter(t => !t.deleted);
  const subjects = getData('subjects', []);

  // Filter Regular Faculty — searchable by name, ID, or department (code or full name)
  const DEPT_NAME_MAP = {
    COED: 'college of education',
    CCJS: 'college of criminal justice',
    CCIS: 'college of computing',
    CON: 'college of nursing',
    CEA: 'college of engineering',
    COM: 'college of management',
    CAT: 'college of agriculture',
    GS: 'graduate school'
  };
  const q = liveSearch.toLowerCase();
  const faculty = teachers.filter(t => {
    if (t.facultyType === 'supervisor') return false;
    const deptCode = (t.dept || '').toLowerCase();
    const deptFull = DEPT_NAME_MAP[(t.dept || '').toUpperCase()] || '';
    const matchesSearch = !q ||
      t.name.toLowerCase().includes(q) ||
      t.tid.toLowerCase().includes(q) ||
      deptCode.includes(q) ||
      deptFull.includes(q);
    // Via deptFilterKey (adminfeatures.js) so "No Dept" matches a blank department.
    const matchesDept = !deptActive || deptFilterKey(t.dept) === deptActive;
    return matchesSearch && matchesDept;
  }).sort(teacherListCompare);          // ID / Name header choice

  const tbody = document.getElementById('teachersTbody');
  if (!tbody) return;

  if (!faculty.length) {
    tbody.innerHTML = `<tr><td colspan="6" style="text-align:center;padding:32px;color:var(--muted);">No matching regular faculty found.</td></tr>`;
    renderSupervisorTable(liveSearch);
    return;
  }

  // Group regular faculty by department
  const deptGroups = {};
  faculty.forEach(t => {
    const dept = t.dept || 'UNASSIGNED';
    if (!deptGroups[dept]) deptGroups[dept] = [];
    deptGroups[dept].push(t);
  });

  const DEPT_NAMES = {
      COED: 'College of Education',
      CCJS: 'College of Criminal Justice & Safety',
      CCIS: 'College of Computing & Info. Sciences',
      CON: 'College of Nursing',
      CEA: 'College of Engineering & Architecture',
      COM: 'College of Management',
      CAT: 'College of Agriculture & Technology',
      GS: 'Graduate School',
      UNASSIGNED: 'No Department Assigned'
  };

  let html = '';
  Object.entries(deptGroups).forEach(([dept, facultyList]) => {
    const deptLabel = DEPT_NAMES[dept] || dept;

    html += `
    <tr class="dept-group-header-row">
        <td colspan="6">
            <div class="dept-group-header">
                <svg width="13" height="13" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24">
                    <path d="M17 21v-2a4 4 0 00-4-4H5a4 4 0 00-4 4v2"/>
                    <circle cx="9" cy="7" r="4"/>
                </svg>
                <span>${escapeHtml(deptLabel)}</span>
                <span class="dept-group-count">${facultyList.length} teacher${facultyList.length !== 1 ? 's' : ''}</span>
            </div>
        </td>
    </tr>`;

```

```
    facultyList.forEach(t => {
      const tSubs = subjects.filter(s => subjectHasTeacher(s, t.id));
      html += `
      <tr class="teacher-row dept-group-student-row" onclick="showAnnexReports('${t.id}')" title="Click to view Annex C & D" style="cursor:pointer;">
        <td><span style="font-family:'JetBrains Mono',monospace;font-weight:600;">${escapeHtml(t.tid)}</span></td>
        <td><strong>${escapeHtml(listName(t))}</strong></td>
        <td><span class="dept-tag-inline">${escapeHtml(t.dept || '—')}</span></td>
        <td>${tSubs.map(s => `<span class="badge badge-primary" style="margin:1px;">${escapeHtml(s.code)}</span>`).join('') || '<span style="color:var(--muted)">None</span>'}</td>
        <td><span class="badge ${t.status === 'active' ? 'badge-success' : 'badge-danger'}">${t.status}</span></td>
        <td><div class="td-actions" onclick="event.stopPropagation()">
          <button class="btn btn-ghost btn-icon btn-sm" title="Edit" onclick="openEditTeacherModal('${t.id}')"><svg width="14" height="14" fill="none" stroke="var(--primary)" stroke-width="2" viewBox="0 0 24 24"><path d="M11 4H4a2 2 0 00-2 2v14a2 2 0 002 2h14a2 2 0 002-2v-7"/><path d="M18.5 2.5a2.121 2.121 0 013 3L12 15l-4 1 1-4 9.5-9.5z"/></svg></button>
          <button class="btn btn-ghost btn-icon btn-sm" title="Move to Supervisor table" onclick="forceSetSupervisor('${t.id}')" style="color:var(--warning);font-size:11px;padding:2px 6px;">⬇️ Sup</button>
          <button class="btn btn-ghost btn-icon btn-sm" title="Toggle Status" onclick="toggleTeacherStatus('${t.id}')"><svg width="14" height="14" fill="none" stroke="${t.status === 'active' ? 'var(--muted)' : 'var(--success)'}" stroke-width="2" viewBox="0 0 24 24"><path d="M18.36 6.64a9 9 0 11-12.73 0"/><line x1="12" y1="2" x2="12" y2="12"/></svg></button>
          <button class="btn btn-ghost btn-icon btn-sm" title="Delete" onclick="deleteTeacher('${t.id}')"><svg width="14" height="14" fill="none" stroke="var(--danger)" stroke-width="2" viewBox="0 0 24 24"><polyline points="3 6 5 6 21 6"/><path d="M19 6l-1 14a2 2 0 01-2 2H8a2 2 0 01-2-2L5 6"/></svg></button>
        </div></td>
      </tr>`;
    });
  });

  tbody.innerHTML = html;

  // Update tab counts
  if (typeof _updateTeacherTabCounts === 'function') _updateTeacherTabCounts();

  // Render supervisor table filtering simultaneously (only if faculty tab is active, to avoid double-render)
  if (window._teacherActiveTab !== 'supervisors') renderSupervisorTable(liveSearch);
};

window.setSupervisorDeptFilter = function(dept) {
  if (typeof filterTeachersByDept === 'function') { filterTeachersByDept(dept); return; }
  renderSupervisorTable();
};

// ===== SUPERVISOR TABLE =====
window.renderSupervisorTable = function(search) {
  // Rebuild supervisor dept filter bar so pills stay in sync
  if (typeof buildSupervisorDeptFilterBar === 'function') buildSupervisorDeptFilterBar();

  // Use the supervisor search input only.
  const supInput = document.getElementById('supervisorSearchInput');
  if (supInput && search !== undefined && supInput.value !== search) {
    supInput.value = search;
  }
  const sq = (supInput ? supInput.value : (search || '')).toLowerCase();

  // Use the supervisor-specific dept filter (set by filterSupervisorsByDept)
  const deptActive = (typeof _supervisorDeptFilter !== 'undefined' ? _supervisorDeptFilter : null) ?? '';

  const teachers = getData('teachers', []).filter(t => !t.deleted);
  const subjects = getData('subjects', []);

  const DEPT_NAME_MAP_SUP = {
    COED: 'college of education',
    CCJS: 'college of criminal justice',
    CCIS: 'college of computing',
    CON: 'college of nursing',
    CEA: 'college of engineering',
    COM: 'college of management',
    CAT: 'college of agriculture',
    GS: 'graduate school'
  };

  const DEPT_NAMES_SUP = {
    COED: 'College of Education',
    CCJS: 'College of Criminal Justice & Safety',
    CCIS: 'College of Computing & Info. Sciences',
    CON: 'College of Nursing',
    CEA: 'College of Engineering & Architecture',
    COM: 'College of Management',
    CAT: 'College of Agriculture & Technology',
    GS: 'Graduate School',
    UNASSIGNED: 'No Department Assigned'
  };

  const supervisors = teachers.filter(t => {
    if (t.facultyType !== 'supervisor') return false;
    const deptCode = (t.dept || '').toLowerCase();
    const deptFull = DEPT_NAME_MAP_SUP[(t.dept || '').toUpperCase()] || '';
    const matchesSearch = !sq ||
      t.name.toLowerCase().includes(sq) ||
      t.tid.toLowerCase().includes(sq) ||
      deptCode.includes(sq) ||
      deptFull.includes(sq);
    // Filter by SUPERVISED departments, matching the pill counts.
    const sup = getSupervisedDepts(t).filter(a => a.dept);
    const matchesDept = !deptActive ||
      (deptActive === 'UNASSIGNED' ? sup.length === 0 : sup.some(a => a.dept === deptActive));
    return matchesSearch && matchesDept;
  }).sort(teacherListCompare);          // ID / Name header choice
  paintTeacherSortArrows();

  const tbody = document.getElementById('supervisorsTbody');
  if (!tbody) return;

  if (!supervisors.length) {
    tbody.innerHTML = `<tr><td colspan="7" style="text-align:center;padding:32px;color:var(--muted);">No matching supervisors found for this criteria.</td></tr>`;
```

```
    return;
  }

  // Group supervisors by department (mirrors faculty grouping)
  const deptGroups = {};
  supervisors.forEach(t => {
    const dept = t.dept || 'UNASSIGNED';
    if (!deptGroups[dept]) deptGroups[dept] = [];
    deptGroups[dept].push(t);
  });

  let html = '';
  Object.entries(deptGroups).forEach(([dept, supList]) => {
    const deptLabel = DEPT_NAMES_SUP[dept] || dept;
    const colorStyle = dept === 'UNASSIGNED' ? 'background:#f1f5f9;color:#475569;' : '';

    html += `
    <tr class="dept-group-header-row">
      <td colspan="7">
        <div class="dept-group-header" style="${colorStyle}">
          <svg width="13" height="13" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24">
            <path d="M12 22s8-4 8-10V5l-8-3-8 3v7c0 6 8 10 8 10z"/>
          </svg>
          <span>${escapeHtml(deptLabel)}</span>
          <span class="dept-group-count">${supList.length} supervisor${supList.length !== 1 ? 's' : ''}</span>
        </div>
      </td>
    </tr>`;

    supList.forEach(t => {
      const tSubs = subjects.filter(s => subjectHasTeacher(s, t.id));
      html += `<tr class="teacher-row dept-group-student-row" onclick="showAnnexReports('${t.id}')" title="Click to view Annex C & D" style="cursor:p
ointer;">
        <td><span style="font-family:'JetBrains Mono',monospace;font-weight:600;">${escapeHtml(t.tid)}</span></td>
        <td><strong>${escapeHtml(listName(t))}</strong></td>
        <td><span class="dept-tag-inline">${escapeHtml(t.dept || '—')}</span></td>
        <td>${supRoleCellHtml(t)}</td>
        <td>${tSubs.map(s => `<span class="badge badge-primary" style="margin:1px;">${escapeHtml(s.code)}</span>`).join('') || '<span style="color:va
r(--muted)">None</span>'}</td>
        <td><span class="badge ${t.status === 'active' ? 'badge-success' : 'badge-danger'}">${t.status}</span></td>
        <td><div class="td-actions" onclick="event.stopPropagation()">
          <button class="btn btn-ghost btn-icon btn-sm" title="Edit" onclick="openEditTeacherModal('${t.id}')"><svg width="14" height="14" fill="non
e" stroke="var(--primary)" stroke-width="2" viewBox="0 0 24 24"><path d="M11 4H4a2 2 0 00-2 2v14a2 2 0 002 2h14a2 2 0 002-2v-7"/><path d="M18.5 2.5a
2.121 2.121 0 013 3L12 15l-4 1 1-4 9.5-9.5z"/></svg></button>
          <button class="btn btn-ghost btn-icon btn-sm" title="Toggle Status" onclick="toggleTeacherStatus('${t.id}')"><svg width="14" height="14" fi
ll="none" stroke="${t.status === 'active' ? 'var(--muted)' : 'var(--success)'}" stroke-width="2" viewBox="0 0 24 24"><path d="M18.36 6.64a9 9 0 11-1
2.73 0"/><line x1="12" y1="2" x2="12" y2="12"/></svg></button>
          <button class="btn btn-ghost btn-icon btn-sm" title="Delete" onclick="deleteTeacher('${t.id}')"><svg width="14" height="14" fill="none" str
oke="var(--danger)" stroke-width="2" viewBox="0 0 24 24"><polyline points="3 6 5 6 21 6"/><path d="M19 6l-1 14a2 2 0 01-2 2H8a2 2 0 01-2-2L5 6"/></sv
g></button>
        </div></td>
      </tr>`;
    });
  });

  tbody.innerHTML = html;
  if (typeof _updateTeacherTabCounts === 'function') _updateTeacherTabCounts();
};

window.toggleTeacherDetails = function(teacherId) {
  const row = document.getElementById(`teacher-details-${teacherId}`);
  if (!row) return;
  const isHidden = row.style.display === 'none';
  row.style.display = isHidden ? 'table-row' : 'none';
  if (isHidden) renderTeacherDetailsContent(teacherId);
};

window.renderTeacherDetailsContent = function(teacherId) {
  const t = getData('teachers', []).find(t => t.id === teacherId);
  const subjects = getData('subjects', []).filter(s => subjectHasTeacher(s, teacherId) && s.loadType !== 'Overload' && !s.isLabSchool);
  const sefEvals = getData('evaluations', []).filter(e => e.teacherId === teacherId && e.evaluatorType === 'supervisor');

  // Same §8.3 computation as every other screen (admin-scoring.js).
  const agg = computeWeightedSET(teacherId, evalInActiveTerm);
  const classBreakdown = agg.classes.map(c => ({
    sub: c.sub, enrolledCount: c.enrolledCount,
    evalCount: c.evalCount, avgScore: parseFloat(c.avgScore)
  }));
  const weightedSET = agg.totalStudents > 0 ? agg.overallSET : '—';
  const latestSEF = sefEvals.length > 0 ? (sefEvals.reduce((a, b) => a + b.totalScore, 0) / sefEvals.length).toFixed(2) : '—';

  const container = document.getElementById(`teacher-details-content-${teacherId}`);
  if (!container) return;

  container.innerHTML = `
    <div style="padding:16px;">
      <div style="display:flex;gap:16px;margin-bottom:14px;flex-wrap:wrap;">
        <div style="background:#f0fdf4;border-radius:8px;padding:12px 20px;text-align:center;">
          <div style="font-size:0.68rem;color:var(--muted);font-weight:600;">Weighted SET</div>
          <div style="font-size:1.4rem;font-weight:800;color:#16a34a;">${weightedSET}${weightedSET !== '—' ? '%' : ''}</div>
        </div>
        <div style="background:#fff7ed;border-radius:8px;padding:12px 20px;text-align:center;">
          <div style="font-size:0.68rem;color:var(--muted);font-weight:600;">Latest SEF</div>
          <div style="font-size:1.4rem;font-weight:800;color:#d97706;">${latestSEF}${latestSEF !== '—' ? '%' : ''}</div>
        </div>
      </div>
      <h4 style="font-size:0.82rem;font-weight:700;margin-bottom:10px;">Class Performance Breakdown (Regular Load Only)</h4>
      ${classBreakdown.length === 0 ? '<p style="color:var(--muted);font-size:0.8rem;">No regular-load subjects assigned.</p>' :
        `<table style="width:100%;border-collapse:collapse;font-size:0.78rem;">
          <thead><tr style="background:var(--bg);">
            <th style="padding:8px;text-align:left;">Subject</th>
            <th style="padding:8px;text-align:center;">Enrolled</th>
            <th style="padding:8px;text-align:center;">Evaluations</th>
            <th style="padding:8px;text-align:center;">SET Avg</th>
            <th style="padding:8px;text-align:center;">Percentage</th>
```

```
          </tr></thead>
          <tbody>${classBreakdown.map(cr => `<tr style="border-bottom:1px solid var(--border);">
            <td style="padding:8px;"><strong>${escapeHtml(cr.sub.code)}</strong> — ${escapeHtml(cr.sub.name)}</td>
            <td style="padding:8px;text-align:center;">${cr.enrolledCount}</td>
            <td style="padding:8px;text-align:center;">${cr.evalCount}</td>
            <td style="padding:8px;text-align:center;">${cr.avgScore.toFixed(2)}</td>
            <td style="padding:8px;text-align:center;"><strong>${Math.min(100, cr.avgScore).toFixed(2)}%</strong></td>
          </tr>`).join('')}</tbody>
        </table>`}
    </div>
  `;
};

// Read / write the split name inputs on the faculty modal
function tchNameParts() {
  const v = id => (document.getElementById(id) || {}).value || '';
  return { title: v('tchTitle'), first: v('tchFirst'), middle: v('tchMiddle'), last: v('tchLast'), suffix: v('tchSuffix') };
}
function fillTchNameFields(t) {
  const p = t && t.lastName
    ? { title: t.title, first: t.firstName, middle: t.middleName, last: t.lastName, suffix: t.suffix }
    : splitName(t && t.name);
  const set = (id, val) => { const el = document.getElementById(id); if (el) el.value = val || ''; };
  set('tchTitle', p.title);  set('tchFirst', p.first);
  set('tchMiddle', p.middle); set('tchLast', p.last); set('tchSuffix', p.suffix);
}

function openAddTeacherModal() {
  editTeacherId = null;
  _pendingFacultyType = 'regular';
  document.getElementById('teacherModalTitle').textContent = 'Add Faculty';
  document.getElementById('saveTeacherBtn').textContent = 'Add Faculty';
  document.getElementById('tchId').value = '';
  fillTchNameFields(null);
  populateDeptDropdown('tchDept', '');
  const catEl = document.getElementById('tchCategory');
  if (catEl) catEl.value = '';
  const rkEl = document.getElementById('tchRank');
  if (rkEl) rkEl.value = '';
  const ftEl = document.getElementById('tchFacultyType');
  if (ftEl) ftEl.value = 'regular';
  const hidden = document.getElementById('tchFacultyTypeHidden');
  if (hidden) hidden.value = 'regular';
  const drEl = document.getElementById('tchDeptRole');
  const drGrp = document.getElementById('tchDeptRoleGroup');
  if (drEl) drEl.value = '';
  renderSupRows(null);
  if (drGrp) drGrp.style.display = 'none';
  const ftGrp = document.getElementById('tchFacultyTypeGroup');
  if (ftGrp) ftGrp.style.display = '';
  // Regular faculty never sign in: hide the password box.
  const pwGrp = document.getElementById('tchPasswordGroup');
  const pwEl  = document.getElementById('tchPassword');
  if (pwGrp) pwGrp.style.display = 'none';
  if (pwEl)  pwEl.value = '';
  _setHomeDeptLabel('regular');
  openModal('addTeacherModal');
}

// Supervisor role cell: every department they oversee.
window.supRoleCellHtml = function(t) {
  const ROLE = {
    dean:       { label: '\ud83c\udf93 Dean',       color: '#7c3aed' },
    chairperson: { label: '\ud83e\ude91 Chairperson', color: '#0369a1' },
    supervisor:  { label: '\ud83d\udc64 Supervisor',  color: '#374151' }
  };
  const list = getSupervisedDepts(t);
  if (!list.length) return '<span style="color:var(--muted);font-size:0.75rem;">\u2014</span>';
  return list.map(a => {
    const r = ROLE[a.role] || ROLE.supervisor;
    return '<div style="font-size:0.75rem;font-weight:600;color:' + r.color + ';white-space:nowrap;">'
         + r.label + ' <span style="color:var(--muted);font-weight:500;">\u00b7 ' + escapeHtml(a.dept) + '</span></div>';
  }).join('');
};

// ===== SUPERVISORY ASSIGNMENTS (multi-department) =====
// supervisedDepts: [{ dept: 'COED', role: 'dean' }, ...]. `dept`/`deptRole`
// mirror the FIRST entry for older code and the Android app.

const SUP_ROLES = [
  { value: 'dean',       label: '\ud83c\udf93 Dean' },
  { value: 'chairperson', label: '\ud83e\ude91 Chairperson' },
  { value: 'supervisor',  label: '\ud83d\udc64 General Supervisor' }
];

// Normalise old or new shape into a list.
// FACULTY of deptCode: regular faculty's dept, or a supervisor's home dept
// unless they also supervise it.
// TABLE ORDER: click ID or Name header to sort/reverse (Name = by last name,
// ID = numeric). Remembered on this computer.
window._teacherSort = (function () {
  try { return JSON.parse(localStorage.getItem('teacherSortPref')) || { key: 'name', dir: 1 }; }
  catch (e) { return { key: 'name', dir: 1 }; }
})();

window.teacherListCompare = function(a, b) {
  const s = window._teacherSort || { key: 'name', dir: 1 };
  const opts = { sensitivity: 'base', numeric: true };
  const r = s.key === 'id'
    ? String(a.tid || '').localeCompare(String(b.tid || ''), undefined, opts)
    : personSortKey(a).localeCompare(personSortKey(b), undefined, opts);
  return (r || String(a.tid || '').localeCompare(String(b.tid || ''), undefined, opts)) * s.dir;
};

// Arrow beside the header that is currently sorting.
```

```
window.paintTeacherSortArrows = function() {
  const s = window._teacherSort;
  document.querySelectorAll('.sort-ind').forEach(el => {
    el.textContent = el.getAttribute('data-sort') === s.key ? (s.dir === 1 ? '\u25B2' : '\u25BC') : '';
  });
};

window.setTeacherSort = function(key) {
  const s = window._teacherSort;
  window._teacherSort = (s.key === key) ? { key: key, dir: -s.dir } : { key: key, dir: 1 };
  try { localStorage.setItem('teacherSortPref', JSON.stringify(window._teacherSort)); } catch (e) {}
  const q = (document.getElementById('teacherSearchInput') || {}).value || '';
  if (typeof renderTeachers === 'function') renderTeachers(q);
  if (typeof renderSupervisorTable === 'function') renderSupervisorTable();
  paintTeacherSortArrows();
};

window.teachesInDept = function(t, deptCode) {
  if (!t || t.deleted || !deptCode || t.dept !== deptCode) return false;
  if ((t.facultyType || 'regular') !== 'supervisor') return true;
  return !getSupervisedDepts(t).some(a => a.dept === deptCode);
};

// Short "Dean · COED" text for a supervisor listed as faculty elsewhere.
window.supervisorRoleSummary = function(t) {
  const label = r => r === 'dean' ? 'Dean' : r === 'chairperson' ? 'Chairperson' : 'Supervisor';
  return getSupervisedDepts(t).map(a => label(a.role) + ' \u00b7 ' + a.dept).join(', ');
};

window.getSupervisedDepts = function(t) {
  if (!t) return [];
  if (Array.isArray(t.supervisedDepts) && t.supervisedDepts.length) {
    return t.supervisedDepts.filter(a => a && a.dept);
  }
  // Legacy record: a single dept + deptRole pair.
  if (t.dept && t.deptRole) return [{ dept: t.dept, role: t.deptRole }];
  if (t.dept) return [{ dept: t.dept, role: 'supervisor' }];
  return [];
};

function _supRowHtml(idx, dept, role) {
  const depts = (typeof getDepartments === 'function') ? getDepartments() : {};
  const deptOpts = '<option value="">-- Department --</option>' +
    Object.entries(depts).map(([code, cfg]) =>
      `<option value="${code}"${code === dept ? ' selected' : ''}>${code} \u2014 ${escapeHtml(cfg.name)}</option>`
    ).join('');
  const roleOpts = '<option value="">-- Role --</option>' +
    SUP_ROLES.map(r => `<option value="${r.value}"${r.value === role ? ' selected' : ''}>${r.label}</option>`).join('');
  return `
    <div class="sup-row" data-idx="${idx}" style="display:flex;gap:8px;align-items:center;margin-bottom:8px;">
      <select class="form-control sup-dept" style="flex:1;">${deptOpts}</select>
      <select class="form-control sup-role" style="flex:1;">${roleOpts}</select>
      <button type="button" class="btn btn-ghost btn-icon btn-sm" title="Remove"
              onclick="removeSupRow(this)" style="color:var(--danger,#dc2626);">&#10005;</button>
    </div>`;
}

window.renderSupRows = function(assignments) {
  const host = document.getElementById('tchSupRows');
  if (!host) return;
  const list = (assignments && assignments.length) ? assignments : [{ dept: '', role: '' }];
  host.innerHTML = list.map((a, i) => _supRowHtml(i, a.dept || '', a.role || '')).join('');
};

window.addSupRow = function() {
  const host = document.getElementById('tchSupRows');
  if (!host) return;
  host.insertAdjacentHTML('beforeend', _supRowHtml(host.children.length, '', ''));
};

window.removeSupRow = function(btn) {
  const host = document.getElementById('tchSupRows');
  const row = btn.closest('.sup-row');
  if (row) row.remove();
  if (host && host.children.length === 0) renderSupRows(null);   // never leave it empty
};

// Collect the rows, dropping blanks and duplicate departments.
window.readSupRows = function() {
  const host = document.getElementById('tchSupRows');
  if (!host) return [];
  const seen = new Set();
  const out = [];
  Array.prototype.forEach.call(host.querySelectorAll('.sup-row'), row => {
    const dept = (row.querySelector('.sup-dept') || {}).value || '';
    const role = (row.querySelector('.sup-role') || {}).value || '';
    if (!dept || seen.has(dept)) return;
    seen.add(dept);
    out.push({ dept, role: role || 'supervisor' });
  });
  return out;
};

window.openAddSupervisorModal = function() {
  editTeacherId = null;
  _pendingFacultyType = 'supervisor';
  document.getElementById('teacherModalTitle').textContent = 'Add Supervisor';
  document.getElementById('saveTeacherBtn').textContent = 'Add Supervisor';
  document.getElementById('tchId').value = '';
  fillTchNameFields(null);
  populateDeptDropdown('tchDept', '');
  const ftEl = document.getElementById('tchFacultyType');
  if (ftEl) ftEl.value = 'supervisor';
  const hidden = document.getElementById('tchFacultyTypeHidden');
  if (hidden) hidden.value = 'supervisor';
```

```
  const drEl = document.getElementById('tchDeptRole');
  const drGrp = document.getElementById('tchDeptRoleGroup');
  if (drEl) drEl.value = '';
  renderSupRows(null);
  if (drGrp) drGrp.style.display = '';
  const ftGrp = document.getElementById('tchFacultyTypeGroup');
  if (ftGrp) ftGrp.style.display = 'none';
  // Show password field, clear it (will auto-set to TID on save if blank)
  const pwGrp = document.getElementById('tchPasswordGroup');
  const pwEl  = document.getElementById('tchPassword');
  if (pwGrp) pwGrp.style.display = '';
  if (pwEl)  pwEl.value = '';
  _setHomeDeptLabel('supervisor');
  openModal('addTeacherModal');
};

window.onFacultyTypeChange = function(val) {
  // Assign the BARE identifier: a top-level `let` is not a window property.
  _pendingFacultyType = val;
  const hidden = document.getElementById('tchFacultyTypeHidden');
  if (hidden) hidden.value = val;

  const grp = document.getElementById('tchDeptRoleGroup');
  if (grp) grp.style.display = val === 'supervisor' ? '' : 'none';
  _setHomeDeptLabel(val);

  // Show password field only for supervisors
  const pwGrp = document.getElementById('tchPasswordGroup');
  if (pwGrp) pwGrp.style.display = val === 'supervisor' ? '' : 'none';

  if (!editTeacherId) {
    const titleEl = document.getElementById('teacherModalTitle');
    const btnEl = document.getElementById('saveTeacherBtn');
    if (titleEl) titleEl.textContent = val === 'supervisor' ? 'Add Supervisor' : 'Add Faculty';
    if (btnEl) btnEl.textContent = val === 'supervisor' ? 'Add Supervisor' : 'Add Faculty';
  }
};

function openEditTeacherModal(id) {
  const t = getData('teachers', []).find(t => t.id === id);
  editTeacherId = id;
  document.getElementById('teacherModalTitle').textContent = 'Edit Teacher';
  document.getElementById('saveTeacherBtn').textContent = 'Save Changes';
  document.getElementById('tchId').value = t.tid;
  fillTchNameFields(t);
  populateDeptDropdown('tchDept', t.dept || '');
  const rkEl2 = document.getElementById('tchRank'); if (rkEl2) rkEl2.value = t.rank || '';
  const catEl = document.getElementById('tchCategory');
  if (catEl) catEl.value = t.category || '';
  const resolvedType = t.facultyType || 'regular';
  _pendingFacultyType = resolvedType;
  const ftEl = document.getElementById('tchFacultyType');
  if (ftEl) ftEl.value = resolvedType;
  const hidden = document.getElementById('tchFacultyTypeHidden');
  if (hidden) hidden.value = resolvedType;
  const drEl = document.getElementById('tchDeptRole');
  const drGrp = document.getElementById('tchDeptRoleGroup');
  if (drEl) drEl.value = t.deptRole || '';
  renderSupRows(getSupervisedDepts(t));
  if (drGrp) drGrp.style.display = (resolvedType === 'supervisor') ? '' : 'none';
  const ftGrp = document.getElementById('tchFacultyTypeGroup');
  if (ftGrp) ftGrp.style.display = '';
  // Password field: only for supervisors, pre-fill with current password so admin can see/change it
  const pwGrp = document.getElementById('tchPasswordGroup');
  const pwEl  = document.getElementById('tchPassword');
  if (pwGrp) pwGrp.style.display = resolvedType === 'supervisor' ? '' : 'none';
  if (pwEl)  pwEl.value = resolvedType === 'supervisor' ? (t.password || t.tid || '') : '';
  _setHomeDeptLabel(resolvedType);
  openModal('addTeacherModal');
}

// HOME vs SUPERVISED DEPARTMENTS: `dept` is where they teach (counted as
// faculty); supervisedDepts is where they're Dean/Chair. Blank home dept
// falls back to the first supervised one.
function _setHomeDeptLabel(type) {
  const sel = document.getElementById('tchDept');
  const lbl = sel && sel.closest('.form-group') && sel.closest('.form-group').querySelector('.form-label');
  if (!lbl) return;
  lbl.textContent = type === 'supervisor' ? 'Home Department (where they teach)' : 'Department';
  const first = sel.querySelector('option[value=""]');
  if (first) first.textContent = type === 'supervisor'
    ? '-- Same as first supervised department --'
    : '-- Select Department --';
}

window.forceSetSupervisor = function(id) {
  const teachers = getData('teachers', []);
  const idx = teachers.findIndex(t => t.id === id);
  if (idx === -1) return;
  const t = teachers[idx];
  if (!confirm('Move "' + t.name + '" to the Supervisors table?\nThis will set their Faculty Type to Supervisor and cannot be undone from here (use E
dit to change back).')) return;
  const before = cloneRecord(teachers[idx]);
  teachers[idx].facultyType = 'supervisor';
  if (!teachers[idx].password) teachers[idx].password = t.tid;
  commitRecords('teachers', teachers, [{ before, after: teachers[idx] }]);
  addAudit('Fix Supervisor', 'Moved ' + t.name + ' (' + t.tid + ') to Supervisor table');
  showToast(t.name + ' moved to Supervisors table!', 'success');
  renderTeachers();
};

// Guards against double-click creating the same teacher twice.
let _savingTeacher = false;

async function saveTeacher() {
```

```
  if (_savingTeacher) return;
  // Captured now: the modal stays open while the database check runs.
  const editingId = editTeacherId;
  const tid = document.getElementById('tchId').value.trim();
  const nameParts = tchNameParts();
  const name = buildName(nameParts);
  const dept = document.getElementById('tchDept').value;
  // #tchCategory may not exist; only read it if present, else keep the stored value.
  const catEl_    = document.getElementById('tchCategory');
  const category  = catEl_ ? (catEl_.value || '') : null;
  const rank = (document.getElementById('tchRank') || {}).value || '';
  const facultyType = _pendingFacultyType || 'regular';

  // `dept`/`deptRole` mirror the first assignment; regular faculty clears
  // both so migrateSupervisorRecords() won't re-promote them.
  const supervisedDepts = facultyType === 'supervisor' ? readSupRows() : [];
  const deptRole = supervisedDepts.length ? supervisedDepts[0].role : '';

  if (!tid || !nameParts.first.trim() || !nameParts.last.trim()) {
    showToast('Teacher ID, first name and last name are required.', 'error');
    return;
  }
  if (facultyType === 'supervisor' && supervisedDepts.length === 0) {
    showToast('Add at least one department for this supervisor to oversee.', 'error');
    return;
  }
  const nameFields = {
    title:      nameParts.title.trim(),
    firstName:  nameParts.first.trim(),
    middleName: nameParts.middle.trim(),
    lastName:   nameParts.last.trim(),
    suffix:     nameParts.suffix.trim()
  };

  const teachers = getData('teachers', []);

  // Read the optional supervisor password field
  const pwFieldVal = (document.getElementById('tchPassword') || {}).value?.trim() || '';

  // Quick check against this browser's copy first.
  if (teachers.find(t => t.tid === tid && !t.deleted && t.id !== editingId)) {
    showToast(editingId ? 'Another teacher already uses that ID.' : 'Teacher ID already exists.', 'error');
    return;
  }
  const priorTid = editingId ? (teachers.find(t => t.id === editingId) || {}).tid : null;
  const tidUnchanged = priorTid != null && tid === priorTid;

  _savingTeacher = true;
  try {
    // Then check the DATABASE.
    let matches = [];
    if (!tidUnchanged) {
      try {
        matches = await findRosterRecords('teachers', 'tid', tid);
      } catch (e) {
        console.error('Duplicate check failed:', e);
        showToast('Could not check the database for this ID. Check your connection and try again.', 'error');
        return;
      }
      if (matches.some(m => !m.deleted && m.id !== editingId)) {
        showToast('That Teacher ID is already in use (it may have just been added on another computer). ' +
                  'Refresh the list to see it.', 'error');
        return;
      }
    }

    let changes;
    let savedRecord;
    if (editingId) {
      const idx = teachers.findIndex(t => t.id === editingId);
      if (idx === -1) { showToast('Teacher not found. Refresh and try again.', 'error'); return; }
      const existing = teachers[idx];
      const before = cloneRecord(existing);
      teachers[idx].tid = tid;
      teachers[idx].name = name;
      Object.assign(teachers[idx], nameFields);
      teachers[idx].dept = dept;
      // null means "no such field on the form" - keep what is already stored.
      if (category !== null) teachers[idx].category = category;
      teachers[idx].rank = rank;
      teachers[idx].facultyType = facultyType;
      teachers[idx].deptRole = deptRole;
      teachers[idx].supervisedDepts = supervisedDepts;
      // Keep the chosen home department (see _setHomeDeptLabel).
      if (!teachers[idx].dept && supervisedDepts.length) teachers[idx].dept = supervisedDepts[0].dept;

      if (facultyType === 'supervisor') {
        if (pwFieldVal) {
          // Save other edits, then reset the password via resetLoginPassword.
          await commitRecords('teachers', teachers, [{ before, after: teachers[idx] }]);
          addAudit('Edit Supervisor', `Updated: ${name} (${tid}) — password reset requested`);
          await resetLoginPassword('supervisor', teachers[idx], pwFieldVal);
          closeModal('addTeacherModal');
          renderTeachers(_teacherSearchText());
          return;
        } else {
          // Keep existing password; if none exists (e.g. promoted from regular), default to TID
          if (!before.password) teachers[idx].password = tid;
          addAudit('Edit Supervisor', `Updated: ${name} (${tid})`);
        }
      } else {
        // Demoted to regular — clear supervisor password
        delete teachers[idx].password;
        addAudit('Edit Teacher', `Updated: ${name} (${tid}) — now regular faculty`);
      }
```

```
      changes = [{ before, after: teachers[idx] }];
      savedRecord = teachers[idx];
    } else {
      const fresh = {
        tid,
        name,
        ...nameFields,
        // Home department as chosen; blank falls back to the first supervised one.
        dept: dept || (supervisedDepts.length ? supervisedDepts[0].dept : ''),
        category: category || '',
        rank,
        facultyType,
        deptRole,
        supervisedDepts,
        status:'active',
        deleted: false
      };
      // Password defaults to TID. Never written to the audit log.
      if (facultyType === 'supervisor') fresh.password = pwFieldVal || tid;

      // A deleted teacher with this ID stays deleted; this is a new record.
      const rec = Object.assign({ id: 'tch' + Date.now() }, fresh);
      teachers.push(rec);
      changes = [{ before: null, after: rec }];
      savedRecord = rec;
      addAudit(facultyType === 'supervisor' ? 'Add Supervisor' : 'Add Teacher', `Added: ${name} (${tid})`);
    }

    // Awaited, so the toast reflects what reached the database.
    const ok = await commitRecords('teachers', teachers, changes);
    if (ok) {
      if (!editingId && facultyType === 'supervisor') {
        showToast(`Supervisor saved! Login password: ${savedRecord.password}`, 'success');
      } else {
        showToast(editingId ? (facultyType === 'supervisor' ? 'Supervisor updated!' : 'Teacher updated!')
                            : 'Teacher added!', 'success');
      }
    }

    closeModal('addTeacherModal');

    const searchInput = document.getElementById('teacherSearchInput');
    if (searchInput) searchInput.value = '';

    const teacherDeptBar = document.getElementById('teacherDeptFilterBar');
    if (teacherDeptBar) teacherDeptBar.dataset.active = '';

    renderTeachers();
  } finally {
    _savingTeacher = false;
  }
}

// What is typed in the Teachers search box, so a redraw keeps the filter.
function _teacherSearchText() {
  const el = document.getElementById('teacherSearchInput');
  return el ? el.value : '';
}

function toggleTeacherStatus(id) {
  const teachers = getData('teachers', []);
  const idx = teachers.findIndex(t => t.id === id);
  if (idx === -1) return;
  const before = cloneRecord(teachers[idx]);
  teachers[idx].status = teachers[idx].status === 'active' ? 'inactive' : 'active';
  commitRecords('teachers', teachers, [{ before, after: teachers[idx] }]);
  addAudit(teachers[idx].status==='active'?'Activate Teacher':'Deactivate Teacher', teachers[idx].name);
  renderTeachers(_teacherSearchText());
  showToast(`Teacher ${teachers[idx].status}!`, 'info');
}

function deleteTeacher(id) {
  const t = getData('teachers', []).find(t => t.id === id);

  // How many subjects they are on, so the warning can say so.
  const onSubjects = getData('subjects', []).filter(s =>
      (typeof subjectHasTeacher === 'function') ? subjectHasTeacher(s, id) : s.teacherId === id);
  const note = onSubjects.length
      ? ` They are assigned to ${onSubjects.length} subject${onSubjects.length === 1 ? '' : 's'}` +
        ` (${onSubjects.slice(0, 3).map(s => s.code).join(', ')}${onSubjects.length > 3 ? '…' : ''}),` +
        ' and will be removed from them.'
      : '';

  // SEF ratings they gave stop counting (countedEvaluations) but are kept.
  const sefGiven = evaluationsSubmittedBy(id, 'teacher').length;
  const sefNote = sefGiven
      ? ` The ${sefGiven} SEF rating${sefGiven === 1 ? '' : 's'} they gave to faculty will stop counting` +
        ' toward faculty scores (kept in the database, not deleted).'
      : '';

  showConfirm('Delete Teacher', `Remove ${t.name}?${note}${sefNote} Evaluations OF this teacher are kept.`, () => {
    const teachers = getData('teachers', []);
    const tIdx = teachers.findIndex(x => x.id === id);
    if (tIdx === -1) return;
    const before = cloneRecord(teachers[tIdx]);
    teachers[tIdx].deleted = true;
    commitRecords('teachers', teachers, [{ before, after: teachers[tIdx] }]);

    // Remove them from every subject too.
    if (onSubjects.length) {
      const subjects = getData('subjects', []);
      let changed = false;
      subjects.forEach(sub => {
        // Use the RAW list; subjectTeachers() already hides deleted teachers.
        const raw = Array.isArray(sub.teachers) && sub.teachers.length
```

```
          ? sub.teachers
          : (sub.teacherId ? [{ id: sub.teacherId, sections: [] }] : []);
      const kept = raw.filter(x => x && x.id !== id);
      if (kept.length === raw.length) return;          // this subject is unaffected
      sub.teachers  = kept.map(x => ({ id: x.id, sections: Array.isArray(x.sections) ? x.sections : [] }));
      sub.teacherId = kept.length ? kept[0].id : '';   // keep the single id in step
      changed = true;
    });
    if (changed) setData('subjects', subjects);
  }

  addAudit('Delete Teacher', `Deleted: ${t.name} — removed from ${onSubjects.length} subject(s)` +
    (sefGiven ? `, ${sefGiven} SEF rating(s) they gave no longer counted` : '') + ', records preserved');
  renderTeachers();
  if (typeof renderSubjects === 'function') renderSubjects();
  showToast('Teacher deleted' +
    (onSubjects.length ? ` and removed from ${onSubjects.length} subject(s)` : '') +
    (sefGiven ? `. Their ${sefGiven} SEF rating(s) no longer count.` : '.') + ' Records preserved.', 'info');
});
}
```

admin/js/reports/admin-annex.js

863 lines

```
// admin-annex.js - Annex C, Annex D, print serialisation and the FEDAF plan.
// Wording matches the QA office's signed forms; treat changes as official.
// Load order matters (see dashboard.html): the last definition wins.

// ===== ANNEX D REPORT MODAL (CMO 19 format) =====
window.showAnnexDReport = function(teacherId) {
    const t = getData('teachers', []).find(t => t.id === teacherId);
    if (!t) return;

    const sy = getActiveSY();
    const setScore = calculateWeightedSETRating(teacherId);
    const sefAgg2 = getSEFForTeacher(teacherId, inTerm);
    const sefScore = sefAgg2.count ? sefAgg2.average.toFixed(2) : '—';


    const subjects = getData('subjects', []).filter(s => subjectHasTeacher(s, teacherId) && s.loadType !== 'Overload' && !s.isLabSchool);
    const evals = getData('evaluations', []).filter(e => e.evaluatorType !== 'supervisor');
    const students = getData('students', []).filter(s => !s.deleted);

    const classBreakdown = subjects.map(sub => {
        // This teacher's ratings and students only - a subject may have several.
        const classEvals = evals.filter(e => e.subjectId === sub.id && evalBelongsTo(e, sub, teacherId));
        const enrolledCount = (sub.enrolledIds||[]).filter(id => {
            const st = students.find(s => s.id === id);
            return st && teacherTeachesStudent(sub, teacherId, st);
        }).length;
        const avgScore = classEvals.length > 0 ? classEvals.reduce((a,b) => a + b.totalScore, 0) / classEvals.length : 0;
        return { sub, enrolledCount, evalCount: classEvals.length, avgScore: avgScore.toFixed(2), percentage: Math.min(100, avgScore).toFixed(2) };
    });

    document.getElementById('annexDModalTitle').textContent = 'Faculty Evaluation & Development Acknowledgement Form';
    document.getElementById('annexDModalBody').innerHTML = `
        <div id="annexDPrintArea" style="font-family:serif;font-size:0.88rem;">
            <div style="text-align:center;font-weight:700;font-size:1rem;margin-bottom:16px;text-transform:uppercase;letter-spacing:0.03em;">
                Faculty Evaluation and Development Acknowledgement Form
            </div>

            <h4 style="background:#f0f0f0;padding:6px 12px;font-size:0.8rem;font-weight:700;margin:0 0 8px;text-transform:uppercase;">A. Faculty Memb
er Information</h4>
            <table style="width:100%;border-collapse:collapse;margin-bottom:16px;">
                <tr><td style="padding:4px 8px;width:40%;font-weight:600;">Name of Faculty</td><td style="padding:4px 8px;border-bottom:1px solid #99
9;">${escapeHtml(t.name)}</td></tr>
                <tr><td style="padding:4px 8px;font-weight:600;">Department/College</td><td style="padding:4px 8px;border-bottom:1px solid #999;">${e
scapeHtml(t.dept || '—')}</td></tr>
                <tr><td style="padding:4px 8px;font-weight:600;">Current Faculty Rank</td><td style="padding:4px 8px;border-bottom:1px solid #999;">
${escapeHtml(t.rank || '—')}</td></tr>
                <tr><td style="padding:4px 8px;font-weight:600;">Semester/Term &amp;amp; Academic Year</td><td style="padding:4px 8px;border-bottom:1px s
olid #999;">${sy ? sy.activeSem + ' / ' + sy.year : '—'}</td></tr>
            </table>

            <h4 style="background:#f0f0f0;padding:6px 12px;font-size:0.8rem;font-weight:700;margin:0 0 8px;text-transform:uppercase;">B. Faculty Eval
uation Summary</h4>
            <table style="width:100%;border-collapse:collapse;margin-bottom:16px;border:1px solid #ccc;">
                <thead>
                    <tr style="background:#e8e8e8;">
                        <th style="padding:8px;text-align:center;border:1px solid #ccc;font-size:0.82rem;">Student Evaluation of Teachers (SET)</th>
                        <th style="padding:8px;text-align:center;border:1px solid #ccc;font-size:0.82rem;">Supervisor's Evaluation of Faculty (SEF)</
th>
                    </tr>
                </thead>
                <tbody>
                    <tr>
                        <td style="padding:14px 8px;text-align:center;border:1px solid #ccc;font-size:1.6rem;font-weight:800;color:#16a34a;">${setSco
re}${setScore !== '0' && setScore !== 0 ? '%' : '—'}</td>
                        <td style="padding:14px 8px;text-align:center;border:1px solid #ccc;font-size:1.6rem;font-weight:800;color:#d97706;">${sefSco
re}${sefScore !== '—' ? '%' : ''}</td>
                    </tr>
                </tbody>
            </table>

            <h4 style="background:#f0f0f0;padding:6px 12px;font-size:0.8rem;font-weight:700;margin:0 0 8px;text-transform:uppercase;">C. Class Perfor
mance Breakdown</h4>
            <table style="width:100%;border-collapse:collapse;margin-bottom:16px;">
                <thead>
                    <tr style="background:#f8f8f8;">
                        <th style="padding:6px 8px;border:1px solid #ccc;font-size:0.78rem;">Subject</th>
                        <th style="padding:6px 8px;border:1px solid #ccc;font-size:0.78rem;text-align:center;">Enrolled</th>
                        <th style="padding:6px 8px;border:1px solid #ccc;font-size:0.78rem;text-align:center;">Evaluations</th>
                        <th style="padding:6px 8px;border:1px solid #ccc;font-size:0.78rem;text-align:center;">SET Avg</th>
                        <th style="padding:6px 8px;border:1px solid #ccc;font-size:0.78rem;text-align:center;">Percentage</th>
                    </tr>
                </thead>
                <tbody>
                    ${classBreakdown.length > 0 ? classBreakdown.map(cr => `
                        <tr>
                            <td style="padding:6px 8px;border:1px solid #ccc;font-size:0.8rem;"><strong>${escapeHtml(cr.sub.code)}</strong> — ${escap
eHtml(cr.sub.name)}</td>
                            <td style="padding:6px 8px;border:1px solid #ccc;text-align:center;">${cr.enrolledCount}</td>
                            <td style="padding:6px 8px;border:1px solid #ccc;text-align:center;">${cr.evalCount}</td>
                            <td style="padding:6px 8px;border:1px solid #ccc;text-align:center;">${cr.avgScore}</td>
                            <td style="padding:6px 8px;border:1px solid #ccc;text-align:center;font-weight:700;">${cr.percentage}%</td>
                        </tr>
                    `).join('') : `<tr><td colspan="5" style="padding:10px;text-align:center;color:#888;font-size:0.8rem;">No regular-load subjects.
</td></tr>`}
                </tbody>
            </table>

            <h4 style="background:#f0f0f0;padding:6px 12px;font-size:0.8rem;font-weight:700;margin:0 0 8px;text-transform:uppercase;">D. Development
Plan</h4>
            <table style="width:100%;border-collapse:collapse;margin-bottom:16px;border:1px solid #ccc;">
```

```
                <tr><td style="padding:8px;font-weight:600;font-size:0.8rem;width:40%;border:1px solid #ccc;">Areas for Improvement</td><td style="pa
dding:30px 8px;border:1px solid #ccc;"></td></tr>
                <tr><td style="padding:8px;font-weight:600;font-size:0.8rem;border:1px solid #ccc;">Proposed Learning and Development Activities</td>
<td style="padding:30px 8px;border:1px solid #ccc;"></td></tr>
                <tr><td style="padding:8px;font-weight:600;font-size:0.8rem;border:1px solid #ccc;">Action Plan</td><td style="padding:30px 8px;borde
r:1px solid #ccc;"></td></tr>
            </table>

            <p style="font-size:0.75rem;margin-bottom:16px;">I acknowledge that I have received and reviewed the faculty evaluation conducted for the
period mentioned above...</p>

            <table style="width:100%;border-collapse:collapse;">
                <tr>
                    <td style="width:50%;vertical-align:top;padding-right:20px;">
                        <div style="background:#f8f8f8;padding:8px;margin-bottom:6px;font-weight:700;font-size:0.78rem;text-align:center;">SUPERVISOR
</div>
                        <div style="margin-bottom:8px;"><span style="font-size:0.75rem;">Signature: </span><span style="border-bottom:1px solid #333;
display:inline-block;width:70%;"></span></div>
                        <div style="margin-bottom:8px;"><span style="font-size:0.75rem;">Name: </span><span style="border-bottom:1px solid #333;displ
ay:inline-block;width:75%;"></span></div>
                        <div><span style="font-size:0.75rem;">Date Signed: </span><span style="border-bottom:1px solid #333;display:inline-block;widt
h:65%;"></span></div>
                    </td>
                    <td style="width:50%;vertical-align:top;padding-left:20px;">
                        <div style="background:#f8f8f8;padding:8px;margin-bottom:6px;font-weight:700;font-size:0.78rem;text-align:center;">FACULTY</d
iv>
                        <div style="margin-bottom:8px;"><span style="font-size:0.75rem;">Signature: </span><span style="border-bottom:1px solid #333;
display:inline-block;width:70%;"></span></div>
                        <div style="margin-bottom:8px;"><span style="font-size:0.75rem;">Name: </span><span style="border-bottom:1px solid #333;displ
ay:inline-block;width:75%;"></span></div>
                        <div><span style="font-size:0.75rem;">Date Signed: </span><span style="border-bottom:1px solid #333;display:inline-block;widt
h:65%;"></span></div>
                    </td>
                </tr>
            </table>
        </div>
    `;
    openModal('annexDModal');
};

// ===== PRINT SERIALISATION =====
// Replaces form controls in a CLONE with plain text of their current values,
// since innerHTML doesn't include typed input.
function _annexPrintHtml(sourceEl) {
    if (!sourceEl) return '';
    const clone = sourceEl.cloneNode(true);
    const src   = sourceEl;

    const asText = (value, placeholder) => {
        const d = document.createElement('div');
        d.style.cssText = 'font-family:inherit;font-size:0.8rem;white-space:pre-wrap;'
                        + 'min-height:1.1em;padding:2px 0;';
        d.textContent = value || placeholder || '';
        return d;
    };

    // Dates print as "15 December 2026" rather than 2026-12-15.
    const prettyDate = v => {
        if (!v) return '';
        const d = new Date(v + 'T00:00:00');
        return isNaN(d) ? v : d.toLocaleDateString('en-PH', { day: 'numeric', month: 'long', year: 'numeric' });
    };

    ['textarea', 'input', 'select'].forEach(tag => {
        const live  = src.querySelectorAll(tag);
        const copies = clone.querySelectorAll(tag);
        // Same query on both trees, so index i refers to the same control.
        copies.forEach((node, i) => {
            const el = live[i];
            if (!el) return;
            let text;
            if (tag === 'select')                text = el.options[el.selectedIndex] ? el.options[el.selectedIndex].text : '';
            else if (el.type === 'checkbox')     text = el.checked ? 'Yes' : 'No';
            else if (el.type === 'date')         text = prettyDate(el.value);
            else                                 text = el.value;
            node.replaceWith(asText(text));
        });
    });

    // Buttons and on-screen-only controls have no place on paper.
    clone.querySelectorAll('.annex-noprint, button').forEach(el => el.remove());
    return clone.innerHTML;
}

// SECTION ROSTER (click column 3 of Annex C): who has submitted, and any
// duplicates. Shows status only - never ratings or comments (CMO 8.1, 6.10).
// teacherId is optional; without it the roster covers every teacher.
window.annexSectionRoster = function (subjectId, sectionLabel, teacherId) {
  const sub = getData('subjects', []).find(x => x.id === subjectId);
  if (!sub) { showToast('Subject not found.', 'error'); return; }

  const students = getData('students', []).filter(s => !s.deleted);
  const inTerm   = (typeof annexEvalInTerm === 'function') ? annexEvalInTerm : () => true;

  const focus = (teacherId && subjectHasTeacher(sub, teacherId)) ? teacherId : null;
  const tIds  = focus ? [focus] : subjectTeacherIds(sub);

  // Scope: this subject, student evaluations, this term, teacher(s) in view.
  const evals = getData('evaluations', []).filter(e =>
      e.subjectId === subjectId &&
      e.evaluatorType !== 'supervisor' &&
      tIds.some(tid => evalBelongsTo(e, sub, tid)) &&
      inTerm(e));

  // Per student: teachers in view and rating count each. Duplicate = two for the SAME teacher.
```

```
  const ratingsFor = (st, tid) => evals.filter(e => e.studentId === st.id && evalBelongsTo(e, sub, tid)).length;
  const owedTo     = st => tIds.filter(tid => teacherTeachesStudent(sub, tid, st));

  // A multi-section label is an unrated class; accept any listed section.
  const wanted = String(sectionLabel || '').split(',').map(x => x.trim()).filter(Boolean);
  const labelOf = st => (typeof _sectionLabel === 'function') ? _sectionLabel(st) : '';
  const inSection = st => !wanted.length || wanted.indexOf(labelOf(st)) !== -1;

  const roster = (sub.enrolledIds || [])
      .map(id => students.find(s => s.id === id))
      .filter(Boolean)
      .filter(inSection)
      .filter(st => owedTo(st).length > 0)          // only students the teacher(s) teach
      .sort(byName);                                // last name A to Z

  // Count submissions per student to expose duplicates.
  const countFor = {};
  evals.forEach(e => { if (e.studentId) countFor[e.studentId] = (countFor[e.studentId] || 0) + 1; });

  const doneFor   = st => owedTo(st).filter(tid => ratingsFor(st, tid) > 0).length;
  const isDup     = st => owedTo(st).some(tid => ratingsFor(st, tid) > 1);
  const submitted = roster.filter(st => doneFor(st) === owedTo(st).length).length;
  const dupes     = roster.filter(isDup);

  // Submissions made BY this section.
  const sectionIds   = new Set(roster.map(s => s.id));
  const sectionEvals = evals.filter(e => sectionIds.has(e.studentId));

  // Stray = not enrolled in the subject at all.
  const subjectIds = new Set((sub.enrolledIds || []));
  const strays = Object.keys(countFor).filter(id => !subjectIds.has(id));

  const rows = roster.map(s => {
    const n     = countFor[s.id] || 0;
    const dup   = isDup(s);
    const owed  = owedTo(s).length;
    const done  = doneFor(s);
    return `<tr${dup ? ' style="background:#fef2f2;"' : ''}>
        <td style="padding:6px 8px;border-bottom:1px solid #eee;font-family:monospace;font-size:0.8rem;">${escapeHtml(s.sid || '')}</td>
        <td style="padding:6px 8px;border-bottom:1px solid #eee;">${escapeHtml(listName(s))}</td>
        <td style="padding:6px 8px;border-bottom:1px solid #eee;text-align:center;">
          ${done === 0
            ? '<span style="color:#b45309;">Not yet</span>'
            : done < owed
              ? `<span style="color:#b45309;">${done} of ${owed} teachers</span>`
              : '<span style="color:#15803d;">Submitted</span>'}
        </td>
        <td style="padding:6px 8px;border-bottom:1px solid #eee;text-align:center;font-weight:${dup ? '700' : '400'};color:${dup ? '#b91c1c' : 'inher
it'};">
          ${n}${dup ? ' &#9888; duplicate' : ''}
        </td>
      </tr>`;
  }).join('');

  const strayRows = strays.map(id => {
    const s = students.find(x => x.id === id);
    return `<tr style="background:#fffbeb;">
        <td style="padding:6px 8px;border-bottom:1px solid #eee;font-family:monospace;font-size:0.8rem;">${escapeHtml(s ? s.sid : id)}</td>
        <td style="padding:6px 8px;border-bottom:1px solid #eee;">${escapeHtml(s ? listName(s) : '(student record not found)')}</td>
        <td style="padding:6px 8px;border-bottom:1px solid #eee;text-align:center;color:#b45309;">Not enrolled</td>
        <td style="padding:6px 8px;border-bottom:1px solid #eee;text-align:center;">${countFor[id]}</td>
      </tr>`;
  }).join('');

  const warn = [];
  if (dupes.length) warn.push(`${dupes.length} student${dupes.length !== 1 ? 's have' : ' has'} submitted more than once`);
  if (strays.length) warn.push(`${strays.length} submission${strays.length !== 1 ? 's are' : ' is'} from someone not enrolled in this subject`);

  const old = document.getElementById('annexRosterOverlay');
  if (old) old.remove();

  const wrap = document.createElement('div');
  wrap.id = 'annexRosterOverlay';
  wrap.className = 'modal-overlay active';
  wrap.style.display = 'flex';
  wrap.innerHTML = `
    <div class="modal" style="max-width:620px;">
      <div class="modal-header">
        <h3>${escapeHtml(sub.code)} &mdash; ${escapeHtml(sectionLabel || 'All sections')}</h3>
        <button class="modal-close" onclick="document.getElementById('annexRosterOverlay').remove()">&times;</button>
      </div>
      <div class="modal-body">
        <div style="display:flex;gap:18px;flex-wrap:wrap;margin-bottom:14px;font-size:0.85rem;">
          <div><strong>${roster.length}</strong> enrolled</div>
          <div style="color:#15803d;"><strong>${submitted}</strong> submitted</div>
          <div style="color:#b45309;"><strong>${roster.length - submitted}</strong> not yet</div>
          <div><strong>${sectionEvals.length}</strong> submission${sectionEvals.length !== 1 ? 's' : ''} ${wanted.length ? 'from this section' : 'in
total'}</div>
        </div>
        ${warn.length ? `<div style="background:#fef2f2;border-left:3px solid #b91c1c;padding:8px 10px;margin-bottom:12px;font-size:0.82rem;color:#7f
1d1d;">
            &#9888; ${escapeHtml(warn.join('. '))}. A repeated submission is counted twice in both the number of submissions and the average, so it s
kews the weighted score.
          </div>` : ''}
        <div style="max-height:340px;overflow:auto;">
          <table style="width:100%;border-collapse:collapse;font-size:0.85rem;">
            <thead><tr style="background:#f2f7f4;">
              <th style="padding:7px 8px;text-align:left;">Student ID</th>
              <th style="padding:7px 8px;text-align:left;">Name</th>
              <th style="padding:7px 8px;text-align:center;">Status</th>
              <th style="padding:7px 8px;text-align:center;">Submissions</th>
            </tr></thead>
            <tbody>${rows || `<tr><td colspan="4" style="padding:18px;text-align:center;color:#888;">No students enrolled in this section.</td></tr>
`}${strayRows}</tbody>
          </table>
```

```
        </div>
        <p style="font-size:0.74rem;color:#777;margin:12px 0 0;line-height:1.5;">
          Submission status only. Individual ratings and comments stay anonymous (CMO 19 &sect;6.10).
        </p>
      </div>
      <div class="modal-footer">
        <button class="btn btn-primary" onclick="document.getElementById('annexRosterOverlay').remove()">Close</button>
      </div>
    </div>`;
  document.body.appendChild(wrap);
};

window.printAnnexD = function() {
    const printContent = document.getElementById('annexDPrintArea');
    if (!printContent) return;
    const w = window.open('', '_blank');
    w.document.write(`<html><head><title>Annex D</title><style>body{font-family:serif;margin:30px;font-size:12px;}table{width:100%;}</style></head><b
ody>${_annexPrintHtml(printContent)}</body></html>`);
    w.document.close();
    w.print();
};

// ===== COMBINED ANNEX C & D VIEWER =====
// Current teacher id for tab switching
window._currentAnnexTeacherId = null;

window.showAnnexReports = function(teacherId, fromReports) {
    window._currentAnnexTeacherId = teacherId;
    // Follows the Reports term only when opened from Reports.
    window._annexTermOverride = fromReports ? null : 'ACTIVE';
    window._currentAnnexFromReports = !!fromReports;
    const t = getData('teachers', []).find(t => t.id === teacherId);
    if (!t) return;

    // Set modal title
    const titleEl = document.getElementById('annexDModalTitle');
    const subtitleEl = document.getElementById('annexModalSubtitle');
    if (titleEl) titleEl.textContent = 'Faculty Evaluation Reports';
    if (subtitleEl) subtitleEl.textContent = t.name + ' — ' + (t.dept || '—');

    // Reset tabs
    const tabC = document.getElementById('annexTabC');
    const tabD = document.getElementById('annexTabD');
    if (tabC) { tabC.className = 'annex-tab annex-tab-active'; }
    if (tabD) { tabD.className = 'annex-tab'; }

    // Default: show Annex C
    buildAnnexCContent(teacherId);
    openModal('annexDModal');
};

window.switchAnnexTab = function(tab) {
    const teacherId = window._currentAnnexTeacherId;
    if (!teacherId) return;
    const tabC = document.getElementById('annexTabC');
    const tabD = document.getElementById('annexTabD');
    if (tab === 'C') {
        if (tabC) tabC.className = 'annex-tab annex-tab-active';
        if (tabD) tabD.className = 'annex-tab';
        buildAnnexCContent(teacherId);
    } else {
        if (tabC) tabC.className = 'annex-tab';
        if (tabD) tabD.className = 'annex-tab annex-tab-active';
        buildAnnexDContent(teacherId);
    }
};

window.printActiveAnnex = function() {
    const printContent = document.getElementById('annexPrintArea');
    if (!printContent) return;
    const tabD = document.getElementById('annexTabD');
    const isD = tabD && tabD.classList.contains('annex-tab-active');
    const w = window.open('', '_blank');
    w.document.write(`<html><head><title>${isD ? 'Annex D — FEDAF' : 'Annex C — IFER'}</title><style>body{font-family:serif;margin:30px;font-size:12p
x;}table{width:100%;border-collapse:collapse;}th,td{border:1px solid #ccc;padding:6px 8px;}</style></head><body>${_annexPrintHtml(printContent)}</bod
y></html>`);
    w.document.close();
    w.print();
};

// Annex C — Individual Faculty Evaluation Report
// Column (2) "Year/Section" (e.g. "BSCE 4A"), built from enrolled students;
// courses[]/yearLevel is only a fallback.
// courseShorthand: full course name -> short code, derived not stored.
window.courseShorthand = function(name) {
    const n = String(name || '').trim();
    if (!n) return '';

    // 1. CODE FIRST, description in brackets: "BPED (Bachelor of ...)" -> BPED.
    const beforeParen = n.split('(')[0].trim();
    if (n.indexOf('(') > 0 && beforeParen.length >= 2 && beforeParen.length <= 22) {
        return beforeParen.replace(/\s+/g, ' ').toUpperCase();
    }

    // 2. CODE IN BRACKETS - "Bachelor of Elementary Education (BEEd)" -> BEED.
    //    Up to 24 chars so majors fit ("BSIT AUTOMOTIVE").
    const paren = n.match(/\(([^)]{2,24})\)/);
    if (paren) return paren[1].trim().replace(/\s+/g, ' ').toUpperCase();

    // 3. Something already short with no spaces is left alone - "BSCE" -> BSCE.
    if (n.indexOf(' ') === -1 && n.length <= 8) return n.toUpperCase();

    // 4. Otherwise initials, skipping of/in/and/the; existing abbreviations
    //    stay whole ("BS Nursing" -> BSN).
    const STOP = ['of', 'in', 'and', 'the', 'for', 'a'];
```

```
    let out = '';
    n.replace(/\([^)]*\)/g, ' ')          // never take initials from a description
     .split(/[\s\-\/]+/).filter(Boolean).forEach(function(w) {
        if (STOP.indexOf(w.toLowerCase()) > -1) return;
        if (!/[A-Za-z]/.test(w[0])) return;
        out += (w === w.toUpperCase() && w.length <= 4) ? w : w[0];
    });
    return out.length >= 2 ? out.toUpperCase() : n.toUpperCase();
};

window.annexYearSection = function(sub, students) {
    const combos = new Set();
    (sub.enrolledIds || []).forEach(function(id) {
        const st = students.find(function(x) { return x.id === id; });
        if (!st) return;
        const course = courseShorthand(st.course);
        const yr  = String(st.year || '').match(/\d+/);
        const sec = String(st.section || '').trim().toUpperCase();
        const label = (course + ' ' + (yr ? yr[0] : '') + sec).trim();
        if (label) combos.add(label);
    });
    if (combos.size) return Array.from(combos).sort().join(', ');

    const c = (sub.courses || []).map(courseShorthand).filter(Boolean).join('/');
    const y = String(sub.yearLevel || '').match(/\d+/);
    return [c, y ? y[0] : ''].filter(Boolean).join(' ') || '\u2014';
};

// The office form letters the comments section "D" too; reproduced as-is.
// Change to 'E' for sequential letters.
window.ANNEX_C_COMMENTS_LETTER = 'D';

window.buildAnnexCContent = function(teacherId) {
    const t = getData('teachers', []).find(t => t.id === teacherId);
    if (!t) return;
    const termInfo = (typeof annexTermInfo === 'function') ? annexTermInfo() : { label: '—' };
    const inTerm = (typeof annexEvalInTerm === 'function') ? annexEvalInTerm : (() => true);
    // Section B lists regular-load classes; the two ways it can come up empty
    // are reported separately below.
    const allSubjects = getData('subjects', []);
    const assigned = allSubjects.filter(s => subjectHasTeacher(s, teacherId));
    const excluded = assigned.filter(s => s.loadType === 'Overload' || s.isLabSchool);
    const subjects = assigned.filter(s => s.loadType !== 'Overload' && !s.isLabSchool);
    const evals = getData('evaluations', []).filter(e => e.evaluatorType !== 'supervisor' && inTerm(e));
    const students = getData('students', []).filter(s => !s.deleted);

    // Why is section B empty? Say so instead of rendering a blank table.
    let annexCNotice = '';
    if (assigned.length === 0) {
        annexCNotice = 'No subject is assigned to this faculty member. Open Subjects and set '
            + escapeHtml(t.name) + ' as the teacher on their classes.';
    } else if (subjects.length === 0) {
        annexCNotice = 'All ' + assigned.length + ' subject(s) assigned to this faculty are marked '
            + 'Overload or Lab School, which CMO 19 excludes from the SET computation.';
    } else if (students.length === 0) {
        annexCNotice = 'No student records are loaded, so enrolment counts show as 0.';
    } else {
        const totalEnrolled = subjects.reduce(
            (n, sub) => n + (sub.enrolledIds || []).filter(id => students.find(st => st.id === id)).length, 0);
        if (totalEnrolled === 0) {
            annexCNotice = 'These subjects have no enrolled students that match the current student '
                + 'records. If the roster was re-imported, re-check enrolment on each subject.';
        }
    }
    const sefEvals = getData('evaluations', []).filter(e => e.teacherId === teacherId && e.evaluatorType === 'supervisor' && inTerm(e));
    const sefScore = sefEvals.length > 0 ? (sefEvals.reduce((a, b) => a + b.totalScore, 0) / sefEvals.length).toFixed(2) : '—';

    // Section C: one row per supervisor, then the average. Supervisor id may be
    // in `studentId` (app) or `supervisorId` (admin modal).
    const allTeachersForSef = getData('teachers', []);
    const sig = getSignatories();          // auto-filled Prepared/Reviewed by
    const sefAggC = getSEFForTeacher(teacherId, inTerm);
    const sefRows = sefAggC.list.map((ev, i) => {
        const supId = ev.supervisorId || ev.studentId || '';
        const sup = allTeachersForSef.find(x => x.id === supId);
        return {
            seq: i + 1,
            name: sup ? sup.name : (supId ? 'Supervisor (record removed)' : 'Not recorded'),
            score: (parseFloat(ev.totalScore) || 0).toFixed(2)
        };
    });


    // Section B uses computeWeightedSET (admin-scoring.js), same as every report.
    const setAgg = computeWeightedSET(teacherId, inTerm);
    const totalStudents = setAgg.totalStudents;
    const totalWeightedScore = setAgg.totalWeighted;

    // ONE ROW PER YEAR/SECTION, built from who responded so rows sum to the
    // TOTAL; unevaluated classes still get a row. 2 decimals, as on the form.
    let _seq = 0;
    const classBreakdown = [];
    setAgg.classes.forEach(c => {
        (c.sections || []).forEach(sec => {
            classBreakdown.push({
                seq: ++_seq,
                sub: c.sub,
                yearSection: sec.yearSection,
                count: sec.count,
                enrolledCount: c.enrolledCount,
                evalCount: c.evalCount,
                // No responses: print a dash, not 0.00.
                avgScore: sec.rated ? sec.avgScore : '\u2014',
                weightedScore: sec.rated ? sec.weightedScore : '\u2014'
            });
```

```
        });
    });


    // Flag course values that aren't registered courses (they print verbatim).
    const knownCourses = new Set();
    Object.keys(getData('customCourses', {})).forEach(function (d) {
        (getData('customCourses', {})[d] || []).forEach(function (c) {
            knownCourses.add(courseKey(c));
        });
    });
    const unknownCourses = new Set();
    let missingCourse = 0;
    subjects.forEach(function (sub) {
        (sub.enrolledIds || []).forEach(function (id) {
            const st = students.find(function (x) { return x.id === id; });
            if (!st) return;
            const c = normalizeCourseName(st.course);
            if (!c) { missingCourse++; return; }
            if (knownCourses.size && !knownCourses.has(courseKey(c))) unknownCourses.add(c);
        });
    });

    // Explain empty Year/Section cells for "Any year level" subjects.
    const noYearSection = classBreakdown
        .filter(cr => cr.yearSection === '\u2014')
        .map(cr => cr.sub.code);
    if (unknownCourses.size && !annexCNotice) {
        annexCNotice = 'Year/Section shows ' + Array.from(unknownCourses).map(escapeHtml).join(', ')
            + ' because that is the Course recorded on an enrolled student, and it is not one of '
            + 'the courses registered for any department. Open Students and correct the course, '
            + 'or add it under Manage Courses - this column is printed on the signed Annex C.';
    } else if (missingCourse && !annexCNotice) {
        annexCNotice = missingCourse + ' enrolled student(s) have no Course recorded, so their '
            + 'Year/Section shows only the year and section (for example "2B" instead of "BSCE 2B").';
    }
    if (noYearSection.length && !annexCNotice) {
        annexCNotice = 'Year/Section is blank for ' + noYearSection.join(', ') + '. That column is '
            + 'built from the students enrolled in each class, so it needs their course, year level '
            + 'and section on record. A subject set to "Any year level" contributes no cohort of its '
            + 'own, and neither does a class with nobody enrolled.';
    }

    const overallSET = totalStudents > 0 ? (totalWeightedScore / totalStudents).toFixed(2) : '0.00';

    // Section D comments (anonymous): student and supervisor comments for this faculty.
    const studentComments = getData('evaluations', [])
        .filter(e => e.teacherId === teacherId && e.evaluatorType !== 'supervisor' && inTerm(e) && e.comment && e.comment.trim())
        .map(e => e.comment.trim());
    const supervisorComments = sefEvals
        .filter(e => e.comment && e.comment.trim())
        .map(e => e.comment.trim());

    const commentRow = (n, text) =>
        `<tr><td style="padding:12px 8px;border:1px solid #ccc;text-align:center;vertical-align:top;">${n}</td>`
        + `<td style="padding:12px 8px;border:1px solid #ccc;white-space:pre-wrap;word-break:break-word;">${text ? escapeHtml(text) : ''}</td></tr>`;
    // At least 5 rows; blanks fill the rest.
    const buildCommentRows = (arr) => {
        const rowCount = Math.max(5, arr.length);
        let out = '';
        for (let i = 0; i < rowCount; i++) out += commentRow(i + 1, arr[i] || '');
        return out;
    };
    const studentCommentRows = buildCommentRows(studentComments);
    const supervisorCommentRows = buildCommentRows(supervisorComments);

    document.getElementById('annexDModalBody').innerHTML = `
    <div id="annexPrintArea" style="font-family:serif;font-size:0.88rem;padding:4px 0;min-width:0;">
        <div style="text-align:center;font-weight:700;font-size:1rem;margin-bottom:4px;text-transform:uppercase;letter-spacing:0.03em;">Individual Fa
culty Evaluation Report</div>
        <div style="text-align:center;font-size:0.75rem;color:#666;margin-bottom:16px;">(ANNEX C — CMO No. 19, Series of 2025)</div>

        <h4 style="background:#f0f0f0;padding:6px 12px;font-size:0.8rem;font-weight:700;margin:0 0 8px;text-transform:uppercase;">A. Faculty Informat
ion</h4>
        <table style="width:100%;border-collapse:collapse;margin-bottom:16px;table-layout:fixed;">
            <colgroup><col style="width:42%"/><col style="width:58%"/></colgroup>
            <tr><td style="padding:4px 8px;font-weight:600;word-break:break-word;">Name of Faculty Evaluated</td><td style="padding:4px 8px;border-bo
ttom:1px solid #999;word-break:break-word;">${escapeHtml(t.name)}</td></tr>
            <tr><td style="padding:4px 8px;font-weight:600;word-break:break-word;">Department/College</td><td style="padding:4px 8px;border-bottom:1p
x solid #999;word-break:break-word;">${escapeHtml(t.dept || '—')}</td></tr>
            <tr><td style="padding:4px 8px;font-weight:600;word-break:break-word;">Current Faculty Rank</td><td style="padding:4px 8px;border-bottom:
1px solid #999;word-break:break-word;">${printField(t.rank, '70%', 'e.g. Assistant Professor I')}</td></tr>
            <tr><td style="padding:4px 8px;font-weight:600;word-break:break-word;">Semester/Term &amp;amp; Academic Year</td><td style="padding:4px 8px;b
order-bottom:1px solid #999;word-break:break-word;">${escapeHtml(termInfo.label)}</td></tr>
        </table>

        <h4 style="background:#f0f0f0;padding:6px 12px;font-size:0.8rem;font-weight:700;margin:0 0 8px;text-transform:uppercase;">B. Summary of Avera
ge SET Rating</h4>
        <p style="font-size:0.75rem;color:#555;margin:0 0 8px;padding:0 4px;">
            <strong>Step 1:</strong> Get the average SET rating for each class. &amp;nbsp;
            <strong>Step 2:</strong> Multiply the number of students in each class with its average SET rating to get the Weighted SET Score per clas
s. &amp;nbsp;
            <strong>Step 3:</strong> Get the total number of students and the total weighted SET score.
        </p>
        ${annexCNotice ? `<div class="annex-noprint" style="border:1px solid #f0c36d;background:#fff8e6;color:#7a5b00;padding:9px 12px;border-radius:
6px;font-size:0.76rem;margin-bottom:10px;">${escapeHtml(annexCNotice)}</div>` : ''}
        <div style="overflow-x:auto;margin-bottom:16px;">
        <table style="width:100%;min-width:480px;border-collapse:collapse;border:1px solid #ccc;table-layout:fixed;">
            <colgroup>
                <col style="width:6%"/>
                <col style="width:16%"/>
                <col style="width:28%"/>
                <col style="width:14%"/>
                <col style="width:18%"/>
                <col style="width:18%"/>
```

```
            </colgroup>
            <thead>
                <tr style="background:#e8e8e8;">
                    <th style="padding:6px 5px;border:1px solid #ccc;font-size:0.72rem;text-align:center;white-space:normal;word-break:break-word;ver
tical-align:top;">Seq</th>
                    <th style="padding:6px 5px;border:1px solid #ccc;font-size:0.72rem;white-space:normal;word-break:break-word;vertical-align:top;">
(1) Course Code</th>
                    <th style="padding:6px 5px;border:1px solid #ccc;font-size:0.72rem;white-space:normal;word-break:break-word;vertical-align:top;">
(2) Year/Section</th>
                    <th style="padding:6px 5px;border:1px solid #ccc;font-size:0.72rem;text-align:center;white-space:normal;word-break:break-word;ver
tical-align:top;">(3) No. of Students</th>
                    <th style="padding:6px 5px;border:1px solid #ccc;font-size:0.72rem;text-align:center;white-space:normal;word-break:break-word;ver
tical-align:top;">(4) Average SET Rating</th>
                    <th style="padding:6px 5px;border:1px solid #ccc;font-size:0.72rem;text-align:center;white-space:normal;word-break:break-word;ver
tical-align:top;">(3x4) Weighted SET Score</th>
                </tr>
            </thead>
            <tbody>
                ${classBreakdown.length > 0 ? classBreakdown.map(cr => `
                <tr>
                    <td style="padding:5px;border:1px solid #ccc;text-align:center;font-size:0.78rem;">${cr.seq}</td>
                    <td style="padding:5px;border:1px solid #ccc;font-style:italic;font-size:0.78rem;word-break:break-word;">${escapeHtml(cr.sub.cod
e)}</td>
                    <td style="padding:5px;border:1px solid #ccc;text-align:center;font-size:0.78rem;word-break:break-word;">${escapeHtml(cr.yearSect
ion)}</td>
                    <td style="padding:5px;border:1px solid #ccc;text-align:center;font-size:0.78rem;cursor:pointer;"
                        onclick="annexSectionRoster('${cr.sub.id}', &quot;${String(cr.yearSection).replace(/"/g, '')}&quot;, '${teacherId}')"
                        title="Click to see who has submitted in this section">
                        <span>${cr.count}</span>
                        <span class="annex-noprint" style="color:#3f6f5b;font-size:0.7rem;margin-left:3px;">&#9432;</span>
                    </td>
                    <td style="padding:5px;border:1px solid #ccc;text-align:center;font-size:0.78rem;">${cr.avgScore}</td>
                    <td style="padding:5px;border:1px solid #ccc;text-align:center;font-weight:600;font-size:0.78rem;">${cr.weightedScore}</td>
                </tr>`).join('') : `<tr><td colspan="6" style="padding:10px;text-align:center;color:#888;font-size:0.8rem;">No regular-load subjects.
</td></tr>`}
                <tr style="background:#f8f8f8;font-weight:700;">
                    <td colspan="3" style="padding:6px 8px;border:1px solid #ccc;text-align:right;font-size:0.78rem;">TOTAL</td>
                    <td style="padding:6px 8px;border:1px solid #ccc;text-align:center;font-size:0.78rem;">${totalStudents}</td>
                    <td style="padding:6px 8px;border:1px solid #ccc;text-align:center;font-size:0.78rem;">TOTAL</td>
                    <td style="padding:6px 8px;border:1px solid #ccc;text-align:center;font-size:0.78rem;">${totalWeightedScore.toFixed(2)}</td>
                </tr>
            </tbody>
        </table>
        </div>

        <h4 style="background:#f0f0f0;padding:6px 12px;font-size:0.8rem;font-weight:700;margin:0 0 8px;text-transform:uppercase;">C. Summary of Avera
ge SEF Rating</h4>
        <table style="width:100%;border-collapse:collapse;margin-bottom:16px;border:1px solid #ccc;table-layout:fixed;">
            <colgroup><col style="width:12%"/><col style="width:58%"/><col style="width:30%"/></colgroup>
            <thead>
                <tr style="background:#e8e8e8;">
                    <th style="padding:8px;text-align:center;border:1px solid #ccc;font-size:0.8rem;">Seq</th>
                    <th style="padding:8px;text-align:center;border:1px solid #ccc;font-size:0.8rem;">Supervisor</th>
                    <th style="padding:8px;text-align:center;border:1px solid #ccc;font-size:0.8rem;">SEF Rating</th>
                </tr>
            </thead>
            <tbody>
                ${sefRows.length ? sefRows.map(r => `
                <tr>
                    <td style="padding:6px 8px;border:1px solid #ccc;text-align:center;font-size:0.78rem;">${r.seq}</td>
                    <td style="padding:6px 8px;border:1px solid #ccc;font-size:0.78rem;">${escapeHtml(r.name)}</td>
                    <td style="padding:6px 8px;border:1px solid #ccc;text-align:center;font-size:0.78rem;">${r.score}</td>
                </tr>`).join('') : `
                <tr><td colspan="3" style="padding:10px;text-align:center;color:#888;font-size:0.8rem;">No supervisor evaluation recorded for this te
rm.</td></tr>`}
                <tr style="font-weight:700;background:#f7f7f7;">
                    <td colspan="2" style="padding:6px 8px;border:1px solid #ccc;text-align:center;font-size:0.78rem;">Average</td>
                    <td style="padding:6px 8px;border:1px solid #ccc;text-align:center;font-size:0.78rem;">${sefScore}</td>
                </tr>
            </tbody>
        </table>

        <h4 style="background:#f0f0f0;padding:6px 12px;font-size:0.8rem;font-weight:700;margin:0 0 8px;text-transform:uppercase;">D. SET and SEF Rati
ngs</h4>
        <p style="font-size:0.75rem;color:#555;margin:0 0 8px;padding:0 4px;">
            <strong>Computation:</strong> Calculate the Overall SET Rating by dividing the total Weighted SET Score by the total number of students
(${totalWeightedScore.toFixed(2)} ÷ ${totalStudents} = ${overallSET}).
        </p>
        <table style="width:100%;border-collapse:collapse;margin-bottom:16px;border:1px solid #ccc;table-layout:fixed;">
            <colgroup><col style="width:50%"/><col style="width:50%"/></colgroup>
            <thead>
                <tr style="background:#e8e8e8;">
                    <th style="padding:8px;text-align:center;border:1px solid #ccc;font-size:0.8rem;">SET Rating</th>
                    <th style="padding:8px;text-align:center;border:1px solid #ccc;font-size:0.8rem;">*SEF Rating</th>
                </tr>
            </thead>
            <tbody>
                <tr>
                    <td style="padding:16px 8px;text-align:center;border:1px solid #ccc;">
                        <div style="font-size:0.7rem;color:#555;margin-bottom:4px;">Student Evaluation of Teachers</div>
                        <strong style="font-size:1.6rem;color:#16a34a;">${overallSET}</strong>
                    </td>
                    <td style="padding:16px 8px;text-align:center;border:1px solid #ccc;">
                        <div style="font-size:0.7rem;color:#555;margin-bottom:4px;">Supervisor's Evaluation of Faculty</div>
                        <strong style="font-size:1.6rem;color:#d97706;">${sefScore !== '—' ? sefScore : '—'}</strong>
                    </td>
                </tr>
                <tr style="background:#f8f8f8;font-weight:700;">
                    <td colspan="2" style="padding:7px;text-align:center;border:1px solid #ccc;font-size:0.78rem;">OVERALL RATING</td>
                </tr>
            </tbody>
        </table>
        <p style="font-size:0.72rem;color:#666;margin-bottom:16px;font-style:italic;">*Note: rating given by the supervisor using the SEF instrument
(Annex B)</p>

```

```
        <h4 style="background:#f0f0f0;padding:6px 12px;font-size:0.8rem;font-weight:700;margin:0 0 8px;text-transform:uppercase;">${ANNEX_C_COMMENTS_
LETTER}. Summary of Qualitative Comments and Suggestions</h4>
        <table style="width:100%;border-collapse:collapse;margin-bottom:10px;border:1px solid #ccc;table-layout:fixed;">
            <colgroup><col style="width:10%"/><col style="width:90%"/></colgroup>
            <thead>
                <tr style="background:#e8e8e8;">
                    <th style="padding:7px 8px;border:1px solid #ccc;font-size:0.78rem;text-align:center;">Seq</th>
                    <th style="padding:7px 8px;border:1px solid #ccc;font-size:0.78rem;">Comments and Suggestions from the Students</th>
                </tr>
            </thead>
            <tbody>
                ${studentCommentRows}
                <tr><td style="padding:6px 8px;border:1px solid #ccc;text-align:center;color:#888;font-style:italic;" colspan="2">(add additional row
s if necessary)</td></tr>
            </tbody>
        </table>
        <table style="width:100%;border-collapse:collapse;margin-bottom:16px;border:1px solid #ccc;table-layout:fixed;">
            <colgroup><col style="width:10%"/><col style="width:90%"/></colgroup>
            <thead>
                <tr style="background:#e8e8e8;">
                    <th style="padding:7px 8px;border:1px solid #ccc;font-size:0.78rem;text-align:center;">Seq</th>
                    <th style="padding:7px 8px;border:1px solid #ccc;font-size:0.78rem;">Comments and Suggestions from the Supervisor</th>
                </tr>
            </thead>
            <tbody>
                ${supervisorCommentRows}
                <tr><td style="padding:6px 8px;border:1px solid #ccc;text-align:center;color:#888;font-style:italic;" colspan="2">(add additional row
s if necessary)</td></tr>
            </tbody>
        </table>

        <div style="display:flex;gap:16px;margin-top:8px;flex-wrap:wrap;">
            <div style="flex:1;min-width:180px;">
                <div style="font-size:0.75rem;font-weight:600;margin-bottom:4px;">Prepared by:</div>
                <div style="margin-bottom:6px;font-size:0.75rem;">Signature of Staff: <span style="border-bottom:1px solid #333;display:inline-block;
width:55%;"></span></div>
                ${sigLine('Name of Staff', sig.preparedName + (sig.preparedRole ? ' /' + sig.preparedRole : ''), '58%')}
                ${sigDate('prepared')}
            </div>
            <div style="flex:1;min-width:180px;">
                <div style="font-size:0.75rem;font-weight:600;margin-bottom:4px;">Reviewed by:</div>
                <div style="margin-bottom:6px;font-size:0.75rem;">Signature of Authorized Official: <span style="border-bottom:1px solid #333;displa
y:inline-block;width:35%;"></span></div>
                ${sigLine('Name of Authorized Official', sig.reviewedName + (sig.reviewedRole ? ' /' + sig.reviewedRole : ''), '38%')}
                ${sigDate('reviewed')}
            </div>
        </div>
    </div>`;
};

// Annex D — Faculty Evaluation and Development Acknowledgement Form
// ===== FEDAF DEVELOPMENT PLAN & ACKNOWLEDGMENT (CMO §6.7, §10.2) =====
// Stored per faculty per rating period.
window._devPlanId = function(teacherId, term) {
    return ('dp_' + teacherId + '_' + (term || '')).replace(/[^a-zA-Z0-9_-]/g, '_');
};

window.getDevelopmentPlan = function(teacherId) {
    const term = (typeof annexTermInfo === 'function' ? annexTermInfo() : { label: '' }).label;
    const id = _devPlanId(teacherId, term);
    return getData('developmentPlans', []).find(p => p.id === id)
        || { id, teacherId, term, areas: '', activities: '', actionPlan: '',
             supervisorName: '', supervisorDate: '', facultyName: '', facultyDate: '',
             acknowledged: false, savedAt: '' };
};

window.saveDevelopmentPlan = function(teacherId) {
    const term = (typeof annexTermInfo === 'function' ? annexTermInfo() : { label: '' }).label;
    const id = _devPlanId(teacherId, term);
    const v = elId => (document.getElementById(elId) ? document.getElementById(elId).value.trim() : '');
    const plan = {
        id, teacherId, term,
        areas:          v('dpAreas'),
        activities:     v('dpActivities'),
        actionPlan:     v('dpAction'),
        supervisorName: v('dpSupName'),
        supervisorDate: v('dpSupDate'),
        facultyName:    v('dpFacName'),
        facultyDate:    v('dpFacDate'),
        acknowledged:   !!(v('dpFacName') && v('dpFacDate')),
        savedAt:        new Date().toISOString()
    };
    const plans = getData('developmentPlans', []);
    const i = plans.findIndex(p => p.id === id);
    if (i > -1) plans[i] = plan; else plans.push(plan);
    setData('developmentPlans', plans);
    if (typeof syncCollectionToFirestore === 'function') syncCollectionToFirestore('developmentPlans', plans);
    const t = getData('teachers', []).find(x => x.id === teacherId);
    if (typeof addAudit === 'function') addAudit('FEDAF Saved', `Development plan${plan.acknowledged ? ' + acknowledgment' : ''} for ${t ? t.name : t
eacherId} (${term})`);
    if (typeof showToast === 'function') showToast(plan.acknowledged ? 'Development plan saved & acknowledged.' : 'Development plan saved.', 'succes
s');
    // Refresh the Annex D panel so the acknowledged badge appears.
    if (typeof switchAnnexTab === 'function') switchAnnexTab('D');
};

window.buildAnnexDContent = function(teacherId) {
    const t = getData('teachers', []).find(t => t.id === teacherId);
    if (!t) return;
    const termInfo = (typeof annexTermInfo === 'function') ? annexTermInfo() : { label: '—' };
    const inTerm = (typeof annexEvalInTerm === 'function') ? annexEvalInTerm : (() => true);
    const setScore = (typeof getTeacherOverallRating === 'function') ? getTeacherOverallRating(teacherId).overallSET : calculateWeightedSETRating(tea
cherId);
    const sefEvals = getData('evaluations', []).filter(e => e.teacherId === teacherId && e.evaluatorType === 'supervisor' && inTerm(e));
    const sefScore = sefEvals.length > 0 ? (sefEvals.reduce((a, b) => a + b.totalScore, 0) / sefEvals.length).toFixed(2) : '—';
```

```
    // Saved development plan / acknowledgment for this faculty & term (FEDAF).
    const _dp = (typeof getDevelopmentPlan === 'function') ? getDevelopmentPlan(teacherId)
        : { areas:'', activities:'', actionPlan:'', supervisorName:'', supervisorDate:'', facultyName:'', facultyDate:'', acknowledged:false, savedA
t:'' };

    document.getElementById('annexDModalBody').innerHTML = `
    <div id="annexPrintArea" style="font-family:serif;font-size:0.88rem;padding:4px 0;min-width:0;">
        <div style="text-align:center;font-weight:700;font-size:1rem;margin-bottom:4px;text-transform:uppercase;letter-spacing:0.03em;">
            Faculty Evaluation and Development Acknowledgement Form
        </div>
        <div style="text-align:center;font-size:0.75rem;color:#666;margin-bottom:16px;">(ANNEX D — CMO No. 19, Series of 2025)</div>

        <h4 style="background:#f0f0f0;padding:6px 12px;font-size:0.8rem;font-weight:700;margin:0 0 8px;text-transform:uppercase;">A. Faculty Member I
nformation</h4>
        <table style="width:100%;border-collapse:collapse;margin-bottom:16px;table-layout:fixed;">
            <colgroup><col style="width:42%"/><col style="width:58%"/></colgroup>
            <tr><td style="padding:4px 8px;font-weight:600;word-break:break-word;">Name of Faculty</td><td style="padding:4px 8px;border-bottom:1px s
olid #999;word-break:break-word;">${escapeHtml(t.name)}</td></tr>
            <tr><td style="padding:4px 8px;font-weight:600;word-break:break-word;">Department/College</td><td style="padding:4px 8px;border-bottom:1p
x solid #999;word-break:break-word;">${escapeHtml(t.dept || '—')}</td></tr>
            <tr><td style="padding:4px 8px;font-weight:600;word-break:break-word;">Current Faculty Rank</td><td style="padding:4px 8px;border-bottom:
1px solid #999;word-break:break-word;">${printField(t.rank, '70%', 'e.g. Assistant Professor I')}</td></tr>
            <tr><td style="padding:4px 8px;font-weight:600;word-break:break-word;">Semester/Term &amp;amp; Academic Year</td><td style="padding:4px 8px;b
order-bottom:1px solid #999;word-break:break-word;">${escapeHtml(termInfo.label)}</td></tr>
        </table>

        <h4 style="background:#f0f0f0;padding:6px 12px;font-size:0.8rem;font-weight:700;margin:0 0 8px;text-transform:uppercase;">B. Faculty Evaluati
on Summary</h4>
        <table style="width:100%;border-collapse:collapse;margin-bottom:16px;border:1px solid #ccc;table-layout:fixed;">
            <colgroup><col style="width:50%"/><col style="width:50%"/></colgroup>
            <thead>
                <tr style="background:#e8e8e8;">
                    <th colspan="2" style="padding:8px;text-align:center;border:1px solid #ccc;font-size:0.8rem;">Overall Rating</th>
                </tr>
                <tr style="background:#f0f0f0;">
                    <th style="padding:8px;text-align:center;border:1px solid #ccc;font-size:0.77rem;white-space:normal;word-break:break-word;vertica
l-align:top;">STUDENT EVALUATION<br>OF TEACHERS (SET)</th>
                    <th style="padding:8px;text-align:center;border:1px solid #ccc;font-size:0.77rem;white-space:normal;word-break:break-word;vertica
l-align:top;">SUPERVISOR'S EVALUATION<br>OF FACULTY (SAF)</th>
                </tr>
            </thead>
            <tbody>
                <tr>
                    <td style="padding:14px 8px;text-align:center;border:1px solid #ccc;font-size:1.6rem;font-weight:800;color:#16a34a;">${setScore}
${setScore !== '0' && setScore !== 0 ? '%' : '—'}</td>
                    <td style="padding:14px 8px;text-align:center;border:1px solid #ccc;font-size:1.6rem;font-weight:800;color:#d97706;">${sefScore !
== '—' ? sefScore + '%' : '—'}</td>
                </tr>
            </tbody>
        </table>

        <h4 style="background:#f0f0f0;padding:6px 12px;font-size:0.8rem;font-weight:700;margin:0 0 8px;text-transform:uppercase;">C. Development Plan
<span style="font-weight:400;font-size:0.72rem;">(to be jointly accomplished by the Supervisor and Faculty)</span>${_dp.acknowledged ? ` <span style
="background:#dcfce7;color:#16a34a;font-size:0.62rem;padding:2px 8px;border-radius:10px;">✓ ACKNOWLEDGED</span>` : ''}</h4>
        <table style="width:100%;border-collapse:collapse;margin-bottom:16px;border:1px solid #ccc;table-layout:fixed;">
            <colgroup><col style="width:38%"/><col style="width:62%"/></colgroup>
            <tr><td style="padding:8px;font-weight:600;font-size:0.8rem;border:1px solid #ccc;vertical-align:top;word-break:break-word;">Areas for Im
provement</td><td style="padding:6px;border:1px solid #ccc;"><textarea id="dpAreas" rows="3" style="width:100%;border:none;font-family:inherit;font-s
ize:0.8rem;resize:vertical;outline:none;">${escapeHtml(_dp.areas)}</textarea></td></tr>
            <tr><td style="padding:8px;font-weight:600;font-size:0.8rem;border:1px solid #ccc;vertical-align:top;word-break:break-word;">Proposed Lea
rning and Development Activities</td><td style="padding:6px;border:1px solid #ccc;"><textarea id="dpActivities" rows="3" style="width:100%;border:non
e;font-family:inherit;font-size:0.8rem;resize:vertical;outline:none;">${escapeHtml(_dp.activities)}</textarea></td></tr>
            <tr><td style="padding:8px;font-weight:600;font-size:0.8rem;border:1px solid #ccc;vertical-align:top;word-break:break-word;">Action Plan
</td><td style="padding:6px;border:1px solid #ccc;"><textarea id="dpAction" rows="3" style="width:100%;border:none;font-family:inherit;font-size:0.8r
em;resize:vertical;outline:none;">${escapeHtml(_dp.actionPlan)}</textarea></td></tr>
        </table>

        <p style="font-size:0.75rem;margin-bottom:16px;font-style:italic;">I acknowledge that I have received and reviewed the faculty evaluation con
ducted for the period mentioned above. I understand that my signature below does not necessarily indicate agreement with the evaluation but confirms
that I have been given the opportunity to discuss it with my supervisor.</p>

        <table style="width:100%;border-collapse:collapse;border:1px solid #ccc;table-layout:fixed;">
            <colgroup><col style="width:30%"/><col style="width:70%"/></colgroup>
            <thead>
                <tr style="background:#e0e0e0;">
                    <th colspan="2" style="padding:8px;text-align:center;border:1px solid #ccc;font-size:0.8rem;">SUPERVISOR</th>
                </tr>
            </thead>
            <tbody>
                <tr><td style="padding:6px 8px;border:1px solid #ccc;font-size:0.78rem;">Name</td><td style="padding:4px 8px;border:1px solid #ccc;">
<input id="dpSupName" value="${escapeHtml(_dp.supervisorName)}" style="width:100%;border:none;font-family:inherit;font-size:0.8rem;outline:none;"/></
td></tr>
                <tr><td style="padding:6px 8px;border:1px solid #ccc;font-size:0.78rem;">Date Signed</td><td style="padding:4px 8px;border:1px solid
#ccc;"><input id="dpSupDate" type="date" value="${escapeHtml(_dp.supervisorDate)}" style="border:none;font-family:inherit;font-size:0.8rem;outline:no
ne;"/></td></tr>
            </tbody>
        </table>
        <table style="width:100%;border-collapse:collapse;margin-top:0;border:1px solid #ccc;border-top:none;table-layout:fixed;">
            <colgroup><col style="width:30%"/><col style="width:70%"/></colgroup>
            <thead>
                <tr style="background:#d0d0d0;">
                    <th colspan="2" style="padding:8px;text-align:center;border:1px solid #ccc;font-size:0.8rem;">FACULTY</th>
                </tr>
            </thead>
            <tbody>
                <tr><td style="padding:6px 8px;border:1px solid #ccc;font-size:0.78rem;">Name</td><td style="padding:4px 8px;border:1px solid #ccc;">
<input id="dpFacName" value="${escapeHtml(_dp.facultyName)}" placeholder="Faculty types name to acknowledge" style="width:100%;border:none;font-famil
y:inherit;font-size:0.8rem;outline:none;"/></td></tr>
                <tr><td style="padding:6px 8px;border:1px solid #ccc;font-size:0.78rem;">Date Signed</td><td style="padding:4px 8px;border:1px solid
#ccc;"><input id="dpFacDate" type="date" value="${escapeHtml(_dp.facultyDate)}" style="border:none;font-family:inherit;font-size:0.8rem;outline:non
e;"/></td></tr>
            </tbody>
        </table>
```

```
        <div class="annex-noprint" style="margin-top:14px;display:flex;align-items:center;gap:10px;">
            <button onclick="saveDevelopmentPlan('${teacherId}')" style="padding:9px 18px;border:none;border-radius:8px;background:var(--primary,#059
669);color:#fff;font-weight:600;font-size:0.82rem;cursor:pointer;">Save Development Plan &amp;amp; Acknowledgment</button>
            ${_dp.savedAt ? `<span style="font-size:0.72rem;color:var(--muted,#777);">Last saved ${new Date(_dp.savedAt).toLocaleString('en-PH')}</sp
an>` : ''}
        </div>
    </div>`;
};

// Fills the password box with the Teacher ID; saving goes through resetLoginPassword.
window.resetPasswordToId = function() {
    const tid = document.getElementById('tchId') ? document.getElementById('tchId').value : '';
    const pwInput = document.getElementById('tchPassword');
    if (pwInput && tid) {
        pwInput.value = tid;
        showToast('Set to Teacher ID. Save to start the reset.', 'info');
    }
};
// REMOVED (dead code): exportEnhancedReport - defined in adminEval.js.
```

admin/js/reports/admin-fer.js

320 lines

```
// admin-fer.js - institutional FER trends and the supervisor list.
// Load order matters (see dashboard.html): the last definition wins.

// ===== INSTITUTIONAL FER GRAPH DATA TRENDS =====
window.renderInstitutionalFER = function() {
    const teachers = getData('teachers', []).filter(t => !t.deleted);
    const deptMap = {};
    teachers.forEach(t => {
        if (!t.dept) return;
        if (!deptMap[t.dept]) deptMap[t.dept] = { setTotal: 0, setCount: 0, sefTotal: 0, sefCount: 0 };
        // Shared term-scoped rating, so cards match the table.
        const r = (typeof getTeacherOverallRating === 'function') ? getTeacherOverallRating(t.id) : null;
        const set = r ? parseFloat(r.overallSET) : 0;
        const sef = (r && r.sefScore !== '—') ? parseFloat(r.sefScore) : 0;
        if (set > 0) { deptMap[t.dept].setTotal += set; deptMap[t.dept].setCount++; }
        if (sef > 0) { deptMap[t.dept].sefTotal += sef; deptMap[t.dept].sefCount++; }
    });

    const stats = Object.entries(deptMap).map(([deptCode, d]) => ({
        name: deptCode,
        avgSET: d.setCount > 0 ? (d.setTotal / d.setCount).toFixed(2) : '—',
        avgSEF: d.sefCount > 0 ? (d.sefTotal / d.sefCount).toFixed(2) : '—',
        count: d.setCount || d.sefCount
    })).sort((a,b) => parseFloat(b.avgSET||0) - parseFloat(a.avgSET||0));

    const container = document.getElementById('institutionalFERContainer');
    if (container) {
        container.innerHTML = `
            <div class="card">
                <div class="card-header-bar"><h3>Institutional Statistical Trends (FER)</h3><span class="badge badge-primary" style="font-size:0.68re
m;">${typeof getReportTermInfo === 'function' ? escapeHtml(getReportTermInfo().label) : ''}</span></div>
                <div class="card-body">
                    <p style="font-size:0.8rem;color:var(--muted);margin-bottom:15px;">SET and SEF per department — displayed separately per CMO 19.
Used by President and VPAA for institutional decision-making.</p>
                    <div style="display:grid;grid-template-columns:repeat(auto-fit,minmax(180px,1fr));gap:15px;">
                        ${stats.map(s => `
                            <div style="padding:15px;border:1px solid var(--border);border-radius:8px;text-align:center;">
                                <div style="font-size:0.75rem;font-weight:700;margin-bottom:8px;">${escapeHtml(s.name)}</div>
                                <div style="display:flex;justify-content:space-around;">
                                    <div>
                                        <div style="font-size:0.62rem;color:var(--muted);">SET</div>
                                        <div style="font-size:1.2rem;font-weight:800;color:#16a34a;">${s.avgSET}${s.avgSET !== '—' ? '%' : ''}</div>
                                    </div>
                                    <div style="border-left:1px solid var(--border);"></div>
                                    <div>
                                        <div style="font-size:0.62rem;color:var(--muted);">SEF</div>
                                        <div style="font-size:1.2rem;font-weight:800;color:#d97706;">${s.avgSEF}${s.avgSEF !== '—' ? '%' : ''}</div>
                                    </div>
                                </div>
                                <div style="font-size:0.62rem;color:var(--muted);margin-top:6px;">${s.count} faculty</div>
                            </div>
                        `).join('')}
                        ${stats.length === 0 ? '<p style="text-align:center;color:var(--muted);">No department data available.</p>' : ''}
                    </div>
                </div>
            </div>
        `;
    }
};

function viewTeacherReport(teacherId) {
  const t = getData('teachers', []).find(t => t.id === teacherId);
  if (!t) return;

  const rating = calculateFinalRating(teacherId);
  const subjects = getData('subjects', []);
  const evals = getData('evaluations', []);
  const tSubs = subjects.filter(s => subjectHasTeacher(s, teacherId));
  const tEvals = evals.filter(e => tSubs.some(s => s.id === e.subjectId) && e.evaluatorType !== 'supervisor');

  document.getElementById('reportModalTitle').textContent = `Report: ${t.name}`;
  document.getElementById('reportModalBody').innerHTML = `
    <div class="info-row"><span class="info-label">Teacher ID</span><span class="info-value">${escapeHtml(t.tid)}</span></div>
    <div class="info-row"><span class="info-label">Department</span><span class="info-value">${escapeHtml(t.dept || '—')}</span></div>
    <div class="info-row"><span class="info-label">Subjects</span><span class="info-value">${tSubs.map(s=>s.code).join(', ')||'None'}</span></div>
    <div class="info-row"><span class="info-label">Total Evaluations</span><span class="info-value">${tEvals.length}</span></div>

    <div class="total-score-display" style="margin-top:20px;">
      <div class="big-score">${rating.hasSEF ? rating.finalPercentage + '%' : (rating.hasSET ? rating.studentPercentage + '%' : '—')}</div>
      <div class="out-of">${rating.hasSEF ? 'Institutional composite: 60% SET + 40% SEF (NOT prescribed by CMO 19 — the CMO reports SET and SEF separ
ately)' : 'SET only — no SEF submitted yet'}</div>
      <div class="remarks-badge" style="background:${rating.remarksColor};">${rating.remarks}</div>
    </div>

    <div style="margin-top:16px; display:grid; grid-template-columns:1fr 1fr; gap:16px;">
      <div style="background:#f0fdf4; padding:12px; border-radius:8px; text-align:center;">
        <div style="font-size:0.7rem; color:var(--muted);">Student SET (CMO 19 §8.3)</div>
        <div style="font-size:1.4rem; font-weight:700;">${rating.hasSET ? rating.studentPercentage + '%' : '—'}</div>
        <div style="font-size:0.7rem;">(weighted avg across ${tSubs.length} class${tSubs.length !== 1 ? 'es' : ''})</div>
      </div>
      <div style="background:#eff6ff; padding:12px; border-radius:8px; text-align:center;">
        <div style="font-size:0.7rem; color:var(--muted);">Supervisor SEF (CMO 19 §9.3)</div>
        <div style="font-size:1.4rem; font-weight:700;">${rating.hasSEF ? rating.supervisorPercentage + '%' : '—'}</div>
      </div>
    </div>

    <div style="margin-top:16px;">
      <div style="font-size:0.78rem;font-weight:700;margin-bottom:10px;">Anonymous Comments</div>
      ${tEvals.filter(e=>e.comment).map(e=>`<div style="background:var(--bg);border-radius:7px;padding:10px 12px;margin-bottom:8px;font-size:0.78rem;
color:var(--muted);font-style:italic;">"${escapeHtml(e.comment)}"</div>`).join('')||'<p style="color:var(--muted);font-size:0.78rem;">No comments.</p
```

```
>'}
    </div>
  `;
  openModal('viewReportModal');
}

// PDF export via the print dialog, from window._allTeacherEvalData so it
// matches the on-screen table (term and department filter included).
function exportReport() {
  const allData = window._allTeacherEvalData || _buildTeacherEvalData();
  const dept    = window._reportsDeptFilter || '';
  const list    = allData.filter(t => !dept || (t.dept || 'UNASSIGNED') === dept);
  const termLabel = (typeof getReportTermInfo === 'function') ? getReportTermInfo().label : '';

  // Prepared by / Reviewed by from Actions > Signatories; blank prints a signing line.
  const sig = (typeof getSignatories === 'function') ? getSignatories()
            : { preparedName: '', preparedRole: '', reviewedName: '', reviewedRole: '' };
  const sigBox = (label, name, role) => `
      <div class="sig">
        <div class="sig-label">${label}</div>
        <div class="sig-name">${name ? escapeHtml(name) : ' '}</div>
        <div class="sig-role">${role ? escapeHtml(role) : ' '}</div>
        <div class="sig-date">Date: _____</div>
      </div>`;

  // All Departments: generic title + Department column. One department: its
  // name as title, no column (see _rptDeptFullName in admin-reports.js).
  const showDept  = !dept;
  const deptName  = (typeof _rptDeptFullName === 'function') ? _rptDeptFullName(dept) : dept;
  const pageTitle = showDept ? 'Faculty Evaluation Report \u2014 SET and SEF Ratings' : deptName;

  const rows = list.map(t => `
    <tr>
      <td>${escapeHtml(t.tid)}</td>
      <td>${escapeHtml(listName(t))}</td>
      ${showDept ? `<td>${escapeHtml(t.dept || 'N/A')}</td>` : ''}
      <td>${escapeHtml(t.facultyType || 'regular')}</td>
      <td style="text-align:center;">${t.overallSET}${t.overallSET !== '—' ? '%' : ''}</td>
      <td style="text-align:center;">${t.sefScore}${t.sefScore !== '—' ? '%' : ''}</td>
    </tr>`).join('');

  const html = `<html><head><meta charset="utf-8"><title>${escapeHtml(showDept ? 'Faculty Evaluation Report' : deptName + ' \u2014 Faculty Evaluation
Report')}</title><style>
      /* The page is a column at least one sheet tall; margin-top:auto on the
         signatures pushes them to the BOTTOM of the sheet. A report longer
         than one sheet keeps them right after the table instead. */
      @page{margin:12mm;}
      html,body{margin:0;}
      body{font-family:serif;font-size:12px;color:#111;}
      .page{min-height:calc(100vh - 2px);display:flex;flex-direction:column;box-sizing:border-box;padding:18px 18px 8px;}
      @media print{.page{padding:0;}}
      h1{font-size:16px;margin:0 0 2px;}
      .sub{color:#555;margin:0 0 16px;font-size:11px;}
      table{width:100%;border-collapse:collapse;}
      th,td{border:1px solid #ccc;padding:6px 8px;text-align:left;}
      th{background:#f2f2f2;}
      /* Signature block. Kept together so the names never split from the
         table onto a page of their own. */
      .sigs{display:flex;justify-content:space-between;gap:40px;margin-top:auto;padding-top:48px;page-break-inside:avoid;}
      .sig{flex:1;max-width:280px;}
      .sig-label{font-size:11px;margin-bottom:34px;}
      .sig-name{border-top:1px solid #111;padding-top:4px;font-weight:bold;text-transform:uppercase;font-size:12px;text-align:center;}
      .sig-role{font-size:11px;text-align:center;color:#333;}
      .sig-date{font-size:11px;margin-top:14px;}
    </style></head><body><div class="page">
      <h1>${escapeHtml(pageTitle)}</h1>
      <div class="sub">${showDept ? '' : 'Faculty Evaluation Report &mdash; SET and SEF Ratings &middot; '}${escapeHtml(termLabel)} &middot; Generate
d ${new Date().toLocaleDateString()}</div>
      <table><thead><tr><th>Teacher ID</th><th>Name</th>${showDept ? '<th>Department</th>' : ''}<th>Faculty Type</th><th>SET Rating</th><th>SEF Ratin
g</th></tr></thead>
      <tbody>${rows}</tbody></table>
      <div class="sigs">
        ${sigBox('Prepared by:', sig.preparedName, sig.preparedRole)}
        ${sigBox('Reviewed by:', sig.reviewedName, sig.reviewedRole)}
      </div>
    </div></body></html>`;

  const w = window.open('', '_blank');
  w.document.write(html);
  w.document.close();
  w.print();

  addAudit('Export Report', 'Exported PDF (SET and SEF separate columns)' + (dept ? ` — dept ${dept}` : ''));
  showToast('Report ready — choose "Save as PDF" in the print dialog.', 'success');
}

// Prints Annex C for every listed faculty as ONE print job, via
// buildAnnexCContent (admin-annex.js); the modal content is restored after.
window.printAllAnnexC = function () {
  const allData = window._allTeacherEvalData || _buildTeacherEvalData();
  const dept    = window._reportsDeptFilter || '';
  // Faculty AND supervisors with an assigned class, a student evaluation,
  // or a supervisor rating.
  const list    = allData
    .filter(t => !dept || (t.dept || 'UNASSIGNED') === dept)
    .filter(t => t.totalClasses > 0 || t.totalEvaluations > 0 || t.sefRaters > 0);

  if (!list.length) {
    showToast('Nothing to print' + (dept ? ' in this department' : '') + ' — no assigned classes, student evaluations, or supervisor ratings.', 'inf
o');
    return;
  }

  const body = document.getElementById('annexDModalBody');
  if (!body) { showToast('Could not open the print view.', 'error'); return; }
```

```
  const savedBody = body.innerHTML;

  // Follow the Reports page's own term selection, same as clicking a row does.
  const savedOverride = window._annexTermOverride;
  window._annexTermOverride = null;

  const sections = list.map(t => {
    buildAnnexCContent(t.id);
    return `<div style="page-break-after:always;">${body.innerHTML}</div>`;
  });

  body.innerHTML = savedBody;
  window._annexTermOverride = savedOverride;

  const w = window.open('', '_blank');
  w.document.write(`<html><head><title>Annex C — All Faculty</title><style>
      body{font-family:serif;margin:30px;font-size:12px;}
      table{width:100%;border-collapse:collapse;}
      th,td{border:1px solid #ccc;padding:6px 8px;}
      button{display:none;}
    </style></head><body>${sections.map(h => _annexPrintHtmlFromString(h)).join('')}</body></html>`);
  w.document.close();
  w.print();

  addAudit('Print All Annex C', `${list.length} faculty and supervisors` + (dept ? ` — dept ${dept}` : ''));
};

// String version of _annexPrintHtml's cleanup.
window._annexPrintHtmlFromString = function (htmlStr) {
  const holder = document.createElement('div');
  holder.innerHTML = htmlStr;
  return (typeof _annexPrintHtml === 'function') ? _annexPrintHtml(holder) : holder.innerHTML;
};

// ===== SUPERVISOR LIST =====
window.renderSupervisorList = function(search = '') {
  const pillBar = document.getElementById('supervisorPageDeptFilterBar');
  if (pillBar) {
    const depts = ['COED','CCJS','CCIS','CON','CEA','COM','CAT','GS'];
    const active = pillBar.dataset.active || '';
    pillBar.innerHTML = `<button class="dept-filter-pill ${active===''?'active':''}" onclick="setSupervisorPageDeptFilter('')">All</button>` +
      depts.map(d => `<button class="dept-filter-pill ${active===d?'active':''}" onclick="setSupervisorPageDeptFilter('${d}')">${d}</button>`).join
('');
  }
  const deptFilter = (document.getElementById('supervisorPageDeptFilterBar') || {}).dataset?.active || '';

  const teachers = getData('teachers', []).filter(t => !t.deleted && t.facultyType === 'supervisor');
  const filtered = teachers.filter(t =>
    // Match on ANY supervised department.
    (!deptFilter || getSupervisedDepts(t).some(a => a.dept === deptFilter)) &&
    (t.name.toLowerCase().includes(search.toLowerCase()) ||
    (t.tid || '').toLowerCase().includes(search.toLowerCase()))
  ).sort(byName);
  const tbody = document.getElementById('supervisorTbody');
  if (!tbody) return;

  if (!filtered.length) {
    tbody.innerHTML = `<tr><td colspan="6" style="text-align:center;padding:32px;color:var(--muted);">No supervisors assigned yet. Go to Teachers and
set Faculty Type to "Supervisor".</td></tr>`;
    return;
  }

  const deptGroups = {};
  filtered.forEach(t => {
    const dept = t.dept || 'Unassigned';
    if (!deptGroups[dept]) deptGroups[dept] = [];
    deptGroups[dept].push(t);
  });

  let html = '';
  Object.entries(deptGroups).forEach(([dept, supervisors]) => {
    html += `<tr style="background:linear-gradient(90deg,var(--primary-light,#eff6ff),transparent);">
      <td colspan="6" style="padding:8px 14px;">
        <div style="display:flex;align-items:center;gap:8px;">
          <svg width="12" height="12" fill="none" stroke="var(--primary)" stroke-width="2" viewBox="0 0 24 24"><rect x="3" y="3" width="18" height="1
8" rx="2"/><path d="M9 9h6M9 13h4"/></svg>
          <span style="font-size:0.78rem;font-weight:700;color:var(--primary);">${escapeHtml(dept)}</span>
          <span style="font-size:0.7rem;color:var(--muted);">${supervisors.length} supervisor${supervisors.length !== 1 ? 's' : ''}</span>
        </div>
      </td>
    </tr>`;

    supervisors.forEach(t => {
      const evals = getData('evaluations', []);
      const lastSef = getSEFForTeacher(t.id).count > 0;

      html += `<tr onclick="showAnnexReports('${t.id}')" title="Click to view Annex C & D" style="cursor:pointer;">
        <td>
          <strong>${escapeHtml(listName(t))}</strong><br>
          <small style="font-family:'JetBrains Mono',monospace;color:var(--muted);">${escapeHtml(t.tid)}</small>
        </td>
        <td>${escapeHtml(t.dept || '—')}</td>
        <td>${supRoleCellHtml(t)}</td>
        <td>
          <span class="badge ${lastSef ? 'badge-success' : 'badge-warning'}">
            ${lastSef ? 'Has Evaluated' : 'No SEF Yet'}
          </span>
        </td>
        <td>
          <div class="td-actions" style="display:flex;gap:6px;flex-wrap:wrap;" onclick="event.stopPropagation()">
            <button class="btn btn-ghost btn-icon btn-sm" title="Edit" onclick="openEditTeacherModal('${t.id}')"><svg width="14" height="14" fill="no
ne" stroke="var(--primary)" stroke-width="2" viewBox="0 0 24 24"><path d="M11 4H4a2 2 0 00-2 2v14a2 2 0 002 2h14a2 2 0 002-2v-7"/><path d="M18.5 2.5a
2.121 2.121 0 013 3L12 15l-4 1 1-4 9.5-9.5z"/></svg></button>
            <button class="btn btn-ghost btn-icon btn-sm" title="Toggle Status" onclick="toggleTeacherStatus('${t.id}')"><svg width="14" height="14"
fill="none" stroke="${t.status === 'active' ? 'var(--muted)' : 'var(--success)'}" stroke-width="2" viewBox="0 0 24 24"><path d="M18.36 6.64a9 9 0 11-
```

```
12.73 0"/><line x1="12" y1="2" x2="12" y2="12"/></svg></button>
            <button class="btn btn-ghost btn-icon btn-sm" title="Delete" onclick="deleteTeacher('${t.id}')"><svg width="14" height="14" fill="none" s
troke="var(--danger)" stroke-width="2" viewBox="0 0 24 24"><polyline points="3 6 5 6 21 6"/><path d="M19 6l-1 14a2 2 0 01-2 2H8a2 2 0 01-2-2L5 6"/></
svg></button>
          </div>
        </td>
      </tr>`;
    });
  });

  tbody.innerHTML = html;
};

// See saveResetPass in admin-students.js.
window.resetSupervisorPassword = async function(supervisorId) {
  const teachers = getData('teachers', []);
  const t = teachers.find(t => t.id === supervisorId);
  if (!t) return;
  const newPass = prompt(`Reset password for ${t.name}.\nLeave blank to reset to their Teacher ID (${t.tid}):`);
  if (newPass === null) return;
  await resetLoginPassword('supervisor', t, newPass);
  renderSupervisorList();
};

window.setSupervisorPageDeptFilter = function(dept) {
  const bar = document.getElementById('supervisorPageDeptFilterBar');
  if (bar) bar.dataset.active = dept;
  renderSupervisorList(document.querySelector('#page-supervisor input[type=text]')?.value || '');
};
```

admin/js/reports/admin-reports.js

926 lines

```
// admin-reports.js - evaluation control window and the Reports & Analytics page.
// Load order matters (see dashboard.html): the last definition wins.

// ===== EVAL CONTROL =====
// Submission Tracking filters; declared outside renderEvalControl so they persist.
window._trackQuery = '';
window._trackDept  = '';
window._trackOpen    = new Set();   // faculty whose subject list is expanded
window._trackShowAll = false;       // "View all" pressed (more than 10 faculty)
const TRACK_PREVIEW_ROWS = 10;

// Today as YYYY-MM-DD in local time (toISOString would give the UTC date).
function _localToday() {
  return new Date().toLocaleDateString('en-CA');
}

// Open, but the start date hasn't arrived yet.
function evalPeriodNotStarted(period) {
  return !!(period && period.open && period.startDate && _localToday() < period.startDate);
}

function renderEvalControl() {
  const period = getData('evalPeriod', { open: false, deadline: '' });
  const notStarted = evalPeriodNotStarted(period);
  document.getElementById('evalControlContent').innerHTML = `
    <div class="card" style="margin-bottom:20px;">
      <div class="card-header-bar"><h3>Evaluation Period Control</h3></div>
      <div class="card-body">
        <div style="display:flex;align-items:center;justify-content:space-between;margin-bottom:20px;flex-wrap:wrap;gap:12px;">
          <div><div style="font-size:0.75rem;color:var(--muted);font-weight:600;text-transform:uppercase;letter-spacing:0.06em;margin-bottom:4px;">Cu
rrent Semester</div>
          <div style="font-weight:700;font-size:1rem;">${getActiveSY()?getActiveSY().year+' - '+getActiveSY().activeSem:'Not set'}</div></div>
          <button class="btn ${period.open?'btn-danger':'btn-success'}" onclick="toggleEvalPeriod()">${period.open?'🔒 Close Evaluation':'🔓 Open Ev
aluation'}</button>
        </div>
        <div class="form-row">
          <div class="form-group"><label class="form-label">Evaluation Start Date</label><input class="form-control" type="date" id="startDateInput"
value="${period.startDate || ''}" max="${period.deadline || ''}" onchange="updatePeriodSettings()"/></div>
          <div class="form-group"><label class="form-label">Evaluation Deadline</label><input class="form-control" type="date" id="deadlineInput" val
ue="${period.deadline || ''}" min="${period.startDate || ''}" onchange="updatePeriodSettings()"/></div>
        </div>
        <div id="periodScheduleNote" style="font-size:0.78rem;color:var(--muted);">${_periodScheduleNote(period, notStarted)}</div>

      </div>
    </div>
    <div id="trackDeptSummary"></div>
    <div class="card">
      <div class="card-header-bar" style="display:flex;align-items:center;justify-content:space-between;gap:12px;flex-wrap:wrap;">
        <h3>Submission Tracking</h3>
        <input class="form-control" id="trackSearch" placeholder="Search faculty, ID, or subject..."
               value="${escapeHtml(window._trackQuery)}" oninput="filterTracking(this.value)"
               style="max-width:280px;margin:0;"/>
      </div>
      <div id="trackDeptBar" style="display:flex;gap:6px;flex-wrap:wrap;padding:12px 16px 0;"></div>
      <div class="table-wrap"><table>
        <thead><tr><th>Faculty</th><th>Subjects</th><th>Ratings Expected</th><th>Submitted</th><th>Pending</th><th>Progress</th></tr></thead>
        <tbody id="trackTbody"></tbody>
      </table></div>
      <div style="padding:10px 16px 14px;font-size:0.74rem;color:var(--muted);">
        Click a faculty member to see the progress of each of their subjects. Click a subject to see who has submitted and who has not.
      </div>
    </div>
  `;
  renderTrackDeptSummary();
  renderTrackDeptBar();
  renderTrackingRows();
}

// DEPARTMENT PROGRESS: one card per department (submitted vs expected);
// click to filter the table. Hidden with only one department.
function _trackDaysLeft(deadline) {
  if (!deadline) return null;
  const end = new Date(deadline + 'T23:59:59');
  if (isNaN(end)) return null;
  return Math.ceil((end - new Date()) / 86400000);
}

// 'danger' / 'warning' when a department is behind with the deadline close.
function _trackRisk(pct, daysLeft) {
  if (daysLeft === null || pct >= 100) return '';
  if (daysLeft <= 7 && pct < 50) return 'danger';
  if (daysLeft <= 14 && pct < 75) return 'warning';
  return '';
}

function renderTrackDeptSummary() {
  const box = document.getElementById('trackDeptSummary');
  if (!box) return;
  const DEPT_CONFIG = getDepartments();
  const period = getData('evalPeriod', { open: false, deadline: '' });
  const daysLeft = _trackDaysLeft(period.deadline);

  const byDept = {};
  let expected = 0, submitted = 0;
  _trackFacultyStats().forEach(s => {
    const d = deptFilterKey(s.teacher.dept);
    const g = byDept[d] || (byDept[d] = { expected: 0, submitted: 0, faculty: 0 });
    g.expected += s.expected; g.submitted += s.submitted; g.faculty++;
    expected += s.expected; submitted += s.submitted;
  });
```

```
  const codes = Object.keys(DEPT_CONFIG).filter(c => byDept[c]);
  if (byDept['UNASSIGNED']) codes.push('UNASSIGNED');
  if (codes.length < 2) { box.innerHTML = ''; return; }

  const colors = { danger: 'var(--danger,#dc2626)', warning: 'var(--warning,#d97706)' };
  const overallPct = _trackPct(submitted, expected);
  const deadlineText = daysLeft === null ? 'No deadline set'
    : daysLeft < 0 ? 'Deadline passed'
    : daysLeft === 0 ? 'Due today'
    : daysLeft + ' day' + (daysLeft !== 1 ? 's' : '') + ' to deadline';

  const cards = codes.map(code => {
    const g = byDept[code];
    const cfg = DEPT_CONFIG[code];
    const pct = _trackPct(g.submitted, g.expected);
    const risk = _trackRisk(pct, daysLeft);
    const fill = risk ? `background:${colors[risk]};` : '';
    const active = window._trackDept === code;
    const name = cfg ? (cfg.short || code) : 'No Dept';
    return `<button type="button" onclick="filterTrackingByDept('${active ? '' : code}')"
        title="${escapeHtml(cfg ? cfg.name : 'No Department')} - click to ${active ? 'show all' : 'filter the table'}"
        style="text-align:left;cursor:pointer;background:var(--white,#fff);border:1.5px solid ${active ? 'var(--primary,#064e3b)' : 'var(--border,#e2
e8f0)'};
               border-radius:var(--radius,10px);padding:12px 14px;font:inherit;color:inherit;">
        <div style="display:flex;justify-content:space-between;align-items:baseline;gap:8px;margin-bottom:6px;">
          <strong style="font-size:0.9rem;">${escapeHtml(name)}</strong>
          <span style="font-weight:700;font-size:0.95rem;${risk ? 'color:' + colors[risk] + ';' : ''}">${pct}%</span>
        </div>
        <div class="progress-bar"><div class="progress-fill" style="width:${pct}%;${fill}"></div></div>
        <div style="font-size:0.72rem;color:var(--muted);margin-top:6px;">
          ${g.submitted.toLocaleString()} / ${g.expected.toLocaleString()} ratings &middot; ${g.faculty} faculty</div>
      </button>`;
  }).join('');

  box.innerHTML = `
    <div class="card" style="margin-bottom:20px;">
      <div class="card-header-bar" style="display:flex;align-items:center;justify-content:space-between;gap:12px;flex-wrap:wrap;">
        <h3>Department Progress</h3>
        <span style="font-size:0.8rem;color:var(--muted);">Overall <strong>${overallPct}%</strong>
          (${submitted.toLocaleString()} / ${expected.toLocaleString()}) &middot; ${deadlineText}</span>
      </div>
      <div class="card-body" style="display:grid;grid-template-columns:repeat(auto-fill,minmax(200px,1fr));gap:12px;">
        ${cards}
      </div>
    </div>`;
}

// SUBMISSION TRACKING, BY FACULTY: one row per faculty, expanding to
// subjects, then to the submission roster. First 10 shown, "View all" for more.

// Progress for every faculty member who teaches at least one subject.
function _trackFacultyStats() {
  const students = getData('students', []).filter(s => !s.deleted);
  const byId = {}; students.forEach(s => { byId[s.id] = s; });
  const teachers = getData('teachers', []).filter(t => !t.deleted);
  const evals    = getData('evaluations', []).filter(e => e.evaluatorType !== 'supervisor');
  const subjects = getData('subjects', []);

  const stats = {};
  subjects.forEach(sub => {
    const enrolledSt = (sub.enrolledIds || []).map(id => byId[id]).filter(Boolean);
    subjectTeacherIds(sub).forEach(tid => {
      const t = teachers.find(x => x.id === tid);
      if (!t) return;
      const mine = enrolledSt.filter(st => teacherTeachesStudent(sub, tid, st));
      const mineIds = new Set(mine.map(st => st.id));
      const submitted = evals.filter(e => e.subjectId === sub.id && mineIds.has(e.studentId)
                                     && evalBelongsTo(e, sub, tid)).length;
      const st = stats[tid] || (stats[tid] = { teacher: t, expected: 0, submitted: 0, subjects: [] });
      st.expected  += mine.length;
      st.submitted += Math.min(submitted, mine.length);
      const secs = (subjectTeachers(sub).find(x => x.id === tid) || {}).sections || [];
      st.subjects.push({ sub, secs, expected: mine.length, submitted: Math.min(submitted, mine.length) });
    });
  });
  return Object.values(stats);
}

function _trackPct(submitted, expected) {
  return expected ? Math.min(100, Math.round((submitted / expected) * 100)) : 0;
}

function _trackBar(pct) {
  return `<div style="display:flex;align-items:center;gap:8px;"><div class="progress-bar" style="flex:1;">
            <div class="progress-fill" style="width:${pct}%"></div></div>
            <span style="font-size:0.72rem;font-weight:700;">${pct}%</span></div>`;
}

// Department pills: faculty counted by their home department.
function renderTrackDeptBar() {
  const bar = document.getElementById('trackDeptBar');
  if (!bar) return;
  const DEPT_CONFIG = getDepartments();
  const list = _trackFacultyStats();

  const counts = {};
  list.forEach(s => { const d = deptFilterKey(s.teacher.dept); counts[d] = (counts[d] || 0) + 1; });

  let html = `<button class="student-dept-filter-btn${window._trackDept === '' ? ' active' : ''}" data-dept="" onclick="filterTrackingByDept('')">
      All <span class="dept-filter-count">${list.length}</span></button>`;
  Object.entries(DEPT_CONFIG).forEach(([code, cfg]) => {
    if (counts[code]) {
      html += `<button class="student-dept-filter-btn${window._trackDept === code ? ' active' : ''}" data-dept="${code}" onclick="filterTrackingByDep
t('${code}')">
```

```
        ${escapeHtml(cfg.short || code)} <span class="dept-filter-count">${counts[code]}</span></button>`;
    }
  });
  if (counts['UNASSIGNED']) {
    html += `<button class="student-dept-filter-btn${window._trackDept === 'UNASSIGNED' ? ' active' : ''}" data-dept="UNASSIGNED" onclick="filterTrac
kingByDept('UNASSIGNED')">
        No Dept <span class="dept-filter-count">${counts['UNASSIGNED']}</span></button>`;
  }
  bar.innerHTML = html;
}

window.filterTracking = function (q) {
  window._trackQuery = q || '';
  renderTrackingRows();
};

window.filterTrackingByDept = function (dept) {
  window._trackDept = dept;
  window._trackShowAll = false;
  document.querySelectorAll('#trackDeptBar .student-dept-filter-btn').forEach(p => {
    p.classList.toggle('active', p.dataset.dept === dept);
  });
  renderTrackDeptSummary();
  renderTrackingRows();
};

// Open or close a faculty member's subject list.
window.toggleTrackFaculty = function (tid) {
  if (window._trackOpen.has(tid)) window._trackOpen.delete(tid);
  else window._trackOpen.add(tid);
  renderTrackingRows();
};

window.toggleTrackShowAll = function () {
  window._trackShowAll = !window._trackShowAll;
  renderTrackingRows();
};

// Rows only - the search box is never re-rendered, so typing does not lose focus.
function renderTrackingRows() {
  const tbody = document.getElementById('trackTbody');
  if (!tbody) return;

  const DEPT_CONFIG = getDepartments();
  const q = window._trackQuery.trim().toLowerCase();

  const rows = _trackFacultyStats()
    .filter(s => {
      if (window._trackDept && deptFilterKey(s.teacher.dept) !== window._trackDept) return false;
      if (!q) return true;
      const t = s.teacher;
      const subjText = s.subjects.map(x => (x.sub.code || '') + ' ' + (x.sub.name || '')).join(' ');
      return ((t.name || '') + ' ' + (t.tid || '') + ' ' + subjText).toLowerCase().includes(q);
    })
    .sort((a, b) => {
      const da = deptFilterKey(a.teacher.dept), db = deptFilterKey(b.teacher.dept);
      if (da !== db) return da.localeCompare(db);
      return byName(a.teacher, b.teacher);           // last name A to Z
    });

  if (!rows.length) {
    tbody.innerHTML = `<tr><td colspan="6" style="text-align:center;padding:24px;color:var(--muted);">No faculty match this filter.</td></tr>`;
    return;
  }

  const shown = window._trackShowAll ? rows : rows.slice(0, TRACK_PREVIEW_ROWS);
  let html = '', lastDept = null;
  shown.forEach(s => {
    const t = s.teacher;
    const dept = deptFilterKey(t.dept);
    if (dept !== lastDept) {
      const cfg = DEPT_CONFIG[dept];
      const label = cfg ? ((cfg.short || dept) + ' — ' + cfg.name) : 'No Department';
      html += `<tr><td colspan="6" style="background:var(--bg,#f2f7f4);font-weight:700;font-size:0.78rem;
                text-transform:uppercase;letter-spacing:0.05em;color:var(--primary,#3f6f5b);padding:8px 12px;">
                ${escapeHtml(label)}</td></tr>`;
      lastDept = dept;
    }

    const open    = window._trackOpen.has(t.id);
    const pending = Math.max(0, s.expected - s.submitted);
    const pct     = _trackPct(s.submitted, s.expected);
    html += `<tr style="cursor:pointer;" title="${open ? 'Hide' : 'Show'} this faculty member's subjects"
                onclick="toggleTrackFaculty('${t.id}')">
      <td><span style="display:inline-block;width:14px;font-size:0.7rem;color:var(--muted);transition:transform .15s;${open ? 'transform:rotate(90de
g);' : ''}">&#9654;</span>
          <strong>${escapeHtml(listName(t))}</strong>
          <span style="font-family:monospace;font-size:0.72rem;color:var(--muted);margin-left:4px;">${escapeHtml(t.tid || '')}</span></td>
      <td>${s.subjects.length}</td>
      <td>${s.expected}</td>
      <td><span class="badge badge-success">${s.submitted}</span></td>
      <td><span class="badge badge-warning">${pending}</span></td>
      <td style="min-width:120px;">${_trackBar(pct)}</td>
    </tr>`;

    if (open) {
      s.subjects
        .slice()
        .sort((a, b) => String(a.sub.code || '').localeCompare(String(b.sub.code || ''), undefined, { numeric: true }))
        .forEach(x => {
          const p = _trackPct(x.submitted, x.expected);
          html += `<tr style="cursor:pointer;background:#fbfdfc;" title="Click to see who has submitted"
                      onclick="annexSectionRoster('${x.sub.id}', &quot;&quot;)">
            <td style="padding-left:40px;font-size:0.82rem;">
                <strong>${escapeHtml(x.sub.code || '')}</strong> - ${escapeHtml(x.sub.name || '')}</td>
```

```
            <td style="font-size:0.78rem;color:var(--muted);">${x.secs.length ? escapeHtml(x.secs.join(', ')) : 'All students'}</td>
            <td style="font-size:0.82rem;">${x.expected}</td>
            <td><span class="badge badge-success">${x.submitted}</span></td>
            <td><span class="badge badge-warning">${Math.max(0, x.expected - x.submitted)}</span></td>
            <td style="min-width:120px;">${_trackBar(p)}</td>
          </tr>`;
        });
    }
  });

  if (rows.length > TRACK_PREVIEW_ROWS) {
    html += `<tr><td colspan="6" style="text-align:center;padding:10px;">
        <button type="button" class="btn btn-ghost btn-sm" onclick="toggleTrackShowAll()">
          ${window._trackShowAll ? 'Show less' : `View all ${rows.length} faculty`}</button></td></tr>`;
  }
  tbody.innerHTML = html;
}

function toggleEvalPeriod() {
  const period = getData('evalPeriod', {});
  period.open = !period.open;
  setData('evalPeriod', period);
  addAudit(period.open?'Open Evaluation':'Close Evaluation', `Evaluation period ${period.open?'opened':'closed'}`);
  renderEvalControl();
  showToast(`Evaluation period ${period.open?'opened':'closed'}!`, period.open?'success':'info');
}

// One line under the dates saying when students can actually submit.
function _periodScheduleNote(period, notStarted) {
  if (!period.open) return period.startDate
    ? 'Evaluation is closed. Open it to let students submit from ' + period.startDate + '.'
    : 'Evaluation is closed.';
  if (notStarted) return '⏳ Scheduled: students can submit from ' + period.startDate
    + (period.deadline ? ' to ' + period.deadline : '') + '.';
  return '✅ Students can submit now' + (period.deadline ? ', until ' + period.deadline : '') + '.';
}

function updatePeriodSettings() {
  const period = getData('evalPeriod', {});
  const startDate = document.getElementById('startDateInput').value;
  const deadline  = document.getElementById('deadlineInput').value;
  if (startDate && deadline && startDate > deadline) {
    showToast('The start date must be on or before the deadline.', 'error');
    renderEvalControl();   // put the inputs back to the saved dates
    return;
  }
  period.startDate = startDate;
  period.deadline  = deadline;
  setData('evalPeriod', period);
  addAudit('Update Eval Settings', `Start: ${period.startDate || 'none'}, Deadline: ${period.deadline || 'none'}`);
  renderEvalControl();   // refreshes the date limits, note and days-to-deadline
}

// ===== REPORTS - CMO 19 COMPLIANT =====
// ===== REPORTS: PILL FILTER + SINGLE TABLE =====

// Active dept filter for reports page ('' = All)
window._reportsDeptFilter = '';

// Build teacher evaluation data — called once, results cached per render
function _buildTeacherEvalData() {
    const teachers = getData('teachers', []).filter(t => !t.deleted);
    const subjects = getData('subjects', []);
    const evals    = getData('evaluations', []);
    const students = getData('students', []).filter(s => !s.deleted);

    return teachers.map(teacher => {
        const teacherSubjects = subjects.filter(s =>
            subjectHasTeacher(s, teacher.id) && s.loadType !== 'Overload' && !s.isLabSchool
        );
        // §8.3 maths from computeWeightedSET (admin-scoring.js).
        const agg = computeWeightedSET(teacher.id, evalInActiveTerm);
        const classRatings = agg.classes.map(c => ({
            subjectCode: c.subjectCode, subjectName: c.subjectName,
            enrolledCount: c.enrolledCount, evalCount: c.evalCount,
            avgScore: c.avgScore,
            avgPercentage: c.percentage
        }));
        const overallSET = agg.totalStudents > 0 ? agg.overallSET : '—';

        const supervisorEvals = evals.filter(e =>
            e.teacherId === teacher.id && e.evaluatorType === 'supervisor'
            && (typeof evalInActiveTerm !== 'function' || evalInActiveTerm(e))
        );
        const sefScore = supervisorEvals.length > 0
            ? (supervisorEvals.reduce((a, b) => a + b.totalScore, 0) / supervisorEvals.length).toFixed(2) : '—';
        // Number of supervisors averaged.
        const sefRaters = supervisorEvals.length;

        return {
            ...teacher,
            sefRaters,
            classRatings,
            overallSET,
            sefScore,
            totalClasses: teacherSubjects.length,
            totalEvaluations: classRatings.reduce((sum, cr) => sum + cr.evalCount, 0)
        };
    }).sort(byName);
}

// Set the active dept pill and refresh both tables
window.setReportsDeptFilter = function(dept) {
    window._reportsDeptFilter = dept;

```

```
    // Update pill active states (now uses student-dept-filter-btn)
    document.querySelectorAll('#rpt-pill-bar .student-dept-filter-btn').forEach(p => {
        p.classList.toggle('active', p.dataset.dept === dept);
    });

    // Re-render both table bodies only (no full page rebuild)
    _renderFacultyTable();
    _renderSupervisorTable();
};

// Build one faculty table row (shared by main table and View All modal)
function _buildFacultyRow(t) {
    return `
        <tr class="report-teacher-row" onclick="showAnnexReports('${t.id}', true)" title="Click to view Annex C &amp;amp; D">
            <td>
                <strong>${escapeHtml(listName(t))}</strong><br>
                <small style="color:var(--muted);">${escapeHtml(t.tid)}</small>
            </td>
            <td><span class="dept-tag-inline">${escapeHtml(t.dept || '—')}</span></td>
            <td style="text-align:center;"><span class="badge badge-info">${t.totalClasses}</span></td>
            <td style="text-align:center;"><span class="badge badge-secondary">${t.totalEvaluations}</span></td>
            <td style="text-align:center;">
                <strong style="color:#16a34a;font-size:1.05rem;">
                    ${t.overallSET}${t.overallSET !== '—' ? '%' : ''}
                </strong>
            </td>
            <td style="text-align:center;">
                <strong style="color:#d97706;font-size:1.05rem;">
                    ${t.sefScore}${t.sefScore !== '—' ? '%' : ''}
                    ${t.sefRaters > 1 ? `<div style="font-size:0.62rem;color:var(--muted);font-weight:600;">avg of ${t.sefRaters} supervisors</div>`
: ''}
                </strong>
            </td>
        </tr>`;
}

// Shared empty-state row for the Faculty and Supervisors tables.
function _rptEmptyRow(colspan, message, hint) {
    return `<tr><td colspan="${colspan}" style="text-align:center;padding:32px;color:var(--muted);">`
        + `<div style="font-size:0.9rem;">${message}</div>`
        + (hint ? `<div style="font-size:0.78rem;margin-top:6px;opacity:0.85;">${hint}</div>` : '')
        + `</td></tr>`;
}

// Rows shown in the Reports summary tables before "View Full List" takes over.
const RPT_PREVIEW_ROWS = 5;

// Footer row noting the table is truncated.
function _rptMoreRow(total, shown, colspan) {
    if (total <= shown) return '';
    return `
        <tr>
            <td colspan="${colspan}" style="text-align:center;padding:11px;background:var(--surface2,#f8fafc);">
                <span style="font-size:0.76rem;color:var(--muted);">
                    Showing ${shown} of ${total}
                </span>
            </td>
        </tr>`;
}

function _renderFacultyTable() {
    const tbody   = document.getElementById('rpt-faculty-tbody');
    const countEl = document.getElementById('rpt-faculty-count');
    if (!tbody) return;

    const dept    = window._reportsDeptFilter;
    const allData = window._allTeacherEvalData || [];
    const list    = allData
        .filter(t => (t.facultyType || 'regular') !== 'supervisor')
        .filter(t => !dept || (t.dept || 'UNASSIGNED') === dept);

    if (countEl) countEl.textContent = list.length;

    if (!list.length) {
        tbody.innerHTML = _rptEmptyRow(6,
            `No faculty found${dept ? ' for this department' : ''}.`,
            dept ? 'Assign faculty to this department to see their SET ratings here.' : '');
        return;
    }

    // Cap the summary table; "View Full List" has the rest.
    const shown = list.slice(0, RPT_PREVIEW_ROWS);
    tbody.innerHTML = shown.map(t => _buildFacultyRow(t)).join('')
        + _rptMoreRow(list.length, shown.length, 6);
}

// Build one supervisor table row (shared by main table and View All modal)
function _buildSupervisorRow(t) {
    return `
        <tr class="report-teacher-row" onclick="showAnnexReports('${t.id}', true)" title="Click to view Annex C &amp;amp; D">
            <td>
                <strong>${escapeHtml(listName(t))}</strong><br>
                <small style="color:var(--muted);">${escapeHtml(t.tid)}</small>
            </td>
            <td><span class="dept-tag-inline">${escapeHtml(t.dept || '—')}</span></td>
            <td style="text-align:center;">
                <strong style="color:#16a34a;">
                    ${t.overallSET}${t.overallSET !== '—' ? '%' : ''}
                </strong>
            </td>
            <td style="text-align:center;">
                <strong style="color:#d97706;">
                    ${t.sefScore}${t.sefScore !== '—' ? '%' : ''}
                    ${t.sefRaters > 1 ? `<div style="font-size:0.62rem;color:var(--muted);font-weight:600;">avg of ${t.sefRaters} supervisors</div>`
: ''}
```

```
                </strong>
            </td>
            <td style="text-align:center;">
                <span class="badge ${t.status === 'active' ? 'badge-success' : 'badge-danger'}">${t.status}</span>
            </td>
        </tr>`;
}

// Render supervisor tbody rows based on current filter
function _renderSupervisorTable() {
    const tbody   = document.getElementById('rpt-supervisor-tbody');
    const countEl = document.getElementById('rpt-supervisor-count');
    const card    = document.getElementById('rpt-supervisor-card');
    if (!tbody) return;

    const dept    = window._reportsDeptFilter;
    const allData = window._allTeacherEvalData || [];
    const list    = allData
        .filter(t => t.facultyType === 'supervisor')
        .filter(t => !dept || (t.dept || 'UNASSIGNED') === dept);

    // Show the card if supervisors match OR a department filter is active.
    if (card) card.style.display = (list.length > 0 || dept) ? '' : 'none';
    if (countEl) countEl.textContent = list.length;

    if (!list.length) {
        tbody.innerHTML = _rptEmptyRow(5,
            `No supervisors found${dept ? ' for this department' : ''}.`,
            dept ? 'Assign a supervisor to this department to see SEF ratings here.' : '');
        return;
    }

    const shownSup = list.slice(0, RPT_PREVIEW_ROWS);
    tbody.innerHTML = shownSup.map(t => _buildSupervisorRow(t)).join('')
        + _rptMoreRow(list.length, shownSup.length, 5);

}

// Build dept pill bar HTML — reuses student-dept-filter-btn for visual consistency
function _buildReportPills(teacherData) {
    const DEPT_CONFIG = (typeof getDepartments === 'function') ? getDepartments() : {};
    const DEPT_ORDER  = ['COED','CCJS','CCIS','CON','CEA','COM','CAT','GS'];

    // Collect depts that actually have teachers
    const deptCounts = {};
    teacherData.forEach(t => {
        const d = t.dept || 'UNASSIGNED';
        deptCounts[d] = (deptCounts[d] || 0) + 1;
    });

    const activeDepts = [
        ...DEPT_ORDER.filter(d => deptCounts[d]),
        ...Object.keys(deptCounts).filter(d => !DEPT_ORDER.includes(d))
    ];

    const current = window._reportsDeptFilter || '';
    let html = `<button class="student-dept-filter-btn ${current === '' ? 'active' : ''}" data-dept="" onclick="setReportsDeptFilter('')">
        All
    </button>`;

    activeDepts.forEach(d => {
        const cfg   = DEPT_CONFIG[d];
        const label = cfg ? escapeHtml(cfg.short || d) : escapeHtml(d);
        html += `<button class="student-dept-filter-btn ${current === d ? 'active' : ''}" data-dept="${d}" onclick="setReportsDeptFilter('${d}')">
            ${label}
        </button>`;
    });

    return html;
}

window.renderReports = function() {
    // Reset filter on full re-render
    window._reportsDeptFilter = '';

    // Populate/refresh the term (history) selector.
    if (typeof _refreshReportTermLabel === 'function') _refreshReportTermLabel();

    // Build & cache all teacher data
    const allData = _buildTeacherEvalData();
    window._allTeacherEvalData = allData;

    const regularFaculty    = allData.filter(t => (t.facultyType || 'regular') !== 'supervisor');
    const supervisorFaculty = allData.filter(t => t.facultyType === 'supervisor');

    document.getElementById('reportsContent').innerHTML = `
        <!-- DEPT PILL FILTER -->
        <div class="rpt-pill-bar" id="rpt-pill-bar">
            ${_buildReportPills(allData)}
        </div>

        <!-- FACULTY PERFORMANCE TABLE -->
        <div class="card" style="margin-bottom:20px;">
            <div class="card-header-bar">
                <div style="display:flex;align-items:center;gap:10px;">
                    <h3>Faculty Performance</h3>
                    <span class="count-badge" id="rpt-faculty-count">${regularFaculty.length}</span>
                </div>
                <div style="display:flex;align-items:center;gap:8px;">
                    <button id="rpt-faculty-viewall-btn" class="btn btn-ghost btn-sm rpt-viewall-btn" onclick="_openRptViewAllPage('faculty')">
                        <svg width="13" height="13" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24" style="margin-right:3px;">
<path d="M18 13v6a2 2 0 01-2 2H5a2 2 0 01-2-2V8a2 2 0 012-2h6"/><polyline points="15 3 21 3 21 9"/><line x1="10" y1="14" x2="21" y2="3"/></svg>View F
ull List
                    </button>
                    <span class="badge badge-primary" style="font-size:0.68rem;">CMO 19 — SET &amp;amp; SEF Displayed Separately</span>
```

```
                </div>
            </div>
            <div class="table-wrap">
                <table class="data-table">
                    <thead>
                        <tr>
                            <th>Faculty Name</th>
                            <th>Department</th>
                            <th style="text-align:center;">Classes</th>
                            <th style="text-align:center;">Evals</th>
                            <th style="color:#16a34a;text-align:center;">SET Rating</th>
                            <th style="color:#d97706;text-align:center;">SEF Rating</th>
                        </tr>
                    </thead>
                    <tbody id="rpt-faculty-tbody"></tbody>
                </table>
            </div>
        </div>

        <!-- SUPERVISORS TABLE -->
        <div class="card" id="rpt-supervisor-card" style="margin-bottom:20px;${supervisorFaculty.length === 0 ? 'display:none;' : ''}">
            <div class="card-header-bar">
                <div style="display:flex;align-items:center;gap:10px;">
                    <h3>Supervisors</h3>
                    <span class="count-badge" id="rpt-supervisor-count">${supervisorFaculty.length}</span>
                </div>
                <button id="rpt-supervisor-viewall-btn" class="btn btn-ghost btn-sm rpt-viewall-btn" onclick="_openRptViewAllPage('supervisors')">
                    <svg width="13" height="13" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24" style="margin-right:3px;"><pat
h d="M18 13v6a2 2 0 01-2 2H5a2 2 0 01-2-2V8a2 2 0 012-2h6"/><polyline points="15 3 21 3 21 9"/><line x1="10" y1="14" x2="21" y2="3"/></svg>View Full
List
                </button>
            </div>
            <div class="table-wrap">
                <table class="data-table">
                    <thead>
                        <tr>
                            <th>Faculty Name</th>
                            <th>Department</th>
                            <th style="color:#16a34a;text-align:center;">SET Rating</th>
                            <th style="color:#d97706;text-align:center;">SEF Rating</th>
                            <th style="text-align:center;">Status</th>
                        </tr>
                    </thead>
                    <tbody id="rpt-supervisor-tbody"></tbody>
                </table>
            </div>
        </div>

        <div id="institutionalFERContainer"></div>
    `;

    // Populate table bodies
    _renderFacultyTable();
    _renderSupervisorTable();
    renderInstitutionalFER();
};

window.toggleClassDetails = function(teacherId) {
    const row = document.getElementById(`class-details-${teacherId}`);
    if (row) row.style.display = row.style.display === 'none' ? 'table-row' : 'none';
};

// ===== REPORTS "VIEW ALL" FULL PAGE =====

// Navigates to the full-page view for either 'faculty' or 'supervisors'
window._openRptViewAllPage = function(tableType) {
    // Stash what we need so renderRptViewAll can read it
    window._rptViewAllType = tableType;
    window._rptViewAllDept = window._reportsDeptFilter || '';
    window._rptViewAllSearch = '';

    // Use showPage — registers the page as active, runs its renderer
    showPage('rptViewAll');
};

// Renderer called by showPage
function renderRptViewAll() {
    const tableType   = window._rptViewAllType || 'faculty';
    const dept        = window._rptViewAllDept || '';
    const allData     = window._allTeacherEvalData || [];
    const DEPT_CONFIG = (typeof getDepartments === 'function') ? getDepartments() : {};

    const isFaculty = tableType === 'faculty';
    const list = allData
        .filter(t => isFaculty
            ? (t.facultyType || 'regular') !== 'supervisor'
            : t.facultyType === 'supervisor')
        .filter(t => !dept || (t.dept || 'UNASSIGNED') === dept);

    const deptLabel = dept
        ? (DEPT_CONFIG[dept] ? escapeHtml(DEPT_CONFIG[dept].short || dept) : escapeHtml(dept))
        : 'All Departments';

    const title = (isFaculty ? 'Faculty Performance' : 'Supervisors') + ' — ' + deptLabel;

    const thead = isFaculty
        ? `<thead>
                <tr>
                    <th>Faculty Name</th>
                    <th>Department</th>
                    <th style="text-align:center;">Classes</th>
                    <th style="text-align:center;">Evals</th>
                    <th style="color:#16a34a;text-align:center;">SET Rating</th>
                    <th style="color:#d97706;text-align:center;">SEF Rating</th>
                </tr>
```

```
            </thead>`
        : `<thead>
                <tr>
                    <th>Faculty Name</th>
                    <th>Department</th>
                    <th style="color:#16a34a;text-align:center;">SET Rating</th>
                    <th style="color:#d97706;text-align:center;">SEF Rating</th>
                    <th style="text-align:center;">Status</th>
                </tr>
            </thead>`;

    document.getElementById('rptViewAllContent').innerHTML = `
        <div class="page-header" style="display:flex;align-items:flex-start;justify-content:space-between;">
            <div class="page-title">
                <h1>${title}</h1>
                <p id="rpt-viewall-subtitle"></p>
            </div>
            <div style="display:flex;gap:8px;align-items:center;">
                <button id="rpt-viewall-export-btn" class="btn btn-ghost btn-sm" onclick="_exportRptViewAllPDF('${tableType}', window._rptViewAllLis
t, '${deptLabel}', '${escapeHtml(dept || '')}')">
                    <svg width="13" height="13" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24" style="margin-right:4px;"><pat
h d="M21 15v4a2 2 0 01-2 2H5a2 2 0 01-2-2v-4"/><polyline points="7 10 12 15 17 10"/><line x1="12" y1="15" x2="12" y2="3"/></svg>Export PDF
                </button>
                <button class="btn btn-ghost btn-sm" onclick="showPage('reports')">
                    <svg width="13" height="13" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24" style="margin-right:4px;"><pol
yline points="15 18 9 12 15 6"/></svg>Back to Reports
                </button>
            </div>
        </div>
        <div class="search-box" style="margin-bottom:12px;">
            <svg class="search-icon" width="14" height="14" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24"><circle cx="11" cy
="11" r="8"/><line x1="21" y1="21" x2="16.65" y2="16.65"/></svg>
            <input type="text" id="rptViewAllSearchInput" placeholder="Search by name or ID..." value="${escapeHtml(window._rptViewAllSearch || '')}"
oninput="_rptViewAllSetSearch(this.value)"/>
        </div>
        <div class="card">
            <div class="table-wrap">
                <table class="data-table">${thead}<tbody id="rpt-viewall-tbody"></tbody></table>
            </div>
        </div>
    `;

    // Full (unsearched) list; the search box narrows it in _rptViewAllRenderRows.
    window._rptViewAllBase = { list, isFaculty, dept, deptLabel };
    _rptViewAllRenderRows();
}

window._rptViewAllSetSearch = function(q) {
    window._rptViewAllSearch = q || '';
    _rptViewAllRenderRows();
};

// Redraws only the rows and subtitle, so the search box keeps focus while typing.
function _rptViewAllRenderRows() {
    const base = window._rptViewAllBase;
    const tbodyEl = document.getElementById('rpt-viewall-tbody');
    if (!base || !tbodyEl) return;
    const { list, isFaculty, dept, deptLabel } = base;

    const q = (window._rptViewAllSearch || '').trim().toLowerCase();
    const shown = q
        ? list.filter(t => (t.name || '').toLowerCase().includes(q) || String(t.tid || '').toLowerCase().includes(q))
        : list;

    const noun = isFaculty ? 'faculty member' : 'supervisor';
    const plural = n => n + ' ' + noun + (n !== 1 ? 's' : '');
    document.getElementById('rpt-viewall-subtitle').textContent =
        q ? `${shown.length} of ${plural(list.length)}` : plural(list.length);

    const cols = isFaculty ? 6 : 5;
    let rows;
    if (shown.length) {
        rows = shown.map(t => isFaculty ? _buildFacultyRow(t) : _buildSupervisorRow(t)).join('');
    } else if (q) {
        rows = _rptEmptyRow(cols, `No ${isFaculty ? 'faculty' : 'supervisors'} match "${escapeHtml(window._rptViewAllSearch.trim())}".`,
            'Try a different name or ID.');
    } else if (isFaculty) {
        rows = _rptEmptyRow(cols, `No faculty yet${dept ? ' in ' + deptLabel : ''}.`,
            'Assign faculty to a department and give them a teaching load to see them here.');
    } else {
        rows = _rptEmptyRow(cols, `No supervisors yet${dept ? ' in ' + deptLabel : ''}.`,
            'Designate a faculty member as a program chair to see them here.');
    }
    tbodyEl.innerHTML = rows;

    const exportBtn = document.getElementById('rpt-viewall-export-btn');
    if (exportBtn) {
        exportBtn.disabled = !shown.length;
        exportBtn.style.opacity = shown.length ? '' : '.5';
        exportBtn.style.cursor = shown.length ? '' : 'not-allowed';
    }

    // Export PDF prints what is on screen, search applied.
    window._rptViewAllList = shown;
}

// "Prepared by" / "Approved by" from Reports > Actions > Signatories.
function _rptSignatureBlock() {
    const sig = (typeof getSignatories === 'function') ? getSignatories() : {};
    const one = (label, name, role) =>
        '<div><div class="lbl">' + label + '</div>'
      + '<div class="name">' + (escapeHtml(name || '') || ' ') + '</div>'
      + '<div class="role">' + (escapeHtml(role || '') || ' ') + '</div></div>';
    return '<div class="sig">'
      + one('Prepared by:', sig.preparedName, sig.preparedRole)
```

```
      + one('Approved by:', sig.approvedName, sig.approvedRole)
      + '</div>';
}

// PDF export via the print dialog.
// Full department name for printed titles.
window._rptDeptFullName = function(code) {
    if (!code) return '';
    if (code === 'UNASSIGNED') return 'No Department';
    const cfg = (typeof DEPT_CONFIG !== 'undefined' && DEPT_CONFIG[code]) || null;
    return (cfg && cfg.name) || code;
};

// deptCode: selected department, or '' for All (adds a Department column).
function _exportRptViewAllPDF(tableType, list, deptLabel, deptCode) {
    list = list || [];
    if (!list.length) {
        showToast('Nothing to export — the list is empty.', 'info');
        return;
    }

    const isFaculty = tableType === 'faculty';
    const termLabel = (typeof getReportTermInfo === 'function') ? getReportTermInfo().label : '';
    const pct = v => (v !== '\u2014' ? v + '%' : 'N/A');

    const showDept = !deptCode;
    const deptTh = showDept ? '<th>Department</th>' : '';
    const deptTd = t => showDept ? `<td>${escapeHtml(t.dept || 'N/A')}</td>` : '';

    const head = isFaculty
        ? `<tr><th>Teacher ID</th><th>Name</th>${deptTh}<th>Classes</th><th>Evals</th><th>SET Rating</th><th>SEF Rating</th></tr>`
        : `<tr><th>Teacher ID</th><th>Name</th>${deptTh}<th>SET Rating</th><th>SEF Rating</th><th>Status</th></tr>`;

    const rows = list.map(t => isFaculty
        ? `<tr><td>${escapeHtml(t.tid)}</td><td>${escapeHtml(listName(t))}</td>${deptTd(t)}`
          + `<td style="text-align:center;">${t.totalClasses}</td><td style="text-align:center;">${t.totalEvaluations}</td>`
          + `<td style="text-align:center;">${pct(t.overallSET)}</td><td style="text-align:center;">${pct(t.sefScore)}</td></tr>`
        : `<tr><td>${escapeHtml(t.tid)}</td><td>${escapeHtml(listName(t))}</td>${deptTd(t)}`
          + `<td style="text-align:center;">${pct(t.overallSET)}</td><td style="text-align:center;">${pct(t.sefScore)}</td>`
          + `<td style="text-align:center;">${escapeHtml(t.status || '')}</td></tr>`
    ).join('');

    const kind  = isFaculty ? 'Faculty Performance' : 'Supervisors';
    const title = showDept ? kind + ' \u2014 All Departments' : _rptDeptFullName(deptCode);

    const w = window.open('', '_blank');
    w.document.write(`<html><head><title>${escapeHtml(title)}</title><style>
        /* The page is a column at least one sheet tall; margin-top:auto on
           the signature block pushes it to the BOTTOM of the sheet. A list
           longer than one sheet keeps it right after the table instead. */
        @page{margin:12mm;}
        html,body{margin:0;}
        body{font-family:serif;font-size:12px;color:#111;}
        .page{min-height:calc(100vh - 2px);display:flex;flex-direction:column;box-sizing:border-box;padding:18px 18px 8px;}
        @media print{.page{padding:0;}}
        h1{font-size:16px;margin:0 0 2px;}
        .sub{color:#555;margin:0 0 16px;font-size:11px;}
        table{width:100%;border-collapse:collapse;}
        th,td{border:1px solid #ccc;padding:6px 8px;text-align:left;}
        th{background:#f2f2f2;}
        .sig{display:flex;justify-content:space-between;gap:60px;margin-top:auto;padding-top:56px;page-break-inside:avoid;}
        .sig > div{flex:1;max-width:280px;}
        .sig .lbl{margin-bottom:34px;}
        .sig .name{border-bottom:1px solid #111;min-height:16px;padding-bottom:2px;text-align:center;font-weight:bold;text-transform:uppercase;}
        .sig .role{text-align:center;font-size:11px;color:#333;margin-top:3px;min-height:14px;}
      </style></head><body><div class="page">
        <h1>${escapeHtml(title)}</h1>
        <div class="sub">${showDept ? '' : escapeHtml(kind) + ' &middot; '}${escapeHtml(termLabel)} &middot; ${list.length} ${isFaculty ? 'faculty member' : 'supervisor'}${list.length !== 1 ? 's' : ''} &middot; Generated ${new Date().toLocaleDateString()}</div>
        <table><thead>${head}</thead><tbody>${rows}</tbody></table>
        ${_rptSignatureBlock()}
      </div></body></html>`);
    w.document.close();
    w.print();

    addAudit('Export View-All PDF', `Exported ${tableType} list for ${deptLabel}`);
    showToast('Report ready — choose "Save as PDF" in the print dialog.', 'success');
}

// ===== SUBJECT DEPT FILTER PILLS =====
window._subjectDeptFilter = '';

function buildSubjectDeptPills() {
  const bar = document.getElementById('subjectDeptFilterBar');
  if (!bar) return;
  const DEPT_CONFIG = (typeof getDepartments === 'function') ? getDepartments() : {};
  const subjects = getData('subjects', []);
  const depts = Object.keys(DEPT_CONFIG);
  const current = window._subjectDeptFilter || '';
  const counts = {};
  subjects.forEach(s => { const d = s.dept || 'UNASSIGNED'; counts[d] = (counts[d]||0)+1; });
  const allCount = subjects.length;

  let html = `<button class="student-dept-filter-btn ${current===''?'active':''}" data-dept="" onclick="setSubjectDeptFilter('')">All <span class="dept-filter-count">${allCount}</span></button>`;
  depts.forEach(d => {
    if (!counts[d]) return;
    const cfg = DEPT_CONFIG[d];
    html += `<button class="student-dept-filter-btn ${current===d?'active':''}" data-dept="${d}" onclick="setSubjectDeptFilter('${d}')">${cfg ? escapeHtml(cfg.short) : d} <span class="dept-filter-count">${counts[d]}</span></button>`;
  });
  if (counts['UNASSIGNED']) {
    html += `<button class="student-dept-filter-btn ${current==='UNASSIGNED'?'active':''}" data-dept="UNASSIGNED" onclick="setSubjectDeptFilter('UNASSIGNED')">No Dept <span class="dept-filter-count">${counts['UNASSIGNED']}</span></button>`;
  }
```

```
  bar.innerHTML = html;
}

window.setSubjectDeptFilter = function(dept) {
  window._subjectDeptFilter = dept;
  buildSubjectDeptPills();
  const searchEl = document.querySelector('#page-subjects input[type="text"]');
  const search = searchEl ? searchEl.value : '';
  renderSubjects(search);
};
```

admin/js/reports/admin-sef.js

196 lines

```
// admin-sef.js - supervisor SEF entry (Annex B), audit log, modal helpers.
// Load order matters (see dashboard.html): the last definition wins.

// ===== SUPERVISOR EVALUATION (SEF) - ANNEX B =====
window.openSEFModal = function(teacherId) {
    const t = getData('teachers', []).find(item => item.id === teacherId);
    if (!t) return;

    // Use the published SEF instrument, not a hardcoded copy.
    const _sefSet = (typeof getPublishedQuestionSet === 'function')
        ? getPublishedQuestionSet('SEF') : null;
    const sefItems = (_sefSet ? _sefSet.questions : []).map(q => ({
        id: q.id, text: q.text, mov: q.mov || ''
    }));
    if (!sefItems.length) {
        showToast('No SEF questions published yet. Open Reports \u2192 Questions to set them up.', 'error');
        return;
    }
    const _sefSetId = _sefSet ? _sefSet.id : '';

    document.getElementById('reportModalTitle').textContent = `Supervisor's Evaluation: ${t.name}`;
    let html = `<div style="padding:10px;">
        <p style="font-size:0.8rem; margin-bottom:15px;">Rate based on 1-5 scale per CMO 19 Annex B.</p>
        <div style="margin-bottom:15px; padding:10px; background:#f0fdf4; border-radius:8px;">
            <strong>Faculty:</strong> ${escapeHtml(t.name)}<br>
            <strong>Department:</strong> ${escapeHtml(t.dept || 'N/A')}<br>
            <strong>Period:</strong> ${getActiveSY() ? getActiveSY().year + ' - ' + getActiveSY().activeSem : 'N/A'}
        </div>`;

    sefItems.forEach(item => {
        html += `
            <div style="margin-bottom:15px; padding-bottom:10px; border-bottom:1px solid #eee;">
                <div style="font-size:0.85rem; font-weight:600;">${item.text}</div>
                <div style="font-size:0.7rem; color:var(--muted); font-style:italic;">MOV: ${item.mov}</div>
                <div style="display:flex; gap:10px; margin-top:5px;">
                    ${[5,4,3,2,1].map(num => `
                        <label style="font-size:0.8rem;"><input type="radio" name="${item.id}" value="${num}"> ${num}</label>
                    `).join('')}
                </div>
            </div>`;
    });

    html += `<button class="btn btn-primary" onclick="saveSEFRating('${teacherId}','${_sefSetId}')">Submit SEF Rating</button></div>`;
    document.getElementById('reportModalBody').innerHTML = html;
    openModal('viewReportModal');
};

window.saveSEFRating = function(teacherId, setId) {
    // Iterate the PUBLISHED instrument; answers keyed by question id, version stamped.
    const set = setId ? getQuestionSetById(setId) : getPublishedQuestionSet('SEF');
    const items = (set && set.questions) || [];
    if (!items.length) { showToast('No SEF questions published.', 'error'); return; }

    const ratings = {};
    let totalScore = 0, answered = 0;

    items.forEach(q => {
        const val = document.querySelector(`input[name="${q.id}"]:checked`);
        if (val) {
            ratings[q.id] = parseInt(val.value);
            totalScore += parseInt(val.value);
            answered++;
        }
    });

    if (answered < items.length) {
        showToast(`Please rate all ${items.length} benchmark statements required by Annex B.`, 'warning');
        return;
    }

    // Percentage of the maximum possible (5 per item), not a fixed /75.
    // formula sa pag rate (SEF): rating = total score / (no. of items x 5) x 100
    const computedRating = ((totalScore / (items.length * 5)) * 100).toFixed(2);
    // Full list: it is written back whole just below.
    const evals = getData('evaluations', [], { includeHidden: true });
    evals.push({
        id: 'sef_' + Date.now(),
        teacherId: teacherId,
        evaluatorType: 'supervisor',
        ratings: ratings,
        questionSetId: set.id,        // which version these answers belong to
        totalScore: parseFloat(computedRating),
        timestamp: new Date().toISOString(),
        // Stamp BOTH semester and schoolYear so term filters match.
        schoolYear: getActiveSY()?.year,
        semester: getActiveSY()?.activeSem
    });

    setData('evaluations', evals);
    addAudit('Supervisor Evaluation', `Completed SEF for Teacher ID: ${teacherId} (${set.id})`);
    showToast('Supervisor Evaluation (SEF) saved successfully!', 'success');
    closeModal('viewReportModal');
    renderReports();
};

// ===== AUDIT LOG =====
function renderAuditLog(search = '') {
  const el = document.getElementById('auditLogList');
  if (!el) return;

```

```
  const q = String(search || '').toLowerCase();
  const raw = getData('auditLog', []);

  // Tolerate entries missing action/detail, and sort newest-first by timestamp.
  const log = raw
    .filter(l => l && typeof l === 'object')
    .filter(l => {
      if (!q) return true;
      return String(l.action || '').toLowerCase().includes(q)
          || String(l.detail || '').toLowerCase().includes(q);
    })
    .sort((a, b) => String(b.timestamp || '').localeCompare(String(a.timestamp || '')));

  if (!log.length) {
    el.innerHTML = raw.length
      ? '<div class="empty-state"><p>No audit entries match &ldquo;' + escapeHtml(search) + '&rdquo;.</p></div>'
      : '<div class="empty-state"><p>No audit logs yet. Entries appear here as you add, edit, or delete records.</p></div>';
    return;
  }
  el.innerHTML = log.map(l => `
    <div class="audit-row">
      <div class="audit-dot"></div>
      <div style="flex:1;">
        <div style="display:flex;align-items:center;justify-content:space-between;gap:12px;flex-wrap:wrap;">
          <span class="badge badge-primary">${escapeHtml(l.action || 'Unknown')}</span>
          <span class="audit-time">${escapeHtml(l.timestamp || '')}</span>
        </div>
        <div style="font-size:0.75rem;color:var(--muted);margin-top:4px;">${escapeHtml(l.detail || '')}</div>
      </div>
    </div>
  `).join('');
}

// ===== MODAL HELPERS =====
function openModal(id) {
    const modal = document.getElementById(id);
    if (modal) modal.classList.add('open');
}
function closeModal(id) {
    const modal = document.getElementById(id);
    if (modal) modal.classList.remove('open');
}

document.querySelectorAll('.modal-overlay').forEach(o => o.addEventListener('click', e => {
    if (e.target === o) o.classList.remove('open');
}));

function showConfirm(title, msg, cb) {
  document.getElementById('confirmTitle').textContent = title;
  document.getElementById('confirmMsg').textContent = msg;
  document.getElementById('confirmActionBtn').onclick = () => {
      closeModal('confirmModal');
      if (typeof cb === 'function') cb();
  };
  openModal('confirmModal');
}

// ===== HELPER FUNCTIONS =====
window.escapeHtml = function(str) {
    if (!str) return '';
    return str.replace(/[&<>]/g, function(m) {
        return { '&': '&amp;', '<': '&lt;', '>': '&gt;' }[m];
    });
};

// Soft Delete Functions
window.softDeleteTeacher = function(id) {
    const teachers = getData('teachers', []);
    const t = teachers.find(item => item.id === id);
    if (!t) return;

    showConfirm('Archive Teacher', `Archive ${t.name}? Evaluation history will be preserved.`, () => {
        const before = cloneRecord(t);
        t.deleted = true;
        t.status = 'archived';
        commitRecords('teachers', teachers, [{ before, after: t }]);
        addAudit('Archive Teacher', `Archived: ${t.name}`);
        if (typeof currentDept !== 'undefined' && currentDept && typeof renderDeptPage === 'function') renderDeptPage(currentDept);
        renderTeachers();
        showToast('Teacher archived successfully.', 'info');
    });
};

window.softDeleteSubject = function(id) {
    const subjects = getData('subjects', []);
    const s = subjects.find(item => item.id === id);
    if (!s) return;

    showConfirm('Archive Subject', `Archive "${s.name}"? Past evaluation data will be preserved.`, () => {
        s.deleted = true;
        setData('subjects', subjects);
        addAudit('Archive Subject', `Archived: ${s.name}`);
        if (typeof currentDept !== 'undefined' && currentDept && typeof renderDeptPage === 'function') renderDeptPage(currentDept);
        renderSubjects();
        showToast('Subject archived successfully.', 'info');
    });
};
```